

—— // POWERLINERS // ——

powerliners

v0.2.18

THE COMPANION

the segments · the daemon · the wire protocol



MENKE
TECHNOLOGIES

Copyright and Attribution

powerliners — the powerline statusline, rewritten as native code

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews and for the source excerpts identified below, which carry their own upstream licenses.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the powerline project, its maintainers, the Rust Foundation, or any other project named in these pages.

The software this book describes

`powerliners` is released under the MIT License, Copyright © 2026 MenkeTechnologies. The source is at github.com/MenkeTechnologies/powerliners.

The upstream this book is a port of

`powerliners` is a Rust port of **powerline-status**, and the upstream sources are vendored in the `powerliners` repository under `vendor/powerline/` as a read-only reference copy. `powerline` is the work of its own authors and contributors and is distributed under its own license terms.

`powerliners` does not invent a statusline format; it reimplements one, and it consumes the same configuration layout the upstream reads. Short excerpts of `powerline`'s source are quoted in this book for the purpose of commentary, criticism, and technical explanation — specifically, to show what a ported Rust segment was validated against. Every such excerpt is cited to the file it came from.



MT PUBLISHING



MT OPEN SOURCE

Contents

Copyright and Attribution	1
Before You Begin	6
A Shell-Prompt Primer	8
Chapter 1 — The Pitch	14
Chapter 2 — The Segment Model	17
Chapter 3 — The Built-in Segment Library	22
Chapter 4 — The Daemon and the Wire Protocol	27
Chapter 5 — The Performance Story	32
Chapter 6 — Themes and Colorschemes	35
Chapter 7 — Renderers	40
Chapter 8 — Bindings	45
Chapter 9 — Fidelity to Upstream	48
Chapter 10 — The Port Methodology	50
Chapter 11 — Packaging and Deployment	55
Chapter 12 — Concurrency, Backpressure, and the Daemon Under Load	58
Chapter 13 — The Lifetime of Warm State	62
Chapter 14 — The Trust Boundary	66
Chapter 15 — Authoring a Line: Latency, Priority, and Density	69
Chapter 16 — Writing a Native Segment	73
Chapter 17 — Debugging a Wrong Prompt	78
Chapter 18 — Status and the Honest Gaps	82
Chapter 19 — Cell Width: What the Terminal Actually Counts	86
Chapter 20 — Sampling, Freshness, and the Aliasing Problem	90
Chapter 21 — Locale, Encoding, and the Bytes That Are Not Text	94
Chapter 22 — Port Artifacts and the Diagnostic Surface	97

Chapter 23 — Adding a Renderer: The Other Extension Point	100
Chapter 24 — Four Architectures for a Prompt	103
Chapter 25 — Version Skew: When a Warm Daemon Outlives Its Binary	106
Chapter 26 — Geometry: Where Width Comes From, and Who Owns It	110
Chapter 27 — The Clock: Time as an Input to a Long-Lived Process	113
Chapter 28 — Where powerliners Fits	117
Chapter 29 — The Three NEAR Files, Opened	118
Chapter 30 — The Parity Harness Without an Interpreter	122
Chapter 31 — The Request Frame, Byte by Byte	126
Chapter 32 — Highlight Resolution, Observed From Outside	130
Chapter 33 — The Segment Registry as a Test Surface	133
Chapter 34 — A Census of the Test Suite	136
Chapter 35 — The Fixture Corpus as an Executable Specification	139
Chapter 36 — The Segment Set, Rendered One at a Time	142
Chapter 37 — The Colorscheme and Theme Resolution Chain	146
Chapter 38 — The Binding Layers As Shipped	151
Chapter 39 — The Config Format and Its Precedence Rules	155
Chapter 40 — The Python Oracle: How the Comparison Is Actually Made	159
Chapter 41 — The Daemon’s Lifecycle and Its Failure Modes	163
Chapter 42 — The Watcher, and Why Its “Debounce” Is Not a Timer	168
Chapter 43 — The Left Side Renders Multi-Chunk Segments Backwards	171
Chapter 44 — Two Segments the Bundled Theme Names and the Registry Does Not	174
Chapter 45 — The Gradient Crash Under the Shipped Configuration	176
Chapter 46 — What time powerline Cannot Show	179
Chapter 47 — Exit Codes, Surveyed	181
Chapter 48 — Degenerate Inputs, Executed	183
Chapter 49 — This Book’s Own Transcripts, Replayed	185
Chapter 50 — The Linter That Passes Everything	187
Chapter 51 — Which Numbers in This Book Are Stable	189
Appendix A — The Full Segment Catalog	191

Appendix B — The Config Schema Reference	197
Appendix C — The Color and Highlight-Group Reference	200
Appendix D — The Renderer Reference	203
Appendix E — The Daemon Protocol and Lifecycle Reference	206
Appendix F — The Parity Harness, the Tiers, and the Bug Log	209
Appendix G — Performance Measurement Methodology	212
Appendix H — A Complete Worked Example: From Config to Bytes	214
Appendix I — The Bindings Cookbook	216
Appendix J — The Custom-Segment Cookbook	219
Appendix K — Listers and <code>segment_list</code> Expansion	221
Appendix L — The Five Binaries in Detail	223
Appendix M — The Migration and Rollback Runbook	225
Appendix N — The Config Cascade, Worked	227
Appendix O — Glossary	229
Appendix P — The Truncation Algorithm, Worked	230
Appendix Q — Platform Notes: macOS, Linux, and the BSDs	232
Appendix R — The Extension Provenance Map	234
Appendix S — The Test Suite Map	236
Appendix T — The <code>segment_info</code> Reference	237
Appendix U — Worked Segment Outputs	239
Appendix V — The tmux Conf Files and Version Dispatch	241
Appendix W — Design-Rationale FAQ	242
Appendix X — The Divider Math, Worked	243
Appendix Y — Upstream Lineage and the Vendor Fixture	245
Appendix Z — Man-Page Cross-Reference	246
Appendix AA — Reading the Bundled Shell Theme	247
Appendix AB — Common Pitfalls and Their Fixes	249
Appendix AC — Quick-Reference Tables	251
Appendix AD — The <code>mergedicts</code> Deep-Merge, Worked	254
Appendix AE — The Divider Glyph Cascade and the Fallback Chain	256

Appendix AF — The Mode Machine, End to End	258
Appendix AG — Signal Handling and Async-Signal-Safety in the Daemon	260
Appendix AH — The poll Loop and What Changed from <code>select.select</code>	262
Appendix AI — The <code>powerline-lint</code> Check Catalog	264
Appendix AJ — The <code>segment_info</code> Environment Contract	266
Appendix AK — Building, Installing, and Cross-Compiling the Suite	268
Appendix AL — The Cost of Absence: None-Returning Segments	270
Appendix AM — The Config Acceptance Envelope	272
Appendix AN — Degenerate Inputs and Boundary Geometry	275
Appendix AO — Remote, Nested, and Containerized Sessions	278
Appendix AP — The Reproducibility Ledger	281
Colophon	283
Reproducing the Book’s Claims	284
Also from MenkeTechnologies	285

Before You Begin

This is the book about powerliners — the one you read in a chair, front to back, to understand not only *what* powerliners does but *why* it is shaped the way it is. It is not a man page. The man pages — `powerline(1)`, `powerline-daemon(1)`, `powerline-config(1)`, `powerline-render(1)`, and `powerline-lint(1)` — enumerate every flag and every subcommand, and you reach for them when you need one fact fast. This book is the other thing: the continuous account of the program, from the keystroke that triggers a prompt redraw to the `#[fg=...,bg=...]...` markup that lands in your statusline, and of the design decisions that let a Rust binary drop straight into a configuration written for a Python program.

powerliners is a Rust port of `powerline-status`, the canonical statusline and prompt renderer used in tmux, zsh, bash, vim, and the ipython REPL. The Python implementation pays a ~50–150 ms interpreter-startup tax on every render — every prompt redraw, every tmux statusline refresh, every window switch. On an interactive shell that redraws after keystrokes, that tax compounds into visible lag. powerliners is a single static binary: zero import cost, no garbage collector, sub-millisecond render, and — this is the point that governs the whole design — **drop-in compatible with the existing `~/.config/powerline` JSON theme and segment files**, so users keep their themes verbatim.

This book is honest about its foundation, because the honesty is the point. powerliners does not invent a statusline format; it reimplements one. The segment model, the theme grammar, the colorscheme cascade, and the daemon wire protocol are all inherited from upstream powerline, documented here faithfully and completely. The original work — the work that makes powerliners a MenkeTechnologies product rather than a rebuild — is the *engineering* of that port: a multi-binary suite of native clients, a UNIX-socket daemon that speaks the upstream C client's wire format, and a parity harness that runs the real Python interpreter on the upstream source and asserts byte-identical output.

Everything here is grounded in the program. Every JSON snippet is config the renderer actually parses; every segment named is one of the 58 adapters wired into the render runtime; every claim about the daemon is grounded in the running binary. Where a number or a behaviour is provisional this book marks it rather than rounding up: powerliners is at version `0.2.18` at the time of writing, with 134 of 137 upstream files at the DONE tier, 2,534 library tests, and 462 parity assertions run against the live Python interpreter. Two qualifications travel with that tier ratio everywhere it appears in this book, because both are measurable and neither is visible in the number: the DONE tier grades *citation* and *doccomment density*, not executed behaviour — `renderers/shell/bash.py` is DONE with a `do_render` that returns `String::new()` — and four of the 137 rows are counted from files that are not in the crate's module tree and therefore never compile at all (Chapters 18, 29, 36). powerliners is the work of MenkeTechnologies, built on the foundation of the powerline authors before him, and this book is grounded in that code.

Provenance: which tree every figure came from

A book whose claims are transcripts is worth exactly as much as the reader's ability to tell which checkout produced them. Several crates in this ecosystem exist as two working copies that drift apart in both directions, so naming the tree is not a formality.

Every measured figure in this book — every test count, every rendered byte, every timing, every source line number — comes from one checkout:

Path	<code>MenkeTechnologiesMeta/powerliners</code>
Commit	<code>8b5a8627bcbcb09aa8c327f9c73faac7b5766e99</code>

Authored	2026-08-02
<code>Cargo.toml</code> version	<code>0.2.18</code>
Binaries	<code>target/debug/</code> (a plain <code>cargo build</code> , not <code>--release</code>)

There is no second checkout. The search that establishes that, and the command that recovers the exact tree:

```
$ find ~/RustroverProjects/MenkeTechnologiesMeta -maxdepth 3 -name Cargo.toml -path  
~ '*powerliners*'  
/Users/.../MenkeTechnologiesMeta/powerliners/Cargo.toml  
  
$ git -C powerliners checkout 8b5a8627bc 66 cargo build
```

Two skews are live at the time of writing and both matter when reading a transcript. The tree above is one commit behind its remote, which has moved to `v0.2.20`; and the `powerline*` binaries on this machine's `$PATH` come from Homebrew and are `0.2.20`, not `0.2.18`. Transcripts in this book therefore invoke `target/debug/` explicitly rather than whatever `$PATH` resolves, except where a chapter says otherwise — and Chapter 46, which is about process cost rather than about the port, deliberately times the release binaries and labels them. Chapter 25 is about exactly this class of confusion; the front matter is not exempt from it.

One unnumbered section sits between this front matter and Chapter 1 — A Shell-Prompt Primer — for readers who have never built a prompt by hand; if you have, skip past it.

Let's start.

A Shell-Prompt Primer

Everything after this section assumes you have customised a shell prompt before. If you have, skip to Chapter 1 — nothing here is new to you. If you have not, this is the twenty minutes that makes the rest of the book legible: what a prompt is as a piece of data, why it is rebuilt from scratch before every command you type, what the word “powerline” names, and why a program whose whole job is drawing one line ships a daemon. It is deliberately short and deliberately not a reference; Appendix A is the segment catalog, Appendix C the colour reference, Appendix O the glossary. Most of what follows is ordinary shell behaviour that powerliners does not change; the last two sections are about what it does.

Every transcript below is real output. The powerliners transcripts come from the Homebrew-installed suite — `/opt/homebrew/bin/powerline` and its four siblings, all reporting `0.2.23`, a **release** build, matching the version in the checkout’s `Cargo.toml`. That is a different tree from the one the front matter names: no `target/` directory exists in the checkout at the time of writing and this section builds nothing, so none of the `0.2.18` debug binaries the provenance table describes could be run, and the timings below are release throughout. `command -v powerliners` finds nothing, which is expected — the crate is `powerliners` and installs five binaries all named `powerline*`:

```
$ for b in powerline powerline-daemon powerline-config powerline-render powerline-lint; do
  $b --version </dev/null; done
powerline 0.2.23
powerline-daemon 0.2.23
powerline-config 0.2.23
powerline-render 0.2.23
powerline-lint 0.2.23
```

Two things could not be shown. Nothing here ran against a real user configuration — every render below uses a four-file config tree in `/tmp` with `HOME` pointed at an empty directory — and a book made of bytes cannot show you a font, so the section on glyphs gives codepoints and byte sequences instead of pictures.

A prompt is a string the shell expands

A prompt is not a feature of your terminal. It is a variable holding a string, which the shell expands into bytes just before it waits for you to type. In `zsh` that variable is `PS1`, and `zshparam(1)` documents `PROMPT` as the same parameter; `bash` uses `PS1` too. Neither shell’s default is interesting, and both are worth seeing raw, because seeing them raw is the whole idea:

```
$ zsh -fic 'print -r -- $PS1' </dev/null
%mm%#
$ /bin/bash --noprofile --norc -i -c 'echo "${PS1-unset}"' </dev/null 2>/dev/null
\s-\v\$
```

Both shells were started with their configuration disabled (`-f`, `--norc`), so those are the built-in defaults, not anybody’s dotfiles. Neither string contains a host name or a username; each contains *instructions* for producing one. `Bash` writes them with backslashes — `man bash` gives `\s` as the name of the shell, `\v` as the `bash` version, and `\$` as “if the effective UID is 0, a `#`, otherwise a `$`”. `zsh` writes them with percent signs, and the ones that carry most real prompts, quoted from `zshmisc(1)`, are these:

%n	\$USERNAME		
%m	the hostname up to the first `.`	%M	the full machine hostname
%~	as %d, but \$HOME is replaced by `~`	%d	current working directory
%#	`#` when privileged, `%` when not	%?	return status of the last command
%j	the number of jobs	%D{...}	formatted using strftime

`print -P` applies exactly the expansion a prompt gets, which makes each of them testable without touching your own configuration:

```
$ zsh -f -c 'cd /tmp/promptrepo; print -P "%~ %# %D{%H:%M} %j"' </dev/null
/tmp/promptrepo % 22:05 0
```

Without `-P` the same string prints as itself: the percent codes are not shell syntax, they are a small language interpreted only during prompt expansion.

It is expanded again before every command

A prompt is a string of instructions rather than text because it is re-expanded from scratch every time the shell is about to read a line. Not once at login: every time. Here is an interactive zsh, started with no configuration, fed three lines on standard input, with its line editor and end-of-line markers switched off so only the prompts remain:

```
$ zsh -f -c 'cd /; printf "cd /tmp\ncd /usr/local\nexit\n" | PS1="[%~]$ " zsh -fi -o nozle
~ -o nopromptsp -o nopromptcr' </dev/null
[/]$ [/tmp]$ [/usr/local]$
```

Three prompts, one per input line, each showing where the shell stood at the moment it printed. `%~` was not evaluated when `PS1` was assigned; it was evaluated three times. Every command, every empty line, every `^C` runs the whole prompt-construction machinery again, and unlike a slow compiler you cannot go and do something else while it happens: you have just pressed Enter and you are waiting for permission to type the next thing. That is why prompt latency is the most-felt latency on a developer machine even though it is nowhere near the largest — single-digit milliseconds, sampled hundreds of times an hour, by someone whose hands are already moving.

The cost of asking other programs questions

A prompt that only asks the shell about itself is nearly free. A prompt that asks *git* what state the tree is in is not, because asking git means forking a process, usually more than one. The two scripts below are the cheap prompt and the naive git prompt, stripped to essentials:

```
# /tmp/prompt-cheap.zsh
cd /tmp/promptrepo
print -P '%~ %#'

# /tmp/prompt-git.zsh
cd /tmp/promptrepo
branch=$(git rev-parse --abbrev-ref HEAD 2>/dev/null)
dirty=$(git status --porcelain 2>/dev/null | wc -l)
print -P "%~ ${branch} ${dirty} %#"
```

`/tmp/promptrepo` is a git working tree of 244 tracked files, committed clean — a small repository by any standard. Under *hyperfine*, with the intermediate shell disabled so the numbers are the processes themselves:

```
$ hyperfine -N --warmup 10 --min-runs 100 'zsh -f /tmp/prompt-cheap.zsh' 'zsh -f
  ^ /tmp/prompt-git.zsh'
Benchmark 1: zsh -f /tmp/prompt-cheap.zsh
  Time (mean ± σ):      3.7 ms ±   0.4 ms    [User: 1.4 ms, System: 1.2 ms]
  Range (min ... max):   3.3 ms ... 11.3 ms    816 runs

Benchmark 2: zsh -f /tmp/prompt-git.zsh
  Time (mean ± σ):      30.8 ms ±   1.1 ms    [User: 20.9 ms, System: 38.5 ms]
  Range (min ... max):   28.5 ms ... 35.0 ms    101 runs

Summary
  zsh -f /tmp/prompt-cheap.zsh ran
    8.27 ± 0.84 times faster than zsh -f /tmp/prompt-git.zsh
```

Twenty-seven extra milliseconds after every command, to learn a branch name and a dirty count on a clean 244-file repository. Nothing in that script is badly written; it is the obvious way to do it, and it is what most hand-rolled prompts do. Scale the repository up and the gap widens; add a container context, a Kubernetes context, a virtualenv, and each is another fork. If the expensive part is *starting things*, the fix is to have something already started and ask it over a socket. Hold that thought until the last section.

Colour is bytes, and the shell has to count them

Terminals have no colour API. A colour is an in-band escape sequence: the byte `0x1B`, then `[`, then parameters, then a letter. `cat -v` prints escape as `^[:`

```
$ printf '\033[1;32mgreen\033[0m\n' | cat -v
^[[1;32mgreen^[[0m
```

Five visible characters wrapped in two directives — “bold, foreground green” and “reset” — and that ratio is the source of the most common prompt bug there is. The terminal prints nothing for the directive bytes, but the *shell* has to predict how many columns its own prompt occupies, because it is the shell, not the terminal, that decides where your typing starts, how a long line wraps, and how far back `^A` goes. Count bytes and you are wrong by the length of every escape sequence:

```
$ zsh -f -c 'p=$(print -P "%F{green}>%f"); print ${#p}' </dev/null
11
```

Eleven characters, one column. A shell that believed the 11 would redraw a wrapped command line on top of its own prompt, leave the cursor in the wrong place after `^R`, or eat the first line of a long paste — the classic corruption everyone has seen and few can explain. Both shells provide the same fix: a bracket meaning “these bytes are zero columns wide”. `zsh` spells it `%{ ... %}`; `bash` spells it `\[ ... \]`, which `man bash` defines as “begin a sequence of non-printing characters”. The markers are stripped during expansion and the bytes inside pass through untouched:

```
$ zsh -f -c 'print -Pn "%{X%}y"' </dev/null | od -c
0000000      X      y
0000002
```

Take one rule from this section: every raw escape sequence in a prompt must be wrapped, and any prompt generator you trust must emit the wrapping for the shell it talks to. Chapter 19 is the deep version, where the number the shell needs is not the byte count or the character count but the column count, and the right answer depends on software the renderer cannot see.

What “powerline” means

“Powerline” is a look with a data model behind it. The line is cut into **segments** — small independent producers, each returning a piece of text plus the name of a colour group: one for the host, one for the working directory, one for the git branch, one for the exit status. Chapter 2 is the segment as powerliners models it.

Between segments sit **dividers**, and the dividers are what people recognise. When two neighbours have different backgrounds the boundary is a solid arrow filled with the next segment’s background — a *hard* divider, the chevron the style is named for. When two neighbours share a background it is a thin mark in a muted colour — a *soft* divider. The theme author does not place them; they fall out of the colours, one decision per boundary.

Those arrows are not ASCII and they are not standard Unicode characters. They live in the Private Use Area — **U+E000–U+F8FF**, the range reserved for private agreement between a font and an application, which Unicode gives the **Co** general category:

```
$ perl -e 'printf "E0B0 Co=%s E000 Co=%s F8FF Co=%s F900 Co=%s\n",
  ~ (\p{Co}/?1:0), (\p{Co}/?1:0), (\p{Co}/?1:0), (\p{Co}/?1:0),
  ~ (\p{Co}/?1:0)'
E0B0 Co=1 E000 Co=1 F8FF Co=1 F900 Co=0
$ perl -CS -e 'print chr(0xE0B0), chr(0xE0B1)' | od -An -tx1
ee 82 b0 ee 82 b1
```

U+E0B0 is the hard divider and **U+E0B1** the soft one for a left-hand line; **U+E0B2** and **U+E0B3** are their mirrors for the right, and those four are exactly what the bundled top-level theme uses. A font without those glyphs renders them as boxes, blanks, or nothing, which is why every powerline setup begins by installing a *patched* font — an ordinary font with the powerline range grafted in — or a Nerd Font, which ships that range among its icon glyphs. Nothing about the renderer changes when the font is wrong: the bytes are correct and the picture is broken. Appendix AE is the fallback chain for that situation.

What is different about powerliners specifically

Four things matter before you start.

The segments are a fixed, compiled registry. Upstream powerline finds a segment by importing a Python module named in your theme; powerliners looks the same dotted name up in a static table — **pub const ADAPTERS: &[(&str, AdapterFn)]** in `src/bin/shared/render_runtime.rs`, 59 keys, 58 distinct adapters plus a short-form **exec** alias. It carries upstream’s names (`powerline.segments.common.env.cwd`, `powerline.segments.common.vcs.branch`, `powerline.segments.shell.last_status`) alongside names that are this port’s own (`powerliners.k8s.kubecontext`, `powerliners.docker.containers`, `powerliners.github.ci_status`, `powerliners.gpu.gpu_usage_percent`). Chapter 3 walks the library; Chapter 33 measures what the registry’s tests certify.

Your configuration is JSON on disk, and it is upstream’s JSON. A whole working config is four files. This one — a literal string segment plus the built-in **cwd** segment, with the divider glyphs from the previous section — is what every render below uses:

```
$ find /tmp/plprimer -type f | sort
/tmp/plprimer/colors.json
/tmp/plprimer/colorschemes/shell/default.json
/tmp/plprimer/config.json
/tmp/plprimer/themes/shell/default.json
$ powerline-lint -p /tmp/plprimer </dev/null; echo "rc=$?"
rc=0
```

config.json says which theme and colorscheme each extension uses; the theme lists the segments and the divider glyphs; the colorscheme maps highlight-group names to colour names; **colors.json** maps colour

names to terminal indices. Chapter 6 is the cascade that merges them, Appendix B is the schema, and Chapter 50 is worth reading before you trust a clean `powerline-lint` run.

One render is one command, and you can look at its bytes. `powerline-render` builds a line and exits. Below, `POWERLINE_CONFIG_PATHS` points it at the demo tree and `HOME` at an empty directory; the divider glyphs are rewritten to their codepoints and the non-breaking padding spaces to dots, so everything the renderer emitted is visible:

```
$ zsh -f -c 'cd /tmp/promptrepo; POWERLINE_CONFIG_PATHS=/tmp/plprimer HOME=/tmp/plhome
  ~ powerline-render shell left' </dev/null | perl -CSD -pe 's/\N{U+E0B0}/<E0B0>/g;
  ~ s/\N{U+E0B1}/<E0B1>/g; s/\N{U+00A0}/./g'
]
  ~ #[fg=colour252,bg=colour240,bold,noitalics,nounderscore].demo.#[fg=colour252,bg=colour240,nobold,noit
```

Everything in this primer is in that one line: `#[fg=...,bg=...]` directives carrying the colours, `.` where a non-breaking space pads a segment, `<E0B1>` soft dividers where the background stays `colour240`, one `<E0B0>` hard divider at the end where the background becomes the terminal's default. Two details are worth catching. The path reads `promptrepo`, `tmp`, `/` — outermost last — which is Chapter 43's defect, turning up here by accident on a shipped release binary rather than being argued for. And the directives are `#[...]` rather than shell escapes: Chapter 7 is renderers, Chapter 18 is the gap list, and Chapter 8's bindings are what turn a line into something a `PS1` can hold.

The daemon is the answer to the section on forks. `powerline-daemon` binds a UNIX socket, loads the config tree once, and answers requests in the wire format upstream's C client speaks — `parse_args` in `src/ported/scripts/powerline_daemon.rs` documents it as `<numargs_hex>\0<arg>\0<arg>...\0<cwd>\0<env=val>\0...`, and Chapter 4 covers the framing and why forwarding the client's whole environment and working directory is what lets a long-lived process render faithfully for a shell it is not related to. The `powerline` client is the other half: it sends that blob, prints the reply, and falls back to `powerline-render` when the socket is unreachable. Started on a private socket, against the same four files, it answers with the same line:

```
$ zsh -f -c 'cd /tmp/promptrepo; POWERLINE_CONFIG_PATHS=/tmp/plprimer HOME=/tmp/plhome
  ~ powerline shell left --socket /tmp/plprimer.sock' </dev/null | perl -CSD -pe
  ~ 's/\N{U+E0B0}/<E0B0>/g; s/\N{U+E0B1}/<E0B1>/g; s/\N{U+00A0}/./g; s/#\[([^\]]*\)]//g'
  ~ .demo.<E0B1>promptrepo.<E0B1>tmp.<E0B1>/.<E0B0>
```

And the honest measurement — both binaries release, both pointed at the same four files, one over the socket and one cold per call:

```
$ hyperfine -N --warmup 20 --min-runs 200 --export-markdown /tmp/pl-bench.md 'env
  ~ POWERLINE_CONFIG_PATHS=/tmp/plprimer HOME=/tmp/plhome powerline shell left --socket
  ~ /tmp/plprimer.sock' 'env POWERLINE_CONFIG_PATHS=/tmp/plprimer HOME=/tmp/plhome
  ~ powerline-render shell left' </dev/null >/dev/null 2>&1
$ cat /tmp/pl-bench.md
| Command | Mean [ms] | Min [ms] | Max [ms] | Relative |
|:---|:---|:---|:---|:---|
| `env POWERLINE_CONFIG_PATHS=/tmp/plprimer HOME=/tmp/plhome powerline shell left --socket
  ~ /tmp/plprimer.sock` | 7.1 ± 0.3 | 6.4 | 9.1 | 1.31 ± 0.11 |
| `env POWERLINE_CONFIG_PATHS=/tmp/plprimer HOME=/tmp/plhome powerline-render shell left`
  ~ | 5.4 ± 0.4 | 4.8 | 7.0 | 1.00 |
```

Read that table carefully, because it does not say what a reader expects a book's own benchmark to say. On a four-file config with two segments the socket round trip is *slower* than starting the one-shot renderer — 7.1 ms against 5.4 ms — and both are far under the 30.8 ms of the hand-rolled git prompt above. The daemon is not there to beat a Rust process start; it is there to beat an interpreter boot and a repeated config parse — the tax the front matter puts at 50–150 ms per render for the Python original — and to amortise a config tree and a segment set much larger than this one. Chapters 5 and 46 measure that properly, and Chapter 46 is specifically about what a naive `time powerline` cannot show.

Where to go from here

To run it, Chapter 11 is packaging and Chapter 8 is the shell and tmux bindings; `powerline-config` writes both sides for you, and it will tell you which client path the bindings are going to use:

```
$ powerline-config shell command </dev/null
powerline
```

To understand it, Chapter 2 is the segment model, Chapter 4 the daemon and the wire format, Chapter 6 the colour cascade. To extend it, Chapter 16 writes a native segment and Appendix J is the cookbook for doing it without touching Rust. And to find out what is missing rather than what works, Chapter 18 is the gap list and Chapter 51 says which of this book's numbers are stable — including the ones in this section.

Chapter 1 – The Pitch

A prompt is, on paper, a solved problem: shells have printed one for fifty years. Powerline made it beautiful — segmented, colored, divider-chevrons, aware of your git branch and your Kubernetes context — and it did so in Python, which was the right call for a configurable, plugin-rich renderer and the wrong call for something invoked on every keystroke. This chapter states the problem powerliners exists to solve and the one constraint that shaped every decision that follows.

The whole argument is one claim: the Python renderer pays its interpreter-startup cost on every invocation, and the daemon pays it once. Stated as a quantity, the warm render costs a UNIX-socket round trip plus the microseconds of the render itself, and Chapter 46 measures that at a median of **0.452 ms** over 400 requests.

An earlier printing of this chapter offered `time powerline shell left` as the demonstration, annotated “sub-millisecond after first render.” That command cannot show it and never could. What `time` measures there is a *process*, and the floor for starting any process on the machine these measurements come from is 7.6 ms — seventeen times the render it is supposed to be revealing. The client against a warm daemon measures 15.7 ms end to end, essentially all of it spawn cost, and it would still measure roughly that if the render were free. The architecture’s win is real and it is exactly where this chapter says it is; the instrument was wrong. Chapter 46 separates the two and gives the commands for each.

The Python startup tax

The tax is not a bug in powerline; it is the unavoidable cost of the runtime it runs on. When a shell renders its prompt, it forks a process, and that process must exist, do its work, and exit before the prompt can print. For a native binary “exist” means the kernel maps a static executable into memory and jumps to `main`, which is microseconds. For a Python program “exist” means starting the CPython interpreter, initializing its object system, and then importing the powerline package — `powerline`, `powerline.renderers`, `powerline.segments`, the color and theme machinery, and whatever `psutil`, `netifaces`, or `vcs` libraries the active segments pull in. Each import is a filesystem walk, a bytecode compile-or-load, and a module-init pass. On a warm page cache this is tens of milliseconds; cold, or on a network filesystem, or under a slow disk, it is well over a hundred. The powerliners README pins the range at ~50–150 ms per render, and that range matches what anyone who has profiled a Python powerline setup has seen.

Crucially, none of that startup work is the *render*. Resolving your git branch, formatting the time, and packing a colored segment into tmux markup is cheap — microseconds of actual computation. The interpreter startup is pure overhead paid before the useful work begins, and it is paid in full every single time, because each prompt render is a fresh process with no memory of the last one. You cannot amortize it within the Python model because the process that paid it has already exited by the time the next prompt needs rendering. This is the structural fact that makes a daemon the answer rather than a faster interpreter or a leaner import graph: the only way to stop paying startup on every render is to stop starting up on every render.

powerliners attacks the tax from both ends. The direct renderer, `powerline-render`, is a native binary, so even its cold path — used as the daemon-less fallback — is roughly **~30 ms** per call rather than a hundred, because there is no interpreter to boot. And the warm path, the daemon, removes startup from the equation entirely: the config tree is parsed once, the segment adapters are instantiated once, the color palettes live in memory, and each render is a socket round-trip plus the microseconds of the render itself. The measurement in the code fence above is the whole pitch compressed to two lines — same command, same arguments, same output, two orders of magnitude apart in latency, and the difference is entirely startup.

Why this matters per-keystroke, not per-session

If a prompt rendered once per shell session, a hundred milliseconds would be invisible and this book would not exist. It renders far more often than that, and the frequency is what turns a tolerable per-invocation cost into a felt one. In zsh, any `precmd` hook fires before each prompt is drawn, and prompts are redrawn not only after a command completes but on many interactive events; with a right-hand prompt (`Rprompt`) that also updates, a single command line can trigger several renders. In tmux the statusline refreshes on a timer — `status-interval`, often every one to five seconds — and it refreshes *per window*, so a session with eight windows multiplies the render count by eight on every tick. In vim the statusline redraws on mode changes, buffer switches, window focus changes, and on old vim versions on every cursor move. The pattern is the same everywhere: the prompt is not a one-time cost, it is a recurring one, and recurring costs are where latency becomes lag.

The per-keystroke case is the sharpest. A shell that redraws its prompt in response to typing — because a `precmd` or a continuation prompt or a completion UI triggers a render — pays the startup tax on the critical path between your keystroke and the screen updating. A hundred milliseconds there is not a background annoyance; it is the difference between a shell that feels instant and one that feels like a slideshow, in the README's phrase. Humans notice input latency above roughly the tens-of-milliseconds mark, and a hundred-millisecond render blows past it. This is the workload powerliners is tuned for, and it is why the daemon is the centerpiece: the goal is not to make one render fast, it is to make the ten-thousandth render of the day as fast as the first.

The multiplication is what makes the numbers stark. A developer with a tmux session of a dozen windows, a `status-interval` of two seconds, and a zsh prompt that redraws on interaction is triggering renders continuously through an eight-hour day. At a hundred milliseconds each that is real wall-clock time and real CPU spent booting an interpreter to do microseconds of work, over and over. At sub-millisecond each it disappears into the noise. powerliners does not make your prompt prettier — it renders the exact same prompt — it makes the prompt stop costing you anything to draw.

The one rule: keep the user's themes verbatim

There is exactly one constraint that outranks performance in powerliners' design, and it is compatibility. A faster prompt renderer that required you to rewrite your themes would be a different program that you would have to learn, migrate to, and maintain; the whole value of powerline is the ecosystem of themes, colorschemes, and segment configurations that people have accumulated and tuned over years. So the governing rule is stated plainly in the README and honored throughout the code: powerliners loads the user's existing `~/.config/powerline` JSON tree and resolves it *exactly* as upstream does. Your `config.json`, your `themes/`, your `colorschemes/`, your `colors.json` — all of it works unchanged, byte-for-byte.

This is not a compatibility *layer* bolted onto a new design; it is the design. The config cascade (`_find_config_files`, `load_json_config`, `mergedicts`) is a port of powerline's own resolution, so the same file in the same place merges the same way. The colorscheme resolution — alias chasing, cterm-to-hex, gradient picking — is a port of powerline's `Colorscheme` class, so a highlight group resolves to the same colors. The theme grammar — `segments.left` / `segments.right`, `extends` inheritance, `segment_data` overrides, per-mode `mode_translations` — is powerline's grammar. Even the segment *names* are the same, so a theme that names `cpu_load_percent` or `branch` finds an adapter of that name in powerliners' registry.

The payoff is a migration that touches nothing you have written. The upstream C client shipped with `pip install powerline-status` talks to powerliners' daemon unchanged, because they speak the same wire protocol; you can leave the Python client in `$PATH` and only swap the daemon. Your `~/.tmux.conf` invocations (`run-shell "powerline-config tmux setup"`) work as-is. Your `.vimrc` sources the same kind of plugin. There are no config edits, no `.tmux.conf` edits, no shell-rc edits — Chapter 11 walks the whole swap, and Chapter 9 is candid about the two or three places where output can legitimately differ (live-sampled numeric values, mainly), which are exactly the places where “identical” was never physically possible.

What “a port,” not “an inspired-by,” commits us to

Calling powerliners a *port* rather than a reimplementation-in-the-spirit-of is a commitment, and the project takes the word seriously. An “inspired-by” statusline would be free to make its own choices — a nicer config format, a cleaner segment API, different divider math — and would ask users to adopt them. A port is not free in that way. It is bound to reproduce the original’s behavior, including the behavior nobody would design on purpose: the exact merge semantics of overlapping config files, the exact fallback order when a highlight group is missing, the exact way `#` is doubled to `##[ ]` in tmux markup, the exact five-tuple shape that `gen_segment_getter` returns. Fidelity to those details is the product.

The commitment shows up structurally in the source tree. Ported code lives under `src/ported/`, mirroring upstream’s module layout — `src/ported/segment.rs` is the port of `powerline/segment.py`, `src/ported/colorscheme.rs` of `powerline/colorscheme.py`, `src/ported/renderers/tmux.rs` of the tmux renderer, and so on down to the scripts under `src/ported/scripts/`. Every function body carries `// py:NNN` citations pinning each line to the upstream source line it ports (Chapter 10), and a drift gate asserts that every ported function name matches `docs/powerline_py_functions.txt`. This is what “a faithful port” means in practice: not a vibe, but a mechanically-checked correspondence between two source trees.

The port also commits us to a boundary, and drawing it honestly is part of the deal. The behavior powerliners inherits is powerline’s; the segments that are *new* — `git_status`, `ci_status`, the cloud-context segments, the cache-stat segments for the sibling MenkeTechnologies tools — live under `src/extensions/`, explicitly outside the ported tree, and this book keeps them separate (Chapter 3 and Chapter 18). The MenkeTechnologies project bar kills “X but faster” and “X but in Rust” as products; powerliners survives that bar not by pretending the segment model is novel but because the *engineering* — the daemon, the wire-compatible client suite, the parity harness, and the new native segments — is the original work built on a faithfully-ported foundation.

Chapter 2 – The Segment Model

A powerline line is a list of *segments*: independent producers of a `{contents, highlight_group}` pair, laid out left-to-right (or split across a center gap), joined by hard and soft dividers, and padded. Everything visible is a segment. This chapter defines the segment as powerliners models it and walks the lifecycle of one from configuration to rendered cell.

Segments, highlight groups, and dividers

A segment is the atom of a powerline line. Conceptually it is a small function of the environment that returns a piece of text plus a name telling the renderer what color to paint it: the `hostname` segment returns your host name and asks for the `hostname` highlight group; the `branch` segment returns your git branch and asks for `branch`. The renderer never decides colors itself — it asks the colorscheme to resolve the segment's highlight group into concrete foreground, background, and attribute values (Chapter 6). This separation is why the same theme renders differently under different colorschemes: the segment says *what* and *which group*, the colorscheme says *what color that group is*, and the renderer says *how to encode that color for this target*.

A highlight group is just a name — `information:regular`, `branch_dirty`, `cpu_load_gradient` — that both the segment and the colorscheme agree on. A segment can offer several groups in priority order, and the colorscheme resolution walks them until one matches (Chapter 6's `get_highlighting`), which is how a segment degrades gracefully: a git segment might ask for `branch_dirty` first and fall back to `branch`, so a colorscheme that never defined `branch_dirty` still paints the branch in a sensible color rather than erroring. Some segments also carry a `gradient_level` — a number from 0 to 100 — that selects a color from a gradient list rather than a flat color, which is how `cpu_load_percent` shades from green through red as load climbs.

Between segments sit *dividers*, and the divider is where powerline's signature look comes from. Every segment pair is joined by either a **hard** divider or a **soft** divider, and which one is used depends on color: if two adjacent segments share a background, the boundary between them is soft (a thin separator drawn in a muted color); if their backgrounds differ, the boundary is hard (the solid chevron *'that fills with the neighboring background, producing the arrow effect'*). The renderer computes this per boundary — `divider_type = 'soft'` if the backgrounds match else `'hard'` in `render_length` — so the theme author does not place dividers manually; they fall out of the coloring. Around the whole line sits *outer padding* and per-segment *spaces*, small amounts of blank cells that keep the text from crowding the terminal edges and the dividers.

The three segment kinds: string, filler, and segment-list

Not every segment runs a function. powerliners, following upstream, recognizes three segment *types*, and `process_segment` (at `src/ported/segment.rs:692`) dispatches on the `type` field of the segment's JSON. The check is explicit: `process_segment` handles `"function"` and `"segment_list"` together at the top, and treats everything else — chiefly `"string"` — as a literal.

A **string** segment is a constant. Its `contents` come straight from the theme JSON, not from any function; it exists so a theme can drop a literal label, a fixed icon, or a separator into the line. In `gen_segment_getter` the string case stamps `type = "string"` and carries the literal contents forward. A **function** segment is the interesting kind: its `function` field names a segment adapter (by bare name like `hostname`, or by dotted path

like `powerliners.k8s.kubecontext`), and at render time that adapter runs against the environment and returns the contents. The vast majority of visible segments are function segments; the whole built-in library of Chapter 3 is function adapters. A **segment_list** (or “filler” / lister) segment is a segment that *expands into more segments*: it runs a lister (Chapter 7’s i3 workspaces, for example) that produces a variable number of sub-segments, each processed in turn. `process_segment_list` (at `src/ported/segment.rs:578`) handles that expansion.

The three types share one output shape but reach it differently, and `gen_segment_getter` is the function that normalizes them. It validates that a segment’s declared type is one of `"function" | "string" | "segment_list"` (it rejects anything else), and it produces, for each, the resolved four-or-five-tuple that the render loop consumes. The filler/lister case is what makes powerline able to render “one segment per tmux window” or “one segment per i3 workspace” without the theme knowing how many there will be — the count is discovered at render time and the lister fans it out.

Each kind is easiest to recognize from a one-line theme entry. A **string** segment is a literal label — `{ "type": "string", "contents": " | ", "highlight_groups": ["separator"] }` — whose `contents` come straight from the JSON, never from an adapter; `gen_segment_getter` stamps `type = "string"` and carries the literal forward, and at render time `process_segment` takes the `string` branch and uses the contents verbatim. It is how a theme drops a fixed icon, a label, or a divider glyph into a line without a function. A **function** segment — `{ "function": "cpu_load_percent" }` — is the common kind: its `function` names an adapter that runs against the environment at render time. A **segment_list** segment — `{ "type": "segment_list", "function": "workspace_lister", "segments": [ ... ] }` — carries a lister and a `segments` template, and expands into one instance of the template per item the lister yields (Appendix K). The key distinction is *when the contents exist*: a string segment’s contents exist at load time (they are in the JSON), a function segment’s exist at render time (the adapter produces them), and a segment_list’s `count` exists only at render time (the lister discovers it). All three converge on the same `{contents, highlight}` record the renderer consumes; they differ only in how that record is reached.

gen_segment_getter and the segment table

`gen_segment_getter` (at `src/ported/segment.rs:934`) is the factory that turns a segment’s JSON spec into a resolved, ready-to-run getter, and it is the hinge of the whole model. Its job is to take the raw segment dictionary from the theme — which might say `{ "function": "cpu_load_percent", "type": "function", "args": { ... } }` — and resolve it into a concrete record: the resolved function, the module it came from, the segment’s name, and (for string segments) its literal contents. It mirrors upstream’s `gen_segment_getter` line for line, including validating the type and routing string segments through the `get_string` path and function / segment_list segments through the `get_function` path.

The resolution of *which function* a segment names is `get_function` (also in `segment.rs`), and its output is the `SegmentGetterResult` struct — the direct port of the Python five-tuple. Its fields are worth naming exactly, because they recur throughout the render path:

```
pub struct SegmentGetterResult {
    pub contents: Option<String>,           // literal contents (string segments)
    pub function_name: Option<String>,      // resolved function name
    pub module: Option<String>,            // resolved module name
    pub function_name_dup: Option<String>,  // duplicate of function_name (py tuple shape)
    pub name: Option<String>,              // segment.get('name')
}
```

The fourth field, `function_name_dup`, is a duplicate of the second, and it exists for exactly one reason: the Python original returns a five-tuple in which the function name appears twice, and a faithful port preserves the shape even where it is redundant (the field’s own doccomment says “preserved for parity with py:70”). This is the port discipline of Chapter 1 made concrete — powerliners does not “clean up” the upstream tuple, because cleaning it up would be a behavioral change, and the goal is byte-and-structure fidelity, not a nicer API.

The “segment table” is where names become adapters. `get_function` resolves the dotted `function` string

to a `(module, function_name)` pair by `rpartition` on the last `.` — `powerliners.k8s.kubecontext` splits into module `powerliners.k8s` and function `kubecontext`, while a bare `hostname` falls back to the default module. That resolved name is then looked up in the `ADAPTERS` registry (Chapter 3) at render time. Upstream did this lookup by Python's dynamic `__import__`; `powerliners` cannot import Python modules, so the dotted path is resolved against a static registry instead — same names, same dispatch, no dynamic import. One consequence, documented as a known divergence, is that for `"type": "function"` segments the port currently uses the resolved `function_name` as the sole highlight group (`gen_segment_getter` at `src/ported/segment.rs:1011`) rather than honoring a theme's explicit `highlight_groups` override; Chapter 3's custom-segment section explains how to work with that.

process_segment and process_segment_lister

`process_segment` (at `src/ported/segment.rs:692`) is the per-segment driver. Its Rust signature carries the pieces the render needs — the side (`left` or `right`), the `segment_info` environment map, the mutable `parsed_segments` accumulator, the segment spec, the mode, the colorscheme, and a `contents_func` closure that actually invokes the adapter:

```
pub fn process_segment<C>{
    pl: &(),
    side: &str,
    segment_info: &Map<String, Value>,
    parsed_segments: &mut Vec<Value>,
    segment: &Map<String, Value>,
    mode: Option<&str>,
    colorscheme: &Colorscheme,
    contents_func: &C,
} where C: Fn(&(), &Map<String, Value>, &Map<String, Value>) -> Option<Value>
```

Its first act is to *copy* the segment (`segment = segment.copy()` at `py:168`), because processing mutates it — stamping the resolved highlight, the computed contents, the divider bookkeeping — and the theme's original must not be scribbled on, since the same theme dictionary may be rendered again on the next tick. It then branches on `type`: a `function` or `segment_list` segment runs its adapter (via `contents_func`) to produce contents and resolves its highlight group through the colorscheme; a `string` segment takes its contents literally. The output — a segment with resolved `contents` and a resolved `highlight` of `{fg, bg, attrs}` — is pushed onto `parsed_segments`, which is the list the renderer will later turn into bytes.

`process_segment_lister` (at `src/ported/segment.rs:578`) is the fan-out sibling. It takes a `lister` closure (which enumerates the things to render one segment each — workspaces, windows), a slice of `subsegments`, a `display_condition` predicate, and a `contents_func`, and for each item the lister yields it filters by `display_condition` and processes each subsegment against that item's patched environment. This is how “render one segment per X” works: the lister discovers the Xs at render time, and for each one the subsegment template is instantiated with that X's data folded into `segment_info`. It is more machinery than a single segment needs, which is why most themes never touch it directly — but it is what makes `powerline`'s window-lister and workspace-lister segments possible, and the port carries it faithfully, generic closures and all.

The two functions together are the segment model's runtime. `gen_segment_getter` prepares the getters once (when the theme is loaded); `process_segment` and `process_segment_lister` run them on every render, producing the flat `parsed_segments` list of `{contents, highlight}` records that Chapter 6's renderer consumes. Everything upstream of the renderer — every adapter in Chapter 3, every colorscheme lookup in Chapter 6 — feeds through this pair. When you read a theme and wonder how a line of JSON becomes a colored chunk on screen, this is the answer: `gen_segment_getter` resolved it, `process_segment` ran it, and the renderer painted its result.

A segment, traced from JSON to `parsed_segments`

Abstractions are easiest to trust once you have watched one concrete case walk the whole path, so here is a single segment doing exactly that. Suppose a theme's left side contains this entry:

```
{ "function": "cpu_load_percent", "type": "function",  
  "args": { "format": "{0:.0f}%" }, "priority": 30 }
```

When the theme loads, `gen_segment_getter` runs once over this dictionary. It reads `type` and validates it against `"function" | "string" | "segment_list"`; `"function"` passes. It routes to the `get_function` path, which `rpartitions` the `function` string on its last `.` — there is no dot, so the module falls back to the default segment module and the function name is `cpu_load_percent`. The result is a `SegmentGetterResult` with `function_name = Some("cpu_load_percent")`, `module = Some(<default>)`, `contents = None` (this is not a string segment), the redundant `function_name_dup = Some("cpu_load_percent")` carried for py-tuple shape, and `name = None` (the theme set no explicit `name`). That record is cached on the getter; the JSON is never re-parsed on later renders.

On each render, `process_segment` takes over. It copies the segment so it can stamp results without scribbling on the theme's original, sees `type == "function"`, and calls `contents_func`, which looks `cpu_load_percent` up in the `ADAPTERS` registry (Chapter 3) and invokes the adapter. The adapter samples CPU utilization — say 71% — formats it through the `{0:.0f}%` argument into `"71%"`, and returns a one-element list whose single dict is `{"contents": "71%", "highlight_groups": ["cpu_load_percent_gradient", "cpu_load_percent"], "gradient_level": 71.0}` — the exact shape the port's `sys.rs::render` builds at `src/ported/segments/common/sys.rs:212`. Note the two highlight groups in priority order and the `gradient_level` equal to the load.

`process_segment` then resolves color. It hands the two groups and the `gradient_level` of 71.0 to `Colorscheme::get_highlighting` (Chapter 6), which walks the groups, resolves `cpu_load_percent_gradient` to a gradient list, and picks the color at 71% of the way along it — a warm orange on a green-to-red gradient. The segment now carries a resolved `contents` of `"71%"` and a resolved `highlight` of `{fg, bg, attrs}`, and it is pushed onto `parsed_segments`. That flat record — text plus concrete colors plus the `priority` of 30 the theme set for truncation — is everything the renderer of Chapter 7 needs; it never looks back at the JSON, the adapter, or the colorscheme again. One line of theme configuration has become one entry in the list the renderer will pack into bytes.

Where a segment's arguments come from

The worked example above passed `args` inline in the segment's own dictionary, which is the simplest case and hides a small cascade worth knowing about, because a theme author who only knows the inline form ends up repeating themselves.

A segment's effective arguments are assembled from two places. The first is the `args` key on the segment entry itself — the form used above — which is local to that one appearance of that one segment. The second is the theme's `segment_data` map, which is keyed by segment name and supplies arguments (along with `before`, `after`, and a display `name`) that apply to every appearance of that segment in the theme. The motivation is repetition: a theme that shows `date` on both the left and the right, or shows the same segment across several derived themes, would otherwise have to restate its `format` argument at each site. `segment_data` states it once.

The interaction between the two is the part to get right, and it follows the same principle as the config cascade of Chapter 6: the more specific statement wins. An inline `args` on a particular segment entry overrides what `segment_data` said for that segment's name, so a theme can set a house default in `segment_data` and then deviate at one site without disturbing the others. This is the same first-hit-wins-per-key shape as `mergedicts`, applied one level down, and it means the mental model transfers: define broadly, override narrowly, and expect the narrow definition to win.

`before` and `after` deserve separate mention because they are not arguments to the adapter at all — the adapter never sees them. They are strings the *renderer* wraps around whatever contents the adapter returned, applied during layout (Chapter 7) rather than during segment execution. This is why they are the right tool for a static icon or label attached to a dynamic value: putting a branch glyph in `before` means the glyph is part of the segment's rendered width and its highlight, but the adapter's own logic never has to know about it, and a user who wants a different glyph edits the theme rather than the segment. It also means that `before` and `after` are applied even when the adapter returned multi-chunk output, wrapping the whole rather than each piece — a detail that matters when a segment like `git_status` produces several chunks and a theme tries to decorate it.

The last source of a segment's effective configuration is inheritance. A theme that `extends` another inherits its `segment_data` along with everything else, subject to the same deep merge, so a derived theme can adjust one argument of one segment by naming just that segment and just that argument. The whole point of the arrangement is that customizing a bundled theme should be a small diff rather than a fork: you say what differs, the cascade supplies the rest, and the segment sees the merged result without knowing which layer any of it came from.

Chapter 3 – The Built-in Segment Library

powerliners ships 58 segment adapters (59 `ADAPTERS` keys counting the bare `exec` alias), wired in `src/bin/shared/render_runtime.rs` and shared between the daemon and the direct renderer. This chapter is the field guide: what each adapter reads, what it emits, and where the Rust port matches or extends the Python original.

The `ADAPTERS` registry

Where upstream powerline resolved a segment name to a callable by importing a Python module and reaching for an attribute, powerliners resolves it against a static table. The `render_runtime.rs` header says so directly: the Rust port “replaces that with a static `ADAPTERS` registry.” Each entry is a `(name, adapter)` pair, and the render path looks a name up three ways, in order: the full dotted path (`powerliners.k8s.kubecontext`), then a bare-name fallback (the trailing component, `kubecontext`), then — for the codegen'd getters — the resolved tail again. The relevant lines in `render_runtime.rs` (`ADAPTERS.iter().any(|(k, _)| *k == full)`, then the bare-name and tail fallbacks around lines 478–514) are exactly this three-tier lookup, and it is what lets a theme write either the full dotted path or the short name and hit the same adapter.

Watching the three tiers resolve a real name makes the fallback concrete. A theme that writes `{"function": "powerline.segments.common.net.hostname"}` presents the string `powerline.segments.common.net.hostname` to the render path. Tier one asks: is that *exact* string a key in `ADAPTERS`? It is, so it resolves immediately. Now consider a theme that writes the short form `{"function": "hostname"}`. Tier one fails — `hostname` is not the full registered key — so tier two takes the trailing component after the last `.` (here the whole string, since there is no dot) and matches it against the registered keys' trailing components, finding `...net.hostname` and resolving to the same adapter. Tier three handles the codegen'd getters by re-resolving the tail. The upshot for a theme author is that the short name and the fully-qualified name are interchangeable — the bundled themes tend to use the qualified form for precision, but a hand-written theme can say `branch` or `cpu_load_percent` and hit the same closure. This is the port's static substitute for upstream's dynamic `__import__`: where Python resolved a dotted string by importing a module and reaching for an attribute, powerliners resolves the same string against a compile-time table, preserving the naming while removing the import.

An adapter is a Rust closure with a fixed signature — the man page states it as `fn(args: &Map<String, Value>, info: &Map<String, Value>) -> Option<Value>` — that takes the segment's `args` (from the theme JSON) and the `segment_info` environment (`cwd`, `environ`, the client's arguments) and returns either `Some` contents or `None` to render nothing. Returning `None` is how a segment *conditionally* appears: the `branch` segment returns `None` outside a git work tree, so it simply is not there rather than showing an empty box; the `kubecontext` segment returns `None` when there is no active context. This “return nothing” convention is load-bearing across the library and is why a busy-looking theme collapses to a clean prompt when its context is absent.

The 58 adapters divide roughly into upstream ports and MenkeTechnologies additions. The ports are the segments a longtime powerline user already knows — `hostname`, `user`, `cwd`, `branch`, `date`, `battery`, the player family, the shell-status family. The additions, all under `src/extensions/`, are the net-new native segments — `git_status`, `ci_status`, the cloud contexts, the cache-stat segments — that Chapter 18 returns to. The registry does not distinguish them at lookup time; a theme names any of them the same way. The

sections below walk the library by function, naming what each adapter reads and emits.

Environment and identity: `hostname`, `user`, `environment`, `internal_ip`, `external_ip`

The identity segments answer “where and who am I,” and they are the cheapest in the library because their data is already in the process. `hostname` emits the machine’s host name — useful the moment you have more than one shell open across more than one box, which is the whole reason it exists; it honors the upstream argument that toggles showing the fully-qualified domain name versus the short name. `user` emits the current user name and, following upstream, can highlight differently when you are root, so a red `root` segment is a standing warning that you are operating with privilege. Neither forks a process or touches the network; both read what the kernel already handed the process at startup.

`environment` is the general-purpose escape valve of the identity group: it emits the value of a named environment variable, chosen by the segment’s `variable` argument. It is how a theme surfaces anything the shell exports — a deployment stage in `$APP_ENV`, a cluster name, a custom marker — without a bespoke adapter. Because it reads the environment that the client forwarded to the daemon over the wire (Chapter 4), it sees the variables as they were in the shell that triggered the render, not as they are in the daemon’s own process, which is the subtle but correct behavior: the render must reflect the *caller*’s environment.

`internal_ip` and `external_ip` answer “what is my address,” and they split along the cost line that runs through the whole library. `internal_ip` reads a local interface address — cheap, local, no network round-trip — and is the one you put in a prompt you want to stay instant. `external_ip` asks the outside world what your public address is, which means a network request, which means latency and the possibility of hanging; it belongs in a statusline you refresh on a timer, not a prompt you redraw on every keystroke. The distinction is not academic: it is the difference between a segment that costs microseconds and one that can cost seconds if the network is down, and Chapter 5’s performance discussion returns to why the daemon’s on-demand sampling makes the placement of segments like this a real decision.

System load: `cpu_load_percent`, `mem_swap`, `disk_io`, `disk_usage`, `gpu_usage_percent`, `gpu_vram`

The system-load segments are the ones people put powerline on for — a live glance at the machine’s vitals in the corner of every window. `cpu_load_percent` emits CPU utilization as a percentage, and it is the canonical gradient segment: it carries a `gradient_level` equal to the load, so the colorscheme shades it from a calm color at idle through a hot color under pressure (Chapter 6’s `pick_gradient_value` is what turns that number into a color). On the sampling side the port diverges deliberately from upstream: where Python powerline leans on `psutil`, powerliners samples via OS subprocess probes — `top`, `vm_stat`, `netstat`, `pmset`, `uptime` on macOS, `/proc` and `/sys` on Linux — so it renders successfully on a machine with no `psutil` installed. The README documents this as an inherent divergence: the framing is identical, the numeric value may differ by a sampling-window’s worth of data.

The memory family — `mem_swap`, `mem_usage`, `mem_swap_percentage`, `mem_usage_percent` — reports RAM and swap in two shapes each: an absolute `USED/TOTAL` in human-readable bytes, and a percentage. These are ports of `mKaloer/powerline_mem_segment` living under `src/extensions/mem_usage.rs`, and their human-byte formatting (the `humanize_bytes` helper, ported from powerline’s own `lib/humanize_bytes.py`) is the same K/M/G rounding upstream uses, so a memory segment reads the same in both.

The storage and GPU segments are MenkeTechnologies additions with a vendor- dispatch pattern worth noting. `disk_usage` and `disk_usage_percent` report a filesystem’s used space for any mount; `disk_io` reports live read/write throughput for a device. `gpu_usage_percent` and `gpu_vram` report GPU compute and VRAM, and they dispatch across vendors in a fallback chain — `nvidia-smi`, then `rocm-smi`, then `intel_gpu_top`, then an `ioreg` fallback on macOS — so one segment works on a machine regardless of whose GPU is in

it. `thermal` rounds the family out with CPU/GPU temperature and fan RPM, from `/sys/class/hwmon` on Linux and `powermetrics/istats` on macOS. All of these live under `src/extensions/` and are wired into `ADAPTERS` as ordinary built-ins — no filesystem lookup, no custom-segment ceremony (Chapter 18).

Version control and orchestration: `branch`, `stash`, `kubecontext`, `containers`

Version control is powerline's most-loved feature and `branch` is its face. The `branch` adapter emits the current git branch name (falling back to a short SHA or tag on a detached HEAD) and returns `None` outside a work tree, so it appears exactly when it is meaningful. `stash` complements it with the count of stashed changes. Alongside the ported `branch`, powerliners ships a heavier native VCS segment, `git_status` (`powerliners.vcs.git_status`), which is p10k-style: a single chunk with the branch glyph, the name, and count badges for unstaged, untracked, staged, conflicted, ahead, behind, and stashed — all from a single `git status --porcelain=v2 --branch` fork. Where `branch` is the faithful upstream port, `git_status` is the richer MenkeTechnologies addition; a theme picks whichever fits.

It is worth seeing what `git_status` actually produces, because the single-fork p10k shape is its whole point. In a work tree on branch `main`, three files edited but unstaged, one new file untracked, two commits ahead of the remote and one stashed change, the adapter runs one `git status --porcelain=v2 --branch` (plus a `git stash list` and a `git config --get remote.origin.url`), parses the porcelain-v2 output in `parse_porcelain_v2`, and assembles a single chunk that reads roughly `main ↑2 *1 !3 ?1` — the branch glyph and name from the header, then the badges in the fixed order action, behind, ahead, stash, conflicts, staged, unstaged, untracked, each emitted only when its count is nonzero (so a clean tree collapses to just `main`). On a detached HEAD the header falls back through `git describe --tags --exact-match` to a tag, or to `@<shortsha>` from `git rev-parse --short HEAD`, and an in-progress operation (a stat of `MERGE_HEAD/rebase-merge/CHERRY_PICK_HEAD` and friends) prepends an action marker. With `status_colors` on, the chunk requests `["git_status_dirty", "git_status", "branch_dirty", "branch"]` when anything is dirty and the `_clean` equivalents otherwise, so a colorscheme paints the whole chunk red-on-dirty without the theme doing anything. The contrast with the ported `branch` is instructive: `branch` produces just the name in its own chunk and defers everything else to separate `stash` and status segments, faithful to upstream's decomposition; `git_status` folds all of it into one fork and one chunk, which is faster and denser but is new work, not a port — exactly the `src/ported` versus `src/extensions` boundary the book keeps drawing.

The orchestration segments bring cloud-native context into the prompt. `kubecontext` (`powerliners.k8s.kubecontext`) emits the current kubectl context and active namespace, honoring the `$KUBECONFIG` cascade and in-context namespace overrides, with a `hide_default` argument that suppresses the namespace when it matches the configured default — so the segment stays quiet when you are in the boring case and speaks up when you have switched to something that could bite you. This is the kind of segment where powerline earns its keep: a namespace you forgot you were in is how a `kubectl delete` hits the wrong cluster, and a persistent reminder in the prompt is cheap insurance.

`containers` (`powerliners.docker.containers`) reports Docker/OCI container counts — running, total, images, stopped — probing via `docker ps` and falling through silently when the daemon is unreachable, which is the right failure mode: a container segment on a machine where Docker is not running should vanish, not error. The broader cloud family (`aws.context`, `gcp.context`, `ci_status`) follows the same discipline — pure filesystem probes where possible, silent fall-through when the underlying tool or context is absent — and Chapter 18 catalogs them in full. The common thread across orchestration is graceful absence: every one of these segments knows how to render nothing, and does so the moment its context disappears.

Time, jobs, and status: `date`, `fuzzy_time`, `jobnum`, `last_status`, `last_pipe_status`

The time segments are the simplest adapters in the library and a good place to see the model with nothing else in the way. `date` emits the current date/time formatted by a strftime-style `format` argument from the theme — no environment read, no subprocess, just the clock and a format string. `fuzzy_time` is its playful sibling: instead of `14:37` it emits a human phrasing — “twenty to three,” “quarter past” — the same rounded-to-the-nearest-five-minutes prose powerline’s `fuzzy_time` produces. Neither costs anything to render, which is why they are safe in a prompt redrawn on every keystroke.

The shell-status segments are where the client’s forwarded arguments matter, and they are why the wire protocol carries `argv` and not just the environment. `last_status` emits the exit code of the previous command — but only when it is nonzero, so a successful command leaves no trace and a failed one paints its exit code in a warning color. It gets that exit code from the shell binding, which passes `--last-exit-code=$?` on the `powerline shell left` invocation (Chapter 7); the segment reads it out of the client’s arguments. `last_pipe_status` is the richer version: it shows the exit status of every stage in a pipeline (`$pipestatus` in `zsh`, `$PIPESTATUS` in `bash`), so a failure in the middle of a `a | b | c` is visible even when the last stage succeeded and `$?` is zero. This is the segment that catches the classic “the pipeline lied to me” bug.

`jobnum` emits the count of background jobs — the `[2]` that reminds you that you suspended two things and forgot them. Like the status segments, it is fed by the shell binding, which knows the job count and forwards it, because the daemon itself has no way to see the calling shell’s job table. This is a recurring theme in the shell-status group: the daemon is a separate process and cannot inspect the shell that called it, so anything shell-local — exit codes, pipe status, job counts, the current `vi` mode for the mode indicator — must be handed over on the wire. The segments read it from `segment_info` where the daemon deposited the parsed client arguments (Chapter 4), and this is the concrete reason the wire protocol forwards `argv` at all: without it, the whole shell-status family would be blind.

Media and the long tail: `itunes`, `cmus`, `dbus_player`, `email_imap_alert`

The player family is a small class hierarchy in upstream and a faithful one in the port. Powerline models “what is playing” through a common `PlayerSegment` base with per-player adapters, and powerliners mirrors that — `itunes`, `cmus`, `mocp`, `mpd`, `rhythmbox`, `clementine`, `spotify`, and the generic `dbus_player` all produce the same shape of output (track, artist, play state) through per-player probes. `dbus_player` is the generic Linux path, querying MPRIS over D-Bus so any compliant player works; the named adapters are the specific integrations for players that predate or sidestep MPRIS. The man page notes the port “faithfully mirroring upstream’s class hierarchy,” which is the point: the shape is inherited, not redesigned.

`email_imap_alert` is the long tail’s most characterful member — a segment that logs into an IMAP mailbox and emits your unread count. It is the archetype of a segment you never put in a prompt: it makes a network connection, it authenticates, and it can block, so it belongs only in a slowly-refreshed statusline where a second of latency now and then is invisible. It also illustrates why powerline’s per-segment placement matters — the same renderer that draws a microsecond `date` segment can draw a segment that talks to a mail server, and the theme author’s job is to keep the expensive ones off the interactive-latency path.

The long tail as a whole is where the library’s breadth shows and where the “return `None`” convention does its quietest work. Most of these segments are absent most of the time — no player running, no mail configured — and they cost nothing when absent because the adapter returns `None` before doing any expensive work. A theme can list a dozen of them and pay only for the ones that have something to say. This is the design that lets powerline be simultaneously feature-rich and fast: features you are not using are segments returning `None`, and a segment returning `None` is close to free.

The `exec` escape hatch

No fixed library can anticipate every segment a user wants, and upstream powerline handled that with dynamic import: drop a `.py` file in `~/.config/powerline/segments/`, reference it by dotted path, and `__import__` made it callable. powerliners is a static binary with no Python runtime, so it cannot import a `.py` at render time — and rather than pretend that limitation away, it provides two subprocess-based escape hatches that reach the same expressive power through a different mechanism, both wired through `src/extensions/exec_segment.rs`.

Pattern A is the explicit `exec` adapter. A theme references the built-in `exec` segment directly and spells out, inline, the command to run, its arguments, a format template, and the highlight groups: `{"function": "exec", "args": {"command": "/usr/local/bin/cpu_temp.sh", "args": ["--unit", "C"], "format": "%s°C", "highlight_groups": ["cpu_load"]}}`. The daemon forks the command, reads its stdout, and wraps the result as a segment. Pattern B is dotted-path filesystem dispatch: reference your segment by dotted path (`{"function": "myseg.cpu_temp"}`) and drop an executable under `<config_path>/segments/myseg/cpu_temp.{sh,py,rb,pl,lua,js}`. The daemon resolves the path against its full config cascade and runs the first match; the search order lives in `SCRIPT_EXTENSIONS` at `src/extensions/exec_segment.rs:31` — `["sh", "py", "rb", "pl", "lua", "js", ""]`, with `sh` winning over `py` when both exist and the empty string matching a bare executable file.

The output protocol is the same for both patterns and it is deliberately forgiving. The daemon inspects the script's stdout at the first non-whitespace byte: if it is `[`, the output is parsed as a JSON array of segment dicts and used verbatim — giving the script full control over `highlight_groups`, `gradient_level`, `divider_highlight_group`, and multi-chunk output; anything else is treated as plain text and wrapped as `[{"contents": <trimmed-stdout>, "highlight_groups": [...]}]`, with the `format` template applied (`%s` → contents, `%%` → literal `%`). So a two-line shell script that echos a number becomes a segment, and a Python script that prints a JSON array becomes a fully-styled multi-chunk segment — same adapter, two levels of ambition.

There is one Rust-port divergence to know when using `exec`, and the README states it plainly: `gen_segment_getter` (at `src/ported/segment.rs:1011`) uses the segment's resolved `function_name` as the *only* highlight group for function segments, ignoring a theme's `highlight_groups` override (upstream respects the override). The practical consequence is to name your colorscheme group to match the function name — a `"exec"` group for Pattern A, or the dotted path's trailing component (`"cpu_temp"`) for Pattern B, which is why Pattern B is the tidier of the two: its natural function name *is* a sensible group name. It is a small wrinkle, honestly documented rather than hidden, and Chapter 10's port methodology explains why a known divergence like this is tracked rather than papered over.

Chapter 4 – The Daemon and the Wire Protocol

The startup tax is defeated by never starting up: `powerline-daemon` binds a UNIX socket, loads the config tree once, and answers render requests over the same wire format the upstream C client speaks. This chapter is the heart of the performance story.

Why a daemon, and why a UNIX socket

The daemon exists because the only way to stop paying startup on every render is to render in a process that is already running. `powerline-daemon` is that process: one per UID, it loads the JSON config tree once, instantiates the segment adapters once, and holds the theme dictionaries and color palettes in memory (the man page's architecture section states exactly this). Every subsequent render is a message to the running daemon rather than a new process, so the config parse, the adapter setup, and — in the Python original — the interpreter boot all happen exactly once, at daemon start, and never again. The client that talks to it can be trivially cheap because all the expensive state lives on the daemon side.

A UNIX domain socket is the right transport for this because the client and daemon are always on the same machine and the message is small and synchronous. A UNIX socket is a local IPC channel with no network stack, no TCP handshake, and kernel-mediated delivery — a round-trip is microseconds, which is the budget a per-keystroke prompt render has to fit inside. It also gives natural per-user scoping: the socket is named per-UID, so your daemon serves your shells and only yours, with filesystem or abstract-namespace permissions doing the isolation. A TCP socket would add latency and a security surface for no benefit, since there is nothing remote to talk to; a named pipe would be clumsier for the request/response shape. The UNIX socket is the upstream choice and the port keeps it.

The single-daemon-per-UID design is what makes the whole thing scale to a real workflow. One `powerline-daemon` process handles your tmux statuslines, your zsh and bash prompts, and every running vim simultaneously — there is no per-vim daemon, no per-shell daemon, no per-window daemon. Start it once at login (`powerline-daemon -q` in `.zshrc` or `.profile`) and every powerline-aware program you launch for the rest of the session finds the socket already bound and warm. This is why the README recommends pre-starting the daemon for vim: with it running, each statusline refresh is a microsecond socket round-trip; without it, every refresh fork-execs `powerline-render` cold at ~30 ms.

The upstream wire format: `argv + cwd + env`

The wire format is the contract that makes powerliners a drop-in replacement, and it is copied exactly from upstream's C client (`vendor/powerline/client/powerline.c`, lines 126–148). The request the client sends is a single flat, NUL-delimited byte blob:

```
hex(argc-1) "\0" argv[0] "\0" argv[1] "\0" ... cwd "\0" env[0] "\0" ... "\0\0"
```

The first field is the argument count minus one, in hexadecimal, as text; then come the arguments themselves, each NUL-terminated; then the current working directory; then the environment as `KEY=VAL` entries, each NUL-terminated; and finally a double NUL to mark end-of-message. There is no length prefix on the whole

blob — the reader accumulates until it sees the terminating double NUL — and no encoding beyond raw bytes, which is what lets it carry arbitrary environment values.

The daemon's `parse_args` (at `src/ported/scripts/powerline_daemon.rs:254`) demarshals this, and its doccomment states the format verbatim: `<numargs_hex>\0<arg>\0<arg>...\0<cwd>\0<env=val>\0....`. It splits on NUL, reads the hex arg count, takes that many arguments, then the cwd, then parses the remaining entries as `KEY=VAL` environment pairs — and, mirroring upstream's `cwd = cwd or environ.get('PWD', '/')`, it falls back to the `$PWD` environment value when the cwd field is empty. The result is a `(Vec<String>, HashMap<String, String>, String)` — `argv`, `environ`, `cwd` — which is precisely the trio a render needs: the arguments say *what* to render (`tmux right, shell left --last-exit-code=1`), the environment says *in what context*, and the cwd anchors path-relative segments like `cwd` and `branch`.

Forwarding the *whole* environment and the cwd is what makes a render in the daemon faithful to the shell that requested it. The daemon is a long-lived process with its own stale environment and its own working directory; if it rendered against those, every segment that reads the environment (`environment`, the shell-status family, `$KUBECONFIG`-aware `kubecontext`) or the path (`cwd`, `branch`) would be wrong. By carrying the caller's `argv`, `cwd`, and `env` over the wire and rendering against *those*, the daemon produces exactly the output the caller would have gotten from a fresh in-process render — which is the entire correctness requirement for a drop-in daemon, and why cross-version clients and servers interoperate as long as they preserve this format.

The wire request, byte by byte

Nothing makes a protocol concrete like a hexdump, so here is exactly what the client writes for the invocation `powerline shell left --last-exit-code=1`, from a shell whose working directory is `/home/j` and whose environment (trimmed to two variables for the example) is `PWD=/home/j` and `TERM=xterm-256color`. The client's marshalling loop (`src/bin/powerline.rs:88-130`) produces, in order:

33 00	"3\0"	argc-1 = 3, in hex
73 68 65 6c 6c 00	"shell\0"	argv[1]
6c 65 66 74 00	"left\0"	argv[2]
2d 2d 6c 61 73 74 2d 65 78 69 74 2d 63 6f 64 65	"--last-exit-code=1\0"	argv[3]
3d 31 00		
2f 68 6f 6d 65 2f 6a 00	"/home/j\0"	cwd
50 57 44 3d 2f 68 6f 6d 65 2f 6a 00	"PWD=/home/j\0"	env[0]
54 45 52 4d 3d 78 74 65 72 6d 2d 32 35 36 63 6f	"TERM=xterm-256color\0"	env[1]
6c 6f 72 00		
00	"\0"	final terminator

Every field worth naming is visible in those bytes. The very first field, `33` = ASCII `3`, is the argument count minus one written in *hexadecimal text*, not a binary integer — a three-argument invocation (`shell`, `left`, `--last-exit-code=1`) writes `3`, and a seventeen-argument one would write `11`. Then come the three arguments, each terminated by a single NUL; then the working directory `/home/j`; then the environment as `KEY=VAL` pairs, each NUL-terminated. The message ends with a NUL, which — following the final NUL of the last env entry — forms the terminating double NUL `\0\0` that `do_read` watches for (`ends_with(b"\0\0")`).

On the receiving side, `parse_args` (`src/ported/scripts/powerline_daemon.rs:249`) runs this in reverse. It splits the blob on NUL and drops empty fragments, reads `usize::from_str_radix("3", 16)` to learn there are three arguments, slices `parts[1..3]` as `argv`, takes `parts[4]` (`/home/j`) as the `cwd`, and parses everything after as `KEY=VAL` by splitting each fragment at its *first* = (so a value containing `=`, like a base64 token in an env var, survives intact). Two guards matter here: an empty request returns `None` (the daemon replies `"malformed request"` rather than crashing), and an empty cwd field falls back to the `PWD` environment value and then to the literal `"/"`, mirroring upstream's `cwd = cwd or environ.get('PWD', '/')`. The result is the `(Vec<String>, HashMap<String, String>, String)` trio — `argv`, `environ`, `cwd` — that a faithful render needs, reconstructed exactly as the calling shell had it.

The reply, and the end of the connection

The request format got a byte-level treatment above; the reply deserves a paragraph because its *asymmetry* with the request is itself a design statement. The request is a structured, delimited, self-describing blob — a hex count, NUL-separated fields, a double-NUL terminator — because the daemon has to demarshal it into three typed things. The reply is none of those. It is the rendered markup, as bytes, and nothing else: no length prefix, no status code, no framing, no trailer.

The reason is that the client is a courier and the response has exactly one consumer shape. Whatever the daemon renders, the client writes to standard output, and whatever lands on standard output is what the shell's command substitution captures or what tmux paints into its statusline. There is nothing for the client to parse, so there is nothing to frame. Connection close *is* the terminator: the client reads until end-of-file, and end-of-file arrives because the daemon is done and the connection is finished with. This is the cheapest possible framing convention, and it is available precisely because the protocol is one-request-one-response over a connection that exists for exactly that exchange.

The absence of a status code has one visible consequence: errors travel in band. When `parse_args` cannot make sense of a request, the daemon's reply is its malformed-request message — text, on the same channel the markup would have used — and the client dutifully copies it to standard output like any other reply. A shell that substituted that into `PS1` would display the error message as its prompt. That sounds like a flaw and is actually the correct behavior for this system: a prompt renderer whose errors are visible in the prompt is a prompt renderer whose errors get noticed and fixed, whereas one that silently rendered an empty line would leave a user staring at a blank prompt with no idea why. The same reasoning explains why a colorscheme that cannot resolve any of a segment's highlight groups errors loudly (Chapter 6) rather than defaulting to some neutral color: loud failure is a feature in a component whose output a human is looking at continuously.

The connection's disposal is equally minimal. There is no keep-alive, no session, and no per-client state on the daemon side — a client that renders a left prompt and then a right prompt makes two connections, and the daemon has no notion that they came from the same shell. This is what makes the daemon's state model as simple as Chapter 13 describes: the only long-lived state is derived from configuration, never from clients, so there is nothing to expire, nothing to garbage-collect, and no way for an abandoned client to leak anything but the socket descriptor that the loop closes when the exchange ends. A protocol with sessions would have bought nothing here — the requests are independent by nature, since each one carries its own complete context — and would have cost exactly the per-client bookkeeping the design is free of.

powerline – the native client and its daemon-less fallback

`powerline` (the binary at `src/bin/powerline.rs`) is a 1:1 port of the upstream C client, `powerline.c` under `vendor/powerline/client/`. It is the thin client that shells and tmux and vim actually invoke, through `$POWERLINE_COMMAND`. Its job is small and fixed: build the wire request from its own `argv`, the current working directory, and the inherited environment; connect to the daemon's UNIX socket; send the request; and copy the daemon's rendered response to standard output. That is the entire happy path — a marshal, a socket write, a read, a copy — and it is why the client can be so much cheaper than the render it triggers. All the cost lives on the daemon side; the client is just a courier.

The client's one interesting decision is what to do when the courier finds no one home. If the daemon socket is not reachable — the daemon was never started, or was killed, or crashed — `powerline` does not error and does not hang; it execs `powerline-render` as a fallback, mirroring the upstream C client's behavior exactly. `powerline-render` is the direct, daemon-less renderer: it parses the config, runs the segments, and emits the same markup, all in one process. It is slower than the warm daemon path (a cold native render, ~30 ms, versus a sub-millisecond socket round-trip) but it is still an order of magnitude faster than the Python interpreter path, and — critically — it is never *broken*. A prompt that falls back to `powerline-render` still renders; you lose speed, not function.

This fallback is the resilience story of the whole design. A daemon can die — a reboot, an OOM kill, a manual `kill` — and if the prompt depended on it absolutely, a dead daemon would mean a broken shell. Because the client transparently degrades to direct rendering, a dead daemon means a *slower* shell that quietly keeps working until the daemon is restarted (or until the next login re-runs `powerline-daemon -q`). The `--socket PATH` option lets the client target a non-default socket, and it must be the first argument, before the extension name, per the C-client convention; absent it, the client uses the platform default socket path (the next section).

Bind, daemonize, pidfile lock, accept loop, EOF shutdown

Starting the daemon is a sequence of careful steps, all ported from upstream's `scripts/powerline-daemon`, and the port keeps each one. First it takes a **pidfile lock** so two daemons cannot race for the same socket; the port implements this both as a `PidLock` RAI guard (released on `Drop`) and by the classic lock-the-pidfile mechanism, so a stale pidfile from a crashed daemon does not permanently block a new one. Then it **binds** the UNIX socket. `check_existing` (at `src/ported/scripts/powerline_daemon.rs:1018`) attempts the bind and returns `Ok(None)` on `EADDRINUSE`, which is how the daemon detects that another instance is already serving that socket — the basis for the `--replace` and plain-startup behaviors.

Then, unless `--foreground` was passed, it **daemonizes**. `daemonize` (at `src/ported/scripts/powerline_daemon.rs:925`) is the textbook double-fork: `fork`, exit the parent; `setsid` to detach from the controlling terminal; `fork` again so the process can never re-acquire a terminal; `chdir /`; `umask 0`; and `dup2` the standard descriptors to `/dev/null`. This is what the `-q` flag's "double-forks and detaches" means — after it, the daemon is a session-leaderless background process with no terminal, no controlling shell, and no stray file descriptors, exactly as a long-lived server should be. `--foreground` skips this entirely and keeps the daemon attached to the terminal, which is what a systemd `Type=simple` unit or an interactive debugging session wants.

With the socket bound and the process detached, the daemon enters its **accept loop** — `main_loop` (at `src/ported/scripts/powerline_daemon.rs:854`), which multiplexes over the listener and the live connections. Where upstream's Python used `select.select`, the port uses `libc::poll` (a documented, behavior-neutral substitution: `POLLHUP` is folded into "readable" so a peer hangup reads as EOF, exactly as `select` reported it). When the listener is readable it accepts a new connection (through `eintr_retry_call`, so an `EINTR`-interrupted `accept` retries rather than dying); when a connection is readable it reads a full request up to the terminating double NUL — `do_read` loops until the buffer `ends_with(b"\0\0")` or a two-second read timeout elapses — checks it, and either renders and replies or shuts down. The **shutdown sentinel** is the special request `EOF = b"EOF\0\0"` (at `src/ported/scripts/powerline_daemon.rs:73`): if a client sends exactly that, `get_answer` returns the exit signal and the daemon exits cleanly with status 0. This is how `powerline-daemon -k` stops a running daemon — it connects and sends the EOF sentinel — and it is a faithful port down to the exact byte sequence, because a client written against the Python daemon must be able to stop the Rust one and vice versa. The daemon exits cleanly on `SIGTERM` — the handler (`sigterm_handler` at `powerline_daemon.rs:1069`) unlinks the pidfile and `_exits` in an async-signal-safe way — and treats `SIGINT` as a clean exit-0, mirroring upstream's `KeyboardInterrupt` → `SystemExit(0)`. One lifecycle detail is an honest omission rather than a port: upstream's `SIGHUP`-driven re-exec (reload config without dropping the socket) is *not* implemented. A config edit is instead picked up by the per-`ext` mtime-staleness reload in `src/bin/powerline-daemon.rs`, which rebuilds the config cache when the underlying files change, and a full restart is available via `--replace`. It is exactly the kind of divergence Chapter 10's methodology tracks in the open rather than papering over.

The socket paths and the two namespaces

Where the socket lives depends on the platform, and the split is inherited from upstream's `USE_FILESYSTEM = not sys.platform.lower().startswith('linux')`. The port's `USE_FILESYSTEM()` (at `src/ported/scripts/powerline_daemon.rs:63`) is `!cfg!(target_os = "linux")` — Linux gets one behavior, everything else the other. On macOS, FreeBSD, OpenBSD, and NetBSD the socket is a real

filesystem path, `/tmp/powerline-ipc-<uid>`, an actual file you can see with `ls`. On Linux the socket is an *abstract-namespace* socket, `\0powerline-ipc-<uid>` — the leading NUL marks it as abstract, meaning it has no filesystem entry at all; it appears only in `ss -x` and `lsof -U` under the NUL marker.

The abstract namespace is not an arbitrary choice; it solves a real problem that filesystem sockets have. A filesystem socket file persists after the daemon dies — a crash or a hard kill leaves the socket file behind, and a `/tmp` cleanup job or a race can delete a *live* socket out from under a running daemon. An abstract socket has no filesystem presence, so it cannot be orphaned by a crash or swept by a `/tmp` cleaner: it exists exactly as long as the daemon holds it and vanishes the instant the daemon exits. The port's bind logic (around `powerline_daemon.rs:1227`) is careful here — it unlinks any non-abstract address before binding regardless of the platform default, so an explicit `--socket /some/path` (a filesystem path) still cleans up a stale file, while the default Linux path uses the abstract namespace and needs no cleanup at all.

Both paths are overridable with `--socket PATH`, which is how the migration workflow runs a throwaway daemon on a probe socket (`--socket /tmp/powerliners-probe`) without disturbing the real one, and how a user with an unusual setup can put the socket wherever they need it. The default per-UID naming (`powerline-ipc-<uid>`) is the same on both namespaces, so the *identity* of the socket — “this UID's powerline daemon” — is stable across platforms even though its physical form differs. The matching pidfile, when used, is `/tmp/powerline-ipc-<uid>.pid`, which is what the migration guide's `ps -p $(cat /tmp/powerline-ipc-$UID.pid)` check reads to confirm the running daemon is the Rust port and not the Python shebang.

The five binaries: `powerline`, `-daemon`, `-render`, `-config`, `-lint`

powerliners ships not one binary but five, and the split mirrors upstream's own division of labor so exactly that each Rust binary names the upstream file it replaces. Together they are the whole CLI surface a powerline user touches, and the Cargo manifest declares them explicitly (`[[bin]]` entries for each) so `cargo install` and `cargo publish` ship the complete suite.

`powerline` (mirrors `client/powerline.c`) is the native client of the previous sections — the thin courier that forwards `argv + cwd + env` to the daemon and falls back to `powerline-render` when the socket is unreachable. `powerline-daemon` (mirrors `scripts/powerline-daemon`) is the server — bind, daemonize, pidfile lock, accept loop, EOF shutdown — rendering real `#[fg=...,bg=...]...` tmux markup from a user's config root. `powerline-render` (mirrors `scripts/powerline-render`) is the direct, daemon-less renderer: `argparse` plus extension lookup plus the full render path through the `Powerline` orchestrator; it is both a standalone tool and the client's fallback.

`powerline-config` (mirrors `scripts/powerline-config`) is the setup and dispatch tool — it is what `run-shell "powerline-config tmux setup"` in your `.tmux.conf` invokes to install the tmux integration, and what `powerline-config vim source-path` invokes to extract the bundled vim plugin (Chapter 7). It handles the tmux/shell known-function dispatch that upstream did in its config script. `powerline-lint` (mirrors `scripts/powerline-lint`) is the config validator: `argparse` plus the full check pipeline — a markedjson loader, the `Spec` checks, and orchestrator integration — that flags unknown segment names, missing color references, and malformed `before/after` formatting strings before they become a broken prompt (Chapter 9's fidelity work leans on it). Five binaries, one config tree, one wire protocol — the suite is the whole of what `pip install powerline-status` gave you, reimplemented as native code.

Chapter 5 – The Performance Story

The daemon is the mechanism; performance is the payoff, and this chapter states it in numbers and in the honest shape of where the wins come from. `powerliners` is fast for a few specific, nameable reasons, and it is worth separating them from each other rather than blurring them into “it’s written in Rust.”

The startup tax, measured

The headline number is the one from Chapter 1, and it bears restating precisely because it is the whole justification for the architecture. Python `powerline` pays ~50–150 ms per render, essentially all of it interpreter startup and module import, before any useful work begins. `powerliners`’ warm daemon path is sub-millisecond: a UNIX-socket round-trip plus the microseconds of the actual render. That is two to three orders of magnitude, and the gap is not from a cleverer render algorithm — the render itself was always cheap — it is from *deleting startup* as a per-render cost.

The cold path matters too, because it is the fallback and because not every invocation is served by a warm daemon. `powerline-render`, the daemon-less direct renderer, runs cold at roughly ~30 ms per call — the README’s figure for the vim statusline case when no daemon is running. That is still three to five times faster than Python’s cold path, because even without a warm process there is no interpreter to boot: a native binary’s cold start is the kernel mapping a static executable, not CPython initializing its object system and walking an import graph. So the performance story has two tiers — sub-millisecond warm, ~30 ms cold — and both beat the thing they replace, with the warm tier beating it by a margin large enough to change how a prompt *feels*.

The reason the tax is so total in Python and so absent in `powerliners` is structural, not incidental, and Chapter 1 made the case: a fresh process cannot amortize its own startup, so a model built on fresh-process-per-render pays startup every time by construction. The daemon breaks that construction. It is not that `powerliners` optimized startup; it is that `powerliners` arranged for startup to happen once. This is why “rewrite it in Rust” alone would not have solved the problem — a Rust binary that still forked per render would pay a Rust binary’s (small) startup every time, which is better than Python’s but not sub-millisecond. The daemon is the load-bearing idea; the native language is what makes the daemon’s client cheap and its cold fallback tolerable.

The warm daemon and its caches

A warm daemon is fast for more reasons than “it is already running.” Its state is built once and reused: the config tree is parsed once (the cascade of Chapter 6), the theme dictionaries and color palettes are held in memory, and the segment adapters are wired once. The daemon also keeps a `Powerline`-instance cache keyed by the things that determine which orchestrator to use — the `PowerlineKey` struct (at `src/ported/scripts/powerline_daemon.rs`) is a port of upstream’s cache tuple: the extension, the renderer module, the config/theme/path overrides, and the `POWERLINE_THEME_OVERRIDES` / `POWERLINE_CONFIG_OVERRIDES` / `POWERLINE_CONFIG_PATHS` environment values. Two renders that share those keys share a warm orchestrator; a render that differs in them gets its own, built once and then cached.

The consequence is that the *second* render of a given kind is even cheaper than the first, because the first built and cached the orchestrator and the second just reuses it. In a real session almost every render is a second render — the same tmux statusline, the same shell prompt, over and over — so the amortized cost settles to the floor: socket round-trip plus render, with all the setup already paid. This is the daemon doing what a daemon is for, keeping warm state warm across the thousands of renders a day produces.

The cache key makes “the same kind of render” precise, and it is worth seeing what shares a warm orchestrator and what does not. `PowerlineKey` (Appendix E.7) is a tuple of the extension, the renderer module, the config/theme/path overrides, and the three environment values `POWERLINE_THEME_OVERRIDES` / `POWERLINE_CONFIG_OVERRIDES` / `POWERLINE_CONFIG_PATHS`. Consider a real session: a `powerline shell left` from `zsh` and a `powerline shell right` from the same `zsh` differ in *side* — but *side* is not part of the key (it is a render argument, not a cache dimension), so both hit the same warm `shell` orchestrator, and only the second one ever paid to build it. A `powerline tmux left` from `tmux` differs in *extension* (`tmux` vs `shell`), so it keys to a different orchestrator, built once on the first `tmux` render and reused thereafter. A `vim` statusline keys to a third. So a developer living in `zsh` under `tmux` with `vim` open warms exactly three orchestrators — one per extension — and every subsequent render of any of the three reuses its warm instance. The key is deliberately coarse in the dimensions that do not change the orchestrator (*side*, the exit code, the job count — all render arguments) and precise in the ones that do (extension, renderer, overrides), which is what lets a handful of warm orchestrators serve an entire day’s tens of thousands of renders. The first render of each kind pays setup; the second through ten-thousandth pay only the socket round-trip and the render, which is the floor the whole architecture drives toward.

There is a deliberate trade-off in how the daemon samples live data, and the README is candid about it. Where Python powerline’s `ThreadedSegment` polls expensive data (network, disk) in a background thread and renders the last-known value, powerliners’ daemon samples on demand at render time. The upside is simplicity and freshness — no background threads, no stale cached value. The downside is a different latency profile: a render that hits a network or disk segment may block briefly while it samples, where the threaded model would have returned instantly with a slightly-old value. The *content* matches; the timing differs. This is why segment placement is a real decision (Chapter 3): an `external_ip` or `email_imap_alert` segment on the interactive-prompt path can occasionally stall the prompt, and a theme author who cares about per-keystroke latency keeps the expensive samplers on the timer-refreshed statusline instead.

Why “in Rust” is necessary but not sufficient

It would be easy to summarize powerliners as “powerline, but in Rust, so it’s fast,” and the summary would be wrong in an instructive way. Rust is necessary — a static binary with microsecond process startup and no garbage collector is what makes the client cheap and the cold fallback tolerable — but Rust is not where the two-orders- of-magnitude win comes from. That win comes from the *daemon*, an architectural choice that could in principle have been made in Python too (and upstream did make it, which is why upstream has a `powerline-daemon` at all). What Rust adds on top of the daemon is that the client and the cold path are both cheap in absolute terms, and that the whole suite is a single static artifact with no runtime to install per machine.

The honest decomposition is three separate wins, and this book keeps them separate the way Chapter 1 promised. First, the daemon deletes per-render startup — the big win, available in principle to any language. Second, the native binary makes the client and the cold render cheap — a Rust win, and the reason powerliners’ fallback path is ~30 ms rather than Python’s ~100. Third, the single static binary makes deployment trivial (Chapter 11) — no interpreter, no `pip` tree, no shared library to keep in sync. Conflating these into “it’s fast because Rust” obscures which lever does what, and the whole design reads more clearly when you keep them apart: the daemon is the architecture, Rust is the implementation, and the static binary is the packaging.

None of this makes the render *algorithm* faster, and it is worth being clear that powerliners does not claim to. The render was never the bottleneck; resolving a branch and packing `tmux` markup is microseconds in any competent implementation. What powerliners removes is the overhead *around* the render — the startup, the import, the per-invocation process cost — which is where all the time actually went. A faithful port that rendered identically but paid Python’s startup would be no faster; a native rewrite that still forked per render would be only somewhat faster. It is the combination — same faithful render, no per-render startup, cheap native client — that produces the number in the code fence of Chapter 1.

The latency budget, accounted

“Sub-millisecond” is a claim worth spending, because a budget accounted line by line is harder to hand-wave than a single headline number. The warm path is a fixed sequence of steps, each with a nameable cost, and it is instructive to walk them in order and see where a microsecond can and cannot hide. The client’s half is a marshal and two syscalls: it builds the `argv + cwd + env` blob in a single owned buffer (`powerline.rs:88-130`), which is a handful of `pushes` and no allocation surprises, then `connect(2)`s the UNIX socket and `write(2)`s the blob. A UNIX-domain `connect` and `write` to a listening peer on the same machine are kernel-local operations — no routing, no handshake, no copy across a network stack — so the pair costs single-digit microseconds on a warm scheduler. The client then blocks in `read(2)` for the reply, which is where it spends most of its wall-clock time: not computing, but waiting for the daemon to answer.

On the daemon’s half the budget divides into three visible slices, and only one of them is the render. The first slice is transport: `poll` returns the listener or the connection as readable, `do_read` accumulates the request until it `ends_with(b"\0\0")` (`powerline_daemon.rs:296`), and `parse_args` splits it on NUL into the `(argv, environ, cwd)` trio (:249). For a typical prompt request — a few arguments and a few dozen environment entries — this is a bounded number of byte scans and small-string allocations, tens of microseconds at most, dominated by the size of the forwarded environment rather than by any computation. It is the one slice that grows with the caller’s `env`, which is why a shell with a pathologically large environment can make even the warm path measurably slower: the daemon must demarshal every `KEY=VAL` the client sent, whether or not any segment reads it.

The second slice is the cache lookup, and in the common case it is nearly free. The daemon builds the `PowerlineKey` (Appendix E.7) — extension, renderer module, the override triple — and probes its orchestrator cache. On the first render of a given kind this misses and the daemon pays the one-time setup (config cascade, colorscheme build, adapter wiring) that Chapter 6 details; on the second through ten-thousandth it hits, and the whole of that setup is skipped for a hash lookup on a small tuple. This is the slice the daemon architecture exists to shrink to zero, and in steady state it succeeds: the amortized cost of orchestrator setup across a day’s renders rounds to the cost of one setup divided by tens of thousands, which is noise.

The third slice is the render itself, and it is smaller than intuition suggests. Each adapter runs — most of them read a value already in `segment_info` (the exit code, the job count, the hostname) and format it, which is microseconds; the probe segments (`cpu_load_percent`, `git_status`) cost a syscall or a subprocess and are the reason segment *placement* is a real decision (Chapter 3, Appendix AB.5). The colorscheme resolves each segment’s groups through `get_highlighting` (a walk of a short list and a dict lookup), the truncation pass measures widths and drops low-priority segments if the line overflows (Appendix P), the divider decision compares adjacent backgrounds (Appendix X), and the target encoder emits the markup byte string. On a line of cheap segments this whole slice is tens of microseconds; the expensive-segment case is bounded by the segment, not the renderer. The daemon then `write(2)`s the reply and the client copies it to stdout. Summed, the cheap-segment warm path is comfortably under a millisecond end to end, and the honest caveat is the one Chapter 5 already made: a network or disk segment on the interactive path lands its own latency *inside* the third slice, because the daemon samples on demand rather than in a background thread. The budget is small everywhere the port controls it, and where it is not small, it is not small for a reason the theme author can see and move.

Chapter 6 — Themes and Colorschemes

This is the compatibility contract made concrete. `powerliners` loads the user's existing `powerline/config`, `themes/`, and `colorschemes/` JSON trees and resolves them exactly as upstream does. This chapter walks the cascade.

The config cascade: `_find_config_files`, `load_json_config`, `mergedicts`

Configuration in `powerline` is not a single file but a *cascade* of files found across a search path and merged, and reproducing the cascade exactly is what lets an existing `~/.config/powerline` tree work unchanged. The search order, documented in the comprehensive man page and honored by the daemon and direct renderer, is: first `$XDG_CONFIG_HOME/powerline/` (falling back to `~/.config/powerline/`); then the colon-separated roots in `$POWERLINE_CONFIG_PATHS`; then system-wide `/etc/powerline/`; and finally the bundled defaults shipped inside the `powerliners` package itself. “First hit wins per key” — a value defined in your user config beats the same key in the system config beats the bundled default.

`_find_config_files` (at `src/ported/mod.rs:104`) is the function that locates a named config file across the search paths. Its logic is a faithful port: append `.json` to the config name, walk the search paths, collect every path where that file exists, and — matching upstream's behavior — raise an error naming the searched paths if none is found. Note that it collects *all* matches, not just the first; the merging happens afterward, so a `themes/tmux/default.json` present in both your user tree and the bundled defaults yields two paths whose contents are then merged, rather than one shadowing the other wholesale.

`load_json_config` (at `src/ported/lib/config.rs:48`) is the loader — it reads a single JSON file and parses it into the in-memory `Value` tree, returning an error on malformed JSON (its tests assert both the happy parse and the error-on-bad-JSON path). It is the narrow, boring function that every config read goes through, and its faithfulness matters precisely because it is boring: the same JSON must parse to the same structure it did under Python, including the handling of the config's quirks, so that everything downstream sees what upstream saw.

`mergedicts` (at `src/ported/lib/dict.rs:117`) is where the cascade becomes a single effective config, and it is a *deep* merge, not a shallow one. For each key in the incoming dict: if both sides hold an object, recurse and merge them; if the incoming value carries the removal sentinel (`__powerliners_remove_this_key__`, the port's spelling of upstream's REMOVE marker) and removal is enabled, delete the key; otherwise overwrite. This recursion is what lets a user config override *one nested field* of a theme — a single segment's `args`, a single color — without restating the whole structure around it, and the REMOVE sentinel is what lets a user *delete* an inherited key rather than only override it. The port also carries `mergedefaults` (fill gaps without overwriting, `d1` wins) and `mergedicts_copy` (a non-mutating variant), the same three merge flavors upstream uses for different phases of config assembly.

Colorscheme alias chasing and cterm color resolution

A colorscheme maps highlight-group names to colors, but the mapping is indirect in two ways, and both indirections are ported faithfully. The first is *named colors*: a colorscheme does not write `#ac4242` on a group, it writes a color name like `red`, and a separate `colors.json` maps `red` to a concrete value. This is why a palette change (edit `colors.json`) restyles every group that uses a named color at once — the groups reference names, the names reference values, and only the values change.

The second indirection is that every color is *two* values, not one — a 256-color terminal index (cterm) and a 24-bit hex value — because the same colorscheme must render on both a truecolor terminal and a 256-color one. The `Colorscheme` constructor (in `src/ported/colorscheme.rs`) builds `self.colors` as a dict of `(cterm_int, hex)` tuples, and it handles both input shapes upstream allows: a color given as `[cterm, "#hex"]` is used directly, while a color given as a bare cterm integer has its hex *derived* from a lookup table. That table is `cterm_to_hex` (at `src/ported/colorscheme.rs`), a 256-entry `const [u64; 256]` that is a line-for-line port of upstream's module-level table — the canonical mapping from an 8-bit terminal color index to its 24-bit RGB value, laid out in the same rows as upstream (`0x000000`, `0xc00000`, `0x008000`, ...). Carrying this table is what lets powerliners render a 256-color-only theme on a truecolor terminal and vice versa without the author maintaining two palettes.

Alias chasing is the third piece. A colorscheme may define a group as an *alias* of another group rather than a set of colors, and resolution must follow the alias chain until it reaches a group with real properties. `get_group_props` (at `src/ported/colorscheme.rs:242`) does this resolution, consulting the mode translations along the way (below), and it is what `get_highlighting` calls per group. The aliasing lets a colorscheme say “the `branch_dirty` group looks like the `warning` group” without duplicating the warning's colors — and if you later restyle `warning`, `branch_dirty` follows, because it was an alias, not a copy.

Colorscheme::get_highlighting and highlight-group lookup

`get_highlighting` (at `src/ported/colorscheme.rs:292`) is the function every segment render calls to turn its list of highlight groups into concrete `{fg, bg, attrs}`, and its structure is a faithful port worth walking. It takes the segment's `groups` (in priority order), the current `mode` (for per-mode overrides), and an optional `gradient_level`. It first looks up the mode's `translations` — the per-mode overrides that let, say, insert mode recolor the whole line. Then it walks the groups in order, calling `get_group_props` for each, and *stops at the first one that resolves*:

```
let mut group_props: Option<Value> = None;
for group in groups {
    let gp = self.get_group_props(mode, &trans, &Value::String(group.clone()), true);
    if gp.is_some() { group_props = gp; break; }
}
let group_props = group_props.ok_or_else(|| format!(
    "Highlighting groups not found in colorscheme: {}", groups.join(", ")
));
```

This first-match-wins walk is the fallback mechanism of Chapter 2 made concrete: a segment offers `["branch_dirty", "branch"]`, and a colorscheme that defines only `branch` skips past the missing `branch_dirty` and resolves `branch`. If *none* of the groups resolves, the function errors with a message naming all the groups it tried — the same loud failure upstream produces, which is what `powerline-lint` (Chapter 9) exists to catch before it reaches a live prompt.

Once a group's properties are in hand, `get_highlighting` extracts the foreground and background color names and resolves each through a `pick` closure: with no gradient level, it looks the color name up in `self.colors` (yielding the `(cterm, hex)` tuple); with a gradient level, it calls `get_gradient` to select a color from the group's gradient list at that level. The attributes — bold, italic, underline — are read from the

group's `attrs` array and packed into an integer flag by `get_attrs_flag`. The function returns a map of `{fg, bg, attrs}`, and that map is what the renderer (Chapter 7) hands to its target-specific encoder. Everything about how a segment gets its color runs through these dozen lines, and their fidelity to upstream is why a colorscheme resolves identically in both implementations.

Per-mode recoloring: `mode_translations` walked

The one piece of `get_highlighting` that a walkthrough clarifies is the *mode* handling, because it is how the same line renders differently depending on what the editor or shell is doing. A colorscheme's `mode_translations` key (Appendix B.5) is a map from mode name to a pair of overrides — a `groups` map that redefines specific groups for that mode, and a `colors` map that remaps color names for that mode. Consider a vim colorscheme with:

```
{ "groups": { "mode": { "fg": "white", "bg": "darkblue", "attrs": ["bold"] } },
  "mode_translations": {
    "i": { "colors": { "darkblue": "darkgreen" },
          "groups": { "mode": { "fg": "white", "bg": "darkgreen", "attrs": ["bold"] } } }
  }
```

In normal mode (`mode = "n"`), `get_highlighting(["mode"], Some("n"), None)` finds no translation for "n" (the `translations` lookup returns an empty map), so `get_group_props` resolves `mode` straight from `self.groups` to white-on-darkblue. In insert mode (`mode = "i"`), the same call first looks up `translations["i"]`, and now two things happen: `get_group_props` consults the mode's `groups` override *first* (recurring with `translate_colors = false`, `colorscheme.rs:242`), finding the insert-mode `mode` group redefined to white-on-darkgreen; and the mode's `colors` remap would rewrite any `darkblue` reference to `darkgreen` for groups that fall through to the base definition. The net effect is that entering insert mode repaints the mode indicator — and, through the `colors` remap, any group that referenced the recolored name — without the theme touching a single segment. This is the mechanism behind the familiar “the whole prompt turns green in insert mode” behavior, and the port carries it faithfully: the mode's `groups` win over the base `groups`, the `colors` remap applies to the base fall-through, and the segment never knows any of it happened — it asked for the `mode` group and got whatever that group means in the current mode.

Truecolor vs 256-color: the `term_truecolor` path

The final resolution — turning a `(cterm, hex)` pair into actual terminal bytes — depends on whether the target terminal supports 24-bit color, and the choice is governed by a `term_truecolor` flag threaded through the renderers. When truecolor is on, the renderer emits the hex value (the second element of the color tuple); when it is off, it emits the cterm index (the first). This is the payoff of carrying both values through the whole pipeline: the same resolved color can be encoded either way, so one colorscheme serves both a modern truecolor terminal and an older 256-color one with no author intervention.

The gradient path respects the same split. `pick_gradient_value` (at `src/ported/colorscheme.rs:72`) selects an index into a gradient list from a 0-to-100 level — `grad_list[int(round(gradient_level * (len(grad_list) - 1) / 100))]`, ported exactly, rounding included — and because the colorscheme built two gradient lists per gradient (a cterm list and a hex list, from the constructor's `(cterm_list, hex_str_list)` handling), the same level picks a matching color in whichever encoding the terminal needs. A `cpu_load_percent` segment at 70% load thus shades correctly whether it lands on a truecolor or a 256-color terminal, because the gradient exists in both encodings and `term_truecolor` chooses between them at the last step.

This is where the tmux renderer's cterm/truecolor branch (Chapter 7) becomes concrete. The README's parity notes list “cterm/truecolor encoding with falsy-hex fallback” among the byte-identical behaviors, and the falsy-hex fallback is a detail worth naming: when a color's hex value is missing or zero (falsy), the renderer falls back to the cterm index even on a truecolor terminal, matching upstream's exact handling of the degenerate

case. The care taken here — two encodings carried through the whole pipeline, chosen at the last moment, with a defined fallback for the missing-hex case — is representative of the port's method: reproduce not just the common path but the edge cases, because the edge cases are where an “inspired-by” reimplementation would quietly diverge and a faithful port must not.

A colorscheme, resolved: from `colors.json` to `{fg, bg, attrs}`

The abstractions of this chapter are best trusted by watching one highlight group resolve all the way to concrete numbers. Start with three small files, each in the shape the port's constructor reads. First `colors.json`, the palette:

```
{
  "colors": {
    "gray0": 236,
    "white": [231, "ffffff"],
    "orange": [166, "d75f00"]
  },
  "gradients": {
    "cpu_load_percent_gradient": [
      [154, 148, 142, 136, 130],
      ["9fd700", "afaf00", "d7af00", "d78700", "d70000"]
    ]
  }
}
```

`Colorscheme::new (src/ported/colorscheme.rs:109)` reads `colors` into `self.colors` as `name → [cterm, hex]` pairs. Note the two input shapes it accepts: `gray0` is a bare cterm integer, so its hex is *derived* from the `cterm_to_hex` table (entry 236 is `0x303030`); `white` and `orange` give both values explicitly, used verbatim. The gradient reads as two parallel lists — a cterm list and a hex list — into `self.gradients["cpu_load_percent_gradient"]`.

Second, the colorscheme's `groups`:

```
{
  "name": "Demo",
  "groups": {
    "cpu_load_percent_gradient": { "fg": "white", "bg": "gray0", "attrs": ["bold"] },
    "cpu_load_percent": { "fg": "white", "bg": "gray0", "attrs": [] },
    "branch_dirty": "warning",
    "warning": { "fg": "white", "bg": "orange", "attrs": [] }
  }
}
```

Now trace `get_highlighting(["cpu_load_percent_gradient", "cpu_load_percent"], None, Some(71.0))` — the call the `cpu_load_percent` segment of Chapter 2's worked example makes. It looks up the mode's `translations` (none, so an empty map), then walks the groups in order. `get_group_props` resolves `cpu_load_percent_gradient` in `self.groups` on the first try, yielding `{fg: "white", bg: "gray0", attrs: ["bold"]}`; the loop stops there and never consults the fallback `cpu_load_percent`. Because a `gradient_level` of `71.0` was passed, `fg` and `bg` are *not* looked up in `self.colors` — instead the gradient path runs. `pick_gradient_value` computes the index `round(71.0 * (5 - 1) / 100) = round(2.84) = 3` (its `py_round` uses banker's rounding to match Python 3's `round`), selecting element 3 of each gradient list: cterm `136`, hex `0xd78700` — a warm orange. The `attrs` array `["bold"]` runs through `get_attrs_flag`, which bit-ORs `ATTR_BOLD` (`1`) to produce the integer `1`. The function returns `{fg: [136, 0xd78700], bg: [136, 0xd78700], attrs: 1}` — concrete `(cterm, hex)` pairs plus a packed attribute flag, which is exactly what the renderer of this chapter turns into terminal bytes.

The alias case is the other half of the mechanism. Resolving `branch_dirty` finds a *string* value, `"warning"`, in `self.groups` — an alias, not a property object — so `get_group_props` recurses on `"warning"`, which resolves to real properties `{fg: "white", bg: "orange", attrs: []}`. That is why restyling `warning` in one place restyles `branch_dirty` too: the dirty-branch group was never a copy of the warning colors, it was a pointer to them, chased at resolution time. One palette file, one colorscheme file, and a dozen lines of resolution turn a segment's request for a highlight group into the four numbers the terminal actually needs.

Chapter 7 – Renderers

A resolved segment list still has to become bytes for a specific target: tmux markup, a zsh P51, a bash prompt, a vim statusline. This chapter covers the renderer loop and the per-target emitters.

The render loop: `do_render`, `_render_segments`, `_render_length`

`do_render` (at `src/ported/renderer.rs:792`) is the top of the render loop — the function that takes a resolved segment list and produces the final byte string for a target. It is a faithful port, and its `// py:NNN` citations map it line-by-line onto upstream's `Renderer.do_render`. Its job is layout: given the segments (already carrying resolved contents and `{fg, bg, attrs}` highlights from Chapter 6), it computes divider widths, measures the line, and — when the line must fit a width — runs the truncation-and-priority logic that decides which segments to drop or shorten. The output is a string of the target's markup with every segment, divider, and pad in place.

The width math runs through `_render_length` (at `src/ported/renderer.rs:1075`), which computes the rendered width of the segment list including contents, dividers, and outer padding, and returns the running length. This is the function `do_render` calls repeatedly as it applies the priority-based truncation: a segment can declare a `priority`, and when the line is too wide for the terminal, low-priority segments are removed or truncated until it fits — so a narrow tmux pane drops the least-important segments rather than wrapping or overflowing. `_render_length` is called before and after each such adjustment (its call sites are threaded all through `do_render`, at `py:347`, `py:353`, `py:358`, `py:374`, `py:401`) to re-measure, which is why the port keeps it a separate function: the loop measures, adjusts, and re-measures until the line fits.

`compute_divider_widths` (at `src/ported/renderer.rs:314`) supplies the per-side hard and soft divider widths that the length math needs. It takes a `get_divider(side, kind)` closure — the port's stand-in for upstream's `theme.get_divider` — and returns a map of `{left: {hard, soft}, right: {hard, soft}}`, each value the display width of the corresponding divider string (via `strwidth`, the port's width function at `renderer.rs:298`). The widths are computed once per render and reused, because the divider strings do not change within a render; only *which* divider (hard or soft) goes at each boundary does, and that is decided per boundary in the length loop.

The width calculations lean on `strwidth`, and its behavior is one of the places where the port is honest about a simplification rather than overselling it. A powerline line is full of characters that are not a single byte — the divider chevrons, powerline glyphs, CJK text, emoji in a media segment — and measuring the line as `str.len()` (a byte count) would be catastrophically wrong for any of them, throwing off the truncation math and producing a line that overflows or under-fills. The port's `strwidth` counts *Unicode scalar values* — `s.chars().count()` — which is correct byte-for-byte for the overwhelming common case (ASCII, the powerline glyphs, Latin text) where every scalar is one cell. What it does not yet do is East-Asian *wide-character* dispatch: upstream's `strwidth_ucs_2/strwidth_ucs_4` count a CJK ideograph or a wide emoji as two cells, and the port's char-count returns one for those, so a line that is heavy with wide glyphs can measure a cell short. This is a documented parity gap, exercised (as a fallback path) by `lib_strwidth_fallback.rs`, and it is the kind of edge the port names rather than hides — the width is exact for the character classes a prompt is actually built from, and the wide-glyph case is a known, bounded divergence rather than a silent one.

Hard and soft dividers, outer padding, side handling

The divider logic is where powerline's signature chevrons come from, and the rule is purely color-driven. Inside the render loop, for each pair of adjacent segments, the divider type is chosen by comparing backgrounds: `divider_type = 'soft' if compare_segment['highlight']['bg'] == segment['highlight']['bg'] else 'hard'` (the port reproduces this exactly at `src/ported/renderer.rs:1186`). Same background → soft divider (a thin separator in a muted color, drawn *within* a color block); different background → hard divider (the solid chevron that fills with the neighbor's background, producing the arrow that visually “points” from one color into the next). The theme author never places these; they emerge from the coloring, which is why re-coloring a theme automatically re-flows its dividers.

Outer padding and spaces frame the line. `theme.get_spaces()` supplies the per-segment `divider_spaces` — the blank cells placed around dividers so text does not touch the chevrons — and `outer_padding` is the blank space at the very ends of the line, keeping the first and last segments off the terminal edges. Both feed into `_render_length`'s width accounting, because padding takes cells and the truncation math must know about every cell. The README's parity list names `outer_padding` and `spaces` explicitly among the byte-identical behaviors, which is the point: even the whitespace is faithful, because a prompt that padded differently would render one cell off and no longer be a drop-in.

Side handling is the last layout concern. A powerline line has a left side and a right side (and, for statuslines, a gap between them), and the two sides render with mirrored divider orientation — the left side's hard dividers point right, the right side's point left, so both “flow” toward the center gap. The renderer handles each side, including the empty-side case (a line with only left segments, or only right), which the README calls out among the parity scenarios (“left/right side handling, empty sides”). The `before/after` wrapping — strings a theme can wrap around a segment's contents — is applied here too, and it too is on the byte-identical list. The whole of this layout logic is inherited; what powerliners adds is that it runs in a native binary in microseconds rather than in a freshly-booted interpreter.

The TmuxRenderer and its `#[...]` markup

The tmux renderer (`src/ported/renderers/tmux.rs`) is the most-used emitter, because the tmux statusline is where most people first meet powerline, and it is a compact, faithful port. Its central function is `hlstyle` (at `src/ported/renderers/tmux.rs:114`), which turns a resolved `{fg, bg, attrs}` into tmux's `#[...]` style directive. The encoding mirrors upstream line-for-line: a foreground of `default` emits `fg=default`; a truecolor foreground emits `fg=#{0:06x}` (the six-hex-digit form); a 256-color foreground emits `fg=colour` followed by the cterm index. Background is the same with `bg=`. The finished directive is `#[` plus the comma-joined attributes plus `]` — for example `[fg=#ac4242,bg=#181818,bold]`.

The attribute handling is its own small port. tmux does not have a “no attributes” state that clears the previous style, so the renderer must *explicitly* turn attributes off when a segment lacks them: `attrs == None` returns `["nobold", "noitalics", "nounderscore"]`, and a segment that is bold-but-not-italic emits `nobold` cleared to `bold` and `noitalics` for the italic slot. This is why the tmux attribute list carries the `no*` forms — without them, a bold segment would leave every following segment bold until something explicitly turned it off, because tmux markup is stateful. The port reproduces this exactly (the `attrs` handling at `tmux.rs:29-64`), because getting it wrong would produce the classic “everything after the bold segment is bold” bug.

Character translation is the last tmux-specific wrinkle, and it is the reason a literal `#` in your prompt does not break tmux markup. In tmux's status format, `#` is the escape character that introduces `#[...]` and `{...}` — so a `#` in segment *contents* (a channel name, a path) must be doubled to survive. The renderer's `character_translations` maps `#` to `##[ ]` (the port's `vec!(['#', "##[ ]")`) at `tmux.rs:104`, citing upstream's `character_translations[ord('#')] = '##[ ]'`, and the README's parity notes list this escaping (`#` → `##[ ]`) among the byte-identical behaviors. It is a one-character rule, but it is exactly the kind of detail that separates a port from an approximation: an implementation that forgot to double `#` would render most prompts correctly and then mangle any prompt containing a hash.

Two segments, rendered to tmux bytes

Take the two-segment left side from the worked examples of Chapters 2 and 6: a `hostname` segment resolved to `{fg: [231, 0xffffffff], bg: [236, 0x303030], attrs: 0}` with contents "jbox", followed by the `cpu_load_percent` segment resolved to `{fg: [231, 0xffffffff], bg: [136, 0xd78700], attrs: 1}` with contents "71%". On a truecolor terminal (`term_truecolor = true`), the renderer walks the list and calls `hlstyle` per segment. For `hostname`, `color_spec` sees a truecolor value and emits `fg=#ffffff` and `bg=#303030`; `attrs` is 0, so `attrs_to_tmux_attrs` returns the three off forms `nobold`, `noitalics`, `nounderscore`; the directive is:

```
#[fg=#ffffff,bg=#303030,nobold,noitalics,nounderscore]jbox
```

For `cpu_load_percent`, the foreground is the same white, the background is the gradient orange `#d78700`, and `attrs` is 1 (`ATTR_BOLD`), so the attribute list is `bold`, `noitalics`, `nounderscore` — bold on, the other two explicitly off:

```
#[fg=#ffffff,bg=#d78700,bold,noitalics,nounderscore]71%
```

Two details are load-bearing and both are faithful ports. First, the `no*` forms are always present because tmux markup is *stateful*: without an explicit `nobold` on the `hostname` directive, a `bold` set by an earlier segment would bleed forward — this is the “everything after the bold segment is bold” bug the port exists to avoid. Second, the boundary between the two segments is a *hard* divider, because their backgrounds differ (`0x303030` versus `0xd78700`): `_render_length` compares the two `bg` values, finds them unequal, and selects `divider_type = "hard"`, so the chevron between them is drawn `fg = 0x303030` (the left segment’s background) on `bg = 0xd78700` (the right segment’s background), producing the signature arrow that appears to point the gray block into the orange one. Had both segments shared a background, the same comparison would have chosen a *soft* divider drawn in the muted `divider_highlight` color instead. On a 256-color terminal the only change is at the last step: `color_spec` emits `fg=colour231` and `bg=colour136/bg=colour236` — the cterm indices — instead of the hex forms, chosen by the same `term_truecolor` flag, from the same resolved color tuple.

Shell renderers and the readline escape markers (\x01 / \x02)

The shell renderers face a problem tmux does not: a shell prompt contains *non-printing* bytes — the color escape sequences — and the shell’s line editor (readline in bash, zle in zsh) needs to know which bytes are non-printing so it can compute the prompt’s visible width correctly. Get this wrong and the shell mis-measures the prompt, and long command lines wrap in the wrong place, overwrite the prompt, or corrupt the display on redraw. The mechanism for telling the line editor “these bytes take no space” is a pair of marker bytes wrapped around each escape sequence, and the shell renderers emit them.

Those markers are `\x01` (start of non-printing) and `\x02` (end of non-printing), and they live in the upstream `renderers/shell/readline.py` as the constants `escape_hl_start = '\x01'` and `escape_hl_end = '\x02'`. In powerliners these are two of the three files still tagged NEAR (Chapter 18) precisely because they are pure lipstick — a class shell holding two string constants and nothing else. The Rust port holds the same two byte constants directly on the equivalent renderer struct, without the Python ceremony of a subclass per shell flavor. The per-flavor renderers then override the markers with what their line editor expects: the zsh renderer (`ZshPromptRenderer`) uses `%{ / %}`, the bash renderer (`BashPromptRenderer`) uses `\[ /\]`, and the base readline pair (`ReadlineRenderer`) stays `\x01 / \x02`. The upstream `zsh.py` is the second of the NEAR-tagged class shells, the same zero-function shape as `readline.py`. Either way the principle is identical: mark the non-printing bytes so the line editor measures the prompt correctly.

The bytes make it concrete. A zsh prompt segment resolved to white (`0xffffffff`) on dark gray (`0x303030`), bold, on a truecolor terminal, produces from `shell/mod.rs`’s `hlstyle` the SGR sequence `\x1b[0;38;2;255;255;255;48;2;48;48;48;1m` — a reset (0), a truecolor foreground

(`38;2;255;255;255`), a truecolor background (`48;2;48;48;48`), and the single bold code (`1`), the mutually-exclusive attribute chain meaning only one of bold/italic/underline is ever emitted. The zsh renderer then wraps that whole escape in its markers, yielding `%{\x1b[0;38;2;255;255;255;48;2;48;48;48;1m%}` followed by the visible contents. The `%{...%}` is what tells zle “these bytes take zero columns,” so a prompt full of such sequences still measures at its visible width. Swap to bash and the identical SGR sequence is wrapped `\[...]` instead; swap to a 256-color terminal and the color codes become `38;5;231 / 48;5;236` (the cterm indices) — same structure, different encoding at the last step, exactly as the tmux renderer chose between hex and `colour<n>` earlier in this chapter.

The reason these files are NEAR rather than DONE is a classifier artifact, not a gap, and it is worth stating because it is exactly the kind of thing an honest port documents rather than rounds away. The DONE tier (Chapter 10) requires a citation density and a `/// Port of <py_fn>() doccomment per Python function` — but these files contain zero functions with bodies (`py_fn_total == 0`), only class shells with constants. There is no behavior to port because the behavior is the two constants, and the port holds them; the classifier simply cannot promote a 0/0 ratio to DONE. The escape markers themselves are fully present and correct — a shell prompt from powerliners measures exactly as one from Python powerline — which is the only thing that matters for the user. Chapter 18 returns to why “3 files not at DONE” overstates the remaining work.

The shell renderer path is also where the other emitters live for completeness: the vim renderer (`src/ported/renderers/vim.rs`) emits vim’s `%#HighlightGroup#` statusline markup and is exercised by `vim_e2e.rs`; the lemonbar, i3bar, pango-markup, and pdb renderers round out the set under `src/ported/renderers/`. Each is a port of the corresponding upstream renderer, sharing the same `do_render` layout core and differing only in the final encoding — tmux `#[...]`, vim `%#...#`, shell `\x01...\x02`, i3bar JSON, and so on. The layout is computed once, faithfully, and each target’s `hlstyle` translates the resolved colors into that target’s bytes.

What the bar targets do to the layout core’s assumptions

The renderers above all target something that draws a *line of cells* — tmux’s statusline, a shell prompt, a vim statusline — and the layout core’s model fits that target exactly: contents occupy cells, dividers occupy cells, padding occupies cells, and truncation drops segments until the total fits a width. The bar and markup targets break some of those assumptions, and watching which ones break is a good way to see which parts of the core are genuinely target-agnostic and which are quietly terminal-shaped.

The i3bar target is the sharpest case because its protocol is not a byte stream at all. Each segment becomes a JSON object in a streaming array, and the bar program — not powerline — decides how to lay those objects out, how much space each gets, and where the boundaries between them are drawn. Two consequences follow. First, the *divider* has no natural representation: the whole hard/soft chevron mechanism exists because a character-cell line needs an explicit glyph to mark a boundary, and a protocol that hands the host discrete blocks does not. The i3bar emitter therefore turns the separator machinery off explicitly at the protocol level rather than emitting divider glyphs. Second, attributes have nowhere to go: the protocol carries a foreground and a background per block and has no notion of bold or underline, so the `attrs` integer that Chapter 6 worked so hard to pack is simply not consumed on this path. Neither of these is a gap in the port — the protocol does not have the concepts — but both are places where a faithful renderer has to decide what to do with information the target cannot express, and the answer both times is to drop it cleanly rather than to approximate it.

The lemonbar target keeps the byte-stream model but changes the coordinate system. Its position markers delimit regions rather than the emitter tracking columns itself, which means the “left side, gap, right side” structure that the terminal renderers implement by counting cells is expressed declaratively instead. The layout core still runs — the segments are still resolved, the colors still resolved, the truncation still applied — but the final placement is delegated. This is the useful distinction between the parts of the render that are about *content* (which segments, what they say, what color) and the parts that are about *geometry* (where they sit, how wide, what fills between them). The first half is shared by every target; the second half is exactly where the targets differ, and it is why the encoders are per-target functions rather than a formatting flag.

The pango-markup target introduces the one concern the terminal targets never have: escaping content into a

markup language whose special characters overlap with ordinary text. The terminal renderers each have a small escape rule — tmux’s `#` doubling is the canonical example — but XML entity replacement is a different scale of problem, because the characters that need escaping are common in the exact strings segments produce (paths, branch names, command output). The ordering matters too: the ampersand must be replaced before the angle brackets, because replacing the brackets first would then have its own ampersands rewritten by the second pass. That ordering constraint is a classic escaping bug and the port carries the upstream order rather than reinventing it, which is the citation discipline of Chapter 10 doing quiet work on a function nobody thinks about.

The pdb target is the smallest and the most instructive about what “faithful” costs. It is a readline-family renderer whose output is ASCII-folded, replacing anything non-ASCII with a placeholder. In isolation that looks like a limitation to fix — why would a renderer throw away Unicode? — and the answer is that upstream’s does, because the debugger prompt it targets historically could not be trusted with anything else, and a port whose job is byte-identity does not get to improve on that. It is a small, clean example of the commitment Chapter 1 described: the port reproduces the behavior including the behavior nobody would design fresh, because the divergence would be invisible until it was not, and “we made it better” is indistinguishable from “we broke compatibility” from the far side of a diff.

The through-line across all five is that the shared layout core is genuinely shared — the segment resolution, the colorscheme lookup, the priority-driven truncation, the before/after wrapping all run identically no matter what the final encoder is — and that the encoders are where target reality intrudes. That split is what makes adding a target tractable and what makes each target’s fidelity checkable in isolation: one function per target, each short, each diffed against upstream’s equivalent by the parity harness.

Chapter 8 – Bindings

powerliners is only useful when a shell, multiplexer, or editor calls it. This chapter covers the integration surface: how each target is wired to the daemon.

tmux: statusline and continuation lines

tmux is the flagship binding, and the wiring is two lines in `~/.tmux.conf` that most users already have from their Python powerline setup and do not need to change:

```
run-shell "powerline-config tmux setup"
run-shell -b "powerline-daemon -q &>/dev/null || exit 0"
```

The first line runs `powerline-config tmux setup`, which installs the tmux integration — it points tmux's `status-left`, `status-right`, and window-status formats at the `powerline` client so tmux invokes it on every statusline refresh. The second line starts the daemon in the background (`-q` double-forks and detaches), so the client's invocations hit a warm socket. From then on, every `status-interval` tick, and every window switch, tmux runs `powerline tmux left/right/window-status`, the client forwards the request to the daemon, and the daemon returns `#[fg=...,bg=...]...` markup that tmux paints directly into the statusline.

The `tmux setup` step is `include_str!`-bundled and install-method-agnostic as of 0.2.3, which is a small but important packaging detail (Chapter 11). Upstream powerline baked config-file paths at install time, which broke when the tool was moved or installed a different way. powerliners instead embeds the eight tmux conf files — `powerline-base.conf` plus seven version-specific variants for different tmux releases — directly into the binary, and `powerline-config tmux setup` extracts them to `$XDG_CACHE_HOME/powerliners/tmux/` (default `~/.cache/powerliners/tmux/`) on first call. So the same command works identically whether powerliners was installed via `cargo install`, `brew install`, or a manual `cp` into `$PATH` — there is no compile-time path to get wrong.

Continuation lines and per-window rendering fall out of the same mechanism. tmux's window-status format invokes the client once per window, so a session with eight windows produces eight renders per tick — which is exactly the multiplication that made the Python startup tax so painful (Chapter 1) and exactly where the warm daemon pays off most. The migration is confirmed by killing and reattaching tmux (`tmux kill-server`; `tmux new-session`): the status bar looks identical, but the `powerline-daemon` in `ps aux` is now the Rust binary rather than the Python shebang, and the renders are sub-millisecond.

zsh: PS1 / R PROMPT and the precmd hook

The zsh binding is a `precmd` hook that rebuilds the prompt before each render by shelling out to the `powerline` client. The canonical shape, from the comprehensive man page's shell-invocation section, is a function registered with zsh's `precmd` machinery:

```
function powerline_precmd() {
    PS1="$(powerline shell left --last-exit-code=$? ...)"
    R PROMPT="$(powerline shell right ...)"
}
```

The key detail is what the invocation *carries*. `powerline shell left` renders the left prompt; `--last-exit-code=$?` forwards the previous command's exit status so the `last_status` segment can show it (and

the pipe-status variant forwards `$pipestatus` for `last_pipe_status`); the job count and the current vi mode are forwarded the same way. The daemon cannot see any of this — it is a separate process with no view into the calling shell's state — so the binding's job is to gather the shell-local facts and hand them over on the wire, where the shell-status segments of Chapter 3 read them back out of `segment_info`. This is the concrete reason the wire protocol forwards `argv` at all.

`RPPROMPT` gets the same treatment for the right prompt, and because `zsh` redraws `RPPROMPT` on interactive events, the right prompt can trigger renders on its own — which is again where the daemon earns its keep, since each of those redraws would otherwise be a Python interpreter boot. The whole `PS1/RPPROMPT` pair is rebuilt in `precmd`, both sides sub-millisecond against the warm daemon, so a `zsh` prompt that updates on interaction stays instant rather than lagging.

The migration story for `zsh` is the same “change nothing” story as `tmux`. The daemon is started at login (`powerline-daemon -q` in `.zshrc`), the `precmd` hook and the `powerline shell left/right` invocations are unchanged from the Python setup, and the only difference is which `powerline/powerline-daemon` resolves in `$PATH`. Because the client speaks the same wire protocol, you can even leave the Python client in place and only swap the daemon; the client does not know or care which language the daemon behind the socket was written in.

bash: PS1 / PROMPT_COMMAND

The `bash` binding is the same idea as `zsh` with `bash`'s own hook name. `bash` rebuilds its prompt through `PROMPT_COMMAND` — a command `bash` runs before each primary prompt — and the binding registers a function there that sets `PS1` from the `powerline` client's output. As in `zsh`, the invocation forwards the previous exit code and the shell-local state the status segments need, and the client forwards it all to the daemon over the wire.

The one `bash`-specific wrinkle is the `readline` escape marking of Chapter 7. `bash`'s line editor is `readline`, and `readline` needs its non-printing bytes wrapped in `\[ / \]` markers to measure the prompt width correctly (`zsh` uses `%{...%}` instead, and the base `readline` renderer uses the raw `\x01\x02` pair those `\[ / \]` sequences stand in for). The shell renderer emits the right markers for the target, so a `bash` prompt wraps its color escapes in `\[...\]` and `readline` computes the visible width correctly — which is what keeps long command lines from wrapping into the prompt or corrupting the display on history navigation. Get this wrong and `bash` line editing breaks in subtle, maddening ways; the port carries the exact upstream constants so it does not.

`bash`'s continuation prompt (`PS2`) and the shell `continuation` subtype are handled by the same client, invoked with the `continuation` argument. The migration, again, is a no-op on the user's config: the `PROMPT_COMMAND` function and its `powerline shell left/right` calls are unchanged, and only the resolved binary differs. The daemon serves `bash`, `zsh`, `tmux`, and `vim` from the same process simultaneously, so a user who lives in `bash` under `tmux` with `vim` open has one warm daemon handling all of it.

vim statusline

The `vim` binding is the one that most visibly benefits from powerliners not needing Python, because the Python `powerline vim` plugin required a `+python3`-compiled `vim` and a `pip install powerline-status` — a real friction point on minimal or remote `vim` installs. powerliners' `vim` support ships in the same `powerline` binary with no `+python3` requirement: the bundled `powerline.vim` plugin is `include_str!`-embedded in the binary and extracted to `~/.cache/powerliners/vim/powerline.vim` on first source, and it works on `vim 7.4+`, `vim 9`, and `neovim`, using only `vim8`-compatible syntax (no `vim9script` lock-in).

Installation is one line in `.vimrc` — `execute 'source' trim(system('powerline-config vim source-path'))` — which extracts the plugin and sources it. The plugin sets `laststatus=2` automatically so the statusline shows even in a single window, and it wires an `augroup powerliners` of autocmds that trigger renders: `VimEnter` for the initial render, `WinEnter/BufWinEnter/BufEnter/TabEnter` for context switches, `ModeChanged` for mode transitions on modern `vim`, and a legacy `CursorMoved/CursorMovedI`

fallback for ancient vim (which is the one to disable if statusline refresh stutters on every keystroke). On each of those events the plugin shells out via `system('powerline vim left ...')`, and — this is the reason the README recommends pre-starting the daemon — with the daemon running that is a microsecond socket round-trip, while without it every refresh fork-execs `powerline-render` cold at ~30 ms.

What the plugin sends per request is chosen to match upstream's `powerline.bindings.vim` exactly, so theme JSON written for the Python vim binding renders identically here. The per-request keys are `mode` (the current vim mode — `n`, `i`, `v`, `R`), `bufnr` (`bufnr('%')`), `winnr` (`winnr()`), and `buf_name` (`expand('%:p')`, omitted when empty). The vim renderer (Chapter 7) turns the resolved segments into vim's `%#HighlightGroup#` statusline markup, and the plugin assigns the result to `statusline`. The statusline theme and colorscheme come from `~/.config/powerline/themes/vim/default.json` and `colorschemes/vim/default.json` — the same tree, same grammar (`segments.left/right`, `extends`, `mode_translations`) as every other target.

ipython / REPL prompts via shell hook

The ipython/REPL binding is the one deliberate non-port in the family, and the README is explicit that it is done “via shell hook, not embedded.” Upstream powerline can embed itself in the ipython process as a Python prompt provider, because upstream *is* Python and lives in the same interpreter. powerliners has no Python runtime and cannot embed in a Python process, so it renders REPL prompts the same way it renders shell prompts — through the external `powerline` client invoked by a hook — rather than as an in-process provider.

This is a place where the port's boundary is honest about a difference in *mechanism* even though the *result* is the same colored prompt. An ipython user gets a powerline prompt; it is produced by the client talking to the daemon rather than by powerline code running inside ipython. For the everyday case this is invisible — the prompt looks and behaves the same — and it keeps powerliners' single-static-binary discipline intact rather than pulling a Python integration back in through a side door. It is a small example of the larger pattern in Chapter 1: where the upstream mechanism depends on being Python, the port reaches the same user-visible result by a different, native path, and says so plainly rather than pretending the mechanisms are identical.

Chapter 9 – Fidelity to Upstream

Compatibility is the product, so this chapter states exactly what “drop-in” means: what is guaranteed identical, what is allowed to differ and why, and how the guarantee is kept honest rather than merely asserted.

What is byte-for-byte identical

The README’s parity notes enumerate the surface that powerliners guarantees identical to Python upstream, and the list is long and specific because vagueness here would defeat the purpose. The markup itself is identical — `tmux #[...]` directives, `vim %#...#`, `shell \x01...\x02` — down to the escaping rules: `#` doubled to `##[ ]` in `tmux`, control characters run through `translate_np`, the non-printing markers around shell escapes. The dividers are identical across every case the code handles: hard, soft, multi- character, empty, and single-character dividers all render the same bytes. The colorscheme resolution is identical: alias chains, fallback groups, gradients, and `cterm/truecolor` encoding with the falsy-hex fallback all resolve to the same colors.

The layout is identical too. Attributes (bold, italic, underline) are bit-packed the same way; `outer_padding`, `spaces`, and left/right side handling produce the same whitespace; empty sides render the same; `before/after` wrapping applies the same; and single-cell Unicode contents measure and truncate the same (with the one bounded exception of East-Asian wide-glyph width, the documented `strwidth` gap of Chapter 7 — the scalar count is exact for every character class a prompt is normally built from). The point of the enumeration is that “identical” is not a hand-wave — it is a checklist of concrete behaviors, each of which the parity harness (Chapter 10) asserts against the live Python interpreter, so that “drop-in” is a claim backed by 462 assertions rather than an aspiration.

The config resolution is identical, which is what makes an existing tree work unchanged. The cascade (`_find_config_files`, `load_json_config`, `mergedicts`) resolves the same files in the same order and merges them with the same deep-merge, REMOVE- sentinel, list-append semantics as upstream; the theme grammar is the same; the segment names are the same. A `~/.config/powerline` tree that renders a certain prompt under Python renders the same prompt under powerliners, and the migration guide’s Step 2 (Chapter 11) is precisely an instruction to *verify* that byte-for-byte before committing to the swap.

Where output may legitimately differ

Honesty about compatibility means naming the places where output can differ, and the README documents exactly three, each an inherent consequence of not running Python’s `psutil`-based sampling rather than a port defect. First, **live-data segments** (`cpu_load_percent`, `network_load`, and the rest) are sampled per-render via subprocess probes in the daemon, while Python uses `psutil` with a different sampling cadence — so the *numeric value* may differ by a sampling-window’s worth of data. The markup framing is identical; the number inside it may be a moment fresher or staler.

Second, **threaded segment caching** differs. Python’s `ThreadedSegment` polls in a background thread and renders the last-known value; powerliners samples on demand. The *content* matches, but the *latency profile* differs — powerliners may block briefly when a network or disk segment samples, where the threaded model would have returned an instant slightly-old value (Chapter 5 discussed the trade-off). Third, **psutil-only features**: Python errors loudly and skips affected segments when `psutil` is missing, while powerliners resolves the same data through OS subprocess probes (`top`, `vm_stat`, `netstat`, `pmset`, `uptime`) and renders successfully. So powerliners is actually *more* robust in the no-`psutil` case — it renders where Python would have skipped — which is a divergence in the user’s favor.

These three are the entire set, and the framing matters: they are the places where byte-identity was never physically possible, because they involve sampling live values at a moment in time, and two different samplers at two different instants will read two different numbers. Everything that is *deterministic* — the markup, the escaping, the colors, the layout, the config resolution — is byte-identical; only the live-sampled numbers, which are inherently timing-dependent, can differ, and only by a sampling window. This is the honest boundary of “drop-in,” stated in the code’s own words.

powerline-lint: catching config errors before they render

Fidelity cuts both ways: powerliners must render a valid config the same as upstream, and it must *reject* an invalid config the same way, so that a config error surfaces as a clear diagnostic rather than a mangled prompt or a silent omission. `powerline-lint` (the binary, mirroring `scripts/powerline-lint`) is the tool for this. It runs the full check pipeline — a `markedjson` loader that preserves source positions, the `Spec` checks that validate structure, and orchestrator integration that resolves the config the way a real render would — and reports authoring errors: unknown segment names, missing color references, malformed `before/after` formatting strings, and the rest.

The lint pipeline lives under `src/ported/lint/` (with `spec.rs`, `checks.rs`, `selfcheck.rs`, `inspect.rs`, `context.rs`, and `imp.rs`), a faithful port of upstream’s lint machinery. Its value in a compatibility story is that it catches the errors that `get_highlighting`’s “groups not found” failure (Chapter 6) would otherwise raise at render time — a colorscheme missing a group a theme references, for instance — and reports them statically, with source positions, before they reach a live prompt. A user migrating a hand-edited config runs `powerline-lint` first and fixes what it flags, rather than discovering the same problems as a broken statusline later.

Linting is also part of how the project keeps its own bundled defaults honest. The test suite includes a self-check path, and the lint checks run against the config the daemon actually resolves, so a bundled theme or colorscheme that referenced a missing group would be caught in CI rather than shipped. Fidelity to upstream includes fidelity in *failure*: the same bad config produces the same class of error, which is exactly what a user who has debugged a Python powerline config before expects, and exactly what makes the migration predictable rather than a source of new mysteries.

Chapter 10 – The Port Methodology

The claim “a faithful port” is only worth what the verification behind it is worth. `powerliners` backs it with a citation rule and a parity harness. This chapter documents how fidelity is *proven*, not asserted.

The citation rule: `// py:NNN` against the upstream source line

The most visible discipline in the `powerliners` source is the citation comment. Nearly every line of a ported function body carries a `// py:NNN` annotation naming the exact upstream source line it ports — you have seen them throughout this book, in `get_highlighting` (py:99, py:100–103, py:104–105, py:107–110), in `mergedicts` (py:38 through py:47), in the `tmux` renderer (py:10, py:16, py:41, py:65), in `daemonize` (sh:321–340). These are not decoration; they are a machine-checkable map from the Rust source to the Python (and C, `sh`: for the shell scripts and daemon) source it descends from, so a reader — or an audit tool — can put the two side by side and verify that the Rust line does what the upstream line does.

A single function shows the discipline in miniature. `mergedicts` (`src/ported/lib/dict.rs:117`) is a deep merge, and its body reads, with citations, roughly:

```
/// Port of `mergedicts()` from powerline/lib/dict.py:33.
pub fn mergedicts(d1: &mut Map<String, Value>, d2: Map<String, Value>, remove: bool) {
    for (k, v2) in d2 {                                     // py:38  for k in d2:
        match d1.get_mut(&k) {
            Some(v1) if v1.is_object() && v2.is_object() // py:40  if k in d1 and ... dict
                => mergedicts(v1.as_object_mut().unwrap(),
                             v2.as_object().unwrap().clone(), remove), // py:41
            _ => mergedicts(...)
                _ if remove && is_remove_sentinel(&v2)           // py:42  elif ... is REMOVE
                    => { d1.remove(&k); }                         // py:43  del d1[k]
                _ => { let mut v = v2; if remove { clear_special_values(&mut v); }
                    d1.insert(k, v); }                             // py:44-47  else: d1[k] = d2[k]
        }
    }
}
```

Set the Rust beside `dict.py` lines 33–47 and every branch lines up: the object/object case recurses (py:40–41), the sentinel case deletes (py:42–43), and the default overwrites after scrubbing nested sentinels (py:44–47). A reader does not have to *trust* that the merge matches upstream — the `// py:NNN` markers say which upstream line each branch descends from, and the two can be read together to confirm it. That is the citation rule doing its job on one function; multiplied across the ported tree, it is a continuous, checkable correspondence between two codebases.

The citations make the port *legible* in a way an uncommented reimplementation never is. When Chapter 6 says `pick_gradient_value` reproduces upstream’s rounding exactly, that is not a claim to take on faith: the function carries the citation `// py:32 grad_list[int(round(gradient_level * (len(grad_list) - 1) / 100))]` right above the Rust that implements it, and you can read both and see they match. Multiply that across the whole ported tree and the citations become a continuous, checkable correspondence between two codebases — which is what turns “faithful port” from a marketing phrase into a property you can inspect line by line.

The citations also serve the port's maintenance. When upstream changes a function, the citations point maintainers at exactly which Rust lines correspond to the changed Python lines, so the port can be updated surgically rather than by re-reading the whole function to find the drift. And they feed the tier classifier below, which measures *citation density* — the fraction of a function's body that is cited — as one of its two criteria for the DONE tier. A function that is genuinely ported line-by-line has high citation density; one that was sketched loosely does not, and the classifier can tell the difference.

The DONE tier: `citation density ≥ 0.5 + a /// Port of <py_fn>() doccomment`

Not every ported function is equal, and the project grades them. The tier classifier (`scripts/gen_port_checklist.py`) assigns each upstream Python function a tier, and the top tier, DONE, requires two things of the corresponding Rust: a `// py:NNN` citation density of at least 0.5 — meaning at least half the function's body lines are pinned to an upstream source line — and a `/// Port of <py_fn>()` doccomment identifying which upstream function this Rust function ports. Both criteria must hold; a function with a doccomment but sparse citations is not DONE, and neither is one densely cited but missing its port doccomment.

The two criteria check different things. The doccomment is a *declaration* — this Rust function claims to port that Python function, by name — which is what the drift gate (below) checks against the canonical function list. The citation density is *evidence* — the port is not just claimed but demonstrated, line by line, to at least the halfway mark. Requiring both means a DONE function is one that says what it ports and shows its work, which is a meaningfully stronger bar than “there is a Rust function with a similar name.” The project's status — 134 of 137 upstream files at DONE — is a count of files where every function clears this bar.

Two limits of that bar have to be stated wherever the number is, because they are properties of the classifier rather than of any one file. The first: **DONE is a documentation metric, not a behavioural one.** Both of its inputs — `// py:` citation density and `/// Port of` doccomment count — are computed by `count_rs` at `scripts/gen_port_checklist.py:79-99` from the *text* of the Rust file, and a body that transcribes every upstream statement as a comment scores maximally whether or not it also executes. `renderers/shell/bash.py` is the worked example: DONE, 80 `// py:` citations, and a `BashPromptRenderer::do_render` whose two return paths are both `String::new()` (`src/ported/renderers/shell/bash.rs:123` and `:135`) under a doccomment that says “**Status:** stub” (`:56`). Its neighbour `renderers/shell/zsh.py` is NEAR with every upstream name present and live. The second: **the classifier walks the filesystem, so it counts files the compiler never sees.** `main()` at `:127` enumerates upstream `.py` files with `os.walk` and scores whatever `.rs` sits at the mirrored path; nothing in that loop asks whether the path is reachable from `lib.rs` or from a `[[bin]]` target. Four rows are scored this way today (Chapter 18 names them, Chapter 29 opens one). Both facts are visible only by reading the classifier, which is why they are restated at every site in this book that quotes a tier count.

The tier system is also what makes the port's honesty auditable rather than self-reported. Anyone can regenerate the per-file tier table from the live source with `python3 scripts/gen_port_checklist.py > docs/PORT_CHECKLIST.md`, and the function-coverage report with `python3 scripts/gen_port_report.py`. The numbers in this book and in the README's badges are not hand-maintained claims; they are outputs of a classifier run over the actual source, which means they cannot drift from reality without the classifier catching it. That is the difference between a project that says it is a faithful port and one that can prove the claim on demand.

The parity harness: 462 assertions run against the live Python interpreter

Citations prove the Rust *looks like* the Python; the parity harness proves it *behaves like* it. The harness — `tests/parity_against_upstream.rs`, `tests/daemon_parity.rs`, and the contract tests — runs 462

assertions in which the real Python interpreter executes the upstream powerline source and the Rust port renders the same input, and the two outputs are compared for byte or value identity. This is not a golden-file comparison against a snapshot someone captured once; it is a live comparison against Python running the actual upstream code, so if upstream's behavior is X, the harness demands the port produce X, and it re-checks that on every CI run.

The daemon parity is the sharpest case, because it exercises the whole pipeline end-to-end. `daemon_parity` covers 45 byte-for-byte scenarios in which a request goes over the wire protocol to the Rust daemon and the rendered markup is compared against what upstream's `powerline-render` produces for the same config and request. The migration guide's Step 2 is the user-facing version of exactly this test: spin up the Rust daemon on a probe socket, send a wire request, and diff the output against `powerline-render tmux right`. If they match for your segments, the port is faithful to *your* config; the harness does the same for 45 representative configs continuously.

The value of testing against the live interpreter rather than a captured snapshot is that it makes the parity *self-correcting*. A snapshot can go stale — upstream changes, the snapshot does not, and the test now asserts old behavior. Running the real interpreter means the harness always compares against current upstream behavior, so the port is held to what powerline actually does today, not what it did when someone last captured a golden file. This is more expensive (the tests need Python and the upstream source, which is why `vendor/powerline` is checked in as a parity fixture) but it is the only way to make “byte-identical to upstream” a claim that stays true as upstream moves.

The 11 port bugs the harness surfaced and fixed

The harness is not ceremony; it found real bugs, and the project counts them. Eleven port bugs were surfaced by the parity harness and corrected in the Rust port — cases where the Rust looked plausibly like the Python but produced different bytes, and only a live diff against the interpreter caught it. The README notes them (see the git log for the full list), and their existence is the argument for the harness: a port checked only by reading code and running its own unit tests would have shipped these eleven divergences, because each was a place where the code looked right and behaved wrong.

This is the deepest reason the harness runs against the live interpreter rather than the port's own idea of correct. A unit test written by the porter encodes the porter's *understanding* of upstream behavior, and if that understanding was wrong, the test is wrong in the same direction as the code — they agree, and the bug survives. A diff against the actual Python cannot be fooled that way: it does not care what the porter thought upstream did, it runs upstream and compares. The eleven bugs are eleven places where the porter's understanding was subtly off and the live diff caught what a self-authored test would have missed.

The count also functions as an honesty signal. A project claiming a flawless port would be either lying or lucky; a project that says “the harness found eleven bugs and we fixed them” is telling you the harness works and the port is now as good as the harness is thorough. Every one of the eleven is a divergence that no longer exists, and the harness that found them still runs on every push, so the twelfth — if the port ever regresses or upstream shifts under it — gets caught the same way. The number is not an embarrassment; it is the evidence that the verification is real.

The drift gate and `docs/powerline_py_functions.txt`

The last piece of the methodology guards against a slow, silent failure mode: the port falling out of correspondence with upstream's set of functions, not just their behavior. `docs/powerline_py_functions.txt` is the canonical list of every function in the upstream Python source, and the drift gate — the test `ported_fn_names_match_py.rs` — asserts that every function name the port claims to have ported (via its `/// Port of <py_fn>()` doccomments) matches a real entry in that list. A doccomment that names a Python function which does not exist, or misspells one that does, fails the gate.

This catches a class of error the parity harness cannot. The harness checks behavior for the inputs it exercises, but it cannot notice that the port *claims* to port a function that upstream renamed, or that a doccomment points at the wrong upstream function. The drift gate does: it holds the port's declared correspondence against the authoritative function list, so the map from Rust functions to Python functions stays accurate as both sides evolve. The README reports the gate as green — every ported function name matches `docs/powerline_py_functions.txt` — which means the port's self-description of *what* it ports is verified, not just its behavior.

Together the four mechanisms — citations, the DONE tier, the parity harness, and the drift gate — form a defense in depth around the word “faithful.” The citations make the port legible and feed the density metric; the DONE tier grades each function on declared-and- demonstrated correspondence; the parity harness proves behavior against the live interpreter; and the drift gate keeps the declared correspondence honest against the canonical function set. No one of them alone would suffice — citations can be wrong, tiers can be gamed, a harness can miss an untested path, a name list can drift — but together they make “1:1 Rust port” a claim that is checked from four independent angles on every CI run, which is what the phrase has to mean to be worth printing on a badge.

What the harness cannot check

Four mechanisms checking fidelity from four angles is a strong position, and the honest way to hold it is to be equally precise about what falls outside all four. A verification story that only enumerates its coverage is marketing; one that enumerates its blind spots is engineering. There are four, and each is bounded in a different way.

The first is **input coverage**. The parity harness compares the port against the live interpreter for the inputs it exercises — a set of configs, a set of requests, a set of scenarios — and it says nothing whatsoever about inputs it does not exercise. A theme using a combination of `extends`, `segment_data`, and per-mode overrides that no fixture combines could diverge without any assertion noticing, not because the harness is weak but because the input space of a config language is unbounded. The mitigation is the citation rule rather than more fixtures: a function whose every branch is pinned to an upstream line is one whose *untested* branches were also ported deliberately. Coverage of behavior comes from the harness; coverage of code comes from the citations; neither alone would be enough, and the reason the methodology has four mechanisms is precisely that each covers a different failure mode.

The second is **the live-sampled boundary**. Appendix AP's ledger states it exactly: a segment whose value is a measurement of a moving quantity cannot be asserted byte-identical against another process sampling the same quantity at another instant. The harness handles this by asserting the *encoding* against injected values rather than the *value* against a live sample, which is the correct move and is also, precisely, an admission that the live path is not what is being compared. The number a user sees on their `cpu_load_percent` segment has been checked for how it is formatted, colored, gradient-bucketed, and laid out; it has not been checked against what Python's `psutil` would have reported, because that comparison is not physically meaningful. Everything structural around the number is verified; the number is measured, not verified.

The third is **the host's interpretation**. The harness compares bytes, and bytes are the correct comparison boundary because they are where powerliners' responsibility ends. What happens after — whether tmux paints the directive the way you expected, whether your terminal renders the divider glyph, whether readline measures the escape markers correctly in your particular shell configuration — is outside the diff entirely. A port that emitted byte-identical output and still looked wrong on a user's screen would pass every assertion in the suite, and correctly so: the divergence would be in the host, not the renderer. This is why Chapter 17's debugging discipline ends at the host as a separate step, and why the correct response to “it looks wrong but the bytes match” is to look at the terminal rather than the port.

The fourth is **the extensions tree**, and this one is a definitional gap rather than a weakness. `src/extensions/` has no Python original, so there is nothing for the parity harness to diff against, nothing for the citation rule to cite, nothing for the tier classifier to grade, and nothing for the drift gate to match against `docs/powerline_py_functions.txt`. Every one of the four mechanisms is inapplicable by construction. What holds the extensions honest instead is ordinary engineering — the library test suite, the pure-functional seams that make

every branch of a resolution chain directly testable, and fixtures for the parsers — which is a good standard but is a *different* standard, and conflating the two would let a reader believe the new segments carry the same evidentiary weight as the ported ones. They do not. The ported tree is verified against an external oracle; the extensions tree is tested against its own specification, like any other well-tested new code.

Stating these four is not a hedge on the fidelity claim; it is what makes the claim falsifiable. “Byte-identical to upstream” means something precise: for deterministic behavior, on exercised inputs, at the byte boundary, in the ported tree. Each of those four qualifiers corresponds to one blind spot above, and each blind spot has a named reason and a named compensating mechanism. A project that dropped the qualifiers would be claiming more than it checks, and the whole point of building four verification mechanisms rather than asserting a slogan is that the qualifiers can be stated without the claim collapsing.

Chapter 11 — Packaging and Deployment

A faithful, fast renderer is only useful if it installs cleanly and swaps in without breaking what was there. This chapter covers the packaging discipline — the single- binary suite, the install paths, and the reversible migration that made the compatibility work of Chapters 6 and 9 pay off in practice.

The five-binary suite as one artifact

`powerliners` publishes as one crate that produces five binaries, declared explicitly in `Cargo.toml` with a `[[bin]]` entry each (`powerline`, `powerline-daemon`, `powerline-config`, `powerline-render`, `powerline-lint`). The explicit declarations exist so `cargo install powerliners` and `cargo publish` ship the *complete* upstream-equivalent CLI surface — install the crate and you get all five, not just a library. The release profile is tuned for a small, fast binary: `lto = "thin"`, `codegen-units = 1`, and `strip = "symbols"`, which is the standard recipe for a production static binary and matches the single-artifact discipline the whole design rests on.

The manifest also carves the repository into what ships and what does not. The `exclude` list keeps `vendor/` (the upstream `powerline` checkout used as a parity fixture), `tests/`, `docs/`, `scripts/` (the Python codegen and audit tools), and `.github/` out of the published crate — so a `cargo install` pulls the Rust source and nothing else, not the megabytes of vendored Python and HTML reports that live in the repo for development. This is the difference between a repository (which carries everything needed to *develop and verify* the port) and a published crate (which carries only what is needed to *build and run* it).

The single-artifact story is the deployment payoff of being native. There is no interpreter to install per machine, no `pip` dependency tree to resolve, no shared library to keep in version-sync with the binary. A container image needs the binaries and your config tree and nothing else; a remote box needs one `scp` or one `brew install`. This is the same deployability that made `awk` and other single-binary tools durable, and `powerliners` keeps it deliberately — the whole point of eliminating the Python runtime is lost if you replace it with a different runtime to install, so the port replaces it with nothing.

Install paths: Homebrew, cargo, and from source

The fastest install is the Homebrew tap, auto-bumped on each release:

```
brew tap MenkeTechnologies/menketech
brew install powerliners
# installs powerline, powerline-daemon, powerline-config, powerline-render, powerline-lint
```

`cargo install powerliners` is the equivalent for a Rust toolchain, and building from source is the third path, useful when testing an unreleased build:

```
cargo build --release --locked \
  --bin powerline --bin powerline-daemon \
  --bin powerline-config --bin powerline-render --bin powerline-lint
```

Release binaries land at `target/release/{powerline,powerline-daemon,powerline-config,powerline-render,powerline-lint}`. All three paths produce the same five binaries; the choice is a matter of which toolchain you already have. The `--locked` flag on the source build pins the exact dependency versions from `Cargo.lock`, which is the reproducibility discipline for a build you intend to trust in place of a working tool.

The install-method-agnostic bundling of Chapter 8 is what makes these paths interchangeable. Because the tmux conf files and the vim plugin are `include_str!`-embedded in the binary and extracted to `~/.cache/powerliners/` on first use, there is no compile-time path baked into the binary that a Homebrew install or a manual `cp` would get wrong. `powerline-config tmux setup` and `powerline-config vim source-path` work identically no matter how the binary got onto the machine — which is a real improvement over upstream’s install-time path baking, and the reason 0.2.3’s move to embedded configs mattered.

The reversible migration

The migration is designed to be tried without commitment and undone without trace, which is the right shape for swapping out a tool you depend on every second. The README’s six steps reduce to: install or build the suite; **verify parity against your own config** before changing anything (spin up the Rust daemon on a throwaway `--socket` and diff its wire-protocol output against `powerline-render tmux right`); stop the Python daemon (`powerline-daemon -k`); symlink the Rust `powerline-daemon` into a `$PATH` directory ahead of the Python one; restart tmux; and confirm the running daemon is the Rust binary (`ps -p $(cat /tmp/powerline-ipc-$UID.pid)`).

The critical property is that the migration touches *only* which binary resolves in `$PATH` — no config edits, no `.tmux.conf` edits, no shell-rc edits. This works because of the wire protocol: the Python C client shipped by `pip install powerline-status` does not need replacing, because it speaks the same protocol to whichever daemon is bound to the socket. You can swap only the daemon and leave the Python client in place, and everything keeps working because the client does not know or care what language the daemon is written in. The compatibility work of Chapters 4, 6, and 9 is what makes a migration this shallow possible — same wire format, same socket path, same config tree, same rendered bytes.

Rollback is symmetric and complete: `powerline-daemon -k` to stop the Rust daemon, `rm` the symlink so `powerline-daemon` re-resolves to the Python script, and `powerline-daemon -q` to start Python again. Because nothing but the symlink changed, there is nothing else to undo — the config is untouched, the tmux and shell config are untouched, and the Python daemon comes back to exactly the state it was in. A migration you can try in a minute and reverse in another is a migration people will actually attempt, and the reversibility is the direct consequence of the port having changed the *implementation* while preserving every *interface*: the wire protocol, the socket path, the config tree, and the rendered output.

Starting the daemon: rc file, login, or a user service manager

The migration above swaps which binary answers to `powerline-daemon`, and deliberately says nothing about *who starts it*, because the answer is the same before and after the swap. It is worth spelling out anyway, because the choice interacts with the socket namespaces of Chapter 4, the absent `SIGHUP` of Chapter 13, and the per-user model Chapter 14 insists on, and because a reader deploying this on a server rather than a laptop will reach for a service manager and should know what the design expects of one.

The default and simplest arrangement is lazy: nothing starts the daemon deliberately, and the first client that fails to reach the socket falls back to `powerline-render` while some other line in the user’s shell rc or tmux conf runs `powerline-daemon -q` to bring one up for next time. This works because of the property Appendix E.9 describes — a client that cannot connect does not fail, it execs the direct renderer — so the window between “no daemon” and “daemon” costs a slower render rather than a broken prompt. The `-q` flag is the

one the rc files use precisely because a startup message printed into an interactive shell's initialization is a nuisance, and because on the second and subsequent shells the daemon is already running and the start is a no-op that should say nothing.

The explicit arrangement starts it once at login rather than on every shell. Functionally this is the same daemon reached by the same socket; what changes is that the start is not racing several shells opening at once. The race is benign — `check_existing` returns the already-running signal on `AddrInUse` rather than erroring, so a second starter simply loses — but a login-time start makes the ordinary case one process instead of a scramble.

The service-manager arrangement is the one worth detailing, because it needs the flag the other two do not. A daemon that forks twice and detaches, which is what `daemonize()` does, is the wrong shape for a supervisor that wants to track a process: the process it started exits immediately and the one doing the work is not its child. That is what `-f / --foreground` exists for. It skips the double fork, keeps the process attached, and makes the daemon a well-behaved supervised service:

```
[Service]
Type=simple
ExecStart=/usr/local/bin/powerline-daemon --foreground --quiet
Restart=on-failure
```

Two properties of the design make this unit shorter than it would otherwise be. The socket identity is derived from the UID rather than configured, so there is nothing to pass and no path to keep in sync between the unit and the shell rc — which is also why this must be a *user* unit rather than a system one, per Chapter 14's insistence that the UID is the scope and not merely a naming convention. And the restart semantics are forgiving: a daemon that dies and is restarted loses only its warm state, and the clients that connected in the gap fell back to the direct renderer, so a restart is a latency event rather than a correctness event. There is no drain, no graceful handover, and none is needed.

The cleanup behavior differs between the two socket namespaces and it matters here. On Linux the socket is in the abstract namespace, so it disappears with the process and a killed daemon leaves nothing behind for the next one to trip over. On the filesystem platforms there is a real socket file and a pidfile, and the port handles both ends: `check_existing` removes a stale socket before binding, the pidfile is held under an RAIL guard whose drop cleans it up, and the `SIGTERM` handler unlinks the pidfile before exiting. A supervisor that stops the service with `SIGTERM` therefore gets the tidy path, and one that kills it harder leaves a stale socket that the next start removes. Neither case requires operator intervention, which is the property you want from a service you will forget you configured.

The last consideration is reload. There is no `SIGHUP` handler, so a supervisor's reload verb has nothing to send; a config edit is picked up by the per-extension staleness check on the next render without any gesture at all, and a full reload is `--replace` or a restart. A unit that maps its reload action onto a signal will silently do nothing, and a reader migrating a unit written for upstream should replace that mapping rather than assume it carries over. This is the lifecycle difference Chapter 13 describes, encountered in the place where it actually costs somebody a confused ten minutes.

Chapter 12 – Concurrency, Backpressure, and the Daemon Under Load

Chapter 4 described the daemon as a bind, a detach, and an accept loop, and Chapter 5 accounted the latency of one render through it. Both descriptions assume a single request at a time, and in the overwhelming majority of a day's renders that assumption holds — a prompt redraw is a lone client connecting to an idle daemon, getting its bytes, and going away. But the daemon is a shared resource by design, one process per UID serving every shell, every tmux pane, and every editor the user has open, and shared resources have to be understood at their worst moment rather than their typical one. This chapter is about that worst moment: what happens when several clients arrive at once, what the daemon does about it, what it deliberately does not do, and why the answer is the right one for this workload even though it would be wrong for a general-purpose server.

One process, one loop, one render at a time

The concurrency model is the simplest one that could work, and stating it plainly is the beginning of understanding its limits. `main_loop` is a single-threaded `poll` over one listening socket and whatever connections are currently open. When `poll` reports the listener readable, the loop accepts and adds the new connection to its set. When `poll` reports a connection readable, the loop reads that connection's request to completion, renders it, writes the reply, and returns to `poll`. There is no worker thread, no thread pool, no `fork` per connection, and no asynchronous executor: the render for connection A runs to completion on the same thread that will later run the render for connection B, and while A's render is running, B's readiness sits in the kernel's socket buffer waiting for the next trip through `poll`.

This means the daemon serializes renders. It does not serialize *connections* — `poll` will happily hold several accepted, half-read connections at once, because accepting and reading are cheap and the loop interleaves them — but the work between reading a complete request and writing its reply is strictly sequential. Two clients that arrive in the same microsecond are served one after the other, and the second one's wall-clock latency is its own render plus however long the first one's took. In the common case that additive term is the tens of microseconds Chapter 5 accounted for, which is invisible. In the uncommon case it is not, and the rest of this chapter is about the uncommon case.

The serialization is not an oversight, and it buys two properties that matter more than parallelism does at this scale. The first is that there is no shared mutable state to protect: the orchestrator cache keyed by `PowerlineKey`, the parsed config tree, the colorscheme palettes, and the adapter registry are all touched by exactly one thread, so there is no lock to acquire on the hot path, no lock to get wrong, and no lock contention to reason about. A render that needs to build and cache a new orchestrator does so without coordination, because nothing else can be building one concurrently. The second is that the daemon's behavior is deterministic in a way a threaded design's would not be: given a sequence of requests, the sequence of renders and the sequence of cache states is fixed, which is exactly the property the parity harness of Chapter 10 depends on when it asserts byte-identical output for a scripted sequence of wire requests.

Where the bursts actually come from

Understanding the load shape matters more than understanding the concurrency model, because the model is adequate or inadequate only relative to the load. The load has three sources and they burst very differently.

Shell prompts are the least bursty. A `precmd` hook fires when a command finishes, which is a human-paced event; even a user typing continuously generates renders at the rate they press Enter, which is single-digit events per second at the absolute most. Two shells in two panes both redrawing after two commands finish at nearly the same instant is the realistic worst case, and it is two requests, not two hundred. The shell path is essentially never the thing that saturates the daemon.

The vim statusline is spikier but still bounded. The autocmd set of Chapter 8 — `WinEnter`, `BufWinEnter`, `BufEnter`, `TabEnter`, `ModeChanged`, and on old vim the `CursorMoved` fallback — can fire several times for one user action, because switching windows in a split can trigger a window event and a buffer event and a mode event in sequence. That produces a short run of requests rather than a sustained rate, and the runs are separated by human think-time. The legacy `CursorMoved` path is the one exception and the reason the book flags it as the autocmd to disable when refresh stutters: it turns cursor motion into renders, which turns a held-down arrow key into a request stream at the terminal's key-repeat rate.

The tmux statusline is the real source of concurrency, and it is the one the whole architecture was shaped around. tmux refreshes on a timer, and it invokes the client once per side and once per window, so a session with many windows produces a burst of client invocations arriving essentially simultaneously on each `status-interval` tick. That is the multiplication Chapter 1 named as the reason the Python startup tax hurt so much, and it is also the reason the daemon's serialization is tolerable: a handful of renders of cheap segments, executed back to back at tens of microseconds each, complete in a small fraction of a millisecond of aggregate work, and the last client in the burst waits for a delay that is still far below anything a human or a statusline refresh cares about. The burst is wide but each item in it is tiny.

Head-of-line blocking, and where it actually bites

The failure mode implied by a serialized loop is head-of-line blocking, and it is worth stating exactly rather than gesturing at. If one request's render is slow, every request behind it waits, regardless of how cheap those requests are. A sub-millisecond `date` render queued behind a render that is blocked on a network read does not get to jump the queue; it gets the network read's latency added to its own.

This is not a hypothetical, and Chapter 5 already named the mechanism that makes it reachable: the daemon samples live data on demand at render time rather than in a background thread. That decision is the right one for freshness and simplicity — no threads, no stale values, no polling machinery — but it has a direct concurrency consequence that the threaded model did not have. Under upstream's `ThreadedSegment` approach, a slow sampler ran off the render path and the render returned the last-known value immediately, so a slow network never delayed a render, only made its value slightly old. Under on-demand sampling the slow network is *inside* the render, and because the render is inside a serialized loop, it is inside every other render queued behind it too. The cost of one expensive segment is therefore not paid once by the line that contains it; it is paid once by that line and again, as queueing delay, by whatever is waiting.

The practical shape of this is narrow but sharp. It requires an expensive segment — one that touches the network or a slow disk — and it requires concurrent demand, which in practice means the tmux burst. A theme that puts `external_ip` or `email_imap_alert` on a tmux statusline refreshed across many windows has arranged for a network round-trip to sit at the head of a deep queue on every tick. Everything the book says about segment placement (Chapter 3's cost line, Chapter 15's latency classes) is usually framed as “this segment makes *its own* line slow,” and that framing is incomplete: on a shared daemon it makes every concurrently-queued line slow too. The placement decision is a system-wide decision, not a per-line one.

The mitigations available to a user are all placement mitigations, which is consistent with the rest of the design. Keep the expensive samplers off the high-frequency surfaces; if a network-derived value must appear, put it where it is rendered rarely rather than where it is rendered per-window per-tick; and prefer a segment that reads a file some other process refreshes over a segment that performs the network read itself, because a filesystem read is bounded and a network read is not. This is the same advice as Chapter 3's, arrived at from the concurrency side rather than the single-render side, and the fact that both roads lead to the same place is a sign the advice is structural rather than incidental.

The read timeout as a backpressure valve

There is one place where the daemon defends itself explicitly, and it is on the read rather than the render. `do_read` accumulates a request until the buffer ends with the terminating double NUL, or until a two-second read timeout elapses. Without that timeout, a client that connected and then went quiet — killed mid-write, wedged, or simply buggy — would leave the loop holding a connection that is readable-then-not, and a naive implementation could block indefinitely waiting for a message that will never be completed. The timeout converts an unbounded wait into a bounded one: the half-written request is abandoned and the loop moves on.

Two seconds is a deliberately generous bound and reading it correctly matters. It is not a render deadline and it is not a latency target; a legitimate client writes its entire request in one `write` and never comes close to the bound. The timeout exists solely to keep a pathological peer from converting itself into a lasting stall of a shared, single-threaded process. The asymmetry — microseconds of expected behavior, seconds of tolerated misbehavior — is correct precisely because the timeout is a safety net rather than a scheduler: setting it tight enough to be a latency control would risk cutting off a slow-but-legitimate write, and the cost of that false positive (a client that gets no prompt) is worse than the cost of a rare stall.

What the daemon notably does not do is impose a *render* timeout. A segment that blocks on a hung network read blocks the loop for as long as the segment blocks; there is no watchdog that abandons a slow render and replies with a partial line. This is a deliberate consequence of the port's structure rather than an omission to be apologized for: introducing a render deadline would mean either killing a render mid-flight (which requires either threads or a cancellation mechanism that the adapters do not have) or returning partial output (which would break the byte-identity guarantee the whole project is built on). Faced with the choice between “always correct, occasionally slow” and “always fast, occasionally wrong,” a port whose product is fidelity has exactly one available answer, and it takes it.

Why not a thread pool

The obvious response to a serialized loop is to hand each connection to a worker thread, and it is worth walking why the port does not, because “we could add threads” is the sort of proposal that sounds free and is not.

The first cost is the shared state. Today the orchestrator cache, the config tree, and the colorscheme palettes are single-threaded and therefore lock-free. A worker pool turns every one of them into shared mutable state that needs synchronization. The orchestrator cache is the sharp case: it is a read-mostly map with a build-and-insert-on-miss path, which is the classic shape where a naive mutex serializes exactly the renders you were trying to parallelize, and where the lock-free alternatives introduce the possibility of two threads building the same orchestrator concurrently and racing to insert it. Solving that properly is real work, and it would be work spent making the *cheap* path concurrent when the cheap path was never the problem.

The second cost is fidelity risk, which for this project outranks the first. The parity harness asserts byte-identical output against the live interpreter for a scripted sequence of requests. A concurrent daemon makes the *order* in which renders observe the world nondeterministic — two renders sampling the same live value at overlapping instants, two renders racing to populate the same cache entry, two renders interleaving their subprocess probes — and nondeterminism is exactly what a byte-identity harness cannot tolerate. The port would be trading a verified property for an unverified performance improvement, on a workload where the improvement is not needed.

The third cost is that threads do not solve the problem people imagine they solve here. Parallelism helps when the work is CPU-bound and there is more of it than one core can chew. The daemon's work is not CPU-bound: Chapter 5's accounting shows the render is tens of microseconds of computation, and the only time the loop is slow is when it is *blocked on I/O* inside an expensive segment. Threads would indeed let a cheap render proceed while an expensive one blocks — that is a real benefit — but the same benefit is available, without any of the costs above, by not putting the expensive segment on the high-frequency path in the first place. Given a choice between a concurrency mechanism that mitigates a placement mistake and a placement discipline that prevents it, the port chooses the discipline and documents it, which is consistent with its refusal to add

machinery that exists to paper over a design problem.

The client's fallback is not a load-shedding mechanism

It is tempting to read the client's `powerline-render` fallback (Chapter 4) as a pressure-relief valve — the daemon is busy, so the client renders locally — and that reading is wrong in a way worth correcting, because it changes what you expect under load.

The fallback triggers on a socket that is *unreachable*: the daemon was never started, was killed, or crashed, and the connect fails. It does not trigger on a daemon that is merely slow. A client whose connect succeeds is committed; it will wait in `read` for however long the daemon takes to work through whatever is ahead of it. There is no client-side timeout that gives up on a warm-but-busy daemon and re-renders locally, and adding one would be a poor trade anyway: a local cold render costs a fresh `powerline-render` process, which is far more than any plausible queueing delay behind a burst of cheap renders, so a client that bailed out under load would usually make itself slower rather than faster.

What the fallback actually guarantees is availability, not latency. A dead daemon degrades every client to a slower path rather than breaking it, and that is the property Chapter 4 called the resilience story. Under load the guarantee is different and weaker: the daemon stays correct and stays available, and its latency degrades gracefully in proportion to the work queued ahead of you. Both are honest guarantees; they are just not the same guarantee, and conflating them would leave a reader expecting a self-healing behavior that the design does not have and does not claim.

The connection's lifetime, end to end

Putting the pieces in order gives the complete life of one request under the loop, and it is short enough to hold in your head. The client connects; the kernel completes the connection and marks the listener readable. On its next trip through `poll` the daemon accepts and adds the connection to its set — `eintr_retry_call` wraps the accept so that a signal arriving mid-syscall retries rather than aborts, which matters because a daemon that dies on a stray `EINTR` is a daemon that dies eventually. The client writes its `argv + cwd + env` blob and the connection becomes readable. On its next trip through `poll` the daemon reads, and `do_read` keeps reading until the double NUL arrives or the two-second bound expires. The complete blob goes to `parse_args`, which either yields the `(argv, environ, cwd)` trio or returns nothing, in which case the daemon replies with its malformed-request message rather than crashing — a hostile or buggy client gets an error, not a denial of service.

With a valid request in hand the daemon checks for the EOF sentinel, and if the request is exactly that byte sequence it exits cleanly rather than rendering; this is the `-k` path, and it is worth noting that it means any client that can reach the socket can stop the daemon, which Chapter 14 returns to as a trust question rather than a bug. Otherwise the daemon builds the `PowerlineKey`, looks up or builds the orchestrator, runs the render, and writes the reply. The connection is then finished with; the client reads the reply, copies it to stdout, and exits, and the loop returns to `poll` for the next event.

Every step in that sequence is either bounded work or bounded-by-timeout waiting, with a single exception: the render itself, whose duration is the sum of its adapters' durations and is therefore bounded only by the slowest segment the theme configured. That exception is the entire concurrency story of the daemon, restated as a property of the loop. Every other part of the system is small and predictable; the theme decides whether the render is too. This is why the book keeps returning to segment placement from different directions — it is not a style preference, it is the one input a user controls that determines whether a shared, serialized, deliberately simple daemon behaves the way its design assumes.

Chapter 13 — The Lifetime of Warm State

A daemon is a bet that state built once can be reused many times, and every such bet has a second half that gets less attention than the first: knowing when the reused state has gone wrong. Chapter 5 covered the payoff — the config parsed once, the orchestrators cached, the amortized cost of setup rounding to nothing across a day’s renders. This chapter covers the other half. What exactly is held warm, what makes a piece of it stale, how the port notices, what upstream did that the port does not, and what a user should expect when they edit a theme while the daemon that is rendering it is still running.

What “warm” actually holds

“The daemon keeps state warm” is three distinct things, and separating them clarifies what can and cannot go stale.

The first is process-level state: the daemon is a running process, so the executable is mapped, the allocator is warm, and — in the Python original — the interpreter is booted and the module graph imported. This is the state whose absence constitutes the startup tax, and it is the least interesting kind because it can never be stale. A mapped executable does not go wrong when a config file changes.

The second is the resolved configuration: the cascade of Chapter 6 walked once, the JSON files parsed, the `mergedicts` deep-merges applied, and the effective config materialized as an in-memory tree. This is the state that *can* go stale, because it is a derived view of files on disk, and the files can change under it. Everything downstream — themes, colorschemes, the palette — is derived from this tree, so staleness here propagates to everything a user sees.

The third is the orchestrator cache: the `PowerlineKey`-keyed map of built `Powerline` instances that Chapter 5 described, each holding its extension’s renderer, its themes, and its wired adapters. This is the state that makes the second render of a given kind cheaper than the first, and it is derived from the second kind, so it inherits its staleness question and adds one of its own — an entry can be stale not only because the underlying config changed but because it was built under environment overrides that a later request does not share.

Only the second and third are invalidation problems. The distinction matters because it tells you what a restart actually accomplishes: restarting the daemon rebuilds all three, but only the last two ever needed rebuilding, and the port’s reload mechanism targets exactly those.

Invalidation: the per-`ext` mtime staleness check

The port’s answer is a staleness check keyed on the modification times of the files the config was built from, applied per extension. When a request arrives for an extension, the daemon compares the mtimes of the underlying config files against what they were when that extension’s config cache was built; if they have moved, the cache for that extension is rebuilt from disk before the render proceeds. A user who edits a theme sees the edit take effect on the next render of the affected extension, without restarting anything.

Three properties of this design deserve naming. It is **pull-based**: nothing watches the filesystem, and no notification mechanism is involved; the check happens because a request arrived and the daemon looked. This means an edit made while no renders are happening costs nothing until the next render, which is the

correct economics for a process that spends most of its life idle. It is **per-extension**: editing a vim theme rebuilds the vim config and leaves the tmux and shell caches untouched, so the cost of an edit is proportional to what the edit affects rather than to the size of the whole tree. And it is **coarse in the right direction**: an mtime that moved without the content changing causes an unnecessary rebuild, which costs one setup; an mtime that did not move despite the content changing would cause a missed reload, which costs correctness. The check errs toward rebuilding, which is the cheap error.

The cost of a rebuild is exactly the cold-path setup of Chapter 5: the cascade re-walked, the JSON re-parsed, the merges re-applied, the colorscheme rebuilt, the adapters re-wired. That is a real cost, measured in whatever a first render costs rather than what a warm one does, but it is paid once per edit rather than once per render, which is the correct place for it. A user in the middle of tuning a theme pays it on each save and never notices, because they are looking at the prompt they just changed rather than timing it.

The `SIGHUP` re-exec, and why its absence is a lifecycle difference

Upstream's daemon supports a `SIGHUP`-driven re-exec: signal the running daemon and it reloads its configuration by replacing itself, without dropping the bound socket. The port does not implement this, and Chapter 4 flags it as one of the handful of deliberate, documented divergences. It is worth being precise about what is and is not lost.

What `SIGHUP` re-exec gives you that the mtime check does not is *explicit, whole-process* reload on demand. It reloads everything, immediately, at a moment the operator chooses, regardless of whether any file's mtime moved — which is the right tool if the thing that changed is not a file at all, or if you want to be certain the reload has happened before you observe the next render rather than as a side effect of it. The mtime check is implicit and lazy: it happens when a render happens, it covers what the mtime covers, and there is no way to demand it.

What the port offers instead is the pair the divergence note names: the automatic per-`ext` mtime staleness reload for the common case (a config edit), and `--replace` for the explicit case. `--replace` starts a new daemon that takes over the socket from the running one, which reloads everything by the most direct means available — a new process — at a moment the operator chooses. The difference from a re-exec is that the socket is rebound rather than carried across, so there is a brief window during the handover; the practical consequence for a prompt renderer is a client or two falling back to `powerline-render` for one render, which is exactly the degradation the fallback exists to make harmless.

So the divergence is a lifecycle difference with no output consequence, and that phrasing is load-bearing for the book's honesty ledger. No rendered byte differs because of it. What differs is the set of operational gestures available: upstream has a signal, the port has a staleness check plus a replace. A reader migrating a service unit or an init script that sends `SIGHUP` on config change needs to know this — the signal will not do what it did — and that is exactly why the port documents the omission in the open rather than quietly approximating it.

Editing a theme while the daemon is warm

The concrete workflow is worth walking because it is the one every user hits within an hour of installing, and the mechanisms above determine what they see.

You open `themes/tmux/default.json`, add a segment, and save. The next tmux `status-interval` tick fires; a client sends a `tmux` request; the daemon notices the tmux config's mtime moved, rebuilds that extension's config from disk, builds a fresh orchestrator, renders, and replies. The new segment appears. Total user-visible effect: the statusline changed on the next tick, one tick's render was slower than usual, and nothing was restarted.

Now suppose the edit had a mistake in it — a segment name that no adapter provides, or a highlight group

no colorscheme defines. The rebuild succeeds (the JSON parsed) but the render fails at the point Chapter 6 described, where `get_highlighting` cannot resolve any of a segment's groups and errors with a message naming everything it tried. This is the moment `powerline-lint` exists for, and the workflow it implies is: lint before you save into the live tree, or at least lint the moment the prompt looks wrong, because the linter reports the authoring error with a source position while the failed render reports only that something did not resolve. The staleness reload makes the edit-render loop fast; the linter is what makes it *diagnosable*, and the two are meant to be used together.

The corresponding workflow for something the mtime check cannot see is worth stating too. The `PowerlineKey` includes the `POWERLINE_CONFIG_PATHS`, `POWERLINE_CONFIG_OVERRIDES`, and `POWERLINE_THEME_OVERRIDES` environment values, which means a change to those does not need invalidation at all — a request carrying different override values simply keys to a different cache entry and gets its own orchestrator, built on demand. That is a nice property that falls out of putting the overrides in the key rather than treating them as ambient state: the environment cannot go stale, because it is not cached, it is *keyed*.

The cache key's coarseness, and the one case it does not see

Chapter 5 praised the `PowerlineKey` for being coarse in the dimensions that do not change the orchestrator — the side, the exit code, the job count — and precise in the ones that do. That is the right trade, and it has a consequence worth naming explicitly rather than leaving as an exercise.

Because the key is a tuple of the extension, the renderer module, the config/theme/path overrides, and the three environment override values, two requests that differ in *nothing else* share an orchestrator. That is the point. But it also means the orchestrator is shared across callers whose environments differ in every other way: two shells in two directories with different `$KUBECONFIG` values and different `$PATHs` hit the same warm `shell` orchestrator. This is correct, and the reason it is correct is the wire protocol. The per-request environment is not baked into the orchestrator; it arrives on the wire, is demarshalled by `parse_args`, and is deposited in `segment_info` for the adapters to read (Chapter 4, Appendix AJ). The orchestrator holds the *configuration*, which is shared; the request carries the *context*, which is not.

The one place a reader can get this wrong is in reasoning about extension segments that read ambient process state rather than `segment_info`. An adapter that consulted the daemon's own environment or its own working directory would silently bind itself to whatever the daemon inherited at start, and it would produce the same wrong answer for every caller, cached or not. This is why Appendix AJ states the environment contract as an obligation on adapters rather than a convenience: adapters read the forwarded environment, never the process environment, and the reason is not style but that the daemon's own environment is a long-lived artifact of login while the caller's is the thing being rendered. The cache key's coarseness is safe precisely because everything that varies per caller travels on the wire instead of living in the cache.

Cold start, and what the first render buys

The last piece of the warm-state story is its beginning: what the very first render pays for, and therefore what a restart costs.

The first request of a given extension after a daemon start walks the whole cascade — `_find_config_files` across the search path, `load_json_config` per file, `mergedicts` to fold them, the `Colorscheme` constructor building the palette with its `cterm_to_hex` derivations and its parallel gradient lists, the theme grammar resolved into segment specs, and `gen_segment_getter` run once per segment to produce the resolved getters. None of this is expensive in absolute terms — it is file reads and JSON parsing — but it is enormously more expensive than the warm path, and it is the entire reason the warm path is worth having.

The shape of the amortization is what makes the daemon worth running rather than just making the renderer fast. Setup cost is paid per *daemon lifetime per extension*; render cost is paid per request. A login session that

starts a daemon and then renders for a working day pays setup once for each of the two or three extensions it uses, against tens of thousands of renders. The ratio is not a tuning result, it is a structural consequence of moving a fixed cost out of the per-request path — which is the same argument Chapter 5 made about interpreter startup, applied one level down to the config parse. Both are fixed costs that the daemon converts from per-render to per-lifetime, and both conversions are worth exactly as much as the render count is large.

This also explains why the port is relaxed about the cost of a staleness rebuild. A rebuild is a cold start for one extension, and cold starts are cheap in absolute terms and rare in practice — a user edits a theme a handful of times in a sitting and then leaves it alone for months. Optimizing the rebuild path would be optimizing something that runs when a human pressed save, which is the definition of a cost nobody can perceive. The engineering effort belongs on the path that runs many thousands of times a day, and that is where it went.

Chapter 14 — The Trust Boundary

Every design decision in the previous chapters was made for speed or fidelity, and several of them have consequences that neither speed nor fidelity would have surfaced. A long-lived process holding a socket, accepting requests that carry the caller's entire environment, and capable of executing programs named in a configuration file is a component with a trust boundary, and a book that documented the wire protocol without documenting who is allowed to speak it would be incomplete. This chapter states the boundary explicitly: what the daemon trusts, what it does not, what a user's threat model should be, and which of the port's design choices a careful operator should think about before adopting.

None of what follows is a vulnerability disclosure or a claim that the design is unsafe. It is the ordinary work of naming a component's assumptions, which is worth doing precisely because the assumptions are reasonable and therefore easy to leave unstated.

Who can talk to the socket

The daemon's identity model is per-UID: one daemon per user, its socket named `powerline-ipc-<uid>`. That naming establishes *identity* — “this is user N's powerline daemon” — and it is important to be clear that identity is not enforcement. What actually constrains who can connect is the socket's access properties, and those differ between the two namespaces of Chapter 4.

On the platforms where `USE_FILESYSTEM()` is true — macOS and the BSDs — the socket is a real file at `/tmp/powerline-ipc-<uid>`, and access to it is governed by ordinary filesystem permissions on that path, evaluated by the kernel at connect time. This is the familiar model: whoever can open the path can speak to the daemon, and the usual questions about the mode bits, the ownership, and the containing directory's semantics all apply.

On Linux the socket is in the abstract namespace, `\0powerline-ipc-<uid>`. An abstract socket has no filesystem entry, and therefore no mode bits and no owning directory: there is nothing for a filesystem permission check to consult. Its scope is the network namespace, so a process in the same network namespace that knows the name can attempt to connect. Chapter 4 presented the abstract namespace as the solution to a real problem — a filesystem socket can be orphaned by a crash or swept by a `/tmp` cleaner, an abstract one cannot — and that is true and remains the right reason for the choice. It is also true that the two namespaces have different access stories, and a reader deploying on a shared multi-user Linux host should know which one they are getting.

The port inherits this choice from upstream rather than making it, which matters for the compatibility argument: a port that “improved” the socket's access model would no longer be speaking the same protocol at the same address as the client it must interoperate with. It is one of the places where fidelity and a hypothetical preference point in different directions, and fidelity wins because the whole product is drop-in compatibility. Documenting the consequence is the honest resolution.

What a connection can do

Given a connection, the set of things a peer can ask for is small and worth enumerating, because the boundary is defined by capability rather than by intent.

A peer can request a render. That means supplying an arbitrary `argv`, an arbitrary `cwd`, and an arbitrary environment, and receiving the rendered markup. The rendering is done against the *supplied* context, which is

the correctness property Chapter 4 established, and which here has a second reading: the peer chooses the context, so the peer can make the daemon render as if it were in any directory, with any environment, running any of the configured segments. On a single-user machine this is uninteresting — you are asking your own daemon to render your own prompt. Its interest is entirely a function of whether anyone else can reach the socket.

A peer can stop the daemon. The EOF sentinel is a plain byte sequence, `EOF\0\0`; any peer that can connect can send it and the daemon exits cleanly with status zero. This is not a flaw — it is exactly how `powerline-daemon -k` works, and the port carries the byte sequence exactly because a Python client must be able to stop a Rust daemon and vice versa — but it does mean “can connect” implies “can shut down.” The consequence is bounded by the fallback of Chapter 4: a stopped daemon degrades every client to a slower local render rather than breaking anything. Availability degrades; correctness does not.

A peer can cause work. Every request costs a render, and Chapter 12 established that renders are serialized. A peer that can connect can therefore consume the daemon’s single thread, and a peer that can name a theme with an expensive segment can make each unit of that consumption larger. The read timeout bounds the misbehaving-write case but there is no request-rate limit, because a prompt renderer serving one user’s shells has no natural need for one. As with the sentinel, the practical severity is entirely determined by who can reach the socket.

What a peer *cannot* do is more interesting than what it can. It cannot make the daemon read files outside what the config cascade resolves, because paths come from configuration rather than from the request. It cannot inject a segment: the request carries an extension and a side, and the segments come from the config the daemon already loaded. And it cannot escalate: the daemon runs as the user who started it and does nothing to change that.

The environment crosses a process boundary

The wire protocol forwards the caller’s entire environment, and that is not a design detail — it is a requirement, for the reason Chapter 4 gave: a daemon rendering against its own stale environment would produce wrong output for every environment-reading segment. But “the entire environment” deserves a moment of thought, because environments accumulate things.

A modern developer shell’s environment contains API tokens, cloud credentials, registry passwords, and session secrets, put there by whatever tool wanted them. When a prompt renders, all of that is marshalled into the wire blob and written across a socket to another process. For the ordinary case — your daemon, your UID, your machine — this is a transfer between two processes that already have equal access to those secrets, and it changes nothing about your exposure. It is worth naming anyway for two reasons. First, it makes the socket’s access properties matter more than they otherwise would: a peer positioned to observe what other clients send would be observing secrets, and so the reachability question above is a confidentiality question and not only an availability one. Second, it means anything that logs or traces wire traffic is logging secrets, which is worth knowing before you point a debugging tool at the socket. Chapter 17’s debugging techniques dump wire requests; a request dump belongs in a scratch buffer, not a pasted issue report.

The mitigation available in the design is not a filter — a filter would break the compatibility guarantee, since upstream forwards everything and segments may read anything — but placement, in the same sense as everywhere else in this book. If a secret does not need to be in the interactive shell’s exported environment, it should not be; and if it must be, its presence on the wire between two of your own processes is the ordinary consequence of exporting it.

exec segments are code, resolved through a cascade

The escape hatches of Chapter 3 are the sharpest part of the boundary and the part most worth understanding before you use them. Both patterns run programs.

Pattern A names a command inline in the theme. The trust question is simple: whoever can edit your theme JSON can choose a program your daemon will execute on every render. That is the same trust as your shell's rc files — the theme is your configuration and you own it — but it is worth stating because a theme feels like data and this makes it code.

Pattern B is the one that rewards a careful reading. A dotted-path segment reference is resolved against the *full config cascade*, and the first match wins, with the extension order in `SCRIPT_EXTENSIONS` deciding among candidates within a directory. The cascade includes the system-wide `/etc/powerline/` root and the roots named in `$POWERLINE_CONFIG_PATHS`. Two implications follow. The first is that a segment script's identity is determined by search order, so a file placed earlier in the cascade shadows one placed later — the same first-hit-wins semantics that makes the cascade useful for overriding a bundled theme is also a mechanism by which a script can be substituted. The second is that `$POWERLINE_CONFIG_PATHS` is an environment variable, it is forwarded on the wire, and it is part of the `PowerlineKey`: a request can name config roots. On a single-user machine, again, this is unremarkable — it is how the probe-socket workflow of Chapter 11 works, and it is a feature. It is worth knowing that the mechanism exists and what it reaches.

The practical guidance is the boring kind and it is the right kind. Treat the config tree as code, because for `exec` segments it literally is: own the directories, do not make them group-writable, and be as careful about a segment script as you would be about something sourced from a shell rc file. If you use Pattern B, know which roots are in your cascade. And prefer a native adapter to an `exec` segment when the segment is going to be permanent, for the reasons Chapter 16 gives — the native path is faster and it removes an executable-resolution step from the render entirely.

Run it as yourself

The last piece of guidance is the shortest and the most important: the daemon is a per-user process and should be started by the user whose prompts it renders. It has no reason to run as root, it gains nothing from running as root, and running it as root would take a component that executes configured programs and reads a caller-supplied environment and hand it privilege it has no use for. The per-UID socket naming assumes the per-user model, the config cascade is rooted at the user's `$XDG_CONFIG_HOME`, and the whole design reads as what it is: a user-session service, started from a login shell or a user-level service manager, living and dying with the user's session.

A system-level unit that starts one daemon for all users would break the model in every direction — the wrong config root, the wrong socket identity, and a privilege boundary crossed for no benefit — and the fact that nothing in the design encourages it is not an accident. When Chapter 4 says “one per UID,” the UID is not merely a naming convention; it is the scope, and the correct deployment is the one that makes the naming true.

The threat model, stated

Collecting the above into a single statement: powerliners' daemon trusts the user who started it, the configuration tree that user's cascade resolves, and any peer that can reach its socket. It does not trust the *content* of a request beyond parsing it defensively — a malformed blob gets an error reply rather than a crash, and an unterminated write gets a timeout rather than a stall — but it does trust that a peer entitled to connect is entitled to ask for a render and to ask for a shutdown, because on the machine it was designed for those are the same person.

That is a reasonable model for a prompt renderer on a personal workstation, which is the overwhelmingly common deployment and the one every design choice was made for. It is worth re-examining in two situations. One is a shared multi-user host, where the socket's access properties determine whether “any peer that can reach the socket” is a larger set than you intended. The other is a container or CI context where a config tree might come from somewhere you did not write, in which case the `exec` cascade is the thing to look at first. Neither situation is unsupported; both are situations where the assumptions above stop being invisible and start being decisions. Naming them is what a trust boundary section is for.

Chapter 15 — Authoring a Line: Latency, Priority, and Density

Every preceding chapter has been about the machinery. This one is about the thing the machinery exists to draw, and it is addressed to the person who actually decides what a prompt says. A powerline theme is a design artifact with hard constraints — a fixed cell budget, a per-keystroke latency budget, and a reader whose attention it is spending — and the constraints interact in ways that are not obvious from the config schema. This chapter is the design discipline: how to think about what a segment costs, what it earns, and how to build a line that stays instant and stays legible when the terminal is narrow and the context is noisy.

Nothing here is a new capability. Everything is an application of mechanisms the book has already established — the `None`-return convention, `priority`, the gradient path, the `segment_info` contract, the divider math — used deliberately rather than incidentally.

The three latency classes

Segments do not have a latency; they have a latency *class*, and the class is determined by what the segment must touch to produce its answer. Sorting the library this way is the single most useful thing a theme author can internalize, because placement decisions follow from it almost mechanically.

Class one: already in hand. The segment’s answer is present in `segment_info` or in the process, and producing it is a map lookup and a format. `last_status`, `last_pipe_status`, `jobnum`, `environment`, `user`, `hostname`, `date`, and `fuzzy_time` are all here. They cost a handful of instructions and they are free in any position, at any refresh rate, in any quantity. A line made entirely of class-one segments has a render cost indistinguishable from an empty line.

Class two: a bounded local probe. The segment must ask the kernel or the filesystem something, and the answer comes back in a bounded number of syscalls. `cwd` is here; `internal_ip` is here; `branch` is here, because deciding whether you are in a work tree is a `stat` walk up the directory tree; the `cache-stat` segments are here, because they `stat` a file. These are cheap and are fine on a prompt, with one caveat that Appendix AL states and that bears repeating in the design context: a class-two segment costs its probe *whether or not it renders anything*, because the probe is how it decides. Ten conditional class-two segments on a prompt are ten probes per keystroke path, whether or not any of them appear.

Class three: a subprocess, a device, or the network. The segment forks something, reads a slow device, or talks to a remote. `git_status` forks git; the load and GPU and thermal segments run OS probes; `external_ip`, `weather`, and `email_imap_alert` go to the network. Costs here range from a fork’s worth of work to an unbounded network wait. These are not “slow segments to use sparingly” — they are segments whose position determines whether your prompt is interactive, and on a shared serialized daemon (Chapter 12) whether *other* lines are interactive too.

The classes are not a quality ranking. A class-three segment can be the most valuable thing on your statusline; `git_status` folding a whole repository state into one chunk is worth a fork. The point is that the class tells you where the segment belongs, and a theme that ignores the classes will be a theme that is mysteriously laggy in a way no amount of tuning the renderer can fix.

Budgeting the keystroke path

The prompt is the latency-critical surface and it deserves an explicit budget rather than an implicit one. The number to design against comes from Chapter 1: humans notice input latency above roughly the tens-of-milliseconds mark, and the whole point of the port is to stay far under it. The warm render itself is not the risk — Chapter 5 accounted it at tens of microseconds for cheap segments — so the entire budget is available to be spent, or squandered, on segment cost.

A workable discipline is to treat the left prompt as class-one and class-two only, with at most one class-three segment and only if it is a fork you have measured and accepted. `git_status` is the canonical example of a defensible class-three prompt segment: it is one fork, it is bounded, and the information density it buys (Chapter 3's badge chunk, replacing several separate segments) is enormous. `external_ip` is the canonical indefensible one: unbounded, remote, and its value does not change between keystrokes.

The right prompt deserves its own budget line because zsh redraws it on interactive events, which makes it a second per-keystroke surface that authors habitually treat as a spillover area for whatever did not fit on the left. Anything you put there is on the same critical path as the left prompt. If the right prompt is where your load average and your Kubernetes context live, you have moved class-three work onto the keystroke path while thinking you were tidying.

The correct home for class three is the tmux statusline, and the reason is refresh economics rather than taste. A statusline refreshes on a timer, so its latency is compared against the interval rather than against a keystroke, and a render that takes a few milliseconds inside a multi-second tick is invisible. The one caveat is the multiplication of Chapter 12: the statusline renders per window, so a class-three segment on the window-status format is that cost times the window count, per tick, through the serialized loop. Class three belongs on `status-left` or `status-right`, which render once per tick, and not on the per-window format.

Priority as an editorial decision

`priority` is documented as a truncation mechanism, and it is, but describing it that way undersells what a theme author is actually doing when they set one. Assigning priorities is deciding, in advance, what your line means at every width — because the terminal will be narrow sometimes, and when it is, the priorities are the only thing that determines what survives.

The useful reframe is to ask, for each segment, “at what width does this stop being worth its cells?” A `cwd` segment is worth its cells at almost any width, because knowing where you are is the prompt's core job. A `date` segment is worth its cells on a wide terminal and is close to worthless on a narrow one, because a clock is available elsewhere and the cells are needed for context. The mode indicator is worth its cells at every width, which is why the book notes it is never truncated: a prompt that hid its mode indicator on a narrow pane would be actively harmful, since the user would be typing into an editor whose state they could no longer see.

Two failure patterns are common enough to name. The first is uniform priority — every segment at the same level, or none set at all — which converts truncation from an editorial decision into whatever the algorithm's default ordering happens to do. You will get a line at narrow widths and it will not be the line you would have chosen. The second is inverted priority, where the segments the author found most interesting to build get the highest priority regardless of their information value; the effect is a narrow prompt that faithfully reports a GPU temperature while having dropped the current directory.

A workable heuristic: give the highest priority to segments whose absence would make you *act wrongly* — the mode indicator, the directory, the root/user marker, the Kubernetes context if you have production access. Middle priority to segments that report state you would want to notice but could survive missing — branch state, exit status, job count. Lowest priority to ambient metrics — clock, load, temperature — that are pleasant on a wide window and are not why you looked at the prompt. That ordering roughly tracks “would I have been better off not running that command,” which is the right question for an interface you glance at rather than read.

The left/right split is a latency split as well as a layout one

Themes conventionally treat the left/right division as spatial — important things on the left, ambient things on the right — and on a tmux statusline that is the whole story. In a shell it is also a cost decision, because the two sides are separate invocations.

`powerline shell left` and `powerline shell right` are two client runs, two wire requests, and two renders. They share a warm orchestrator (side is not part of the `PowerlineKey`, as Chapter 5 established) so they do not double the setup, but they do double the round-trips and they each pay their own segments' costs. This is the concrete reason a right prompt is not free real estate: adding a segment to the right side adds its cost to a second render that happens on the same interactive events as the first.

There is a genuine design use for the split beyond layout, and it is worth naming because it is the sort of thing that only becomes obvious after you have tuned a slow prompt. If your shell binding renders the right prompt on fewer events than the left — or if you can arrange for it to — then the right side becomes a legitimate home for a middling-cost segment whose value does not change between keystrokes. Absent that, treat both sides as one budget: everything either side draws is on the interactive path, and the terminal does not care which half of the line made it slow.

Designing for absence

Appendix AL makes the case that `None`-returning segments are how a static configuration describes a prompt whose shape changes continuously. The design corollary is that the *shape* of your line — its width, its color rhythm, its divider pattern — is not a fixed thing you can lay out once. It is a family of shapes, and you are designing the family.

The practical consequence is that a theme should be evaluated in its states, not in its best state. Look at the prompt in a git repository and outside one; with a nonzero exit code and after a success; with a virtualenv active and without; with background jobs and with none. Each of those toggles one or more segments, and each toggle changes the divider pattern, because dividers are computed from adjacent backgrounds (Chapter 7). A pair of segments that look handsome adjacent may look wrong when the segment that normally sits between them disappears and their backgrounds turn out to match, collapsing a hard divider into a soft one and merging two blocks into an undifferentiated bar.

The mitigation is to design the color sequence so that *any* subset of the optional segments still alternates. If you have three optional segments in a row, giving them three different backgrounds guarantees that whichever survive still produce hard dividers between them and against their fixed neighbors. Giving them all the same background produces a line that looks segmented when they are all present and looks like one long block when only two are.

Density is the other half of designing for absence. Because absent segments cost nothing to have configured (class one and, with the probe caveat, class two), a theme can afford to name many more segments than it typically shows, and the resulting prompt is quieter than a naive reading of the config suggests. This is the correct way to use the library's breadth: configure the segments that matter *when they apply*, let them stay silent the rest of the time, and let the prompt's visible complexity track the situation's actual complexity. A prompt that is three chunks wide in a plain directory and nine chunks wide mid-rebase in a production Kubernetes context is doing exactly what it should.

Color as information

The colorscheme machinery of Chapter 6 supports two distinct uses of color and the difference is worth being deliberate about. The first is identification: a segment has a stable color so you can find it without reading

it — the directory is always blue, the branch is always purple, and your eye goes to the right part of the line without parsing. The second is state signalling: a segment's color changes to tell you something — the `_dirty` variants, the root user marker, the nonzero exit code.

These two uses compete for the same limited perceptual budget, and a theme that uses many colors for identification leaves no headroom for signalling. If every segment is a different bright color, the one segment that turned red because something is wrong does not stand out. The disciplined approach is a muted, closely-related palette for identification and a small number of reserved, high-contrast colors that *only* ever mean state. Then a red block means exactly one thing, and it means it loudly.

Gradients are the third use and they are a specialized one. A gradient segment (`cpu_load_percent` and its family) encodes a continuous quantity as color, which is genuinely more legible at a glance than a number: you learn the color of “fine” and stop reading the digits entirely. That is a real win, and it is why the gradient machinery exists. It works best when exactly one or two segments on a line use it, because a gradient's value is that its color *changes* and a line full of changing colors reads as noise. A single gradient segment on an otherwise-stable line is a gauge; four of them are decoration.

A line, designed

Putting the discipline together on a concrete case. Suppose the goal is a shell prompt for someone who works in git repositories, sometimes against a production Kubernetes cluster, on a machine they reach over ssh from elsewhere.

The left prompt gets, in order: the mode indicator (class one, highest priority, never truncated); `user` and `hostname` (class one, high priority because the whole point is that this is not the local machine, and the `user` segment's root highlighting is a state signal you want at any width); `cwd` (class two, highest information value, high priority); and `git_status` (class three, one fork, accepted because its density is worth it, middle priority so a narrow pane drops it). The `kubecontext` segment (class two — it reads config files) sits next to `cwd` with high priority and `hide_default` on, so it is silent in the boring case and speaks up when you have switched to something that could bite; the argument for high priority is precisely the heuristic above, since its absence is the kind that makes you act wrongly.

The right prompt gets `last_status` and `jobnum` — both class one, both silent in the common case, both things you want to notice immediately when they appear. Nothing else: the right prompt is on the keystroke path and there is no ambient metric worth putting there.

Everything ambient goes to the tmux statusline: the clock, the load gradient, memory, the battery. These are class one and class three, refreshed on a timer, rendered once per tick on the `status-left/status-right` formats rather than per window. The per-window format stays cheap.

The colors: a muted family — grays and desaturated blues — for the identification segments, with exactly two reserved signals, a red for nonzero exit and root, and the gradient for load on the statusline. The optional segments (`git_status`, `kubecontext`, `jobnum`) get distinct backgrounds from their neighbors so the line stays segmented in every subset. Priorities descend from mode and directory through context and repository state down to the ambient statusline metrics, so a narrow pane sheds ornament before it sheds meaning.

That is not the only good line, and it is not a template to copy. It is what falls out of applying the constraints deliberately: latency class determines the surface, information value determines the priority, the absence family determines the color sequence, and the reserved-signal discipline determines the palette. Every one of those constraints comes from a mechanism described earlier in this book; the design work is choosing to let them decide.

Chapter 16 – Writing a Native Segment

The `exec` escape hatches of Chapter 3 cover the case where a user needs a segment the library does not have, and they cover it well: two patterns, a forgiving output protocol, and no build step. But they are subprocess-based by construction, and a subprocess on the render path is a cost the rest of the architecture spent considerable effort eliminating. For a segment that is going to be permanent — one you will render many thousands of times a day for years — the right home is a native adapter in `src/extensions/`, and this chapter is the contract and the discipline for writing one.

This is deliberately positioned as the last technical chapter before the status accounting, because writing a native segment is where all the preceding machinery has to be held simultaneously: the adapter signature from Chapter 3, the `segment_info` contract from Chapter 4, the highlight-group and gradient model from Chapter 6, the latency classes from Chapter 15, and the ported/extensions provenance boundary that Chapter 18 draws.

When an `exec` segment should become a native one

The decision is not primarily about performance, though performance is the visible part. Three signals each independently argue for promotion.

The first is frequency. An `exec` segment on a tmux statusline that refreshes every few seconds is paying a fork per render forever, and on the per-window format it is paying a fork per window per tick. A fork-and-exec is orders of magnitude more expensive than a native probe of the same data. The arithmetic favors promotion long before it becomes perceptible, and it becomes perceptible sooner than authors expect because of the multiplication.

The second is the failure surface. A script segment depends on the script existing, being executable, having a working interpreter, and producing parseable output. Each is a thing that can break independently of powerliners and each breaks at render time, in a prompt, where diagnosis is awkward. A native adapter's failure modes are compile-time or are the ordinary `Option<Value>-returns-None` path, which degrades to a segment that is simply not there.

The third is the highlight-group wrinkle. Chapter 3 documents the divergence honestly: a `"type": "function"` segment uses its resolved `function_name` as its sole highlight group. Pattern A's practical consequence is that your colorscheme group must be named `exec`, which is workable but collides if you have more than one Pattern A segment wanting different colors. A native adapter registered under its own name gets its own group name for free, which removes the whole class of workaround.

Against those, the cost of promotion is that you are now maintaining Rust in a fork of the crate rather than a script in your config directory, which is a real cost and the reason the escape hatches exist at all. The honest rule is: prototype in a script, promote when the prototype has proven it is permanent.

The adapter contract, restated as an obligation

Chapter 3 gives the signature as it appears in the man page:

```
fn(args: &Map<String, Value>, info: &Map<String, Value>) -> Option<Value>
```

Read as a type it is unremarkable. Read as a contract it carries four obligations, each of which has bitten someone.

Read context from `info`, never from the process. This is the `segment_info` contract of Appendix AJ and it is the obligation with the sharpest failure mode. Your adapter runs inside a long-lived daemon whose own environment is a fossil of whatever login started it and whose own working directory is `/` after `daemonize` chdir'd there. Reading the process environment or the process working directory in an adapter does not produce an error; it produces a plausible wrong answer, identically wrong for every caller, forever. Everything a segment needs about the caller arrived on the wire and is in `info`.

Return `None` rather than a placeholder. The graceful-absence convention is load-bearing across the library (Appendix AL) and a new adapter that returns a dash or an empty chunk instead of `None` is not merely stylistically off — it is occupying cells and a divider boundary in every render where its data does not exist, which changes the line's shape and its truncation behavior. The correct behavior for “I have nothing to say” is to say nothing.

Do not panic. An adapter runs inside the daemon's single-threaded loop, and the loop is the process. Every unchecked unwrap on a parse of external output, every index into a vector whose length came from a subprocess, and every assumption that a file has the format you expect is a way to convert a malformed input into a dead daemon. Adapters parse defensively and return `None` on anything unexpected, which is both the safe behavior and the correct one: an unparseable probe result means you do not know the value, and not knowing the value means the segment has nothing to say.

Be fast, or be honest about not being. Chapter 15's latency classes are a taxonomy of existing segments, but they are also a specification for new ones. Know which class you are writing, and if you are writing class three, document it, because a theme author placing your segment needs to know whether it belongs on a prompt.

Returning chunks

The return is `Option<Value>`, and the `Value` is the same list-of-dicts shape the whole library produces — the shape Chapter 2 traced for `cpu_load_percent`, a list whose entries carry `contents`, `highlight_groups` in priority order, and optionally `gradient_level`.

The list shape is the part new authors under-use. Returning several entries lets one adapter produce several independently-colored chunks, which is what `git_status` does when it emits its branch name and its badges as a single logical segment with internal structure. The alternative — several separate segments in the theme, each forking their own probe — is what the ported `branch` plus `stash` plus status decomposition does, and Chapter 3 draws the contrast: upstream's decomposition is faithful to upstream, and the single-fork multi-chunk approach is denser and cheaper. If your segment naturally produces several related values from one probe, produce them as several chunks from one adapter rather than several adapters that each probe.

The `highlight_groups` list is the fallback mechanism of Chapter 6 and it should always have more than one entry when the segment has states. The pattern the library uses is state-first, generic-second: a request for the most specific group that describes the current state, then progressively more generic ones, so a colorscheme that never heard of your segment still paints it something sensible by falling through to a group it does know. `git_status`'s `["git_status_dirty", "git_status", "branch_dirty", "branch"]` is the model. A new adapter that offers exactly one group is a new adapter that errors on every colorscheme except the ones written for it.

`gradient_level` should be supplied whenever the segment's value is a bounded continuous quantity, because it costs nothing and it enables the gauge behavior Chapter 15 described. The convention is a 0-to-100 scale, which `pick_gradient_value` maps onto whatever length the colorscheme's gradient list happens to be.

Arguments, and not trusting them

The `args` map comes from theme JSON, which means it comes from a human editing a file, which means every access to it is a place where a missing key, a wrong type, or a nonsense value can arrive. The library's convention is to treat `args` as advisory: read what is there, apply a documented default when it is not, and never fail the render because an argument was malformed.

Two patterns recur and both are worth copying. The first is the `format` argument, which lets a theme control presentation without a new adapter — a format template applied to the segment's value, with the adapter supplying a sensible default template so an argument-free `{"function": "myseg"}` still renders something. The second is the boolean toggle with a quiet default, of which `hide_default` on `kubecontext` is the model: the argument controls whether the segment suppresses itself in the uninteresting case, and the default is chosen so the common configuration is the quiet one.

The thing to avoid is an argument that changes what the segment *is* rather than how it presents. Adapters that switch between fundamentally different data sources based on an argument are two adapters wearing one name, and they defeat the registry's whole purpose: a theme names a segment and gets a known thing, and a colorscheme paints a group whose meaning is stable. Two behaviors want two registry entries and two group names.

Probing the OS

Most interesting extension segments have to ask the operating system something, and the library's existing extensions establish the order of preference.

Read a file if you can. `/proc` and `/sys` on Linux are file reads, which are cheap, bounded, and free of the fork cost entirely. The cache-stat segments for the sibling stack tools are the purest case — they stat a path and read its size, which is a syscall or two, and they are correspondingly placed as class-two segments that are safe anywhere.

Call a syscall if there is one. Anything the kernel will tell you directly is better than anything you have to ask a program to ask the kernel for you.

Fork a probe only when there is no alternative, and know what it costs. The macOS side of the system segments is the honest example: without `/proc`, the port samples via `top`, `vm_stat`, `netstat`, `pmset`, and `uptime`, which is a fork per probe. Chapter 3 presents this as a divergence from Python's `psutil` dependency and a favorable one — it renders on a machine with no `psutil` — and it is also, straightforwardly, the expensive path. A new adapter that forks should do so once per render and parse everything it needs from that one invocation, never fork twice for two related values.

Never make a network call in a segment that a reasonable person might put on a prompt, and if your segment must make one, say so in its documentation in the same breath as its name. The concurrency consequences of Chapter 12 make this a system-wide courtesy rather than a per-line one.

The vendor-dispatch fallback chain

The GPU segments establish a pattern worth generalizing, because “the same information lives behind different tools on different machines” is a recurring shape. `gpu_usage_percent` and `gpu_vram` dispatch across vendors in a chain — `nvidia-smi`, then `rocm-smi`, then `intel_gpu_top`, then an `ioreg` fallback on macOS — so one segment name works regardless of whose hardware is installed. The `thermal` segment does the same across `/sys/class/hwmon` on Linux and `powermetrics/istats` on macOS.

The design rule the pattern encodes is that the *segment* is the abstraction, not the tool. A theme says “show me GPU utilization” and the adapter's job is to find some way to answer on this machine or return `None`. The alternative — separate per-vendor segments — pushes a machine detail into the theme, which breaks the

property that makes powerline configs portable: the same theme should render on the user's laptop and their server.

Two implementation notes follow from the pattern. Order the chain by cost and likelihood, cheapest and most likely first, since every failed probe is wasted work. And fail the whole chain to `None`, not to an error: a machine with no GPU at all is a completely normal machine, and the correct rendering for “this machine has no GPU” is no segment.

Testability: the pure-functional seam

The extensions tree establishes a testing pattern that the book has already named in passing and that deserves stating as a rule, because it is the difference between an adapter that can be tested and one that can only be tried.

The pattern is to split every adapter into a pure core and a thin impure shell. The core takes its inputs as parameters — including the things that would normally be ambient, like environment lookups and filesystem existence checks, passed as function arguments — and returns its result with no side effects. The shell supplies the real implementations of those parameters. The cache-stat segments' `default_path_with(get_env, path_exists)` seam is exactly this: the resolution logic that decides which of four candidate paths to use is a pure function of an environment-getter and a path-existence predicate, so a test can pin every precedence level by passing closures, without setting environment variables, creating files, or mutating anything a parallel test could observe.

The value is not merely that tests become possible; it is that they become *exhaustive*. A four-level resolution chain has four interesting cases plus their combinations, and with the seam every one of them is a short test with no setup and no teardown. Without the seam, testing the third precedence level means arranging a filesystem state in which the first two candidates are absent and the third is present, which is enough friction that the third level goes untested and stays subtly wrong until someone's machine happens to hit it.

The same split applies to parsing. A segment that forks `git` should have a pure `parse_porcelain_v2`-style function taking the output text and returning the structured result, tested against captured fixtures of real output including the awkward cases — detached HEAD, in-progress merge, unusual branch names — while the forking lives in a thin wrapper that is exercised once. Every interesting bug in a probe segment is a parsing bug, and parsing bugs are only cheap to find when the parser is callable without the probe.

Registering, and the provenance rule

The final step is a `(name, adapter)` entry in the `ADAPTERS` registry in `src/bin/shared/render_runtime.rs`, which is shared between the daemon and the direct renderer so a new segment works on both paths automatically. The three-tier lookup of Chapter 3 means the name you register determines both the fully-qualified form and the bare-name fallback, so a name registered as a dotted path is reachable both ways and its trailing component becomes its natural highlight group name.

The rule that outranks all of the above is the provenance boundary. New segments go in `src/extensions/`, never in `src/ported/`. The ported tree is graded against the upstream Python by citation density, the `/// Port of <py_fn>()` doccomment, the tier classifier, and the drift gate against `docs/powerline_py_functions.txt`; a new function in that tree with no upstream counterpart either fails the gate or, worse, dilutes the meaning of the tier numbers that Chapter 18 reports. The extensions tree exists precisely so that new work has a home where it is held to the same test discipline without being measured against a Python original that does not exist.

This is the same discipline the whole project rests on, applied to the one activity most likely to erode it. A port's fidelity claim is only as good as the boundary around what is ported, and the boundary is only as good as the habit of respecting it while adding the thing you actually wanted to add. Put the new segment in `extensions`,

register it in the shared registry, give it a pure core and a tested parser, and the fidelity numbers keep meaning what they say.

Chapter 17 — Debugging a Wrong Prompt

Prompts fail in a characteristic way: something looks wrong, and the pipeline that produced it has five stages in three processes, any of which could be responsible. The natural response is to guess, change something, and look again — which works often enough to be a habit and wastes enough time to be worth replacing. This chapter is the replacement: a bisection discipline that identifies the responsible stage in a small number of steps, using tools the suite already ships, and a short catalogue of the symptoms whose causes are nearly always the same.

Bisect the pipeline, do not guess

The pipeline is: a binding invokes the client with arguments; the client marshals `argv + cwd + env` and writes it to a socket; the daemon demarshals, resolves configuration, runs adapters, resolves colors, lays out, and encodes; the client copies the reply to stdout; and the host — `tmux`, the shell's line editor, `vim` — interprets those bytes. Five stages, and the crucial observation is that each boundary between them is directly observable. You do not have to reason about which stage is wrong; you can look.

The bisection has a natural order because the stages get progressively harder to observe. Start at the outside and work in: first establish which *process* is producing the output you dislike, then which *stage* within it, then which *input* to that stage. Each step below is one command and one observation.

Step one: is the daemon in the picture at all?

The first question is not “why is the output wrong” but “who produced it,” and the answer is frequently not what the user assumes — particularly after a migration, where the classic outcome is a correct swap that did not take effect because a stale daemon is still bound to the socket or because `$PATH` resolves to the old binary.

Two checks settle it. Ask the shell what `powerline` and `powerline-daemon` actually resolve to, because a symlink placed in the wrong directory or a shell function shadowing the name will produce a setup that looks migrated and is not. Then check what is holding the socket: on the filesystem-socket platforms the pidfile at `/tmp/powerline-ipc-<uid>.pid` names the process, and inspecting that process tells you whether the running daemon is the Rust binary or the Python script — the exact check the migration runbook prescribes for confirming a swap. The book puts that check in the migration steps because it is the one that catches the silent no-op, and it is equally the first check when something is unexpectedly wrong afterwards.

If a daemon is running and the output is still wrong, the second sub-question is whether the client is even reaching it. A client whose connect fails execs `powerline-render` and produces a *correct* prompt more slowly, which is a genuinely confusing failure mode: nothing looks broken except the latency. If your prompt is right but sluggish after a migration, you are almost certainly on the fallback path, and the cause is a socket that is not where the client is looking — a `--socket` argument in one place and not the other, or a daemon started under a different UID than the shell that is rendering.

Step two: is it the config, or is it the render?

With the responsible process established, the next boundary to observe is between configuration and rendering, and the suite ships two tools that sit exactly on it.

`powerline-lint` answers “is my configuration well-formed and internally consistent,” and it answers it statically, with source positions, before a render is involved. Chapter 9 describes what it checks: unknown segment names, missing color references, malformed `before/after` formatting strings. Its value in debugging is that it converts a class of render-time mystery into an authoring error with a line number. A segment that silently does not appear and a colorscheme lookup that fails at render time both usually show up in lint output as something concrete.

`powerline-render` answers “what does this configuration render to, right now, with no daemon involved,” and it is the more powerful of the two for isolation. Because it is the direct, daemon-less renderer running the same code the daemon runs, comparing its output against what the daemon produced splits the problem cleanly. If `powerline-render` produces the same wrong output, the daemon is exonerated and the problem is in the configuration or the segments — go back to lint and to the config cascade. If `powerline-render` produces *correct* output while the daemon produces wrong output, the problem is daemon state, and the likely cause is staleness: a warm orchestrator built from configuration that has since changed in a way the mtime check did not catch, or an orchestrator keyed to override values that differ from the ones you think are in play. Restarting the daemon with `--replace` resolves the state question definitively, and if the wrongness survives a restart it was never state.

This pair — lint for authoring errors, direct render for isolation — covers most configuration problems between them, and neither requires reading any Rust.

Step three: is it the resolution, or the encoding?

If the segment is present but looks wrong, the remaining question is whether the *values* are wrong or the *bytes* are wrong, and they are distinguishable because the wrongness has a different texture.

Wrong values look like wrong information: the branch name is stale, the exit code is from the wrong command, the directory is not the one you are in, an environment segment shows nothing. These are almost always `segment_info` problems, which means they are almost always binding problems, and the place to look is what the binding forwards. The shell-status family is the usual suspect because it is the family that *depends* entirely on forwarded state: `last_status` is wrong when the binding captured the exit code after something else already clobbered it, which is an ordering bug in the `precmd` or `PROMPT_COMMAND` function rather than anything powerliners did. Appendix AJ’s contract is the reference — the daemon knows only what the wire carried, so a wrong shell-local value means the wire carried a wrong value, and the binding is what put it there.

Wrong bytes look like wrong *rendering*: colors bleeding across segments, a line that is one cell off, literal escape text visible in the prompt, a chevron in the wrong direction. These are encoding problems and the diagnostic is to look at the actual bytes rather than the rendered result. Capturing the client’s output and viewing it with control characters made visible turns “the colors are weird” into a concrete question about which directives were emitted. A directive that names the colors you expected means the encoding is right and the terminal is not; a directive that names colors you did not expect sends you back to the colorscheme.

The one thing to be careful about here connects to Chapter 14: a wire request dump contains your entire environment. Inspect it locally; do not paste it into a bug report.

Step four: is it the host?

The last stage is the one powerliners does not control, and a surprising fraction of “wrong prompt” reports live here. `tmux`, the shell’s line editor, and `vim` each interpret the bytes they are given according to their own rules and their own configured state, and each has failure modes that look like renderer bugs.

The terminal's color support is the first candidate. A colorscheme that resolves to truecolor values on a terminal that does not do truecolor produces colors that are wrong in a specific way — approximated or ignored — rather than absent, which reads as a bug in the palette. The two-value color model of Chapter 6 exists exactly so one colorscheme serves both cases, and the branch is taken on the `term_truecolor` flag, so a colorscheme that looks right in one terminal and wrong in another is a strong hint the flag does not match the terminal's actual capability.

Font coverage is the second. The dividers are glyphs from a patched font, and a terminal without one renders them as replacement boxes or as the wrong glyph entirely, which looks like broken divider math and is not. The ASCII divider fallback exists for this case and the fix is a font, not a config change.

The third is width, and it is the place where a real port divergence and a host behavior can be confused. If a line is consistently one cell off and the line contains wide CJK glyphs or emoji, that is the documented `strwidth` gap of Chapter 7 — the port counts Unicode scalar values, and a wide glyph occupies two cells while counting as one. If the line is one cell off and contains no wide glyphs, it is much more likely the host: a shell prompt that mis-wraps usually means the non-printing escape markers were not what that line editor expected, which is a binding-and-renderer mismatch rather than a math error.

Watching the daemon work

For the problems that survive all of the above, the daemon can be run in the foreground. `--foreground` skips the double-fork of Chapter 4 entirely, leaving the process attached to the terminal — which is what a service manager wants for a simple-type unit and what an interactive debugging session wants for exactly the same reason: the process stays where you can see it, and it stays in your job control so you can stop it without hunting a pid.

The complementary technique is the probe socket. `--socket PATH` targets a non-default socket, and running a foreground daemon on a throwaway path lets you experiment against a daemon that is not the one your live shells depend on. This is the same mechanism the migration runbook uses to verify parity before committing to a swap, and it is the right shape for any experiment: your real prompts keep working against the real daemon while you interrogate a disposable one. Combining the two — a foreground daemon on a probe socket, with a client pointed at that socket — gives a fully isolated instance of the whole pipeline that you can start, break, and kill without consequence.

The symptoms whose causes are nearly always the same

A short catalogue, because pattern recognition beats bisection when the pattern is strong enough.

Everything after one segment is the wrong color or is bold. Attribute state leaked. tmux markup is stateful and the renderer emits the explicit `no*` forms precisely to prevent this (Chapter 7), so seeing it usually means something downstream of the renderer is producing markup — a hand-written tmux format string, or an `exec` segment emitting its own escapes — that did not clean up after itself.

A custom segment renders in the wrong color. The `gen_segment_getter` highlight-group divergence: a function-type segment uses its resolved function name as its group. Name the colorscheme group to match, which for Pattern B is the dotted path's trailing component and for Pattern A is `exec`.

A live number differs from what Python showed. Expected, documented, and not a bug: the port samples through OS probes on demand and Python samples through `psutil` on a different cadence, so the framing is identical and the number can differ by a sampling window. Appendix AP's ledger names the whole affected set.

A prompt stalls intermittently. A class-three segment on the interactive path, plus the serialized daemon of Chapter 12. The stall is the segment's own I/O, and if the timing correlates with the tmux tick rather than with your keystrokes, it is queueing behind the statusline burst. Move the expensive segment.

A segment vanished and nothing is wrong. It returned `None`, which is the design (Appendix AL). Before debugging why a segment is missing, verify that its condition is actually true — no branch outside a work tree, no kubecontext without an active context, no jobs when there are none.

The migration “did not do anything.” The daemon check of step one. A correct symlink with a stale daemon still bound to the socket produces exactly the old behavior, and restarting the client’s host without stopping the old daemon does not fix it.

The list is short because the pipeline is short and each stage has few failure modes. That is a property worth noticing on its own: a five-stage pipeline in which every boundary is directly observable and every stage has a tool that exercises it in isolation is a system that can be debugged by looking rather than by guessing, and the tools that make that true — the linter, the direct renderer, the foreground flag, the probe socket — were all inherited from a design that took the same view. The port kept them for the same reason it kept everything else: they were right.

Chapter 18 – Status and the Honest Gaps

Every book in this series ends by walking its own boundaries. This chapter states where powerliners stands — 134/137 upstream files at DONE, 2497 lib tests (Chapter 34) — and is candid about the three remaining files. Chapter 29 opens all three and reports what each one actually contains.

134 / 137 files at DONE (97.8%)

The headline status is 134 of 137 upstream Python files at the DONE tier — 97.8% — with 2497 library tests passing (0 failing, 0 ignored), 462 parity tests against the live interpreter, and 11 port bugs surfaced and fixed. These numbers are not asserted; they are outputs of the tier classifier and the test runner over the actual source, regenerable on demand (Chapter 10). “At DONE” means every function in the file clears the citation-density-plus-doccomment bar, so 134/137 is a count of files where the port is both declared and demonstrated to be faithful, function by function.

Two measured qualifications belong immediately next to that ratio, and this section is where they are stated in full.

What DONE certifies. The tier is computed from two text properties of the Rust file — the count of `// py:` citations divided by upstream LOC, and the count of `/// Port of` doccomments divided by upstream function count (`scripts/gen_port_checklist.py:112-122`). Neither input observes execution. A file that transcribes every upstream statement into a `// py:` comment and returns a default value scores at the top of the tier. `renderers/shell/bash.py` does exactly that: DONE, 76 upstream LOC, 6 `/// Port of` doccomments, 80 `// py:` citations — a density above 1.0 — and `BashPromptRenderer::do_render` returns `String::new()` on both of its paths (`src/ported/renderers/shell/bash.rs:123, :135`) beneath its own “Status: stub” doccomment at `:56`. The file directly below it in the checklist, `renderers/shell/zsh.py`, is tagged NEAR and its port is complete: `escape_hl_start = "%{", escape_hl_end = "%}"`, the `'%' → "%%"` translation, and the `renderer` alias, all present and all live (`src/ported/renderers/shell/zsh.rs:23, :28, :40, :48`), and depended on by `tcsh.rs:24`. The tier ordering does not track completeness here, because completeness is not what the tier measures. Behaviour is certified by the parity harness, the daemon scenarios, and the library tests — three independent instruments — and a reader who takes “134/137 DONE” to mean “134 files behave like upstream” has read a documentation metric as a behavioural one.

Four of the 137 rows never compile. The classifier finds its Rust side by mirroring the upstream path (`gen_port_checklist.py:127-136`), which is a filesystem walk; it never checks whether the resulting file is in the crate’s module tree. `src/ported/bindings/mod.rs` declares eight children — `config`, `ipython`, `pdb`, `qtile`, `tmux`, `vim`, `wm`, `zsh` — and four directories beside them are declared nowhere. Every path in the File column below is relative to `src/ported/bindings/`:

File	LOC	/// Port of	// py:	#[test]	Checklist tier
<code>i3/powerline-i3.rs</code>	126	4	24	3	NEAR
<code>bar/powerline-bar.rs</code>	132	3	25	3	DONE

File	LOC	/// Port of	// py:	#[test]	Checklist tier
<code>lemonbar/powerline-lemonbar.rs</code>	209	3	21	4	DONE
<code>awesome/powerline-awesome.rs</code>	54	1	7	1	DONE

Each filename contains a hyphen, which is not a legal Rust module name, so none of them can be reached by a `mod` declaration at all; the only routes left are a `#[path]` attribute or a `[[bin]]` target, and neither exists — `Cargo.toml:35-54` declares five binaries, all of them under `src/bin/`, and all seven `#[path]` attributes in the tree point at `src/bin/shared/`. The consequence is checkable rather than argued: the four files hold 11 `#[test]` functions between them, and none of the 11 appears in the library’s test list.

```
$ cargo test --lib -- --list | tail -1
2497 tests, 0 benchmarks
$ cargo test --lib -- --list | grep -c
  ~ 'i3powerline\|build_bar_command\|main_runs_without_panic'
0
```

Three of the four are counted as DONE, so they are inside the 134. The 490 lines, 11 doccomments, 77 citations, and 11 tests they contribute are read by the classifier and by the drift gate — both of which walk `src/ported/` from the filesystem — and by nothing else. A file that inflates a coverage metric while being unreachable is precisely the failure the metric exists to catch, which is why the fact is stated here, in the front matter, in Chapter 10, in Chapter 36, and in Appendix F rather than only in the chapter that discovered it.

The README states the shape of the remainder precisely: 3 lipstick-only files, 0 files in a “partial/sparse” degraded state, and a green drift gate. The zero degraded files is the number to notice — there is no tier of half-ported files with real behavior missing. Every file with a function body is at DONE; the only files not at DONE are the three below, and they contain no function bodies to port. So “97.8%” understates the behavioral coverage: the behavioral surface — every Python function with a body — is at 100%, and the missing 2.2% is a classifier artifact on files that have nothing to classify.

The test count is worth reading as a claim of its own. 2497 library tests exercise the ported functions in isolation — the merge semantics, the colorscheme resolution, the divider math, the wire parsing, the width fallbacks — and they pass with none ignored, which is the discipline that keeps the port from rotting between parity runs. The parity suite proves behavior against upstream; the library tests prove the port’s own invariants hold; together they are why the status numbers are stable rather than aspirational.

The 3 NEAR files and why 0/0 can’t promote to DONE

The three files not at DONE are `renderers/shell/readline.py` (14 LOC), `renderers/shell/zsh.py` (16 LOC), and `bindings/i3/powerline-i3.py` (52 LOC), and each is NEAR for a classifier reason rather than a port gap. `readline.py` is a class shell holding two string constants — `escape_hl_start = '\x01'`, `escape_hl_end = '\x02'` — and a module-level alias; `py_fn_total == 0`. `zsh.py` is the same shape with different escape constants. Both were discussed in Chapter 7: the escape markers they hold are fully present and correct in the Rust port, carried directly on the renderer struct without the Python subclass ceremony, so a shell prompt measures identically.

The classifier tags them NEAR because the DONE tier is a *ratio* — cited-and-doccommented functions over total functions — and these files have zero functions with bodies. A 0/0 ratio cannot be promoted to DONE no matter how complete the port is, because there is nothing in the denominator to be complete *about*. The arithmetic is worth seeing plainly: a normal file with, say, twelve functions all cited above the 0.5 density and all doccommented scores `12/12 = 1.0`, comfortably DONE; a file with eight of twelve cited scores `8/12 = 0.67`, still DONE if each cited function also clears its own density bar; but `readline.py` has `py_fn_total == 0`, so its score is `0/0`, which is not a number the classifier can compare to the `0.5` threshold at all. It is

not that the port fell short of the bar — there is no bar to clear, because the file’s entire behavior is two string constants (`escape_hl_start = '\x01'`, `escape_hl_end = '\x02'`) that the Rust holds directly on the renderer struct. The behavior these files carry is not in functions; it is in constants, and the port holds the constants. A classifier amendment could resolve the tag cosmetically, but the project chose to leave it honestly reported — 3 files not at DONE — rather than special-case the classifier to inflate the number, which is itself a small demonstration of the honesty the whole port is built on.

`powerline-i3.py` is the subtler case: it contains one `render` function which is already 100% ported (the port report confirms it), but the file’s structure trips the classifier’s class-only branch, so it is tagged NEAR despite the function being done. These exist in upstream because Python’s renderer registry needs a class per shell flavor to subclass `ShellRenderer` — a language-level requirement of the Python design, not a behavior. The Rust port has no such registry requirement, so it holds the same escape constants and the same `render` logic without the class ceremony. The two shell-renderer files are the places where Python needed structure that Rust does not, and the classifier — measuring Python-shaped structure — cannot score the absence of ceremony as completeness. Neither has any behavioral surface left. The i3 binding is the exception, and Chapter 29 states its status precisely: the `render` function is ported, the driver around it is not, and no `powerline-i3` binary ships.

Zero Python runtime: what “native binary” buys and costs

The defining constraint — zero Python runtime — is worth accounting honestly, because it buys most of powerliners’ value and costs a few specific things. What it buys is the whole performance and packaging story: no interpreter startup on the cold path, sub-millisecond renders on the warm path, a single static binary with no `pip` tree, and a deployment that needs nothing but the binaries and the config. Everything in Chapters 5 and 11 is downstream of there being no Python to boot, import, or install.

What it costs is dynamic extensibility and in-process embedding, and the port pays those costs openly. Upstream’s `__import__`-based custom segments — drop a `.py` in `segments/` and reference it by dotted path — cannot work without a Python runtime, so powerliners replaces them with the two subprocess-based `exec` escape hatches of Chapter 3: same expressive power, different mechanism, and one documented highlight-group wrinkle. Upstream’s in-process ipython prompt provider cannot embed in a Python REPL without being Python, so powerliners renders REPL prompts through the external client hook (Chapter 8) instead — same result, external mechanism. These are the two places where “native binary” is not a pure win, and the port names both rather than pretending the mechanisms are identical.

The trade is heavily favorable and deliberately chosen. Dynamic Python-module extensibility is a feature a small minority of users exercise, and the subprocess escape hatches cover essentially all of what it was used for; in-process REPL embedding is a niche whose result is reproduced externally. Against those two costs stands the elimination of the startup tax that motivated the entire project. powerliners bets that a shell user would rather have a sub-millisecond prompt and script-based custom segments than a hundred-millisecond prompt and Python-module custom segments, and the whole design follows from taking that bet seriously — including the honesty about what the bet gives up.

Where the port ends and net-new MenkeTechnologies segments begin

The cleanest way to see the boundary between ported and original work is the source tree: `src/ported/` is the faithful port of upstream, and `src/extensions/` is the net-new MenkeTechnologies work, explicitly carved out of the port’s fn-name rules by `docs/PORT.md`. The ported tree is graded by the DONE tier and the parity harness; the extensions tree is new code held to the same test discipline but not measured against a Python original, because there is no Python original to measure against. The registry (`ADAPTERS`) merges

both into one namespace, so a theme names an extension segment the same way it names a ported one, but the provenance is unambiguous in the layout.

The extensions are a substantial catalogue and they cluster into families. The system-metrics additions — `git_status`, `disk_usage/disk_io`, `gpu_usage_percent/gpu_vram`, `thermal`, `mem_usage`, `process_count` — are richer or more portable takes on what powerline's `psutil`-based segments did, sampled through OS probes so they work without Python's dependencies. The cloud-native additions — `kubecontext`, `containers`, `aws.context`, `gcp.context`, `ci_status` — bring orchestration context into the prompt, mostly through pure filesystem probes of the tools' own config files, with silent fall-through when the tool or context is absent. Each lives in its own `src/extensions/<module>.rs` and follows the same adapter contract, so adding another is the same pattern rather than a special case.

The most distinctive additions are the ones that tie powerliners into the rest of the MenkeTechnologies stack — the cache-stat segments for the sibling tools. `fusevm.jit_cache` reports the Cranelift JIT cache stats (entry count and bytes) for the shared fusevm runtime that powers `zshrs`, `stryke`, `awkrs`, `vimlrs` and `elisprs`; and `zshrs.rkyv_cache`, `stryke.rkyv_cache`, `awkrs.rkyv_cache`, `vimlrs.rkyv_cache` and `elisprs.rkyv_cache` each stat that tool's authoritative rkyv bytecode archive. All five language runtimes of the fleet are covered by the same two segment shapes — a `bin_version` TTL probe for `{version}`, and the four-level cache-path resolution for `{rkyv_cache}` — with `icons::vimlrs` and `icons::elisprs` completing the glyph set. They share a uniform four-level resolution chain and an identical `{size}/{bytes}/{logical_size}/{logical_bytes}` token surface, are pure filesystem probes (no subprocess, no daemon RPC), and are unit-tested through pure-functional `default_path_with(get_env, path_exists)` seams so every precedence level is pinned without mutating the environment. A user of the wider stack can put a segment in their prompt that reports the health of their zshrs or awkrs cache — a capability that only makes sense inside this ecosystem, and a clean example of where the faithful port ends and the original, stack-aware work begins.

Chapter 19 – Cell Width: What the Terminal Actually Counts

Chapter 7 introduced `strwidth` and named its one bounded divergence — the port counts Unicode scalar values, so an East-Asian wide glyph measures one cell instead of two — and Appendix AB.2 tells a user what to do about it. Both treat width as a small known gap in an otherwise settled quantity. This chapter argues that width is not a settled quantity at all: it is the least well-defined number in the entire render path, the only one whose correct value depends on software the renderer cannot see, and the one place where two faithful implementations can agree with each other completely and still both be wrong about what the user’s screen will do. Understanding why matters more than memorizing the gap, because the same reasoning tells you which glyphs are safe in a width-constrained line and which are a coin flip regardless of whose renderer you run.

Three different numbers, all called length

A segment’s contents arrive at the layout core as a string, and there are at least three defensible answers to “how long is it,” which differ from each other on almost every interesting input.

The first is the byte count — what a Rust `str::len()` returns, and what a naive C implementation would use. For anything outside ASCII this is simply the wrong question: the divider chevron encodes as three bytes in UTF-8 and occupies one column, a CJK ideograph encodes as three bytes and occupies two, and an emoji encodes as four and occupies two. Chapter 7 dismisses the byte count in one clause, correctly, and the book never returns to it. It is worth naming anyway because it is the failure mode that produces the most spectacular misalignment — a line that measures several times its true width and therefore truncates almost everything — and because it is what you get by default in any language whose string type is a byte sequence.

The second is the scalar count: the number of Unicode code points, which is what `s.chars().count()` returns in Rust and what `len()` returns on a Python 3 string. This is what the port uses. It is exactly right for ASCII, for Latin text with precomposed accents, and for the powerline glyphs themselves, and it is what makes the divider math of Appendix X come out even for every theme built from those character classes — which is to say, for nearly every theme anyone has.

The third is the display width: the number of terminal cells the string will occupy when the terminal emulator draws it. This is the number the truncation loop actually needs, because the constraint it is enforcing is a column budget on a grid of fixed-width cells. It is also the only one of the three that is not a property of the string. It is a property of the string *and* the terminal *and* the font, and no function that takes only a string can compute it in the general case. Every implementation of a width function in every statusline in existence is an approximation of an unknowable, and the interesting question is which approximation, and how the errors are distributed.

The property the standard supplies, and what it does not settle

The Unicode Character Database carries a property named `East_Asian_Width`, and it is the input every serious width function reaches for. Querying it directly makes the shape of the problem concrete; the values below are

what `unicodedata.east_asian_width` reports:

```
>>> import unicodedata as u
>>> u.east_asian_width('a')      # 'Na' - narrow
>>> u.east_asian_width(cjk)     # 'W' - wide
>>> u.east_asian_width(fw_A)    # 'F' - fullwidth
>>> u.east_asian_width(hw_kana) # 'H' - halfwidth
>>> u.east_asian_width(pm)      # 'A' - ambiguous
>>> u.east_asian_width(zwj)     # 'N' - neutral
```

Six values, of which four are unambiguous instructions. **Na** and **H** are one cell, **W** and **F** are two. A width function that dispatches on those four and defaults the rest to one handles CJK text, fullwidth punctuation, and the bulk of emoji correctly, and that dispatch is exactly what Chapter 7 says upstream's `strwidth_ucs_2` and `strwidth_ucs_4` perform and the port's scalar count does not.

The remaining two values are where the standard declines to answer. **N** — neutral — covers characters that simply do not occur in East Asian legacy encodings and so have no inherited width; treating them as one cell is conventional and almost always right. **A** — ambiguous — is the genuinely hard class, and it is much larger than its name suggests. It covers characters that were wide in the East Asian legacy encodings and narrow everywhere else, which means their correct width depends on the locale and the font the terminal is using, not on the character. A terminal configured for a CJK locale may draw them at two cells; the same terminal in a Western locale draws them at one; two terminals on the same machine can disagree. There is no answer a renderer can compute. There is only a default, and every implementation picks narrow because narrow is right in the overwhelming majority of deployments.

Why the powerline glyphs are safe

This is where the port's approximation stops being a compromise and starts being defensible, and the reason is a fact about the glyphs powerline is built from. The divider chevrons and the branch glyph live in the Private Use Area, and the database classifies that entire range as ambiguous. Querying the four divider code points and the branch code point returns **A** for every one of them, and their general category is **Co** — private use.

Private-use code points have no assigned meaning, so the database cannot say anything about them; **A** here is the database declining to commit, and **Co** is it saying the characters belong to whoever assigned them, which in this case is a patched font. Their real width is whatever the patched font draws, which for the powerline glyph set is one cell by construction — the chevrons are designed to fill exactly one column so that a divider costs one cell of budget.

The consequence is that a scalar count and an `East_Asian_Width` dispatch that defaults ambiguous to narrow produce the same number for every divider, every branch glyph, and every other patched-font glyph a powerline theme uses. The port's simplification is not approximating the divider math; it is exact there. The same holds for the ellipsis character a truncating segment might use, which the database also reports as ambiguous and which every non-CJK terminal draws in one column. What the simplification actually costs is confined to text the *user* put in a segment: CJK content in a directory name, a wide emoji in a media segment's track title, a fullwidth character in a hostname. That is a real cost and Chapter 7 is right to name it, but it is a cost on segment payloads, not on the frame the renderer builds around them, and that distinction is why a line can be one cell off without the chevrons ever coming apart.

The errors both implementations make together

The more interesting half of the width story is the set of cases where the port's scalar count and upstream's dispatch agree with each other and both disagree with the terminal. These are inherited errors, and for a project whose product is fidelity they are not bugs to fix but behavior to preserve — fixing them would be the divergence.

Combining marks are the clearest case. A decomposed accented letter is two code points, one base and one nonspacing mark, and it draws in one cell. Python's `len()` on the decomposed form returns two, so the scalar count is two. The combining acute is classified ambiguous by `East_Asian_Width`, so a dispatch that defaults ambiguous to narrow also counts two. Both overcount by one, and they overcount identically. A width function that handled this correctly would have to consult the general category and treat nonspacing marks as zero width, which is a different property lookup than the one either implementation performs.

Emoji built from sequences are the same story at larger scale. A profession emoji joined with a zero-width joiner is three code points that draw as one two-cell glyph on a terminal that supports the sequence, and as three separate glyphs on one that does not. A flag is a pair of regional indicator code points that draws as one glyph. The port counts three and two respectively. An `East_Asian_Width` dispatch counts four and two — the two pictographs in the first sequence are `W` and the joiner between them is `N`, while both regional indicators are `N`. So here the two implementations disagree with each other and both disagree with a terminal that renders the sequence as one glyph, and a terminal that does not render the sequence agrees with neither. Variation selectors add a further layer: the emoji-presentation selector is a nonspacing mark that changes whether the preceding character is drawn in text or emoji presentation, which is to say it changes the width of a character while occupying no space itself.

The lesson is not that width functions are bad. It is that beyond the ASCII-plus-patched- font core, display width is a negotiation between the string, the terminal's rendering engine, the font's coverage, and the terminal's locale configuration, and the renderer is party to exactly one of those four. A statusline that stays cell-exact does so by staying inside the character classes where all four parties agree, and a theme author has more control over that than the renderer does.

Which direction to be wrong in

Given that error is unavoidable at the margins, it is worth knowing which sign of error costs more, because it is not symmetric.

Undercounting — measuring a string as narrower than it draws — makes the truncation loop believe a line fits when it does not. The line overflows the terminal width. On a shell prompt this means the prompt wraps or the line editor's cursor arithmetic goes wrong; on a `tmux` statusline it means `tmux` truncates the line itself, at a position `powerline` did not choose, potentially in the middle of a color directive's payload. The visible result is a line that is cut off arbitrarily rather than degraded gracefully by priority.

Overcounting — measuring a string as wider than it draws — makes the loop drop a segment it did not need to drop, or align a right-hand side one cell in from the edge. The visible result is a slightly emptier line. Nothing is corrupted; a segment that should have fit is simply absent, and the absence is one the priority ordering already ranked as the most expendable.

The port's scalar count errs in both directions depending on input — it undercounts wide glyphs and overcounts combining sequences — which is why the guidance in Appendix AB.2 is placement guidance rather than a workaround. The reliable move is to keep the width-constrained parts of a line inside the character classes where the count is exact, and to let the classes where it is not exact live in segments that carry a priority, so that a miscount at the margin degrades into a dropped decorative segment rather than an overflow. A `cwd` segment that can contain arbitrary path text is the usual carrier of surprise glyphs, and it is a good candidate for a priority even in themes that otherwise pin everything.

Why this cannot be fixed by being more careful

There is a temptation, on reading a documented width gap, to treat it as a straightforwardly fixable defect: link a width table, dispatch on it, close the gap. The dispatch is genuinely worth having and closing the wide-glyph case would be an improvement. But it is worth being clear that it would close one case, not the category, and that the category has a floor no implementation reaches.

Consider what a maximally careful width function would need. It would need the `East_Asian_Width` property, and it would need to know the terminal's locale to resolve the ambiguous class. It would need the general category to zero out nonspacing marks. It would need grapheme cluster segmentation to collapse joiner sequences and regional indicator pairs. It would need to know whether the terminal implements the emoji sequence rules at all, because a terminal that does not will draw the pieces separately and the collapsed count will be wrong. And it would need to know which glyphs the active font actually covers, because a missing glyph is drawn as a replacement box whose width is the terminal's choice rather than the character's. The last two are not properties of Unicode; they are properties of a program on the other side of a pipe, and there is no interface through which a statusline renderer can ask.

This is why the honest framing in Chapter 7 — a bounded, named divergence rather than a silent one — is the right framing rather than a hedge. A width function is a model of another program's behavior. The port's model is the same one upstream uses minus one dispatch, both models are exact over the character classes powerline's own visual grammar is built from, and both are approximations over arbitrary user text. Knowing which half of your line is which is the practical skill; knowing that no version of the function is ever going to be exact over the second half is what stops you from chasing it.

Chapter 20 – Sampling, Freshness, and the Aliasing Problem

Chapter 9 lists live-sampled numeric values as the first of the three places where powerliners' output may legitimately differ from Python's, and describes the difference as a sampling window's worth of data. Chapter 12 explains why the sampling happens on the render path and what that costs under concurrency. Neither examines the thing being sampled, and that turns out to be where most of a user's confusion about statusline numbers actually lives. A CPU percentage on a statusline is not a measurement of anything instantaneous; it is an estimate over an interval, produced by one process, displayed by another, refreshed on a third schedule. This chapter takes that apart, because a theme author who understands it makes better choices about which metrics to display and where, and a user who understands it stops filing bugs about numbers that disagree.

A percentage is a ratio of two observations

The first thing to internalize is that most of the interesting metrics on a statusline have no instantaneous value at all. CPU utilization is the fraction of some interval during which the processor was not idle. It is defined by two readings of a monotonically increasing counter and the elapsed time between them; a single reading of the counter tells you nothing about utilization, only about the total work done since boot. The same is true of disk throughput, network throughput, and every other rate. Only the level metrics — memory in use, disk space consumed, battery charge, temperature — have a value you can read once and report.

This distinction determines what a segment must do to answer. A level segment reads a number and formats it; that is the whole of its work, and it is why the memory and disk usage segments are cheap in a way a utilization percentage structurally cannot be. A rate segment needs two observations separated in time, and there are only two ways to obtain them. It can hold the previous observation between renders and difference against it, in which case the reported interval is “since the last render” and varies with how often you render. Or it can obtain both observations within a single invocation, in which case the invocation must span a real interval — which is precisely why the OS probes for this class of metric have sampling arguments, and why running them is measured in the milliseconds rather than the microseconds that Chapter 5 accounts for the render itself.

Both answers have consequences the user sees. The differencing approach makes the number depend on refresh rate: the same machine under the same load reports over one window on a tmux statusline and over a different, longer, irregular window on a prompt that redraws when you press Enter, and those are different numbers describing the same load. The sample-within-the-call approach makes the segment expensive in exactly the way Chapter 15's class-three definition describes, and puts that expense on the render path, and — on the serialized loop of Chapter 12 — in front of everything queued behind it. There is no third option that is both cheap and interval-defined, which is the structural reason rate metrics are the awkward citizens of any statusline.

What the documented divergence is really saying

Chapter 9's phrasing deserves a closer reading in this light. Two implementations sampling the same rate metric will report different numbers not because either is wrong but because they are answering slightly different questions: over which interval, ending at which instant, through which counter. A difference of a sampling window's worth of data is not a rounding discrepancy that a better port could eliminate. It is the irreducible consequence of the metric being interval-defined and the two implementations not sharing an interval.

This is why the divergence sits in the ledger as inherent rather than as a bug with an open fix. Everything deterministic in the render — the markup, the escapes, the color resolution, the layout, the config merge — is a pure function of inputs the harness can hold fixed, and the harness asserts byte-identity over it. A rate metric is a function of inputs that include when the question was asked, and no harness can hold that fixed across two processes. The honest boundary of a byte-identity guarantee is exactly the boundary between pure functions of the request and functions of the world at an instant, and the live-sampled set is the second category.

It also explains why the divergence is bounded rather than open-ended. The framing — the segment's markup, its highlight groups, its format string, its placement, its contribution to the width math — is deterministic and is asserted identical. Only the digits inside the frame move, and they move by an amount bounded by how much the underlying quantity can change in a sampling window. A load average cannot swing arbitrarily between two samples a moment apart; an instantaneous utilization percentage can.

Refresh cadence and what it is capable of showing

The second half of the story is the display side, and it introduces an error that has nothing to do with the sampler. A statusline shows a value that was sampled at some instant and then holds it until the next refresh. The refresh cadence therefore sets a hard limit on what the display can represent, and the limit is not intuitive.

A tmux statusline on a `status-interval` of a couple of seconds produces one observation of a metric every couple of seconds. Any feature of the underlying signal that lives and dies inside that window is invisible: a compile that pegs every core for a fraction of a second may land entirely between two samples and never appear on the statusline at all, or may land on top of one sample and appear as a sustained spike that was never sustained. Both failures are the same phenomenon — a signal observed at a rate slower than it varies is not merely displayed coarsely, it is displayed wrongly, with structure the signal does not have. This is the reason a statusline load readout and a dedicated monitor watching the same machine will tell visibly different stories, and neither is lying.

The prompt case is worse, because the cadence is not a cadence. A prompt redraws when you press Enter, which means the sample instants are determined by your typing rhythm. A metric on a prompt is sampled at irregular intervals correlated with your own activity — you tend to press Enter right after starting something expensive — which is a sampling scheme essentially designed to produce misleading numbers. A load readout in a prompt is a snapshot taken at close to the least representative moment available.

The design conclusion is the one Chapter 15 reaches from the latency side, arrived at from the measurement side: rate metrics belong on a timed surface, not on an event-driven one, because a timed surface at least samples uniformly. And the metrics that survive an irregular cadence best are the level metrics and the state metrics — disk space, battery, branch name, orchestration context — because their value is meaningful at whatever instant you happen to look, and does not encode an interval that the display cadence can distort.

Gradients amplify sampling noise

There is an interaction between sampling and the gradient path of Chapter 6 that is worth naming because it makes an invisible problem visible in an unpleasant way.

A gradient maps a segment's `gradient_level` onto a color list, and the mapping is typically arranged so that the interesting part of the range — the transition from calm to alarming — occupies a narrow band of the input. That is the whole point: a gradient exists to make a threshold legible without a number. But it also means the color is a high-gain function of the sampled value in exactly the region where the sampled value is noisiest, because a machine under partial load is a machine whose utilization fluctuates. A metric that wobbles across the middle of its range on successive samples produces a color that wobbles across the middle of the gradient, and the eye is far more sensitive to a color change in the corner of the screen than it is to two digits changing.

The result is a segment that flickers between hues on every refresh while nothing meaningful is happening, and the flicker reads as urgency because that is what a color change is for. The instinct is to blame the gradient,

but the gradient is faithfully reporting what it was given. The problem is that the input is a noisy estimate and the display function has no smoothing, and smoothing is not something the colorscheme can add — `pick_gradient_value` maps one number to one color, statelessly, which is the correct design for a resolution function and the reason Chapter 6 can describe it in a paragraph.

The place smoothing belongs is the sampler, and the practical form it takes for a theme author is choosing a smoother metric. A load average is already a smoothed quantity by construction — it is a weighted mean over a window rather than an instantaneous reading — which is why a load average on a gradient is calm where an instantaneous utilization percentage is frantic. Choosing the pre-smoothed metric costs nothing and removes the problem at its source, and it is a better answer than tuning the gradient's color stops to hide the wobble, which merely reduces the gradient's resolution everywhere in order to suppress noise in one band.

Level metrics have their own trap: thresholds and rounding

Rate metrics get the attention because their definition is subtle, but level metrics — disk usage, memory in use, battery charge — have a failure mode of their own, and it is the one that produces the complaint “my statusline keeps changing color for no reason.”

The mechanism is quantization at a boundary. A level metric that sits near a threshold will cross it in both directions on successive samples, because the underlying quantity genuinely fluctuates by small amounts and the display has no memory. A filesystem hovering at the edge of a warning band will alternate between two colors every refresh while nothing of consequence is happening; a battery reading that rounds up and down across a digit boundary will alternate between two strings. Both are correct readings, and both are noise presented with the visual weight of a signal.

Rounding in the format string compounds it. A format that renders a fractional percentage to zero decimal places turns a quantity that varies smoothly into one that jumps by whole units, and it does so at the moment the underlying value crosses the halfway point of a digit — which is a boundary the user cannot see and did not choose. The digit changes, the eye catches the change, and the change carries no information the user needed.

The general shape of the answer is hysteresis: a display that changes state should require more evidence to change back than it did to change in the first place. Nothing in the render path provides it. Highlight-group resolution is stateless by design, gradient picking is stateless by design, and both of those are correct decisions for functions whose job is to map a value to a color reproducibly — a resolution function that depended on history would be unstable and would break the determinism the parity harness needs.

So the available mitigations are all authoring mitigations, and they are worth knowing because they are cheap. Choose a coarser format when the extra digit is noise rather than information; a disk usage rendered without a fraction changes far less often than one rendered with two decimal places, and nobody was going to act on the second decimal. Place threshold-driven color changes where the threshold means something — a band boundary at ninety percent of a disk is a decision point, a boundary at fifty is decoration that will flap. And prefer a metric whose interesting range is wide to one whose interesting range is a knife edge, because a display that spends most of its time far from any boundary is a display that is quiet when nothing is happening, which is the entire job of a statusline.

The connecting principle with the rate metrics of the previous sections is that the render path is a faithful, stateless function of a sampled value, and every technique for making a noisy value read calmly has to live either in the choice of value or in the choice of format. The renderer will not smooth for you, and the reason it will not is the same reason its output can be asserted byte-identical against another implementation.

The many-windows problem

The last piece is a consequence of Chapter 12's per-window multiplication that shows up as a specific, confusing symptom, and it is worth stating because the symptom looks like a bug.

A tmux session with several windows renders the statusline once per window per tick. Each of those renders runs the segment adapters independently. If the theme's window-status format includes a live-sampled segment, that segment samples the machine once per window, at slightly different instants, and each window displays its own answer. The user sees a statusline whose windows disagree about the CPU load — several windows reporting different percentages about the same machine at what appears to be the same moment.

Nothing is wrong. Each number is a correct sample; they differ because they were taken a short time apart on a quantity that changes, and because a differencing sampler in each render would be differencing over slightly different intervals. But the display invites the reading that they should agree, because they are all on one line describing one machine.

The fix is placement, and it is the same placement rule Chapter 15 gives for a different reason. A machine-wide metric belongs on the once-per-tick sides of the statusline, not on the per-window format, which renders once per window. On the once-per-tick surface there is exactly one sample and exactly one displayed value, and the disagreement cannot arise. The per-window format is for facts about the window — its index, its name, its activity flag — which are per-window by nature and for which several different answers is the correct output rather than a confusing one. That the same placement rule falls out of the latency analysis, the concurrency analysis, and now the sampling analysis is the strongest evidence available that it is a structural property of the design rather than a stylistic preference.

Chapter 21 – Locale, Encoding, and the Bytes That Are Not Text

Chapter 4 established the principle that the render must reflect the caller's context, not the daemon's, and derived the wire protocol from it: `argv`, `cwd`, and the full environment travel across the socket because a long-lived process cannot otherwise know where the question came from. Locale is part of that context and is worth its own treatment, because it reaches further into the render than the environment-reading segments the earlier chapters used as examples, and because the encoding questions underneath it are the one place where a language whose string type guarantees valid Unicode and the Unix API's actual data model do not line up.

Locale is request context, not process context

The daemon's own locale is an artifact of whatever environment the login shell that started it happened to have. It was fixed at daemon start, it may be hours or days stale, and it has no necessary relationship to the shell that is asking for a prompt right now. A user who switches locale in one terminal, or runs a `tmux` session under one set of locale variables and a detached shell under another, has two different contexts and one daemon.

That makes locale exactly the same kind of input as the working directory, and it must be treated the same way: taken from the request rather than from the process. The wire protocol already carries it, since locale is expressed entirely in environment variables and the protocol forwards the whole environment. The obligation Appendix AJ states for adapters — read the forwarded environment, never the process environment — therefore covers locale without needing a separate rule. But the obligation is easier to violate for locale than for anything else, because locale is usually consumed implicitly. An adapter that calls a formatting routine which consults the process locale internally has read ambient state without ever writing the name of an environment variable, and the resulting bug is a segment that renders correctly for whoever started the daemon and subtly wrong for everyone else, on a machine where the developer is usually the person who started the daemon.

What locale actually reaches

It is worth enumerating the surfaces, because the list is longer than the obvious one.

Date and time formatting is the obvious one. A `date` segment with a `strftime`-style format string containing a weekday or month name produces different bytes under different locales, and it produces different widths, which feeds directly into the truncation math of Chapter 19. A format that fits comfortably in one locale can overflow in a locale whose month names are longer, and the overflow manifests as a dropped segment somewhere else on the line — which is to say the locale of the caller can change which segments appear. `fuzzy_time` is the sharper case: it produces prose, and prose is language, so a `fuzzy_time` segment is either in one language or it is a translation problem, and upstream's decision on that is what the port inherits.

Number formatting is the quiet one. Decimal separators and digit grouping are locale properties, and any segment reporting a percentage, a byte count, or a load average is emitting numbers. A humanized byte count that renders with a period in one locale and a comma in another is a one-byte difference that no user would notice and that a parity harness would flag immediately — which is a nice illustration of the harness being more sensitive than a human reader, and of why byte-identity is a stronger claim than “looks the same.”

Collation is narrower but real. Anything that sorts — a lister enumerating workspaces or buffers, a segment listing container names — sorts according to an order that is a locale property, and a locale-sensitive sort produces a different order than a byte-order sort for anything outside ASCII.

The surface that catches people is the one that has nothing to do with formatting: the output of subprocess probes. Chapter 15's class-three segments fork tools — the OS metric probes, git, container tooling — and parse what those tools print. Those tools localize their output. A parser written against one language's output will fail, or worse will silently match nothing and produce a plausible zero, when the same tool runs under a different locale and prints translated field labels. The forwarded environment is what the subprocess inherits, so the caller's locale reaches the probe. The mitigation is the standard one for machine-parsed subprocess output — invoke the probe with the locale pinned to the portable `C` locale so its output is stable, while leaving the caller's locale intact for the formatting that should honor it — and the reason to state it here rather than leaving it as folklore is that it is the difference between a segment that works everywhere and a segment that works on the author's machine.

Three encoding layers that do not agree

Underneath locale sits a more fundamental mismatch, and it is worth laying out as three layers because conflating them is how the difficult bugs happen.

The configuration layer is Unicode text by definition. JSON is defined over Unicode; a theme file, a colorscheme, a `colors.json` are text, they are decoded once at load, and after that everything derived from them is a string in the ordinary sense. Nothing in the config cascade has to worry about undecodable input beyond rejecting a malformed file.

The environment and filesystem layer is not text. On Unix, environment variable values and path components are byte strings with exactly two constraints: no NUL anywhere, and no slash within a path component. There is no encoding requirement. A directory whose name is a sequence of bytes that is not valid UTF-8 is a perfectly legal directory, a shell can be placed inside it, and a prompt has to render something for it. This is not a hypothetical corner: it is what happens to a filename copied off a filesystem written under a different encoding, or an archive extracted with the wrong assumption, and it survives indefinitely because nothing forces it to be repaired.

The output layer is bytes again, but bytes with an expectation: whatever the terminal will decode. In practice that means UTF-8, and the powerline glyph set assumes it, but the renderer's job ends at writing bytes and the decoding happens in a program it cannot see — the same boundary Chapter 19 described for width, seen from the encoding side. Appendix D.5 documents the one renderer that refuses to assume: the `pdb` renderer folds its output to ASCII, mapping anything non-ASCII to a placeholder, because its consumer's encoding cannot be relied upon. That is the exception that shows the rule the others are operating under.

The wire is byte-transparent, and that is load-bearing

The wire format's design turns out to be exactly right for the middle layer, and it is worth noticing why. The request is a NUL-separated blob terminated by a double NUL. NUL is the one byte that cannot appear in an environment value or a path component, which means every other byte can travel unexamined. The protocol does not need to know the encoding of what it carries, and it does not need an escaping scheme, because the framing byte is the byte the payload cannot contain. That is a property worth admiring rather than passing over: a great many protocols acquire an escaping layer, and every escaping layer is a place where a round-trip can fail to be a round-trip.

That transparency is what allows a request to carry a `cwd` or an environment value that is not valid UTF-8 at all. The transport is not the problem. The problem, if there is one, is what a program written in a language whose string type guarantees UTF-8 does with those bytes after demarshalling them. Rust's `String` and `str` carry that guarantee; the types that do not are `OsString` and `OsStr`, which is exactly why the standard library uses them at every boundary where the operating system hands over bytes with no encoding promise. A program

that converts too eagerly has to decide what to do with input that will not convert, and the choices are to fail, to substitute replacement characters, or to keep the bytes.

Python 3 solved the same problem differently, and its choice is worth knowing because it is the behavior the upstream implementation exhibits. Its filesystem and environment encoding is UTF-8 with a surrogate-escape error handler, which maps undecodable bytes onto lone surrogate code points that round-trip back to the original bytes on encode:

```
>>> import os, sys
>>> sys.getfilesystemencoding(), sys.getfilesystemcodeerrors()
('utf-8', 'surrogateescape')
>>> os.fsdecode(b'/tmp/\xff')
'/tmp/\udcff'
>>> os.fsencode(os.fsdecode(b'/tmp/\xff'))
b'/tmp/\xff'
```

So a Python program can hold an undecodable path in an ordinary string, pass it around, and write it back out unchanged. What it cannot do is give it a sensible display width — a lone surrogate counts as one code point, and what the terminal draws for a byte it cannot decode is the terminal's choice. Which is to say: the encoding problem and the width problem of Chapter 19 are the same problem seen twice, and both have a floor set by the consumer rather than the producer.

What this means in practice

The user-facing consequences are small in number and worth stating plainly, because the machinery above is invisible until one of them bites.

A path segment is the carrier. `cwd` is the segment most likely to encounter bytes that are not text, because directories are named by whoever made them. If a prompt behaves oddly in one specific directory and normally everywhere else, the directory's name is the first thing to look at, and listing the parent through a filter that makes non-printing bytes visible is the fastest way to see it. This is a useful addition to Chapter 17's bisection: between “is it the config” and “is it the render” sits “is it this one input,” and a path is the input most likely to be strange.

Locale belongs in the environment you actually use. Because the caller's locale is what renders, setting locale variables in a shell rc affects that shell's prompts from the next render, with no daemon restart involved. There is nothing to reload, because locale is not cached: like the config-path overrides of Chapter 13, it travels on the wire rather than living in the warm state, and the environment cannot go stale because it is not held.

And a segment that parses tool output should pin its subprocess locale even though a segment that formats a date should not. Those two rules point in opposite directions and both are correct, because they concern different consumers: the probe's output is read by a parser that wants stability, and the segment's output is read by a human who wants their own conventions. Getting them backwards produces either a segment that reports nothing on a translated system or a prompt that renders dates in a language the user did not choose.

Chapter 22 – Port Artifacts and the Diagnostic Surface

Chapter 2 quotes the `SegmentGetterResult` struct and points out that one of its fields is a duplicate of another, carried because the Python original returns a tuple in which the function name appears twice. It calls this the port discipline made concrete. It is also an example of a general category that runs through the whole ported tree and that a reader benefits from having a name for: a *port artifact* — a structure preserved from the original that carries no content of its own, kept because preserving shape is how the port stays diffable against its source. This chapter collects the artifacts the book has already exhibited, argues for why they are kept, draws the line where preserving shape would become preserving a bug, and then follows one particular artifact — the logger parameter — into a discussion of where powerliners’ diagnostics actually live.

What counts as an artifact

The duplicated tuple field is the cleanest specimen. Nothing reads it that could not read its twin; removing it would change no behavior; and it is kept anyway, with a doccomment saying so, because the shape of the returned record is part of what the citation discipline of Chapter 10 is pinning. When a maintainer opens the Rust and the Python side by side to check a `// py:NNN` citation, a record whose fields line up against the tuple’s elements is checkable at a glance. A tidier record would be a better API and a worse port, and the project has been explicit from Chapter 1 about which of those it is building.

The second specimen is visible in the `process_segment` signature quoted in Chapter 2. Its first parameter is `pl: &()` — a reference to the unit type, which carries no data and can carry none. Upstream threads a logger object through the segment machinery as its first argument, so every ported function that takes one in Python takes one in Rust, and the port satisfies the type by passing the only value the unit type has. The parameter’s arity is real; its content is empty.

A third is visible in the escape-hatch machinery, and it is the counterexample that sharpens the category. `SCRIPT_EXTENSIONS` ends with an empty string, which looks like a placeholder and is not — it is what makes a bare executable with no extension match, as Chapter 3 says when it introduces the table. Telling a genuine artifact from a load-bearing oddity is the skill this section is trying to build, and the test is behavioral: delete it in your head and ask whether any input now behaves differently. Deleting the duplicated field changes nothing; deleting the empty extension makes extensionless segment scripts stop resolving.

And there is an inverse category worth naming, which the book exhibits in Chapter 18: ceremony upstream needed that the port does not reproduce at all. The per-flavor shell renderer classes exist in Python because a registry needs a class per flavor to subclass a base; the port holds the same escape constants directly on the renderer struct and skips the class. That is not an artifact preserved, it is structure declined — and the tier classifier’s inability to score its absence is exactly the NEAR-tag story of Chapter 18. Preserving shape is a default, not an absolute, and the shell renderers are where the default is overridden.

Why preserve a shape with no content

The argument for artifacts is entirely about the maintenance model, and it is worth making explicitly because the alternative — clean it up, it costs nothing — is superficially persuasive.

The port's fidelity claim rests on a mechanical correspondence: every ported function name matches a manifest, function bodies carry citations into upstream source lines, and a drift gate fails the build when the correspondence breaks. That machinery works because the two trees are shaped alike. Every place the Rust deviates structurally from the Python is a place where a reviewer has to hold a translation in their head instead of reading a diff, and translations in reviewers' heads are where errors live. One deviation is free; many small tasteful deviations compound into a port nobody can audit against its original, which is precisely the failure the tier classifier and the drift gate exist to prevent.

The second argument is about the future. Upstream is a real project with a history, and Appendix Y describes what tracking it commits the port to. A change upstream lands against a shape; if the port shares that shape, the change is a local edit, and if it does not, the change is a redesign. A duplicated record field costs one machine word and buys the property that a future upstream change touching that tuple is a mechanical port rather than a judgement call.

The line where this stops is behavior. An artifact is a shape with no content; a bug is content that is wrong. Preserving a shape costs nothing a user can see; preserving a bug costs a user something. The port's own documented divergences show where the project drew the line in the other direction: `select` was replaced by `poll` because Appendix E.6 could establish that the substitution is behavior-neutral for the cases the code exercises, and the socket-path handling under an explicit `--socket` deliberately differs from upstream, which prepends the abstract-namespace prefix even for an explicit path where the port honors it literally. Shape is preserved by default; behavior is preserved absolutely; a deviation in either requires an argument, and the project writes the argument down.

The logger parameter, and what a unit type says

Return to `pl: 6()`, because it is the artifact with the most interesting consequences.

In the original, that parameter is how a segment reports trouble. A segment that cannot reach its data source, that got a malformed answer, or that hit an error it chose to swallow has a place to say so, and the daemon has somewhere to route it. That is a real facility, and its absence from the type — a unit reference cannot carry a sink, so nothing can be logged through it — means the ported functions cannot report anything through that channel. This is not hidden: it is in the signature, in the book, on the page where the segment model is introduced, which is the right place for a limitation that a reader of the segment API needs to know about.

Two things follow, and they pull in opposite directions. The first is that the port's diagnostic story has to live somewhere other than the segment API, which the next section takes up. The second is that the artifact is precisely what makes the story fixable later: the parameter exists at every call site, in every signature, in the right position. Introducing a real logger is a change to one type and to the construction of one value; it is not a change to the arity of any function or to the shape of any call. Had the port cleaned up the empty parameter, adding a logger would mean touching every ported signature and every call site in the tree, and every one of those edits would be a place to introduce a divergence. The artifact is load-bearing for maintenance even though it is inert at runtime, which is the strongest form of the argument in the previous section.

Where diagnostics actually live

Given that the segment API carries no logger, it is worth collecting the diagnostic surface that does exist, because the book has introduced its pieces separately and never assembled them.

The first and most useful is static: `powerline-lint`. Chapter 9 makes the case that authoring errors should be caught before they render, and the linter is the only part of the system that reports a problem with a source position. Everything else in this list reports a symptom; the linter reports a location. That asymmetry is why the workflow Chapter 13 recommends — lint on save, not after the prompt looks wrong — is worth adopting even though the staleness reload makes the edit-render loop fast enough to make guessing tempting.

The second is the render-time resolution failure. When highlight-group resolution cannot match any of a

segment's groups it fails with a message naming everything it tried, which is a genuine diagnostic and names the right thing. But it arrives at render time, in whatever channel the render's caller is reading, which for a daemonized process is not the user's terminal.

The third is the protocol's own error path. A malformed request gets an error reply rather than a crash, which Chapter 12 counts as the defensive-parsing property and which is also a diagnostic: a client that receives that reply knows its blob was wrong, and a person debugging a hand-built wire request gets told rather than getting silence.

The fourth is the daemon's own console output, and it is where the constraints get interesting. The `-q` flag exists to suppress diagnostic prints, which establishes that diagnostic prints exist on the startup path; and `-f` exists to keep the daemon attached to a terminal, which is what makes them visible. Together they are the debugging affordance: running the daemon in the foreground, unquiet, is how you see what it says. After a normal daemonize, standard output and standard error are pointed at the null device (Appendix E.5), so those same prints go nowhere. That is correct for a background service — a daemon writing to an inherited terminal is a nuisance — but it means the foreground mode is not merely a convenience for init systems, it is the diagnostic mode.

The fifth is the client's silence, and it deserves naming as a diagnostic gap rather than a diagnostic. When the socket cannot be reached the client execs the direct renderer and the prompt appears normally. That is the resilience property and it is the right default, but it means the most common daemon failure — it is not running — is invisible by design. Chapter 17's first debugging step is to establish whether the daemon is in the picture at all, and the reason that step is first is exactly this: the system is built to make its own absence undetectable from the output.

What adding a real logger would have to respect

It is worth closing with the constraints a future diagnostic channel would face, because they are more specific than “add a logger” suggests and they explain why the current answer is defensible.

The reply is sacred. The render's output is the product, it is asserted byte-identical against the live interpreter, and anything that writes into it breaks the guarantee the whole project exists to make. A logger cannot share the channel the render uses, which rules out the laziest implementation immediately.

The standard streams are gone. By the time the daemon is serving requests it has forked twice and pointed its first three descriptors at the null device. A file sink would have to be opened deliberately, and the path it opens is a decision with its own consequences: it lives as long as the daemon, it grows, and — per Chapter 14's observation that the forwarded environment carries secrets — anything that logs request context is writing secrets to disk. The trust-boundary chapter's warning about pointing a debugging tool at the socket applies with more force to a persistent log than to a scratch buffer, because a scratch buffer is closed and a log file is not.

And the harness must stay able to run. The parity suite depends on the daemon's behavior being deterministic for a scripted sequence of requests. A diagnostic channel that only writes elsewhere does not threaten that; one that changes timing, ordering, or the render path does. The narrow shape that satisfies all three constraints — an opt-in file sink, opened explicitly, writing nothing on the default path — is a small feature, and the reason to state its constraints rather than its design is that the constraints are the durable part.

Chapter 23 – Adding a Renderer: The Other Extension Point

Chapter 16 walks the process of writing a native segment, and it is the extension point most users will ever touch. There is a second one, and the book has described its outputs thoroughly — Appendix D enumerates what every shipped renderer emits — without ever describing what it would take to add one. This chapter does that, and the reason it is worth a chapter rather than a paragraph is that a renderer is a categorically riskier thing to add than a segment, for reasons that have nothing to do with how much code it is.

A segment adds a fact; a renderer adds a target

The asymmetry is worth stating first because it explains everything that follows. A segment produces contents and asks for a highlight group. It participates in the render as one entry in a list, its failure mode is bounded — it returns nothing, or it returns wrong text in the right frame — and it cannot affect any other segment except through the width it occupies. The blast radius of a bad segment is one segment.

A renderer decides how every segment in the line is encoded. It runs after all the layout decisions are made, it touches every byte that reaches the consumer, and its failure modes are global: a missed escape mangles the line from that point on, a missing attribute reset bleeds styling forward through every subsequent segment, a wrong width marker breaks the line editor’s arithmetic for the entire prompt. Appendix D.6 makes exactly this point about the shipped encoders, and closes by calling them short and saying that they are the port. Adding one puts you in that position for a new target, with none of the verification the shipped ones have.

What the core gives you, and what it asks for

The division of labor is clean, and knowing it precisely is most of the work. The layout core owns everything geometric: which segments there are, what order they are in, which boundaries get hard dividers and which get soft, how wide everything is, what gets dropped when the line is too narrow, and how leftover width is distributed among expanding segments. None of that is per-target, which is why it lives once in `renderer.rs` and every renderer inherits it, and why a bug in truncation is a bug in every target simultaneously.

What a renderer owns is translation. Given a resolved highlight — a foreground, a background, and an attribute bitfield — produce the bytes that make the consumer display that. Given a piece of segment text, produce the bytes that make the consumer display that text rather than interpreting part of it as markup. Those are the two obligations, and Appendix D’s per-target sections are, read structurally, a catalogue of how differently they can be discharged.

The four decisions

Working through the shipped renderers as a design space rather than a reference surfaces four questions that any new target forces, and they are the right checklist for someone considering one.

Is the target’s markup stateful? tmux’s is: a style directive persists until something changes it, which is why the tmux renderer emits every attribute slot on every directive, including the negative forms. Without the explicit off-forms, a bold segment makes everything after it bold. The shell renderers take the opposite

approach and lead with a reset. The question a new target poses is which model it follows, and the failure mode of getting it wrong is the one Appendix D.6 names — invisible on a simple line, glaring on a real one, because it only manifests when a styled segment is followed by a differently-styled one.

What characters are structural? Every markup language has a lead-in character that segment text must not be able to impersonate, and each shipped renderer carries the translation rule its target needs: tmux doubles its hash, vim doubles its percent, the markup renderer does XML entity replacement, and the shell flavors escape what their line editor treats specially — or, where the target treats nothing specially, carry no rule at all, which is why the readline flavor's translation column in Appendix D.4 is empty. This is the single most important function in a new renderer, because it is a total function over arbitrary user text — a directory name, a branch name, a track title — and any input it does not handle is an injection into the consumer's markup. It should be the first thing written and the most heavily tested.

Does the consumer measure the line? Shell line editors do, which is why the shell renderers wrap escape sequences in non-printing markers so the editor's cursor arithmetic stays right. tmux does not need them; a bar consumer does not either. If the target measures, the markers are mandatory and their exact form is dictated by the consumer rather than chosen by the renderer.

Does the target take bytes at all? Two of the shipped renderers answer that differently from the rest, and they are the proof that the layout core is not byte-oriented at its boundary. The i3bar renderer emits a JSON object per segment; the ipython renderers hand token names and style rules to a host library rather than escape sequences to a terminal. A new target that is a GUI widget, a structured log line, or an API payload is therefore not an unnatural fit — the core produces a list of styled spans, and what those spans become is entirely the renderer's business.

The invariant no renderer may break: markup is not width

There is one rule that binds every renderer, that none of the per-target sections in Appendix D states outright because each of them obeys it silently, and that a new renderer is most likely to violate. The layout core measures *visible* content — segment contents, dividers, padding — and nothing else. The encoder runs after the measurement, in the emit half, and every byte it adds is a byte the width accounting never saw.

That arrangement is what makes the design work. If markup counted toward width, the truncation loop would have to know how many bytes each target's encoding costs, and the answer would depend on the resolved colors of each segment, which are not known until after the segments to be dropped have been chosen. The dependency would be circular. By declaring markup weightless, the core keeps layout a pure function of visible content and lets each target encode whatever it needs at whatever length it needs.

The obligation this places on a renderer is absolute and easy to state: nothing it emits may occupy a cell. Every byte of an escape sequence, every character of a style directive, every bracket of a JSON object must be consumed as structure by the target and never drawn. A renderer that emits a stray printable character — a separator, a space, a fallback glyph when a color cannot be encoded — has silently added width that the truncation math already committed to not having, and the line will be that much too long. This is a subtle failure because it is invisible on a wide terminal and appears only when the line is near its budget, which is exactly when a user is least inclined to suspect the encoder.

The shell renderers show the same invariant enforced from the other side. There, the consumer measures — a line editor computes the visible prompt width to place its cursor and to wrap long command lines — and the consumer has no way to know which bytes are escapes. The non-printing markers exist precisely to tell it, and they are the target's mechanism for the same rule the core enforces internally: these bytes are structure, do not count them. A renderer for a measuring consumer that omits the markers has not broken powerline's width math, which was correct; it has broken the consumer's, which is worse, because the symptom appears in the shell's cursor handling rather than in the prompt's appearance and sends the user looking in the wrong place entirely.

The oracle problem

Here is the part that makes adding a renderer different in kind from adding a segment, and it follows directly from the project's verification model.

Every shipped renderer is a port. There is an upstream implementation emitting bytes for the same target, and the parity harness runs the real interpreter and diffs the actual output. When the tmux renderer's hash-doubling is right, that is not a claim, it is an assertion that passed against the original. The encoders are short and load-bearing, and the harness is what makes "load-bearing and correct" checkable rather than asserted.

A renderer for a target upstream does not support has no oracle. There is nothing to diff against, so the harness cannot say anything about it, and by the provenance rule of Chapter 18 it belongs outside the ported tree with the extension segments — new work, held to the project's test discipline, but not measured against a Python original because there is none. This is the same category the net-new segments occupy, and it carries the same obligation to be labelled as such rather than blurred into the port's fidelity claim.

That changes what verification has to look like. Without an oracle the burden shifts onto three things. The escaping function needs property tests rather than examples: for arbitrary input, the encoded form must decode back to the input under the target's own rules, and no input may produce a byte sequence the target reads as structure. Golden outputs need to be checked against the real consumer rather than against a reading of its documentation, because the documentation is a model of the consumer and the consumer is the thing that has to be satisfied — which in practice means feeding output to the actual bar, editor, or terminal and looking at it. And the stateful-markup question needs a test that renders two adjacent segments with different attributes and asserts the second is unaffected by the first, because that is the bug class the whole negative-attribute discipline exists to prevent and it cannot appear in a single-segment test.

When not to add one

The last piece of guidance is the one that resolves most cases before any code is written. Many targets accept a markup that a shipped renderer already emits. A consumer that understands ANSI select-graphic-rendition sequences can be fed the shell renderer's output; a consumer that reads a line of text and displays it verbatim needs no styling renderer at all. Reaching for an existing renderer costs nothing and inherits a verified encoder; writing a new one costs an unverifiable encoder and a permanent maintenance obligation on a fidelity surface.

The case for a new renderer is when the target's model is genuinely different — structured output rather than a styled line, named styles rather than inline colors, a protocol rather than a stream. Those are the cases the shipped set already covers at its edges, and they are exactly the cases where reusing an encoder would mean encoding into a language the consumer does not speak. Chapter 16 gives a similar test for segments: reach for the native path when the thing is permanent and the escape hatch is costing you. The renderer version of the test is stricter, because the escape hatch here is "a renderer that already exists is close enough," and close enough is usually true.

Chapter 24 – Four Architectures for a Prompt

Every chapter so far has argued from inside powerliners' design, taking the daemon as given and working out its consequences. This chapter steps outside and asks what the alternatives are, because the daemon is one of a small number of structurally distinct ways to put a rich prompt on a screen, and the trade-offs among them are more interesting than a performance comparison would suggest. The four architectures below are archetypes rather than products; the point is the shape of each, and what the shape forces.

Archetype one: the prompt lives in the shell

The oldest answer is to write the prompt in the shell's own language. The shell expands its prompt string on every redraw, and if the prompt string contains shell code, that code runs in the shell's own process with access to everything the shell knows.

The advantages are real and are easy to undersell. There is no process to start for the parts that need no external data, because the shell is already running: an exit code, a job count, a working directory, and the current time are all in hand, and reading them costs nothing. State persists trivially, because the shell has variables and they survive between prompts, which means a differencing sampler of the kind Chapter 20 describes is available for free. And the integration with shell-local facts is perfect by construction — there is no marshalling problem for the exit code because there is no boundary to marshal across, which is a genuine advantage over every other architecture here, all of which have to forward that information somehow. Chapter 3's observation that the shell-status family exists only because the wire protocol carries `argv` is, read from this angle, a cost this archetype does not pay.

The costs are equally structural. Anything the shell does not already know requires forking a process, and a shell's fork is not cheaper than anyone else's; a prompt that consults version control forks `git` the same way a native renderer does. The implementation is bound to one shell's language, so the same prompt cannot be shared with a second shell, and it cannot serve `tmux` or an editor at all, because those are separate programs that will not evaluate your shell functions. And the language itself is the weak point for anything nontrivial: string processing, arithmetic, and error handling in shell are exactly the operations that make shell code hard to get right, and prompt code is unusually load-bearing because a bug in it makes the terminal unpleasant to use rather than failing quietly in a log.

The archetype is therefore excellent for a prompt that is only a prompt, in one shell, with modest logic, and structurally incapable of being the answer for a statusline system that must also serve `tmux` and an editor from one configuration.

Archetype two: a fresh native process per render

The second answer runs a compiled binary on every redraw. It has no memory between invocations, so it reads its configuration, does its work, writes its output, and exits.

Its virtues are simplicity and isolation. There is no state to invalidate, no lifetime to manage, no socket to secure, no shared resource to serialize on. Every render is independent, so a render that fails affects one prompt and the next one is fine. It is portable across consumers by construction: the same binary serves a

shell, tmux, and an editor because they all just run it. And its failure model is the simplest of the four — the binary is there or it is not.

Its cost is that every render pays the full setup. For a native binary the process cost itself is small, but the config cost is not free: the cascade must be walked, the JSON parsed, the merges applied, the colorscheme built, and the segment getters resolved, on every single render. This is the cold path Chapter 13 accounts, and it is what makes the direct renderer substantially more expensive than a warm daemon round-trip even though there is no interpreter involved. It also has no way to hold sampler state between renders, which by Chapter 20's argument means rate metrics must sample within the call or not at all — the one place where the stateless archetype is strictly worse at producing a correct number rather than merely slower at producing it.

Archetype three: a fresh interpreted process per render

The third is the second with an interpreter underneath, and it is upstream powerline's shape. It inherits every property of archetype two and adds the startup tax that Chapter 1 opens with: before any configuration is read, the interpreter must boot and the module graph must import.

It is worth being precise about what this archetype buys, because the book's framing — correctly — emphasizes what it costs. Dynamic import is the extensibility mechanism the port cannot reproduce, and it is genuinely more expressive than a subprocess escape hatch: a custom segment is a function in the same process, with access to the same objects, callable with the same protocol as a built-in. In-process embedding in a Python REPL is possible only in this archetype. The cost is that the mechanism which makes those possible — a runtime capable of loading arbitrary code at render time — is the same mechanism whose startup dominates every render. The archetype's strength and its weakness are the same property, which is why “make it faster” was never available as an incremental fix and why Chapter 1 frames the daemon as the only structural answer.

Archetype four: a warm server and a thin client

The fourth splits the work in two. A long-lived process holds the configuration and does the rendering; a small client marshals the request, sends it, and copies the answer to standard output. This is powerliners, and it is also upstream's own daemon, which is why the wire protocol is a compatibility surface rather than an invention.

What it buys is the elimination of per-render fixed costs, and Chapter 13 makes the argument precisely: setup becomes per-daemon-lifetime rather than per-render, and the conversion is worth exactly as much as the render count is large. Everything the book says about sub-millisecond renders is downstream of that one structural move.

What it costs is a list the reader has now seen at length, and collecting it here is the honest summary of this book's middle chapters. It costs a state lifetime, which brings an invalidation problem and the mtime staleness machinery of Chapter 13. It costs a trust boundary, because there is now a socket, and Chapter 14 is entirely about what that implies. It costs a shared resource, and Chapter 12 is entirely about what serialization does to a burst. It costs a second failure mode — the daemon can be absent while the client is present — which is why the fallback exists and why the client's silence about that fallback is the diagnostic gap of Chapter 22. And it costs the sampling coherence problem of Chapter 20, where one shared renderer serving many windows produces many answers to one question.

The daemon does not remove complexity from the system. It moves complexity out of the per-render path, where it was being paid thousands of times a day, and into the process lifetime, where it is paid once and reasoned about carefully. That is a good trade, and it is a trade rather than a free win, and a substantial part of this book is the accounting.

What actually decides the choice

Reading the four together suggests that the deciding factor is not performance, which is the thing the archetypes are usually compared on. It is how many consumers share one configuration.

A prompt for one shell can live in that shell, and the in-shell archetype's disadvantages barely apply. The moment the same segment definitions, colorschemes, and layout rules have to serve a shell prompt, a tmux statusline, and an editor's statusline, the in-shell archetype is out structurally — not because it is slow but because tmux will not run your shell functions — and the question becomes how an external program is invoked. Once it is an external program invoked many times per second across several consumers, the fixed setup cost becomes the dominant term, and the daemon is what removes it. Powerline's own architecture, and therefore this port's, is the consequence of the multi-consumer requirement rather than of a performance ambition; the performance work follows from the requirement.

powerliners is archetype four with archetype two as its fallback, and that pairing is the design's load-bearing choice rather than an implementation detail. The daemon supplies the latency; the direct renderer supplies the availability; the client chooses between them without the user knowing. Each archetype covers the other's weakness — the daemon's weakness is that it can be absent, the fresh process's weakness is that it is slow — and running both is what makes the fast path safe to depend on. That is the argument for the shape, and it is why the honest description of powerliners' architecture is not "it has a daemon" but "it has both, and the client picks."

Chapter 25 – Version Skew: When a Warm Daemon Outlives Its Binary

Every chapter so far has treated the daemon as a single thing that either is or is not running. It is time to notice that a daemon is a *process*, that a process is a snapshot of an executable taken at the instant it was started, and that on any machine where software gets updated the executable on disk and the executable in memory drift apart. A package manager replaces a file; it does not reach into a running process and rewrite it. The consequence is a class of confusion with a specific shape: you upgraded, you can see the new binary on disk, and your prompt is still behaving exactly as it did before — because the thing rendering your prompt is the old code, still running, still holding its socket, and nothing in the system is designed to tell you so.

This chapter is about that gap. It is not a bug report. Every property that produces the skew is a property this book has already defended on other grounds, and none of them should change. What is missing is not a mechanism but a statement, and this chapter is the statement: what skew is, why the protocol structurally cannot detect it, how to see it, and where the responsibility for resolving it actually sits.

The daemon is the old code, and nothing says so

Start from the socket name. It is `powerline-ipc-<uid>` — a user identity and nothing else. There is no version in it, no build hash, no ABI generation, no start timestamp. That is the correct name for the thing it identifies, because what it identifies is “user N’s powerline daemon,” and a user has exactly one of those. But a name that encodes only identity cannot encode change, so the socket a freshly-installed client connects to is the same socket the previously-installed client connected to, and the daemon on the far end has no obligation to be the same generation as the client on the near end. It is not even a coincidence that they usually are; it is a consequence of the fact that most people restart their session between upgrades and never notice the question exists.

Now add the wire format. The request is a hexadecimal argument count, the arguments, the working directory, and the environment, all NUL-separated and terminated by a double NUL. Read that field list again and notice what is absent: there is no protocol version, no client identifier, no capability list, no feature flags, no reserved word for future use. The reply is even barer — raw markup bytes, no framing, no status code, terminated by connection close. Chapter 4 called that asymmetry a design statement and it is; here it has a second consequence. **A protocol with no version field cannot negotiate, cannot refuse, and cannot warn.** A client from any generation and a daemon from any generation will complete a transaction successfully as long as both can parse the byte layout, and both always can, because the byte layout is the one thing that has never changed.

Finally add the warm state. Chapter 13 enumerated what a daemon holds: the mapped executable, the resolved configuration, and the `PowerLineKey`-keyed orchestrator cache. The mtime staleness check invalidates the second and third when config files change. It does not and cannot invalidate the first, because the first is not derived from anything the daemon can stat — it is the daemon. A config edit reaches a running daemon within one render. A code change does not reach it at all, ever, under any circumstance short of the process exiting.

Put the three together and the skew is fully determined: the socket does not name a version, the protocol does not carry one, and the reload path does not touch code. The daemon runs the binary it was started from until it dies. This is not a defect in any of the three; it is what each of them being correct produces jointly.

Why the port cannot fix this, and why that is the right answer

The obvious engineering response is to add a version handshake. A first field carrying a protocol generation, a client that refuses to talk to a daemon it does not recognize, a daemon that replies with an “upgrade me” message instead of markup. Every one of those is a normal, sensible thing to build into a client-server protocol, and every one of them is forbidden here.

The reason is the whole product. `powerliners` is drop-in compatible, and drop-in compatible means a *four-way* interoperability matrix, not a two-way one. The upstream Python client can talk to the Rust daemon. The Rust client can talk to the Python daemon. Both same-language pairs obviously work. That matrix is what makes the migration of Appendix M reversible: you can swap the daemon and keep the old client, or swap the client and keep the old daemon, and verify each half independently. A version field would have to be sent by a client that predates it and understood by a daemon that predates it, which is a contradiction — the upstream C client marshals exactly the fields it marshals, and it will never marshal one more. Any handshake the port added would be a field the Python client does not send, and a daemon that required it would reject its own compatibility guarantee at the first request.

So the port inherits versionlessness the same way Chapter 14 says it inherits the socket’s access model: not because versionlessness is preferable in the abstract, but because the alternative is a divergence in the one place the product cannot afford one. The honest resolution is the same in both cases too — document the consequence rather than break the contract. A reader who understands that the daemon is a frozen snapshot of a binary will never be confused by skew. A reader who assumed the protocol was policing generations will be confused exactly once, at the worst possible moment, immediately after an upgrade, when every instinct says the new code must be running because the new code is right there on disk.

What skew actually changes, and what it cannot

It is worth bounding the blast radius, because “the daemon is old” sounds more alarming than it usually is.

Skew cannot corrupt anything. The daemon renders with the code it has, against the request it was given, using config it reads from disk at request time subject to the staleness check. An old daemon serving a new client produces exactly the output the old daemon would always have produced. That output is not wrong in any sense other than “not what the new version produces,” and if the versions render identically for your config — which, for a port whose entire discipline is byte-identity against a fixed upstream, is the common case — skew is *invisible and harmless*. Most of the time nothing happens, which is precisely why the case that does bite is so disorienting.

Skew shows up when a release changes rendered output or adds a capability. A new segment adapter in the registry is the sharpest example: you upgrade, you add the new segment to your theme, and the daemon cannot resolve a segment name that is absent from the `ADAPTERS` table its binary was compiled with. The config is new, the config reload works perfectly, and the failure looks like a config problem because everything you touched was config. It is not; it is the code behind the config. The same shape appears for a bug fix — you upgrade to get a fix and the fix does not appear — and for a change in a default, an icon tier, or a format string baked into an adapter’s defaults.

Skew also interacts with the diagnostic story in an unhelpful way. Chapter 17’s bisection tells you to determine first whether the daemon is in the picture, then whether the problem is config or render. Both of those steps assume the render path under test is the render path you installed. Under skew, the direct-render path and the daemon path are *different programs*, so `powerline-render` produces the new behavior and the client produces the old, and a reader following the bisection sees the daemon and the direct renderer disagree. That disagreement normally means something interesting about caching or context. Under skew it means nothing except that the two paths are different builds, and the correct next step is not to investigate the daemon’s state but to restart it.

That is worth elevating into a rule, because it saves a debugging session: **if the daemon path and the direct-render path disagree and the config is identical, restart the daemon before investigating anything else.** It is a one-line test that eliminates an entire class of cause for the cost of a couple of seconds.

Seeing which binary is actually serving you

The system does not report its own generation, so the observation has to come from the process table and the filesystem. The techniques differ by platform for the same reason everything in Appendix Q differs by platform.

On the filesystem platforms — macOS and the BSDs — the pidfile is the entry point, since the daemon writes its pid there under its lock during startup. Given the pid, the useful question is not “which binary is this” but “when did this process start,” compared against when the binary on disk was last written:

```
pid=$(cat /tmp/powerline-ipc-$(id -u).pid)
ps -o lstart= -p "$pid"
ls -l "$(command -v powerline-daemon)"
```

If the binary’s modification time is later than the process’s start time, the running daemon predates the installed binary and you have skew. This is a coarse test — it can be fooled by a package manager that preserves timestamps, and it says nothing about *which* older build is running — but it answers the only question that matters operationally, which is whether a restart is owed.

On Linux there is no pidfile, because Linux uses the abstract namespace and skips pidfile locking entirely, so the pid has to come from the socket or the process table instead. The abstract socket is visible in the UNIX socket listing but has no filesystem entry, which is why Appendix I’s confirmation step uses a socket listing rather than a path check. Once you have the pid, Linux offers the exact answer the timestamp comparison only approximates: the kernel maintains a symlink to the executable a process was started from, and when that file has been replaced the link reports it as deleted. A daemon whose executable link is marked deleted is definitively running code that is no longer installed.

Neither technique is elegant and neither needs to be, because both exist to answer a yes/no question that is asked rarely and whose remedy is cheap. The value of writing them down is that a reader who has not thought about skew does not know to ask the question at all, and the failure it produces does not announce itself.

--replace is the whole reload story, and it costs a couple of seconds

The port’s five daemon flags include exactly one that addresses this, and it is worth being precise about what it does. The replace flag loops a kill of the running daemon and a two-second sleep until the old daemon is gone, then starts fresh. That is the complete mechanism. It is a full process restart, so it rebuilds all three kinds of warm state, which means it is the only thing that picks up new code — and also, incidentally, the heaviest possible way to pick up new config, which the mtime check already handles for free.

Two properties of that loop deserve attention. First, it is not instantaneous: the sleep is a real wait, and a shell rc line that unconditionally replaces the daemon on every login is buying a multi-second stall in every new session in exchange for a guarantee it almost never needs. The rc line should start the daemon, not replace it; replacement is an operator action, not a session action. Second, the interval between the old daemon exiting and the new one binding is a window in which no daemon exists, and Chapter 4’s fallback is exactly what covers it — a client that connects during the gap fails to connect and execs the direct renderer instead. The prompt is slower for one render and correct throughout. This is the same resilience property that makes the daemon safe to depend on, doing quiet work in a situation nobody designed it for.

The absence of a signal-driven reload matters here in a way Chapter 13 did not need to emphasize. Upstream’s re-exec on hangup would have been the natural upgrade path — signal the daemon, it replaces its own image with the newly-installed executable, the socket stays bound and no client ever sees a gap. The port does not

implement it, which Chapter 13 frames as a lifecycle difference, and this is the concrete cost of that difference: the port's only route to new code is a restart with a visible gap, where upstream's was a re-exec with none. The gap is covered by the fallback and is therefore an availability blip rather than a failure, but "you must restart, and restart means a window" is the accurate description of the port's upgrade story, and a reader deploying powerliners in an rc file should know that the two-second sleep is the shape of it.

The upgrade rule, and where it belongs

Collecting the above into practice: after installing a new version of powerliners, restart the daemon. Not because anything is broken if you do not, but because until you do, the version you are running is the version you were running before, and every conclusion you draw from the prompt's behavior is a conclusion about the old build.

The interesting question is who should do it. Three answers, in decreasing order of attractiveness and increasing order of how often they are actually available.

The package manager is the right place in principle. A post-install step that replaces the running daemon puts the restart adjacent to the event that made it necessary, which is where lifecycle actions belong. It is also the answer least under the port's control, because it depends on the packaging channel — a package format with post-install hooks can do it, and a plain build-from-source or a binary drop cannot.

A user service manager is the next best. If the daemon runs under a supervisor rather than from a shell rc, restarting the unit is an ordinary operation the supervisor already understands, and the foreground flag exists precisely so the daemon can run that way without double-forking away from its supervisor. This is the deployment Chapter 14 already recommends for other reasons — a per-user session service, started and stopped as a unit — and version management is one more argument for it.

The user's hands are the fallback, and they are fine. Running the replace flag once after an upgrade is a two-second operation that a reader who has read this chapter will remember to do because they now know what it is for. What does *not* work is expecting the system to notice: the socket does not name a generation, the protocol cannot carry one, and the mtime check watches config rather than code. The daemon will keep serving the binary it was born with, faithfully and forever, which is exactly what a long-lived process is supposed to do and exactly why the restart has to come from outside it.

Chapter 26 – Geometry: Where Width Comes From, and Who Owns It

Appendix P runs the truncation loop on a narrow pane and shows a five-segment line becoming a three-segment line. It opens by saying that the loop is called with a width, and then it uses that width for everything. This chapter is about the sentence Appendix P skips: where does the number come from? The answer turns out to explain a set of behaviors that otherwise look arbitrary — why a prompt is briefly wrong after you drag a window edge, why the right side is harder to lay out than the left, why some targets ignore width entirely, and why a renderer that cannot see a terminal is nevertheless the correct design.

A renderer with no terminal

The daemon has no controlling terminal. It gave up the one it had during startup: the double-fork sequence detaches it from the session, and standard input, output, and error are all replaced with the null device before the accept loop begins. This is deliberate and it is what makes the daemon a daemon. It also means the daemon cannot answer, even in principle, the question “how wide is the terminal?” There is no terminal to ask. There is no descriptor to interrogate. The daemon does not know whether the machine has a display, let alone how many columns are on it.

The point generalizes past the daemon. Even the direct renderer, which does run in the foreground with the caller’s descriptors intact, is frequently rendering for a surface that is not its own standard output. A tmux statusline is not the width of the shell that invoked the client. A vim statusline is the width of a window inside a terminal, not of the terminal. A right-hand prompt occupies part of a line the left-hand prompt is also on. In every one of those cases the geometry of the render target is a fact only the *host* has, and asking the process’s own terminal would give the wrong answer even when there is one to ask.

So the design falls out. Width is not discovered, it is supplied. It arrives as an ordinary command-line flag on the render invocation, travels across the wire as one more argument in the NUL-separated argv, and lands in the argument parse alongside the exit code and the job count. It is exactly the same kind of input as the last exit code: a fact about the caller’s world that the callee cannot observe and the caller must therefore state.

This is worth naming as a principle because it is the same principle three other chapters arrive at from different directions. Chapter 4 derives the environment’s presence on the wire from the daemon’s inability to know the caller’s context. Chapter 21 derives locale’s treatment from the same place. Appendix AJ turns it into an obligation on adapters. Geometry is the fourth member of that family, and the only one that is not an environment variable — which may be why it is the one that goes unstated.

Width is not in the cache key, and it should not be

A detail in Appendix E.7 becomes interesting once geometry is on the table. The `PowerlineKey` selects which warm orchestrator serves a request. Its components are the extension, the renderer module, the three override channels, and three environment values. Width is not among them, and neither is the working directory, nor the environment at large.

That is the correct design and the reason is worth stating precisely, because “the cache key omits an input that affects output” sounds like a bug. It is not, because the key does not cache *output*. It caches the machinery:

the parsed config, the built colorscheme, the wired adapters, the renderer instance. Two requests that differ only in width need exactly the same machinery, and running that machinery against different widths produces different results without any of the machinery being different. A key that included width would partition the orchestrator cache by a value that changes every time a user drags a window edge, which would convert a cache that is warm essentially always into one that misses constantly, in exchange for nothing.

The general rule the key encodes: **cache what is expensive to build and invariant across requests; recompute what is cheap and varies per request.** Config resolution is expensive and varies only when files or overrides change, so it is keyed. Layout is cheap and varies per request, so it is not. Width sits firmly in the second category — it is consumed by a measure-and-drop loop that runs over an already-resolved segment list — which is why it can be a plain argument rather than a cache dimension.

Who knows the width, target by target

Because width is supplied, every binding has to answer the question for itself, and the answers differ in ways that explain the bindings' shapes.

tmux knows its own geometry exactly and it is the cleanest case. The statusline's available width is a property tmux computes from the client's terminal size and its own layout, and the conf files that Appendix V describes are what wire the client invocation into that context. The window-status format renders per window, which is why Chapter 20's many-windows problem exists; the left and right status render once per tick against the whole statusline's width. The important structural fact is that tmux is a program with a complete model of the surface it is drawing on, which is the ideal position for the party that owns geometry to be in.

A shell prompt is the awkward case, and it is awkward in a way that repays attention. The shell knows its terminal's column count, and the bindings of Appendix I forward the facts a prompt needs — the exit code, the pipe status, the job count. Width is the fact whose forwarding is *optional* in a way the others are not, because a prompt that is too wide does not fail: it wraps. A wrapped prompt is ugly and a wrapped prompt with a right-hand component is worse, but neither is an error, and so the natural default for a shell binding is to say nothing about width and let the terminal do what terminals do. The consequence is that priority truncation — the mechanism Appendix P calls the render feature most authors under-use — is not merely under-used on the shell path, it is frequently *not engaged at all*, because nothing supplied the width that triggers it. An author who has carefully assigned priorities across a prompt and never seen a segment drop should check whether the width is being passed before concluding the priorities are wrong.

vim knows the width of the window, which is not the width of the terminal, and the distinction is the whole reason the vim binding forwards per-request values through the renderer-argument channel rather than relying on anything ambient. A split window is narrower than the terminal; two vertical splits each get their own statusline and each has its own width; the numbers change on every split, close, and resize. This is the target where “the host owns geometry” is least arguable, because the host is the only party that can even enumerate the surfaces being drawn.

The bar targets do not have a width at all, and Chapter 7 already explains why in the language of layout: the i3bar protocol hands the host discrete blocks and the host decides placement, and the lemonbar target delegates positioning to position markers rather than counting columns. Geometry adds one observation to that. Since truncation lives in the shared layout core rather than in the encoders, a bar target still *runs* the truncation loop — Appendix P notes that a short lemonbar drops the same segments in the same order — but what it truncates against is a width somebody supplied, not a width the protocol implies. A bar render with no width supplied never drops anything and hands the bar program every block, which is usually right, because the bar program is the one that knows how much room there is.

The resize race

Now the behavior this chapter exists to explain. A terminal resize is an event delivered to foreground processes as a signal. The daemon does not receive it: it has no controlling terminal, so there is nothing to signal it about.

Appendix AH mentions the resize signal in passing, noting that the retry wrapper keeps such a signal from perturbing the accept loop. That is true and it is the smaller half of the story. The larger half is that the daemon has no notion of a terminal to be resized in the first place, so the signal is not merely handled harmlessly, it is *structurally irrelevant* — there is no state in the daemon that a resize could invalidate, because the daemon holds no geometry between requests. Every render is told its width and forgets it.

That statelessness is what makes the race benign and also what makes it visible. The sequence after a resize is: the host learns its new size, the host redraws on whatever schedule it uses, the redraw invokes the client, the client sends the new width, the render reflects it. Between the resize and the redraw, the display shows a line laid out for the old width. On tmux the interval is bounded by the status interval, so a resize followed by a wrong-looking statusline that corrects itself a second or two later is exactly the expected behavior and not a bug. On a shell prompt the interval is unbounded, because nothing redraws a prompt until something asks it to — resize a terminal and stare at an idle prompt and it will stay as it was until you press Enter.

The asymmetry between those two is worth internalizing, because it produces one genuinely confusing symptom: after a resize, the tmux statusline is correct and the prompt above it is wrong, on the same screen at the same instant, and both are behaving as designed. The statusline is on a timer and the prompt is on an event, and the event has not happened. Nothing is stale in the daemon; what is stale is the pixels the host has not repainted.

The reader-facing rule is short. Geometry problems that resolve themselves on the next redraw are not problems. Geometry problems that persist across a redraw are, and the first question for those is whether the width being supplied is the width of the surface being drawn on — which, given how many surfaces are not the terminal, is the assumption most likely to be wrong.

Two parties, two ways to be wrong

The last piece connects this chapter to Chapter 19, and it is the reason both chapters are necessary rather than one being a section of the other.

Chapter 19 is about *measurement*: given a string, how many cells will the terminal spend drawing it, and why can no producer answer that with certainty. This chapter is about *budget*: how many cells are available, and who says so. The truncation loop compares those two numbers, and they are computed by different parties from different information. The budget comes from the host — tmux, the shell, vim — and describes a surface. The measurement comes from the renderer and describes a string. Neither party checks the other.

So a line can be laid out wrongly in two independent ways, and the distinction is diagnostic. If the budget is wrong — the host reported a width that does not match the surface, or nobody reported one — the line is the wrong *length*: it wraps, or it truncates when it did not need to, or it never truncates when it should. If the measurement is wrong — a glyph the renderer counted as one cell that the terminal draws as two — the line is off by a small number of cells, the dividers do not meet, and the error scales with how many unusual glyphs are on the line. The symptoms are different enough to tell apart at a glance: a length error is dramatic and a measurement error is a couple of cells of drift, and knowing which you are looking at tells you whether to go read the binding or go read the theme's glyph choices.

That both classes exist, that neither is detectable by the renderer, and that each has a different owner is the honest summary of geometry in a program that draws lines for surfaces it cannot see. The design's answer is the same one it gives for encoding and for width measurement: state the contract, put the responsibility with the party that has the information, and be precise in the documentation about where the program's knowledge stops.

Chapter 27 – The Clock: Time as an Input to a Long-Lived Process

The two preceding chapters took inputs the daemon cannot observe — its own generation, the geometry of the surface it draws on — and traced the consequences. There is a third, and it is the one with the most interesting failure modes, because unlike the other two it is an input the process *can* read at any moment and still be wrong about. The clock is available to the daemon. It is also not one thing, it does not always move forward, and a process that lives for weeks will observe it doing things a process that lives for milliseconds never sees.

Chapter 20 examines what happens when a *quantity* changes between two observations. This chapter examines what happens when the *clock itself* is unreliable — when the interval between two observations is not what a naïve subtraction says it is, or when a boundary the display depends on is crossed in a way the refresh schedule cannot show. The two chapters share a subject and do not overlap: Chapter 20's sampler is correct and the world moved; here, the world may have been still and the timekeeping moved.

Two clocks doing two jobs

Every statusline that shows time is using the clock for at least one of two purposes, and they want opposite properties.

The display purpose wants wall-clock time: what a human's watch says, adjusted for the local zone, including whatever discontinuities civil time inflicts on itself. A date segment must show the time that is *correct socially*, not the time that has elapsed. Wall clock is authoritative for this and is the only acceptable source, discontinuities included, because the discontinuities are the point — the whole purpose of a civil clock is to agree with other people's clocks.

The elapsed purpose wants a monotonic clock: a counter that only increases, is unaffected by zone changes, is unaffected by an administrator setting the date, and is unaffected by time synchronization stepping the system clock. Chapter 20 establishes that every rate metric is defined by two counter readings and the elapsed time between them, and “the elapsed time between them” is precisely the quantity that a wall clock computes wrongly whenever it moves for a reason other than time passing.

Getting these backwards produces a specific, memorable bug: a utilization percentage that is correct all year and produces one absurd reading at the instant a zone transition or a clock adjustment happens, because the divisor in a rate calculation briefly became an hour, or a negative number, or a value close enough to zero to send the result to infinity. It is a bug that survives review easily, because the wrong code and the right code look identical except for which clock they read, and it is invisible in testing unless the test steps a clock.

Zone is request context, and the twist locale does not have

Time zone is carried in the environment, so everything Chapter 21 establishes about locale applies to it without modification: the daemon's own zone is whatever the login shell that started it had, it may be days stale, and the render must honor the requester's zone rather than the process's. That is the same argument and this chapter does not repeat it.

What zone adds is a second, independent channel. The date and time segments accept a zone argument in the theme, which means a configured zone can override the environment's. That possibility does not exist for locale in the same form, and it changes the reasoning, because a fact that can arrive through two channels needs a stated precedence and a stated purpose for each.

The purposes are genuinely different and both are legitimate. The environment channel answers “what time is it *here*,” where here is wherever the requesting shell thinks it is — which is the right answer for the primary clock on a prompt, and the reason it must be request context rather than process context. The configured channel answers “what time is it *there*,” where there is a fixed place the author chose: a second clock showing the zone of a team, a production region, a home time zone while travelling. That segment must *not* follow the requester's environment, because following it would defeat the entire purpose of configuring a fixed zone.

So a theme with two time segments has one that should track the caller and one that should ignore it, and both are correct. The rule is the same one Chapter 21 reaches for locale and subprocess probes, in a new instance: a value that exists to reflect the user follows the request, and a value that exists to be stable is pinned. Both are configured in the same file, next to each other, and the difference between them is which channel supplied the zone.

Suspend, resume, and the daemon that was asleep

Here is the case that a short-lived process can never encounter and a warm daemon encounters routinely. A laptop closes at the end of a day and opens the next morning. The daemon does not exit — nothing sent it a signal, nothing closed its socket, and it was blocked in its poll loop with a long timeout when the machine stopped executing instructions. It resumes into a world where every temporal assumption it might have been carrying is hours out of date, and its first render after the lid opens is the interesting one.

Take the categories in turn. Process-level state is completely unaffected: the executable is still mapped, the socket is still bound, the connection table is still empty because it is always empty between requests. Configuration state is unaffected and correct, because the staleness check compares modification times rather than an interval — a theme edited during the suspend has a modification time later than the recorded one, and the comparison notices that regardless of how much time passed on either side. This is a case where the pull-based, mtime-keyed design of Chapter 13 is quietly better than a time-to-live cache would have been: a cache expiring after a fixed interval would treat a nine-hour suspend as nine hours of expiry and rebuild everything for no reason, and a cache that recorded a deadline in wall time would rebuild or fail depending on which direction the clock moved. Comparing timestamps of files rather than measuring elapsed time makes the whole question moot.

What is not fine is any rate computed in the differencing arrangement Chapter 20 describes — a previous reading held from before the suspend, differenced against a reading taken after — which produces a number describing an interval that contains a long period during which the machine was not running. Chapter 20 notes that the differencing approach makes a number depend on refresh rate; suspend is the extreme case of that dependence, and the reading on the first render after resume is not a slightly-off number but a meaningless one. It corrects itself on the following render, since the interval after that contains only running time, so the visible symptom is a single implausible value immediately after a resume that disappears before anyone can investigate it. Knowing that is the whole remedy: it is not a bug, it does not recur, and it is the arithmetic of a differencing sampler meeting an interval that was mostly not happening.

The poll timeout deserves a brief note in the same context. The accept loop waits with a long timeout — a full minute — which exists to give the loop a periodic wakeup rather than to schedule anything. Across a suspend that timeout is not a promise about anything: the wait ends when the machine resumes and the kernel gets around to it, not sixty seconds after it started. The loop does not care, because it does no work on timeout expiry other than going around again. A loop that had used its timeout as a timer — to expire caches, to refresh data, to schedule housekeeping — would have inherited every problem in this section, and the fact that the port's loop does nothing on expiry is what keeps suspend from being a lifecycle event at all.

Steps, and the direction the clock is not supposed to move

Suspend is the common discontinuity; there are others, and they arrive without warning. A time synchronization daemon correcting a drifted clock steps it. An administrator setting the date steps it. A virtual machine restored from a snapshot resumes with a wall clock from whenever the snapshot was taken. A container starting with a different zone configuration than its host shows a different civil time for the same instant.

For display purposes all of these are fine and none of them need handling: the prompt shows whatever the clock says, and if the clock was wrong and got corrected, the prompt showing the corrected value is exactly right. The correction may make the displayed time appear to jump backward, which is startling, and is also the truthful rendering of a clock that was ahead.

For elapsed purposes they are the failure modes the monotonic clock exists to prevent, and they include the one case that arithmetic does not survive: a backward step makes a naive wall-clock difference *negative*. A rate computed with a negative divisor is not a large number or a small number, it is a sign error, and the visible result is a percentage that is negative or wildly out of range. This is the sharpest available argument for the two-clock discipline, because it is the one where using the wrong clock does not degrade the answer, it inverts it.

Because a prompt renderer is not a monitoring system, none of this needs elaborate handling — the correct posture is that display reads the wall clock and takes it at face value, elapsed reads a monotonic source, and no code anywhere subtracts two wall-clock readings to learn how much time has passed. That is a short rule and it covers every case in this section.

Boundaries, ticks, and the values that change on an instant

The last piece is a display problem rather than a correctness one, and it is specific to time in a way no other segment shares.

Most segments show a quantity that drifts. A time segment shows a value that changes *on an instant* — at the top of a minute, at midnight, at the moment a fuzzy description stops being “quarter past” and starts being “twenty past.” The display is refreshed on a schedule that has no relationship to those instants, and the interaction produces artifacts that look like bugs to anyone who has not thought about it.

A statusline refreshing on an interval that does not divide evenly into a minute shows each minute for a slightly variable number of ticks, and a minute boundary can be displayed late by up to one refresh interval. A prompt is worse, because a prompt’s refresh is an event, not a schedule: the time in a prompt is the time at which that prompt was rendered, and it stays on screen unchanged for as long as you leave it there. A user who reads the time off an old prompt is reading a timestamp, not a clock — which is arguably the more useful thing to have, since it tells you when your last command finished, but it is not what the segment appears to be offering.

The fuzzy variant sharpens this. A segment that renders time as prose has coarse buckets, and coarse buckets change rarely, so a fuzzy time segment is stable for long stretches and then changes all at once. Combined with a refresh that is not aligned to bucket boundaries, the transition lands wherever it lands, and the display can sit one bucket behind for a while without anything being wrong. This is the honest reason a fuzzy time segment belongs on a timer-refreshed statusline and not on an event-refreshed prompt: its value is only meaningful if it is being refreshed on a cadence finer than its own resolution, and a prompt has no cadence at all.

Which leads to the structural fact this chapter closes on. **Nothing in powerliners schedules anything.** The daemon has no timer, no background refresh, no periodic task, and no thread waking up to re-sample anything. The accept loop’s timeout does no work. Every clock read in the entire system happens because a request arrived, and the request arrived because a host — tmux on its interval, a shell on its hook, an editor on its

event — decided it was time to redraw. The refresh cadence of every time-varying value on your statusline is therefore entirely a property of the host's configuration and not of powerliners at all, which is why the answer to “make this segment update more often” is always a change to a status interval or a hook, never a change to a theme. A purely reactive renderer has no opinion about how often it is asked, and holding that line is what keeps the daemon's cost proportional to the work it is actually asked to do rather than to the time it spends existing.

Chapter 28 — Where powerliners Fits

A closing chapter should place the program in its world, and powerliners' world is two overlapping ones: the powerline ecosystem it descends from, and the MenkeTechnologies stack it belongs to. Understanding it means seeing it in both.

In the powerline ecosystem, powerliners is the answer to a question the ecosystem has lived with for a decade: how do you keep everything powerline gives you — the segmented, colored, context-aware prompt and statusline, and the whole accumulated culture of themes and colorschemes — without paying Python's startup tax on every render? Every prior answer traded something away. Caching helped but did not eliminate startup. Simpler prompts were faster but gave up the features. Rewrites in other languages usually meant a new config format and a migration that abandoned your themes. powerliners' answer is to change nothing you can see and everything you cannot: the same themes, the same grammar, the same wire protocol, the same rendered bytes, running in a native daemon instead of a per-render interpreter. It is a port, not a replacement, and that is precisely why it can drop into a decade-old config without a single edit.

The honesty about that lineage is not a hedge; it is what makes the originality claim survivable. The MenkeTechnologies project bar kills “X but faster” and “X but in Rust” as products, and a faster powerline would die on that bar. powerliners survives it because the faster render is the *floor*, not the pitch — the pitch is the engineering that made a faithful port into a deployable native suite: the wire-compatible client, the double-forking daemon with its abstract-namespace socket, the parity harness that diffs against the live interpreter and caught eleven real bugs, the tier classifier and drift gate that keep the fidelity claim auditable, and the new native segments that extend the library beyond what upstream offers. The segment model is inherited and this book says so on every page; the machine that runs it at native speed, verified against the original, is the original work.

In the MenkeTechnologies stack, powerliners is the prompt and statusline layer, and its extension segments are where that placement becomes concrete. The cache-stat segments for `fusevm`, `zshrs`, `stryke`, and `awkrs` let the prompt report on the health of the very tools the user is running — a fusevm JIT cache size, an awkrs bytecode archive — which is a capability that only makes sense because those tools share an author and an ecosystem. The prompt that powerliners draws is the frame around a session that is likely running `zshrs` as its shell, `zterminal` as its terminal, and `awkrs` in its pipelines; powerliners is the thin, fast, always-present line at the edge of all of it, and its stack-aware segments are how it participates in the ecosystem rather than merely decorating it.

What powerliners finally is, stated plainly, is a demonstration that “faithful port” and “original engineering” are not opposites. The faithfulness is total where it matters — the markup, the colors, the layout, the config resolution, the wire protocol are byte-identical to upstream, proven against the live interpreter — and the originality is real where it counts — the daemon architecture, the native binary suite, the verification harness, and the new segments are work that did not exist before. The Python powerline authors built the model and the culture; powerliners carries both forward, unchanged where a user can see and rebuilt where a user cannot, so that the prompt you spent years tuning renders in under a millisecond and looks exactly the same. That is the whole of it: same prompt, no Python, sub-millisecond, proven. Prompt locked, native.

Chapter 29 – The Three NEAR Files, Opened

Chapter 18 states the port’s status as a ratio — 134 of 137 upstream files at DONE, three at NEAR — and describes the three as classifier artifacts on files with no behavior left to port. That is a summary, and a summary is exactly the kind of claim this book is supposed to distrust. This chapter opens the three files, reads the classifier that scored them, and reports what is in each one, what is missing from each one, and what the tier actually certifies. Two of the three are what the summary says they are. The third is not, and the difference is instructive rather than embarrassing: it is a demonstration that a file-granularity metric measures what it measures and nothing else.

The classifier’s arithmetic, quoted

The tiers come from `scripts/gen_port_checklist.py`, and the whole decision is one function. Its shape is worth reading verbatim rather than paraphrased, because the branch that produces all three NEAR tags is the *first* one a class-only file hits, before any density arithmetic runs at all:

```
def classify(stat: FileStat) -> str:
    total_fns = stat.py_fns + stat.py_methods
    if total_fns == 0 and stat.py_classes == 0:
        return "DONE" if not stat.rs_is_stub else "STUB-HEAVY"
    if stat.rs_is_stub:
        return "STUB-HEAVY"
    if total_fns == 0:
        return "NEAR" if stat.rs_port_docomments >= stat.py_classes else "STUB-HEAVY"
    citation_density = stat.rs_py_citations / max(1, stat.py_loc)
    port_ratio = stat.rs_port_docomments / max(1, total_fns)
    if not stat.rs_is_stub and citation_density >= 0.5 and port_ratio >= 1.0:
        return "DONE"
```

Read the third branch closely. A file whose Python side declares at least one class and zero functions *and* zero methods returns from that branch unconditionally. The best available outcome is NEAR; DONE is not reachable, not because the ratio fails a threshold but because control flow never reaches the line where DONE is returned. Chapter 18 describes this as a 0/0 ratio that cannot be compared to the 0.5 threshold, which is the right intuition stated slightly wrong — the ratio is never computed. The tag is decided by the *shape* of the Python file, and the only thing the Rust side can influence is whether it carries at least as many `/// Port of` doccomments as the Python has classes.

That branch is what the three NEAR rows have in common. Their measured columns, read out of `docs/PORT_CHECKLIST.md` — Python LOC, then functions (*fns*), classes (*cls*) and methods (*meth*), then Rust LOC and the two doccomment counts. Every path in the Python file column is relative to `powerline/`:

Python file	Py LOC	Py fns	Py cls	Py meth	Rs LOC	/// Port of	// py:
<code>renderers/shell/readline.py</code>	8	0	1	0	22	4	4

Python file	Py LOC	Py fns	Py cls	Py meth	Rs LOC	/// Port of	// py:
<code>renderers/shell/zsh.py</code>	9	0	1	0	29	5	4
<code>bindings/i3/powerline-i3.py</code>	34	0	1	0	40	4	24

Every one of them: zero functions, zero methods, one class. Same branch, same outcome, three different underlying realities.

What DONE certifies, and what it does not

Before opening the three, it is worth being precise about the tier they failed to reach, because the neighboring file makes the point better than any argument. `renderers/shell/bash.py` sits at DONE with 76 Python LOC, one class, one method, and a Rust port carrying 6 `///
Port
of` doccomments and 80 `//
py:` citations — a citation density above 1.0, comfortably past the 0.5 bar. Now read the method the port claims. `BashPromptRenderer::do_render(src/ported/renderers/shell/bash.rs)` carries the entire upstream body as `// py:` comments, line for line, from the cursor-save escape through the right-prompt embedding, and both of its return paths are `String::new()`. Its own doccomment says so: “**Status:** stub.”

That is not a scandal and it is not hidden — the file announces it in the sentence above the function. It is a fact about the metric. The DONE tier measures *citation and doccomment density*, which is a measure of how thoroughly the upstream source has been transcribed and attributed, not a measure of whether the transcribed lines were also executed. The port’s own rules are consistent about this: Rule 6 in `docs/PORT_CHECKLIST.md` requires a `// py:NNN` citation on every line that mirrors a Python statement, and a file that mirrors every statement as a comment satisfies Rule 6 maximally. Behavior is certified elsewhere — by the parity harness (Chapter 30), by the daemon scenarios, and by the library tests — and the two certifications are independent. A reader who takes “134/137 DONE” as “134 files behave identically to upstream” is reading a documentation metric as a behavioral one.

The consequence for this chapter is a mild inversion. `bash.rs` is tier DONE with an unexecuted body; `zsh.rs` is tier NEAR with every line of its upstream file fully present and live. The tier ordering does not track completeness for these files, because completeness is not what the tier measures.

`readline.rs` and `zsh.rs`: complete, live, and unpro-motable

Upstream’s `readline.py` is a class with two string constants and a module-level alias. The port (`src/ported/renderers/shell/readline.rs`) holds `escape_hl_start = "\x01"`, `escape_hl_end = "\x02"`, a `character_translations()` returning an empty map, and `pub type renderer = ReadlineRenderer` for the module-level `renderer = ReadlineRenderer` binding. There is nothing else in the Python file. The port is complete in the strict sense: every name upstream defines exists, with the same value.

`zsh.py` is the same shape with `%{ / %}` and one translation entry, `'%' → '%%'`, and the port holds all three. The evidence that this is live code rather than a data museum is one directory over: `src/ported/renderers/shell/tcsh.rs:24` defines its own `escape_hl_start` as `ZshPromptRenderer::escape_hl_start`, and `tcsh.rs:35` builds its translation table by starting from `ZshPromptRenderer::character_translations()` and adding the tcsh-specific entries — which is exactly what upstream’s `TcshPromptRenderer(ZshPromptRenderer)` does with its parent’s table. The NEAR-tagged file is the base of an inheritance chain whose derived file is tagged DONE.

Both files are also pinned by tests that do not need Python at all. The parity harness carries `parity_readline_renderer_escape_markers`, `parity_zsh_renderer_escape_markers`, and `parity_ksh_renderer_escape_markers` among the 116 tests that assert without calling out to the interpreter (Chapter 30 counts them), and each renderer file carries its own in-module tests — `zsh_escape_markers_use_prompt_subst_safe_form` and `percent_is_doubled_in_translations` in `zsh.rs`, `escape_markers_match_upstream_readline` in `readline.rs`. For these two files, Chapter 18’s characterization holds exactly: there is no behavioral surface left, and the tag is an artifact.

powerline-i3.rs: the file the compiler never sees

The third file is different, and the difference is not visible from the checklist row.

`src/ported/bindings/i3/powerline-i3.rs` is the only file in its directory. There is no `mod.rs` beside it, and `src/ported/bindings/mod.rs` declares eight child modules — `config`, `ipython`, `pdb`, `qtile`, `tmux`, `vim`, `wm`, `zsh` — and does not declare `i3`. A grep for its public names across the rest of the source finds exactly one hit, and that hit is a comment in `src/ported/bindings/wm/mod.rs:48` mentioning the upstream script by name. The file is not in the crate’s module tree. It does not compile; its `use crate::ported::bindings::wm::{...}` line is never type-checked; and its three `#[cfg(test)]` tests never run, which is confirmable by listing the library’s tests and finding no `i3powerline` entry among them.

There is a defensible reason for the layout. Upstream’s `powerline-i3.py` is a *script*, not an importable module — it is invoked as a program and its filename contains a hyphen, which is not a legal Rust module name either. The port’s mirror discipline says every upstream `.py` gets a same-stem `.rs` at the same relative path, and this file honors that rule at the cost of being unreachable from `lib.rs`. What it does not have is the other half: `src/bin/` contains `powerline.rs`, `powerline-daemon.rs`, `powerline-config.rs`, `powerline-render.rs`, `powerline-lint.rs`, and the `shared/` module directory, and no `powerline-i3.rs`. The port’s own doccomment states the intent — “The Rust analog is `src/bin/powerline-i3.rs` (TBD)” — and that binary does not exist in this tree.

So the file’s contents are worth stating precisely rather than summarizing. It holds `I3Powerline::default_log_stream = "stderr"`, a faithful port of the class attribute that redirects logging away from the stdout channel i3bar reads. It holds a `render()` function that takes the rendered string as a closure result and returns `format!("{}",[{ }]", trimmed)` after stripping one trailing newline — a correct port of the inner closure’s `print(',', '[' + powerline.render()[:-1] + '']')`. And it holds a `main()` whose doccomment says “**Status:** stub”, which prints the three i3bar protocol header lines, calls `get_i3_connection()` and `i3_subscribe(...)` with placeholder arguments, and then *returns before the render loop*, with a comment saying so. Chapter 18 says the file’s `render` function “is already 100% ported,” which is true of `render` and not true of the driver around it: the i3bar event loop, the interval sleep, the lock, and the `Powerline` instantiation are all present as citations and absent as code.

The honest one-line status for this file is therefore not “classifier artifact” but “mirror placeholder”: the i3bar driver is not shipped, no binary provides it, and the source that describes it is outside the build. That is a real gap of exactly one upstream script, and naming it costs nothing, because i3bar output is also reachable through the `i3bar` renderer (`src/ported/renderers/i3bar.rs`, tier DONE) driven by the ordinary render path.

The audits see a file the compiler does not

There is a second-order observation here that generalizes past this one file. The port has two kinds of verification: things the compiler and test runner do, and things a script does by walking the filesystem. The drift gate (`tests/ported_fn_names_match_py.rs`) is in the second category — it walks `src/ported/` with `WalkDir` and checks every `fn` name it finds against `docs/powerline_py_functions.txt` and the allowlist. The tier classifier is in the second category too. Both of them read `powerline-i3.rs`; the compiler does

not. Its 4 `Port of` doccomments and 24 `py:` citations count toward the checklist row above, and its function names are checked by the gate, and none of that involves the file being valid Rust.

This is not a flaw in either tool — a filesystem walk is the correct implementation for both — but it is a boundary a reader should know about, because it is the one place where a documentation metric and a build artifact can disagree without anything failing. The generic form of the rule: **a metric computed by walking source files measures files, and the set of files is not the same as the set of compiled units.**

`powerline-i3.rs` is not the only row on the wrong side of that boundary; it is the only one that also lands in the NEAR tier, which is what made it visible from the checklist. The same `src/ported/bindings/mod.rs` that omits `i3` also omits `bar`, `lemonbar`, and `awesome`, and each of those directories holds a same-shaped hyphenated file that no `mod` declaration can name and no `[[bin]]` target builds. Chapter 18 tabulates all four with their citation counts and tiers; the short version is that three of the four are tagged DONE and are therefore inside the 134, and the 11 `#[test]` functions the four contain do not appear among the 2497 the library runs. So the two sets coincide for 133 of 137 files, not 136 — a correction this chapter's own method produces when it is run over the whole `bindings/` directory instead of the one row that happened to be NEAR.

The classifier cannot currently be re-run

One more measured fact, because Chapter 10 and Appendix F both lean on the classifier's re-runnability as the reason its numbers cannot silently drift. In this tree it does not run:

```
$ python3 scripts/gen_port_checklist.py | sed -n '/^## Tier counts/,/Total/p'
## Tier counts

| Tier | Count |
|---|---|
| DONE | 0 |
| NEAR | 0 |
| PARTIAL | 0 |
| SPARSE | 0 |
| STUB-HEAVY | 0 |
| **Total** | **0** |
```

The script walks `vendor/powerline/powerline`, and the vendored upstream checkout was removed from the repository — the commit that did it deleted 336 files and 37,049 lines. `docs/PORT_CHECKLIST.md` is therefore a frozen artifact: accurate as of the tree that had a vendor directory, regenerable by anyone who restores one, and not regenerable in place. The 134/3/0/0/0 figures in this book are read *out of that file*, not recomputed, and a reader who wants them recomputed needs the same upstream checkout the parity harness needs — which is the subject of the next chapter.

Chapter 30 – The Parity Harness Without an Interpreter

The parity harness is the load-bearing claim of the whole project. Everything else — the tier counts, the drift gate, the library tests — verifies the port against itself or against a list. Only the harness verifies it against the thing it is a port of, by running the actual Python and diffing. Appendix F describes how it is built. This chapter measures what it does in the tree as it stands, which is a different question and has a different answer, and then states the rule that follows.

The shape of the file

`tests/parity_against_upstream.rs` is 13,734 lines. Counted mechanically:

Quantity	Count	How counted
<code>#[test]</code> functions	462	<code>grep -c '^#[test\]'</code>
Lines containing <code>assert</code>	897	count of matching lines
<code>assert_eq!</code> call sites	738	<code>grep -c 'assert_eq!'</code>
<code>py_eval</code> call sites	368	<code>grep -c 'py_eval('</code>
<code>python_available()</code> call sites	349	count of matching occurrences
Tests whose body mentions <code>py_eval</code>	346	split on <code>#[test]</code> , test each chunk
Tests that never call <code>py_eval</code>	116	the remainder

The figure this book and the README have carried as “462 parity assertions” is a count of test *functions*, not of assertions; there are 897 lines carrying an assertion inside them. Both numbers are real and they measure different things, and the distinction matters for the next section, because the unit that can skip is the test function, not the assertion.

The 116 tests that never call `py_eval` are worth naming as a category, because they are the part of the file that is not, strictly speaking, parity testing. They pin values the porter transcribed from upstream by hand and then froze: the readline, zsh, and ksh escape markers of Chapter 29; the `ScalarNode` / `SequenceNode` / `MappingNode` `id` class attributes; the `mergedicts` and `mergedefaults` semantics; the config-layout invariants. They assert the same constants a Python-backed test would assert, without asking Python. They are unconditional, and they run everywhere.

Three outcomes, not two

The harness's helper is `py_eval` (`tests/parity_against_upstream.rs:22`), and its first act is not to spawn Python. It is to check for the vendored checkout:

```
fn py_eval(expr: &str) -> Option<String> {
    let repo = repo_root();
    let vendor = repo.join("vendor").join("powerline");
    if !vendor.exists() {
        return None;
    }
}
```

`None` means “no answer available,” and every call site treats it the same way — print a diagnostic to `stderr` and `return` from the test. The test then ends without asserting, and the test runner reports it as passing, because a Rust test passes by not panicking. The same is true of the second guard, `python_available()` (:51), which returns early when `python3` cannot be spawned at all.

That gives a parity test three outcomes where a normal test has two: it can pass having compared two implementations, it can fail having found them different, and it can pass having compared nothing. The third outcome is indistinguishable from the first in the summary line. Measured on this tree:

```
$ cargo test --test parity_against_upstream
running 462 tests
test result: ok. 462 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  1.14s
```

Four hundred and sixty-two passing tests, and 346 of them compared nothing, because `vendor/powerline` does not exist in this tree. The diagnostic is printed but captured by default; asking for it makes the situation visible:

```
$ cargo test --test parity_against_upstream -- --nocapture parity_version
running 2 tests
test parity_version_get_version_falls_back_to_constant ... ok
parity_version_constant: skipped (vendor not present)
test parity_version_constant ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 460 filtered out; finished in
  0.02s
```

The wall-clock time is the other tell, and it is the more reliable one because it needs no flags. Three hundred and forty-six subprocess spawns of a Python interpreter, each importing the `powerline` package, cannot complete in 1.14 seconds. A suite that runs the interpreter is slow in a way that is obvious from the summary line alone; a suite that skips it is suspiciously fast.

The daemon suite says it out loud

`tests/daemon_parity.rs` is the sharper of the two harnesses — it runs upstream’s `powerline-render` and the Rust daemon against the same fixture and diffs the response bytes — and it handles the same absence more loudly:

```
$ cargo test --test daemon_parity -- --nocapture
running 45 tests
[skip] python3 + vendor/powerline not available for scenario_all_attrs
[skip] python3 + vendor/powerline not available for scenario_before_after
[skip] python3 + vendor/powerline not available for scenario_both_sides
[warn] python3 + vendor/powerline NOT available - parity_* tests will silently skip
...
test result: ok. 45 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  0.03s
```

The suite emits its own warning that it is skipping — which is the project being honest with itself at the only moment the honesty is useful — and finishes 45 end-to-end daemon scenarios in 0.03 seconds, which is roughly the time it takes to not do anything 45 times.

The fixtures those scenarios use are real and remain useful whether or not Python is present. `tests/data/e2e/` holds 44 scenario directories, each a self-contained config tree — a `config.json`, a `colors.json`, a `colorschemes/<ext>/default.json`, and a `themes/<ext>/default.json`. The 45 daemon-parity tests reference 42 distinct scenarios, three of them twice (`scenario_hostname`, `scenario_both_sides`, `scenario_width_drop`, each exercised from two directions), and the two scenarios no parity test names — `scenario_default_colors` and `scenario_full` — are used by `daemon_e2e.rs` instead. Chapters 32 and 35 render several of them directly, which is a use for the fixtures that needs no interpreter at all.

What survived the vendor removal

The vendored checkout is gone, but the port did not lose all of its upstream oracles, because three of them had already been extracted into text files that live in `docs/`:

Artifact	Lines	Role
<code>docs/powerline_py_functions.txt</code>	631	every upstream function name — the drift gate's allowlist
<code>docs/powerline_py_functions_with_locations.txt</code>	1222	the same names with their <code>file:line</code> origins
<code>docs/powerline_py_classes.txt</code>	182	every upstream class name — Rule 1's oracle
<code>tests/data/fake_fn_allowlist.txt</code>	1971	maintainer-approved exemptions with citations

Those four files are why the drift gate still means something in a tree with no Python in it:

```
$ cargo test --test ported_fn_names_match_py
running 2 tests
test allowlist_files_are_well_formed ... ok
test every_ported_fn_name_exists_in_python_allowlist ... ok
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
  0.16s
```

The gate walks `src/ported/`, extracts every non-test `fn` name, and checks each against the 631-name upstream list or the exemption file. It is a name-level check and it never claimed to be more, but it is a *live* check: it reads real files, computes a real answer, and can fail. It is the one piece of upstream-derived verification that the vendor removal left fully operational, and its survival is the direct consequence of someone having flattened the oracle into a text file rather than reading it out of the checkout every time.

The rule this produces

Collecting the measurements into a statement a reader can carry:

A green parity suite is evidence of parity only if the interpreter was present when it ran. The suite's own summary line does not distinguish the two cases, so the summary line is not the evidence — the runtime is, the `--nocapture` diagnostics are, and the presence of `vendor/powerline` is. On a machine or a CI job with the checkout in place, 462 tests exercise 346 live comparisons against Python and the number means what Appendix F says it means. On a tree without it, the correct reading of “462 passed” is “462 tests are available, 116 of them asserted, and the other 346 are waiting for an interpreter.”

This is not a defect to be fixed by making the skips fail. A test that hard-fails when Python is missing would make the suite unrunnable for anyone who installs the crate from the registry, where the vendor directory is excluded from the package by design — `Cargo.toml`'s `exclude` list names `vendor/**` along with `tests/**`, `docs/**`, `scripts/**`, and `.github/**` — and it would fail on any CI runner without the interpreter. Skipping is the right behavior for a suite that must run in both worlds. What skipping cannot do is announce itself in the place people read, so the announcement has to live here: check for the checkout, or read the clock.

Chapter 31 – The Request Frame, Byte by Byte

Chapter 4 tells the daemon's story and Appendix E specifies its protocol. This chapter does neither. It captures a real frame off a real socket, decodes every byte of it against the code that wrote it, and then feeds the daemon a series of frames that are wrong in different ways to find out what it says back. The protocol is small enough that this is exhaustive rather than illustrative, and the failure behaviors are the part no specification states.

One frame, captured

The capture technique is a listener that is not the daemon: bind a UNIX socket, accept one connection, read until the buffer ends in a double NUL, and dump it. Running the real client against that listener from `/tmp`, with a deliberately small environment so the frame fits on a page:

```
$ env -i PATH=/usr/bin:/bin TERM=xterm powerline --socket /tmp/pl-sniff2.sock shell left
~ --jobnum 1
total bytes: 69
00000000 34 00 73 68 65 6c 6c 00 6c 65 66 74 00 2d 2d 6a |4.shell.left.--j|
00000010 6f 62 6e 75 6d 00 31 00 2f 70 72 69 76 61 74 65 |obnum.1./private|
00000020 2f 74 6d 70 00 50 41 54 48 3d 2f 75 73 72 2f 62 |/tmp.PATH=/usr/b|
00000030 69 6e 3a 2f 62 69 6e 00 54 45 52 4d 3d 78 74 65 |in:/bin.TERM=xte|
00000040 72 6d 00 00 00                                |rm...|
fields: ['4', 'shell', 'left', '--jobnum', '1', '/private/tmp', 'PATH=/usr/bin:/bin',
~ 'TERM=xterm']
```

Sixty-nine bytes for a complete render request. Decoded against the marshaller in `src/bin/powerline.rs`:

- `34 00` — the ASCII digit `4` and a NUL. This is `format!("{:x}", argv.len() - 1)` (`powerline.rs:104`), the count of arguments *after* `argv[0]`, in lowercase hexadecimal text. Four here: `shell`, `left`, `--jobnum`, `1`. A client passing sixteen arguments would write `10`, two bytes of text, which is why the field is hex rather than binary — it is human-readable in a dump and needs no byte-order agreement.
- The four argument fields, each written by its own `write_all` followed by its own `write_all(b"\0")` in the loop at `powerline.rs:119-127`. The daemon does not care that they arrive as separate writes; the stream is a stream. It is worth noticing anyway, because it means a request with a large environment is a large number of small writes rather than one vectored one — a faithful mirror of the C client's loop, and a place where the port declined to be cleverer than its original.
- `2f 70 72 69 76 61 74 65 2f 74 6d 70 00` — the working directory, from `std::env::current_dir()` written as raw `OsStr` bytes (`powerline.rs:129-133`). No UTF-8 validation happens here in either direction, which is what makes a path with undecodable bytes survive the round trip.
- The environment, one `KEY=VAL` field per variable, assembled byte-wise with an explicit `=` and a trailing NUL per entry (`powerline.rs:134-143`). The frame carries exactly the two variables the invocation had.
- `00 00` — the terminator (`powerline.rs:145-147`). The last three NULs in the dump are the final environment entry's own terminator followed by the double NUL, which is why a well-formed frame always ends in three consecutive zero bytes when it carries at least one environment variable.

The reply direction has no framing at all: the daemon writes markup bytes and closes, and the client copies whatever arrives to stdout in 4 KiB chunks ([powerline.rs:150-163](#)) until it reads zero. This asymmetry is the protocol's whole error-handling budget, and the next section is about how it spends it.

What the daemon says to frames that are wrong

The parser is `parse_args` ([src/ported/scripts/powerline_daemon.rs:254](#)). It splits on NUL, drops empty fragments, reads `usize::from_str_radix(parts[0], 16)`, and then does something upstream's Python did not have to do:

```
let numargs = usize::from_str_radix(&parts[0], 16).ok()?;
if parts.len() < 1 + numargs + 1 {
    return None;
}
```

That length guard ([powerline_daemon.rs:266](#)) carries no `// sh:` citation because it has no upstream counterpart: Python's slice `args[1:numargs + 1]` silently truncates and its `args[numargs + 1]` raises `IndexError`, and the port needed a bounds check where Rust would otherwise index a slice out of range. `None` propagates to `get_answer` ([powerline_daemon.rs:463](#)), which replies with the fixed string at `:473`.

Feeding a live daemon on a probe socket four different malformations, and reading the reply bytes directly:

Frame sent	Reply
<code>\0\0</code>	<code>malformed request</code>
<code>zz\0not-an-ext\0\0</code>	<code>malformed request</code>
<code>ff\0tmux\0right\0/tmp\0\0</code>	<code>malformed request</code>
<code>1\0bogus\0/tmp\0\0</code>	<code>expected one argument</code>

The four cases are three different mechanisms. The first has no non-empty fields at all, so the `parts.is_empty()` guard fires. The second has a first field that is not hexadecimal, so `from_str_radix` fails. The third is well-formed hex — `ff` is 255 — and fails the length guard, which is the case that would have been an index panic without it. The fourth is a *valid* frame that parses cleanly: one argument, `bogus`, with `/tmp` as the working directory. It gets past the protocol layer entirely and dies in the argument parser, whose message comes from [src/ported/commands/main.rs:241](#) — the port of upstream's `parser.error('expected one argument')`.

That is the useful distinction for anyone debugging a client: `malformed request` is the wire parser rejecting your bytes, and any other message is your bytes having been accepted and your *request* being wrong. The two errors come from different layers and mean different things, and neither is distinguishable from a successful render by any framing, because there is none — the error text arrives on the same channel as markup would have.

The robustness of that surface is pinned by [tests/daemon_fuzz.rs](#), whose six tests name the categories directly: `random_wire_bytes_keep_daemon_alive`, `mutated_valid_payload_keeps_daemon_alive`, `overlong_argc_value_does_not_oom_or_panic`, `embedded_null_storm_does_not_break_parser`, `partial_write_then_eof_does_not_hang_daemon`, and `extreme_env_count_handled`. The invariant those tests hold is not “the daemon returns the right answer” but “the daemon is still there afterwards,” which is the correct invariant for a long-lived server reading from a socket any local process can open.

The fallback boundary, measured

The client's resilience story is one branch: if `UnixStream::connect` returns an error, the client does not report it, it `execvp`s `powerline-render` with its own arguments ([powerline.rs:94-100](#), calling `exec_render_`

`fallback` at :171). Three ways of not having a daemon, each rendered against a fixture config so the output is deterministic:

```
$ export POWERLINE_CONFIG_PATHS=$PWD/tests/data/e2e/scenario_mode_xlate

$ rm -f /tmp/pl-none.sock; powerline --socket /tmp/pl-none.sock tmux right
#[fg=colour1,bg=default,nobold,noitalics,nounderscore]#[fg=colour231,bg=colour1,...] MODAL

$ ls -l /tmp/pl-book.sock          # stale socket file, daemon exited
srwxr-xr-x 1 ... 0 ... /tmp/pl-book.sock
$ powerline --socket /tmp/pl-book.sock tmux right
#[fg=colour1,bg=default,nobold,noitalics,nounderscore]#[fg=colour231,bg=colour1,...] MODAL

$ : > /tmp/pl-regular.file        # not a socket at all
$ powerline --socket /tmp/pl-regular.file tmux right
#[fg=colour1,bg=default,nobold,noitalics,nounderscore]#[fg=colour231,bg=colour1,...] MODAL
```

The `export` is load-bearing and an earlier printing omitted it. The fallback path runs `powerline-render` in a fresh process, and that process resolves its own config cascade — so without the fixture pinned, these three commands render the machine's real statusline and the `MODAL` shown here is unreachable. Isolating the fallback means isolating the *renderer the fallback execs*, not just the client.

All three produce the correct line. A missing path, an orphaned socket inode whose server is gone, and a path that is a regular file all fail `connect` for different reasons — `ENOENT`, `ECONNREFUSED`, `ECONNREFUSED` on a non-socket — and the client does not distinguish them because it does not need to. The fallback is keyed on the error being *any* error.

There is exactly one hole in that coverage and it is worth naming, because it is the case that produces a blank prompt rather than a slow one. Every write after a successful connect is guarded by `write_failed()` (`powerline.rs:194`), which prints `powerline: write() to daemon failed` to stderr and exits 1 — it does not fall back. That path is reachable: connecting to a daemon in the middle of shutting down succeeds and then the write fails, which is what a client run during the `--replace` window of Chapter 25 can hit. It does not have to be waited for. A nine-line listener that accepts and immediately hangs up puts the client in exactly that state on demand:

```
$ python3 -c 'import socket,os
p="/tmp/pl-dead.sock"
try: os.unlink(p)
except FileNotFoundError: pass
s=socket.socket(socket.AF_UNIX,socket.SOCK_STREAM); s.bind(p); s.listen(5)
print("READY",flush=True)
c,_=s.accept(); c.close()' &
READY

$ powerline --socket /tmp/pl-dead.sock tmux right; echo "rc=$?"
powerline: write() to daemon failed
rc=1
```

Empty stdout, the diagnostic on stderr, and exit 1 — which is the one number the earlier printing of Chapter 41's failure table got wrong in the other direction. The reason the branch is written that way is provenance — the C client treats a failed write as a hard error too — and the consequence is a one-render blank prompt in a narrow window, not a persistent failure, because the next invocation finds the socket unconnectable and falls back normally. The practical form of the rule: **the fallback covers a dead daemon, not a dying one.**

The kill path

The last piece of the protocol is not a render at all. `EOF` is the byte string `b"EOF\0\0"` (`powerline_daemon.rs:73`) — a valid frame under the wire grammar whose first field is not hex, recognized before

parsing and answered by exiting. `powerline-daemon -k` sends exactly those bytes. Against a socket that never existed:

```
$ powerline-daemon --socket /tmp/pl-none.sock -k
No running daemon found
$ echo $?
1
```

The exit code is the useful half: a script that needs to know whether a daemon was running can branch on it, which is the only place in the entire protocol where a status is reported out of band rather than as text on the response stream.

Chapter 32 – Highlight Resolution, Observed From Outside

Chapter 6 walks the colorscheme code and explains what `get_highlighting` does. This chapter does the complementary thing: it takes fixture configs small enough to hold in your head, runs the real binaries against them, and reads the bytes that come out — so that the resolution rules are confirmed from outside the program rather than described from inside it. The fixtures are the ones the parity suite uses, which makes this an alternate use for test data that is otherwise dormant on a tree without an interpreter (Chapter 30).

Isolating the config, which `-p` does not do

The first thing to get right is which config is under test, and there is a trap here that costs a measurement if you miss it. `powerline-render` accepts `-p PATH` to add a config search path, and adding is not replacing: a render with `-p tests/data/e2e/scenario_mode_xlate` still renders the machine's own installed theme, because the user's config remains in the cascade and wins the merge. The environment variable is the one that isolates, and the code says so at `src/bin/shared/render_runtime.rs:35-51`:

```
// When POWERLINE_CONFIG_PATHS is set it REPLACES the default cascade
// entirely – no bundled fallback. Test fixtures rely on this to pin a
// self-contained config tree.
if let Ok(pcp) = std::env::var("POWERLINE_CONFIG_PATHS") {
```

Every transcript below sets `POWERLINE_CONFIG_PATHS` to one fixture directory. That is also what `daemon_parity.rs` does for the Rust side of each scenario, while passing `-p` to the Python side, and the asymmetry is not sloppiness — upstream's `get_config_paths` treats `--config-path` as a replacement, and the port treats it as an addition, so the two flags reach the same state by different routes.

First group wins, and a missing group is not an error

`scenario_hl_fallback` is a theme with one string segment naming two highlight groups, and a colorscheme that defines only the second:

```
"highlight_groups": ["sg_missing", "sg_present"]
```

```
"groups": {
  "sg_present": { "fg": "yellow", "bg": "red", "attrs": ["bold"] },
  "background": { "fg": "white", "bg": "gray0", "attrs": [] },
  "background:divider": { "fg": "gray2", "bg": "gray0", "attrs": [] }
}
```

Rendered — the output is one line, split here at the boundary between the two tmux directives:

```
$ POWERLINE_CONFIG_PATHS=tests/data/e2e/scenario_hl_fallback powerline-render tmux right
#[fg=colour1,bg=default,nobold,noitalics,nounderscore]
#[fg=colour11,bg=colour1,bold,noitalics,nounderscore] FB
```

`fg=colour11` is yellow, `bg=colour1` is red, and bold is present — `sg_present` resolved and `sg_missing` cost nothing. The loop responsible is four lines (`src/ported/colorscheme.rs:306-312`): walk the group

list, call `get_group_props` on each, and break at the first that returns `Some`. A name with no entry returns `None` from the string branch of `get_group_props` (`colorscheme.rs:242`) and the walk continues. Only an *exhausted* list is an error, and it is a real one — `get_highlighting` returns `Err("Highlighting groups not found in colorscheme: ...")` naming every group it tried (`colorscheme.rs:314-320`).

This is why a theme can name a specific group and a generic one in the same list and get graceful degradation for free: the specific name is optional by construction. It is also why a typo in a highlight group is silent when a fallback follows it and loud when it stands alone — the same mechanism producing opposite diagnostics depending on how the list was written.

Mode translation, seen twice

`scenario_mode_xlate` has one segment in group `sg_main` (`fg: white, bg: red`) and a colorscheme carrying a translation:

```
"mode_translations": { "insert": { "colors": { "red": "green" } } }
```

Two renders, one flag apart:

```
$ POWERLINE_CONFIG_PATHS=tests/data/e2e/scenario_mode_xlate powerline-render tmux right
#[fg=colour1,bg=default,...]#[fg=colour231,bg=colour1,nobold,noitalics,nounderscore] MODAL

$ POWERLINE_CONFIG_PATHS=tests/data/e2e/scenario_mode_xlate powerline-render tmux right -m
  ~ insert
#[fg=colour2,bg=default,...]#[fg=colour231,bg=colour2,nobold,noitalics,nounderscore] MODAL
```

`bg=colour1` becomes `bg=colour2` — the `red` → `green` substitution — and the foreground, `white`, is untouched because the translation names one color. The divider ahead of the segment follows it, since the divider's foreground is the next segment's background.

The same request through a warm daemon, with the mode arriving as a renderer argument on the wire rather than as a command-line flag:

```
$ powerline-daemon --foreground --socket /tmp/pl-book.sock &      # POWERLINE_CONFIG_PATHS
  ~ set
$ powerline --socket /tmp/pl-book.sock tmux right
#[fg=colour1,bg=default,...]#[fg=colour231,bg=colour1,nobold,noitalics,nounderscore] MODAL
$ powerline --socket /tmp/pl-book.sock tmux right -R mode=insert
#[fg=colour2,bg=default,...]#[fg=colour231,bg=colour2,nobold,noitalics,nounderscore] MODAL
```

Byte-identical to the direct renders. That is the observable form of a claim Chapter 27 makes structurally: mode is request context, not process state. The daemon holds a warm orchestrator keyed by extension and overrides (Appendix E.7) and the mode is not part of that key, so two requests one second apart can render the same theme in two modes without the daemon rebuilding anything or remembering which mode came last.

The mechanism behind the substitution has two channels and only one of them is exercised here. `get_group_props` (`colorscheme.rs:242`) consults `trans["groups"]` *first* — a mode may replace a group's properties wholesale — and recurses with `translate_colors = false` when it does, which is what stops a group substituted by a mode from also having its colors renamed. The `trans["colors"]` rename runs only on the non-substituted path. A fixture that used both would show a group override winning outright; this one uses the color channel, which is the common case in real colorschemes because it is the one that scales to a whole palette with one entry.

Truecolor changes the encoding, not the resolution

`scenario_truecolor` is the same machinery with 24-bit colors configured:

```
$ POWERLINE_CONFIG_PATHS=tests/data/e2e/scenario_truecolor powerline-render tmux right
#[fg=#c41e3a,bg=default,nobold,noitalics,nounderscore]#[fg=#ffffff,bg=#c41e3a,bold,...]
  ~ RGB1#[fg=#228b22,bg=#c41e3a,...]#[fg=colour16,bg=#228b22,...] RGB2
```

Two things are visible. The color tokens are `#rrggbb` rather than `colourN`, which is the renderer’s encoding decision downstream of resolution. And `fg=colour16` appears in the middle of a truecolor line: a color configured as a bare cterm index has no hex to emit, so the renderer falls back to the indexed form for that channel only. Resolution produced a `(cterm, hex)` pair either way — the constructor at `colorscheme.rs:109` builds one for every color, deriving the hex from the 256-entry `cterm_to_hex` table when the config gave only an index — and the renderer chose per channel based on what it got.

The unguarded index

There is one place where this machinery will take a process down, and it is worth documenting precisely because the port inherited it deliberately and its own source says so. `pick_gradient_value` (`colorscheme.rs:72`) is four lines:

```
pub fn pick_gradient_value(grad_list: &[u64], gradient_level: f64) -> u64 {
    // py:32 grad_list[int(round(gradient_level * (len(grad_list) - 1) / 100))]
    let raw = gradient_level * (grad_list.len() as f64 - 1.0) / 100.0;
    let idx = py_round(raw) as usize;
    grad_list[idx]
}
```

Its doccomment states the contract at `colorscheme.rs:65-66`: “gradient level is not checked for being inside `[0, 100]` interval (matches Python behaviour).” The index is not bounds-checked because upstream’s is not, and the rounding is `round_ties_even` because Python 3’s `round` is half-to-even — the fix from Appendix F’s bug log, still in place. What follows arithmetically is that a level above `100 * (1 + 0.5/(len-1))` selects an index past the end. For a hundred-entry gradient that threshold is a level of about 100.51.

Observed on a heavily loaded machine:

```
$ powerline-render tmux right
thread 'main' panicked at src/ported/colorscheme.rs:76:5:
index out of bounds: the len is 100 but the index is 100
```

An earlier printing of this chapter called that a load-dependent edge and added that “the bundled palette is not the source,” on the grounds that `colors.json` defines ten gradients whose long ones carry 101 entries. That reading counted one of the two lists in each gradient and drew the wrong conclusion from it. A gradient is a *pair* of parallel lists — a cterm ramp and a hex ramp — and in the bundled palette seven of the ten pairs have sub-lists of different lengths, including one whose hex ramp is exactly 100 entries long. The bundled palette is the source. Chapter 45 takes the crash apart end to end: the palette table, the segment that feeds the out-of-range level, the exact threshold, the exit code, and a command that reproduces it on demand rather than by waiting for load.

The exposure is on the producing side. Two segment families feed a level in: `system_load` clamps, explicitly, in a three-way branch that returns `100.0` in the else arm (`src/ported/segments/common/sys.rs:104-109`), while `cpu_load_percent` forwards the sampler’s value unmodified — “`gradient_level`”: `cpu_percent` at `sys.rs:293`, a faithful port of upstream’s “`gradient_level`”: `cpu_percent` — and the extension segments do the same with their own percentages. Nothing between the segment and the index re-checks the range. So the invariant the whole system depends on is *held by convention in the segments* rather than enforced at the point of use, and a segment or a custom `exec` script that reports 101 percent of anything, against a gradient whose length makes the arithmetic round up, ends the process.

Stating it as a rule for theme and segment authors: **a `gradient_level` is a contract, not a hint**. Anything wired into a gradient group must clamp to `[0, 100]` at the source, because the consumer will not.

Chapter 33 – The Segment Registry as a Test Surface

Chapter 3 introduces the `ADAPTERS` registry as the replacement for upstream's dynamic `__import__`, and Chapter 16 walks through adding an entry to it. This chapter treats the table as an object of measurement rather than a mechanism: how many entries it has, how they are distributed across provenance, exactly what the tests assert about it, and — the part that matters for reading any status claim — what those assertions do and do not certify.

The census

The table is `pub const ADAPTERS: &[(&str, AdapterFn)]` at `src/bin/shared/render_runtime.rs:2441`. Parsed mechanically, with comment lines stripped so prose inside the table is not counted:

Quantity	Count
Rows	59
Distinct keys	59
Distinct adapter functions	58
Keys under <code>powerline.segments.*</code> (upstream names)	29
Keys under <code>powerliners.*</code> (extension names)	25
Keys under <code>powerlinemem.*</code>	4
Bare keys with no dotted prefix	1 (<code>exec</code>)

The one function appearing twice is `ad_exec`, reachable both as `powerliners.exec.exec` and as the bare `exec`, which exists so a theme can write `"function": "exec"` without a module prefix. Every other function has exactly one key.

The three prefixes are a provenance map. `powerline.segments.*` are the dotted paths upstream themes already use, so a decade-old theme naming `powerline.segments.common.net.hostname` resolves without an edit. `powerlinemem.*` mirrors the third-party memory-segment package upstream users installed separately. `powerliners.*` is the net-new extension tier, and its 25 rows are the largest single block in the table — the GPU, disk, thermal, VCS-status, container, Kubernetes, process-count, CI, AWS, and GCP segments, plus the eleven cache and version segments for the fusevm runtime and the five language runtimes built on it.

Earlier printings of Chapter 3 carried an older size for this table — 54 adapters and 55 keys — which was accurate when it was written and stopped being so when the `vimlrs` and `elisprs` additions brought four more rows. The count is now 59 keys over 58 functions, and Chapter 3 has been brought in line with it. The number is a property of a table that grows, which is the argument for deriving it with a command rather than quoting it — the parse above is three lines and it cannot go stale. Chapter 51 reports which of this book's other figures survived a replay and which did not; the table sizes are in the class that did.

What the tests assert

The registry's tests live in the same file, in the `#[cfg(test)]` module that compiles into both the daemon and the one-shot renderer. Six of them concern the table directly, and they fall into two groups.

Two are structural invariants over the table itself:

- `adapters_table_keys_are_unique` (`render_runtime.rs:3128`) inserts every key into a `HashSet` and asserts each insert is new. A duplicate key would not be a compile error — the table is a slice of tuples, not a map — and the lookup is a linear scan that takes the first match, so a duplicate would silently shadow whichever row came second. This test is the reason that cannot happen unnoticed.
- `adapters_table_keys_have_no_leading_or_trailing_dots` (`:3138`) rejects keys that start or end with `..`. That is a guard against the specific way the resolution cascade can be fooled: the segment getter assembles an id as `<default_module>.<function_name>`, and an empty component on either side produces a key that looks resolvable and matches nothing.

Four are behavioral, and they test the resolver rather than the table:

- `adapter_id_full_dotted_path_matches` (`:3061`) asks the resolver for the pair `("powerline.segments.common.net", "hostname")` and asserts the exact concatenation resolves.
- `adapter_id_bare_name_fallback_hits_short_alias` (`:3069`) asks for a module that does not exist — `("powerline.segments.nonexistent", "exec")` — and asserts the resolver falls through to the bare `exec` key.
- `adapter_id_extension_segments_register` (`:3086`) walks a literal list of 25 `(module, name)` pairs — every `powerliners.*` and `powerlinemem.*` segment — and asserts each resolves to the id formed by joining them, with a panic message naming the missing pair.
- `adapter_id_unknown_returns_none` asserts the negative case: a name that is in neither the table nor the filesystem resolves to `None` rather than to something plausible.

The extension-registration test is the interesting design choice, because it is a hand-maintained list of what the table should contain, checked against what it does contain. A row deleted from `ADAPTERS` fails this test by name. A row *added* does not, which is the correct asymmetry: the test is a floor, not a lock.

The resolution cascade the tests exercise

`adapter_id` (`render_runtime.rs:476`) is four attempts in order, and reading it explains why the two fallback tests exist:

1. Exact match of `<module>.<name>` in `ADAPTERS`.
2. Exact match of the bare `<name>` in `ADAPTERS` — the `exec` alias path.
3. A filesystem probe: `exec_segment::resolve_dotted_path` (`src/extensions/exec_segment.rs:45`) for the full dotted path under each config search directory's `segments/` tree.
4. The same probe for the bare name.

Only if all four miss does it return `None`. `invoke_adapter` (`:499`) repeats the shape at call time, adding a retry on the trailing component so an id that arrived already prefixed by a default module — `powerline.segments.tmux.exec` — still finds the short alias.

The consequence for a theme author is that segment names have four different things they can mean, resolved in a fixed order, and the first two are compiled into the binary while the last two are files on disk. The consequence for a reader of the tests is that the registry tests cover the first two rungs, and the filesystem rungs are covered separately by `resolve_dotted_path`'s own five tests in `exec_segment.rs`, which pin extension preference (`.sh` over `.py`), the no-extension fallback, the multi-directory walk, and the missing case.

What a registry test cannot tell you

The tests above certify *reachability*: that a name a theme can write arrives at a function. They say nothing whatsoever about what that function returns. `adapter_id_extension_segments_register` passing means all 25 extension segments are addressable; it does not mean any of them produces correct output, or output at all — every adapter returns `Option<Value>` and `None` is a legal answer that renders as an absent segment (Appendix AL).

Output correctness lives in three other places, and it is worth naming them so the layering is explicit. The per-segment unit tests inside `src/extensions/*.rs` and `src/ported/segments/**` exercise each adapter's pure core with injected inputs — the four-level path resolution of the cache segments is tested through a `default_path_with(get_env, path_exists)` seam precisely so the precedence can be pinned without touching the real environment. The daemon parity scenarios diff whole rendered lines against upstream, which is the only check that covers the composition of segments rather than segments individually — and on a tree without the interpreter those are skipping (Chapter 30). And the fixture renders of Chapters 32 and 35 exercise the full path for the handful of segments those fixtures use.

Stated as a rule, because it is the same rule Chapter 29 reaches from the other direction: **a test that checks a table checks the table**. The registry is verified to be well-formed, unique, complete against a named floor, and reachable through four resolution rungs. Whether the thing on the other end of a row draws the right pixels is a different claim with different evidence, and the two should not be quoted as if they were one number.

Chapter 34 — A Census of the Test Suite

This book cites test counts in several places, and the counts move. The figure carried in earlier chapters until this measurement was 2473 library tests; the tree these measurements were taken on reports a different one, and those chapters now carry the re-derived figure. Nothing is wrong in either place — the number is a property of a repository at a moment, and it goes up when someone writes a test. This chapter does what a book should do with a moving quantity: give the command instead of the constant, run it, and publish the whole distribution rather than the headline.

The library

```
$ cargo test --lib
test result: ok. 2497 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
~ 11.19s
```

Two thousand four hundred ninety-seven in-module tests, none ignored, eleven seconds. “None ignored” is the part worth reading twice: a suite with ignored tests has a hidden backlog, and a count that includes them overstates itself. This one has no such gap.

Where they are is as interesting as how many. They live inside `src/` in `#[cfg(test)]` blocks beside the code they test, which is a deliberate consequence of the port’s structure — a ported function and its test are in the same file as the `// py:` citations that tie both to upstream, so a reader checking a claim reads one file rather than three.

The integration targets

Each file under `tests/` is its own binary. Run individually, with the runtime reported by the harness:

Target	Tests	Time	What it covers
<code>parity_against_upstream</code>	462	1.14s	the live-interpreter parity harness (Chapter 30)
<code>daemon_parity</code>	45	0.03s	byte-for-byte daemon scenarios against upstream
<code>bin_powerline_client</code>	12	0.24s	client wire marshalling and fallback
<code>contract_round4_misc</code>	9	0.00s	assorted pinned contracts
<code>contract_colorscheme_theme</code>	8	0.00s	colorscheme and theme resolution

Target	Tests	Time	What it covers
<code>contract_dict_merge_gradient</code>	8	0.00s	deep merge and gradient picking
<code>daemon_fuzz</code>	6	3.84s	malformed and hostile wire input
<code>daemon_e2e</code>	5	2.71s	socket lifecycle end to end
<code>lib_edge_cases_round5</code>	5	0.21s	numeric and encoding edges
<code>daemon_stress</code>	4	4.27s	concurrency and backpressure
<code>lib_strwidth_fallback</code>	4	0.00s	the <code>N/Na</code> width fallback
<code>vim_e2e</code>	3	2.22s	the vim renderer end to end
<code>lib_edge_cases_boundary</code>	3	0.00s	saturation and boundary geometry
<code>ported_fn_names_match_py</code>	2	0.16s	the drift gate
Total	576		

The runtimes carry information the counts do not. The three slowest targets — `daemon_stress`, `daemon_fuzz`, `daemon_e2e` — are slow because they spawn real daemons on real sockets and wait for real process lifecycles, which is what makes them the tests that would catch a hang. The targets at 0.00s are pure-function contracts. And `daemon_parity` at 0.03 seconds for 45 end-to-end scenarios is the anomaly Chapter 30 explains: those tests are skipping.

The binary targets

Two of the five shipped binaries carry tests of their own:

```
$ cargo test --bin powerline-daemon -- --list | tail -1
26 tests, 0 benchmarks
$ cargo test --bin powerline-render -- --list | tail -1
26 tests, 0 benchmarks
```

The identical counts are not a coincidence: both binaries include `src/bin/shared/render_runtime.rs`, so the 26 tests in that module — the six `ADAPTERS` and `adapter_id` tests of Chapter 33, the ten `colorspec` color-classification tests, and the ten config-reload staleness tests — compile and run twice, once per host binary. Listing either binary's tests shows the same 26 names, all prefixed `render_runtime::tests::`. `powerline`, `powerline-config`, and `powerline-lint` report zero; the client's coverage lives in `tests/bin_powerline_client.rs` instead, which is the right place for it since testing a client means running it against a socket.

The total, and how to read it

Summing what `cargo test` executes: 2497 library, 576 integration, 52 binary — **3125 test functions**, with the caveat stated rather than buried, that 26 of those 52 are the same 26 tests compiled into a second binary. A reader who prefers to count distinct tests rather than executions subtracts 26 and gets 3099.

Alongside them are 21 shell scripts under `tests/`, which are not Rust tests and do not appear in any `cargo test` count. They are repository hygiene gates — final newlines on every shipped artifact, no trailing whitespace in man pages, a synopsis section in each man page, badges and an H2 in the README, no inline event handlers or placeholder hrefs or plain-HTTP links in the docs tree, `rel="noopener"` on every `target="_blank"`. They exist because the project publishes HTML and man pages as artifacts, and an artifact with a broken invariant is a defect the compiler cannot see.

Why the number moved, and what to do about it

The 2473 this book carried before this chapter and the 2497 measured here differ by 24 tests, which is what a few commits of ordinary work looks like — the vimlrs and elisprs segments alone added their own unit tests, and Chapter 33 records that they added four registry rows. Neither figure is wrong for the tree it was taken on, and both will be wrong for some future tree.

The generalizable practice is the one this book uses elsewhere for the divider math and the config cascade: publish the command beside the number. `cargo test --lib` is nine keystrokes and it is authoritative in a way no transcribed constant can be. Where a count appears in a status claim — a README badge, a chapter's opening summary — the honest form carries the date or the version it was measured at, because a number without a provenance is indistinguishable from a number that was carried forward because nobody re-ran it.

That principle is the through-line of these chapters, and it is worth stating once plainly. The tier counts are read out of a file that cannot currently be regenerated. The parity suite reports 462 passes and compares nothing without a vendored interpreter. The registry has grown past the size this book first recorded. The library test count moved by 24. Every one of those is a small thing, none of them is a defect, and all four were found the same way — by running the command instead of quoting the sentence. A book about a port whose entire claim is verifiability should be held to the standard it describes, and the standard is that a figure is worth exactly as much as the command that produced it.

Chapter 35 – The Fixture Corpus as an Executable Specification

Chapter 30 establishes that the 45 daemon-parity scenarios skip when the interpreter is absent, and notes in passing that the fixtures themselves remain useful. This chapter takes that seriously. `tests/data/e2e/` is 44 self-contained config trees, each isolating exactly one rendering behavior, and each of them can be rendered directly by the shipped binary in one command. Read that way the corpus stops being test data and becomes what it structurally is: a specification of the renderer's behavior, written in configuration rather than prose, with the expected output obtainable by running it.

The shape of a scenario

Every directory holds the same four files:

```
$ find tests/data/e2e/scenario_hl_fallback -type f | sort
tests/data/e2e/scenario_hl_fallback/colors.json
tests/data/e2e/scenario_hl_fallback/colorschemas/tmux/default.json
tests/data/e2e/scenario_hl_fallback/config.json
tests/data/e2e/scenario_hl_fallback/themes/tmux/default.json
```

The `sort` is not decoration. `find` emits directory order, which on this filesystem puts `colors.json` last rather than second, so the four lines an earlier printing showed were in an order the command does not produce. It is the smallest possible instance of the thing this corpus exists to prevent — a printed output that was tidied rather than captured — and it is cheap to make honest by pinning the order rather than describing it.

`config.json` names the extension, the colorscheme, and the theme, and in almost every scenario sets `term_truecolor: false` so the output is indexed and stable. `colors.json` is a palette of six to eight named colors and usually an empty `gradients` map. The colorscheme defines the handful of groups the scenario needs, always including `background` and `background:divider` because the renderer needs both to draw an edge. The theme holds the segment list and the divider glyphs.

The whole tree is small by design — the largest of these files is a few dozen lines — and that smallness is the property that makes the corpus readable. A scenario that isolates one behavior with three segments and six colors can be held in the head while reading its output; a scenario built on a real theme could not.

Rendering the corpus

Setting `POWERLINE_CONFIG_PATHS` to a scenario directory replaces the config cascade entirely (Chapter 32), so one command per directory produces that scenario's bytes with nothing from the machine leaking in. Running twelve of them and reporting the byte count of each:

Scenario	Output bytes	What the bytes show
<code>scenario_empty</code>	0	an empty theme renders nothing at all — not a divider, not a space
<code>scenario_combining_mark</code>	118	a base character plus a combining mark occupying one cell

Scenario	Output bytes	What the bytes show
<code>scenario_hash_escape</code>	120	<code>ID#42</code> emitted as <code>ID##[]42</code> — the tmux <code>#</code> escape
<code>scenario_padding0</code>	225	<code>spaces: 0</code> — segment contents flush against their dividers
<code>scenario_soft_divider</code>	229	a soft divider between same-background neighbors
<code>scenario_emoji</code>	231	a wide glyph counted as two cells
<code>scenario_padding2</code>	232	<code>spaces: 2</code> — two spaces each side
<code>scenario_spaces3</code>	234	<code>spaces: 3</code>
<code>scenario_no_dividers</code>	279	empty divider strings — segments abut with only color changes
<code>scenario_width_trunc</code>	348	three fixed-width segments, unconstrained
<code>scenario_priority_drop</code>	458	four segments with three priorities
<code>scenario_palette16</code>	776	the 16-color indexed palette exercised in one line

The `scenario_empty` row is the one worth pausing on. Zero bytes is a decision, not an absence of one: a renderer could reasonably emit a background-colored blank, and this one emits nothing, which is why the scenario exists. The corpus is full of decisions of that kind, each frozen as a directory whose name states the question.

Two scenarios that answer questions prose cannot

Some behaviors are only legible as a series. `scenario_priority_drop` is a theme of four segments, three carrying `priority` values of 10, 50, and 100, and one carrying none. Rendering it at descending widths, with the highlight directives stripped so only the visible text remains:

```
$ POWERLINE_CONFIG_PATHS=tests/data/e2e/scenario_priority_drop \
  powerline-render tmux right -w 16 | perl -pe 's/#\[([^\]]*)\]/g'
P10 P50 PNO
$ ... -w 12
P10 PNO
$ ... -w 8
PNO
$ ... -w 4
PNO
```

Three facts fall out of four commands. Segments are dropped in descending order of their `priority` number, so a *larger* priority value means dropped *sooner* — `P100` goes first, then `P50`, then `P10`. A segment with no priority is never dropped, at any width, including a width narrower than the segment itself. And truncation stops when only the undroppable segments remain rather than continuing to cut characters, which is why `-w 4` and `-w 8` produce the same line.

`scenario_width_trunc` answers the complementary question with three equal-width segments and no priorities:

```
$ ... -w 30 → AAAAA BBBB CCCCC
$ ... -w 20 → AAAAA BBBB CCCCC
$ ... -w 10 → CCCCC
```

The line survives at 20 columns and collapses to its last segment at 10. On a right-hand statusline the segments that go are the leading ones, which is the correct direction — the right edge is anchored and the line is eaten from the left.

Neither of those behaviors is stated in a config schema, and neither is obvious from reading the truncation code, because both are emergent from the interaction between the width budget, the divider costs, and the priority ordering. They are, however, immediately obvious from four invocations of a shipped binary against a fixture that already lives in the repository.

Using the corpus without an interpreter

That makes the corpus a regression harness that works in the tree as it stands. The parity suite needs upstream Python to say whether the bytes match *powerline's*; it does not take an interpreter to say whether the bytes match *yesterday's*. Rendering every scenario and diffing against a stored capture catches any change in dividers, padding, escape handling, color encoding, or truncation, in one loop over 44 directories, at whatever the binary's startup cost is.

What that check cannot do is establish parity, and the distinction is the same one Chapter 33 draws about the registry. A self-diff pins the renderer to its own history: it catches regressions and it cannot catch a divergence that was present in both captures. Only the live comparison does that, and only when the interpreter is there. The two are complementary and neither substitutes for the other — which is precisely why the same 44 directories serve both, and why keeping them small enough to read is worth more than making them realistic.

Chapter 36 – The Segment Set, Rendered One at a Time

Chapter 33 counts the `ADAPTERS` registry and measures what its tests certify: reachability. It closes by naming the thing a registry test cannot tell you — whether the function on the other end of a row produces anything. This chapter answers that question the only way it can be answered, by rendering every row individually and reading what comes back.

The method is mechanical. For each of the 59 keys in `ADAPTERS` (`src/bin/shared/render_runtime.rs:2441-2548`), write a `tmux` theme whose right side is that one segment and nothing else, render it with `POWERLINE_CONFIG_PATHS` pinned to a self-contained config tree, strip the `#[...]` markup, and record the visible length. Fifty-nine renders, one segment each, same machine, same minute.

Extracting the row set

The table is not machine-readable by `grep` alone, because `rustfmt` wraps the long rows onto three lines each. Pulling every string literal out of the table's line range instead of matching a single-line pattern gives the whole set:

```
$ perl -ne 'next unless $.>=2442 && $.<=2548; next if m{^\s*//};
           print "$1\n" while /"([^\"]+)"/g' src/bin/shared/render_runtime.rs | wc -l
59
$ ... | sort -u | wc -l
59
```

Fifty-nine keys, all distinct, which reproduces Chapter 33's census from the other direction. The naive single-line `grep` yields 51 and silently drops the eight wrapped rows — a small reminder that a count taken from formatted source is a count of what the formatter did, unless the extraction is written to survive it.

The harness, and the trap in it

The config tree is a copy of the bundled `src/ported/config_files/` with `themes/tmux/` emptied, so each iteration writes exactly one theme file:

```
{"segments":{"right":[{"function":"powerline.segments.common.time.date"}]}}
```

and renders it:

```
$ POWERLINE_CONFIG_PATHS=$WORK powerline-render tmux right
#[fg=colour247,bg=colour235,...] 2026-08-09 #[fg=colour235,bg=default,...]
```

The first version of this sweep returned an empty string for all 59 rows, and the reason is worth stating because it is the difference between measuring segments and measuring a colorscheme. A function segment's highlight group is not taken from the theme entry; `prepare_segment` sets it unconditionally from the function's own name when `segment_type == "function"` (`src/ported/segment.rs:903-916`), so a spec that writes `"highlight_groups": ["background"]` next to `"function": "..."` has no effect at all, and a spec that renames the segment with `"name"` does not move the group either. If the colorscheme has no group of that name the segment is dropped, and dropped silently — the behaviour Chapter 32 establishes for string

segments, reached here from the function-segment side. A one-group colorscheme therefore renders every segment as nothing, which looks exactly like 59 functions all returning `None`.

The fix is to sweep twice against two colorschemes and read the difference, which turns the trap into a measurement.

Two colorschemes, two counts

The first pass uses the bundled cascade for `ext=tmux: colorschemes/default.json` (52 groups) merged with `colorschemes/tmux/default.json` (10 groups), 61 distinct groups after the merge — there is no `colorschemes/tmux/__main__.json` in the bundled tree, so the middle level of the three-level cascade contributes nothing. The second pass uses a synthesized colorscheme holding 240 groups: every group name defined by any bundled colorscheme (133 across the whole `colorschemes/` tree) unioned with every plausible group name appearing near a `highlight_groups` literal in `src/`, plus `fuzzy_time`, `email_imap_alert`, and `last_status`, which no bundled colorscheme defines.

Colorscheme	Groups	Segments producing output	Segments producing nothing
bundled cascade (<code>default</code> + <code>tmux/default</code>)	61	22	37
synthesized all-groups	240	32	27

Ten segments move between the two runs, and the ten are a coherent set:

<code>powerlinemem.mem_usage.mem_usage</code>	<code>powerliners.disk.disk_usage</code>
<code>powerlinemem.mem_usage.mem_usage_percent</code>	<code>powerliners.disk.disk_usage_percent</code>
<code>powerlinemem.mem_usage.mem_swap</code>	<code>powerliners.k8s.kubecontext</code>
<code>powerlinemem.mem_usage.mem_swap_percentage</code>	<code>powerliners.aws.context</code>
<code>powerliners.gpu.gpu_vram</code>	<code>powerliners.gcp.context</code>

Every one is an extension segment — `powerlinemem.*` or `powerliners.*` — and none of them has a group in the bundled colorschemes, because the bundled colorschemes are upstream powerline’s and predate the extension tier. The practical statement for a user: **the extension segments are wired, tested, and reachable, and half of them render as nothing against the shipped default colorscheme until the user adds a group for them.** That is not a defect in the adapters; it is the cost of keeping the bundled colorscheme faithful to upstream while adding segments upstream never had. It is worth knowing before concluding that `disk_usage` is broken.

What the 32 produced

Sampled from the all-groups pass, with the markup stripped:

Key	Rendered
<code>common.net.hostname</code>	<code>codelabs-arm</code>
<code>common.time.date</code>	<code>2026-08-09</code>
<code>common.env.user</code>	<code>wizard</code>
<code>common.env.cwd</code>	<code>/ private tmp pl-book-pass2</code>
<code>common.sys.cpu_load_percent</code>	<code>100%</code>
<code>common.sys.system_load</code>	<code>121.3 98.2 85.7</code>
<code>common.sys.uptime</code>	<code>13d 2h 34m</code>

Key	Rendered
<code>common.net.external_ip</code>	75.133.214.101
<code>common.net.internal_ip</code>	10.59.0.175
<code>common.bat.battery</code>	⚡ 100%
<code>common.wthr.weather</code>	29°C
<code>powerliners.disk.disk_usage</code>	855.4GiB/926.4GiB
<code>powerliners.thermal.thermal</code>	34°C
<code>powerliners.zshrs.version</code>	0.12.32
<code>powerliners.fusevm.jit_cache</code>	773 3.01M

The `cwd` row is the one to look at twice, because it is the only sampled segment that returns more than one chunk: four path components, each its own segment with its own soft divider, produced by a single theme entry. Everything else in the sample returns one chunk, which is why a naive reading of “one theme entry, one segment” holds for 58 of the 59 rows and does not hold for `cwd`.

What the 27 did not produce, and why

An empty render is not one condition; it is at least five, and separating them is the only way the sweep says anything. Taking them in turn, with the check that distinguishes each:

Missing input, supplied. `common.env.environment` needs a `variable` arg and returns nothing without one. Supplying it resolves the row:

```
$ SOMEVAR=hello powerline-render ...
  { "function": "...env.environment", "args": { "variable": "SOMEVAR" } }
  hello
```

The `exec` adapter behaves the same way: it reads `args.command` and returns `None` on the first line if the key is absent (`render_runtime.rs:1214`), so `{ "function": "exec" }` with no command is a no-op and `{ "function": "exec", "args": { "command": "/bin/echo", "args": [ "hello-from-exec" ] } }` renders `hello-from-exec`.

Wrong working directory. `common.vcs.branch` and `powerliners.vcs.git_status` are empty when the render runs in `/private/tmp`, and both produce output when the same theme is rendered from inside a git checkout — `main` and `main` respectively. Nothing about the segment changed; the input did.

Wrong extension. `shell.jobnum`, `shell.last_status`, and `shell.last_pipe_status` read values the shell passes on the command line, and the `tmux` ext never passes them. Rendered through the `shell` ext with the flags supplied, all three light up at once:

```
$ cat $W/themes/shell/default.json
{ "segments": { "left": [
  { "function": "powerline.segments.shell.jobnum" },
  { "function": "powerline.segments.shell.last_pipe_status" },
  { "function": "powerline.segments.shell.last_status" } ], "right": [] } }

$ POWERLINE_CONFIG_PATHS=$W powerline-render shell left \
  --jobnum 3 --last-exit-code 1 --last-pipe-status "0 1 2" | perl -pe 's/#\[([^\]]*)\]/g'
  3  2  1  0  1
```

The theme has to be shown, because the bundled `themes/shell/default.json` puts `last_pipe_status` on the *right* side and carries no `last_status` at all, so the printed command alone renders `wizard 3` and the transcript cannot be replayed from it. With the three segments named explicitly the result is five chunks, not four: the job count `3`, then the three pipeline stages, then the exit code `1`.

The stage order is the part worth stopping on. The input was `"0 1 2"` and the output reads `2 1 0`. That is

not a formatting choice — it is the left-side chunk reversal that Chapter 43 isolates, and it is why the same segment on the right side of a line prints `0 1 2`. Without the flags the render is empty, which is correct: no jobs and a zero exit status are conditions the segments deliberately do not draw (Appendix AL).

Missing group. The ten already tabulated.

Genuinely absent state. The remainder, and the largest single block: the seven media players (`spotify`, `cmus`, `mocp`, `rhythmbox`, `itunes`, `clementine`, `dbus_player`, plus `mpd`), `docker.containers`, `github.ci_status`, `proc.process_count`, `mail.email_imap_alert`, and the `vimlrs` / `elisprs` version rows. Nothing is playing, no daemon is answering, no credentials are configured, and the tools those rows query are not installed or not running. These are the rows a sweep on this machine cannot promote, and saying so is more useful than inventing an environment that would.

One row is in a category of its own, and it is the row the sweep was worth running for. `common.net.network_load` is empty in both passes despite having a group in the bundled colorscheme and despite the same machine's real statusline showing network throughput seconds earlier. The obvious hypothesis is that it is a rate segment needing two samples, and that a cold one-shot process only has one. That hypothesis is wrong, and the way to find out is to hold the process warm and ask again:

```
$ # one-shot, three cold processes
#1: [] #2: [] #3: []
$ # warm daemon, three successive client requests one second apart
#1: [] #2: [] #3: []
```

Still nothing. The actual cause is in the adapter's first four lines (`render_runtime.rs:1388-1392`): the `interface` argument defaults to the string `"auto"`, and the macOS reader matches it literally against column zero of `netstat -ibn (:1415)` — `cols.first().map(|c| *c == interface)`. No interface is named `auto`, so the lookup always misses and the adapter returns `None`. Naming a real one resolves it immediately, and on the *first* call, which also disposes of the two-sample theory:

```
$ ... "args": {"interface": "en0"}
#1: [ ↓ 32 KiB/s ↑ 10 KiB/s ] #2: [ ↓ 49 KiB/s ↑ 1.6 MiB/s ]
$ ... "args": {"interface": "lo0"}
#1: [ ↓ 0 B/s ↑ 0 B/s ] #2: [ ↓ 0 B/s ↑ 0 B/s ]
```

`lo0` at zero is the control: the segment is reading, and reading zero, rather than declining to render. So `network_load` requires an explicit `interface` in the theme, its documented `"auto"` default resolves to nothing on this platform, and a user who copies the segment out of a theme without setting the argument gets silence. That is a one-line finding that no amount of reading the registry would have produced and that two competing explanations had to be tested against before it was safe to state.

The reading

Three numbers, and they are three different claims. Fifty-nine segments are reachable (Chapter 33, proven by the registry tests). Thirty-two render on this machine against a colorscheme that names their groups. Twenty-two render against the colorscheme the project actually ships. The gap between the second and the third is a colorscheme-authoring gap, the gap between the first and the second is an environment gap, and neither is a port gap — but a status line that quoted only the first number would let a reader assume all three.

The same shape recurs in Chapter 18's tier discussion. A filesystem walk counts files; a registry test counts rows; a render counts renders. Each is exactly as informative as its own instrument and no more, and the useful thing a book can do is publish all three side by side so the differences are visible instead of averaged.

Chapter 37 — The Colorscheme and Theme Resolution Chain

A rendered statusline is the product of two independent lookups that meet at the last moment: the theme decides *what* is on the line, and the colorscheme decides *what colour each thing is*. Each is assembled from a cascade of JSON files, and the cascades are not the same shape. This chapter follows both from the moment `build_configs` is entered to the moment a `Colorscheme` and a `Theme` exist, using the code that does it.

Where the names come from

Everything starts with two strings, and both default to "default" (`src/bin/shared/render_runtime.rs:227-244`):

```
if let Some(s) = ext_block.get("colorscheme").and_then(|v| v.as_str()) { cs =  
    ↪ s.to_string(); }  
if let Some(s) = ext_block.get("theme").and_then(|v| v.as_str()) { th =  
    ↪ s.to_string(); }
```

`ext_block` is `config.ext.<ext>` from the merged main config. So `powerline-render tmux right` reads `ext.tmux.colorscheme` and `ext.tmux.theme`, and a tree that omits the `ext.tmux` block entirely still renders — with `default` and `default` — rather than failing. The bundled `config.json` sets both explicitly for `tmux`, which means the defaults are rarely exercised in practice and are there for the tree a user assembles by hand.

The colorscheme cascade: three levels, one merge

The colorscheme is loaded from three probe levels in order (`render_runtime.rs:246-252`):

```
let cs_levels = vec![  
    format!("colorschemes/{}", cs_name),  
    format!("colorschemes/{}/__main__", ext),  
    format!("colorschemes/{}/{}", ext, cs_name),  
];
```

Read left to right: a cross-extension colorscheme of that name, then a per-extension file of defaults shared by every colorscheme of that extension, then the extension-specific file of that name. In the bundled tree the first and third exist for `tmux` and the second does not:

```
$ ls src/ported/config_files/colorschemes/  
default.json ipython pdb shell solarized.json tmux vim  
$ ls src/ported/config_files/colorschemes/tmux/  
default.json solarized.json
```

`colorschemes/default.json` carries 52 groups, `colorschemes/tmux/default.json` carries 10, and the merged result for `ext=tmux`, `colorscheme=default` is 61 distinct groups. The middle level is not dead weight in general — `colorschemes/vim/__main__.json` and `colorschemes/shell/__main__.json` and `colorschemes/ipython/__main__.json` do exist — it is simply absent for `tmux`, which is why a `tmux` colorscheme override has one fewer layer to think about than a `vim` one.

The merge is `load_cascade` (`render_runtime.rs:96-122`), and its shape is the part that matters:

```
for level in levels {
    if let Ok(matches) = _find_config_files(paths, level) {
        for p in &matches {
            ... mergedicts(&mut out, o, true);
        }
    }
}
```

Two nested loops. The outer walks the three levels; the inner walks *every* search path that has a file at that level, not just the first. So the merge order is level-major and path-minor: level one from every root, then level two from every root, then level three from every root. A user file at `colorschemes/default.json` therefore loses to a bundled file at `colorschemes/tmux/default.json`, because level beats path. That is upstream's order and it is the reason the per-extension file is the right place to put an override that must win.

The comment above the loop records why the inner loop exists at all: an earlier version loaded only `matches.first()` and “silently dropped bundled fallback groups (e.g. `tmux/window:current` highlight)” (`:99-102`). Loading only the first match makes the user's file *replace* the bundled one rather than merge over it, and the symptom is a group that quietly stops resolving.

The theme cascade: three levels, and one of them is computed

The theme's three levels look parallel but the first is not a name the user typed (`render_runtime.rs:280-284`):

```
let theme_levels = vec![
    format!("themes/{}", top_theme),
    format!("themes/{}/__main__", ext),
    format!("themes/{}/{}", ext, theme_name),
];
```

`top_theme` is the *top theme*, a cross-extension layer that carries `dividers`, `spaces`, and `segment_data` — the presentation defaults every extension shares. It is resolved by a two-step fallback (`:261-271`): `common.default_top_theme` from the main config if set, otherwise `get_default_theme(encoding.starts_with("utf") || encoding.starts_with("ucs"))`, which returns “`powerline_terminus`” on a UTF-8 locale and “`ascii`” otherwise (`src/ported/mod.rs:314-321`).

That single boolean is the whole of the Unicode-versus-ASCII decision, and it is made from the process's preferred output encoding rather than from a config key. The consequence is a theme file swap, not a glyph substitution: on a UTF-8 terminal the render loads `themes/powerline_terminus.json` and gets `~`-family dividers; on a non-UTF-8 one it loads `themes/ascii.json` and gets something the terminal can draw. Reading the bundled top theme confirms what that layer carries and what it does not:

```
$ python3 -c "import json; d =
~ json.load(open('src/ported/config_files/themes/powerline.json'));
~ print(list(d.keys()))"
['dividers', 'spaces', 'segment_data']
```

Three keys: `dividers`, `spaces` (1), and `segment_data` (19 entries). No `segments`. The top theme can therefore never contribute a segment to a line, which is what makes it safe to merge under every extension's theme — it sets how things are drawn and spaced, never what is drawn.

What the two cascades have in common, and what they do not

Both use `load_cascade`, so both merge level-major and path-minor with `mergedicts`. The difference is what the third level means. For the colorscheme, level three is `colorschemes/<ext>/<name>`, a file the user names in `ext.<ext>.colorscheme`. For the theme, level three is `themes/<ext>/<name>`, named in `ext.<ext>.theme` — but level *one* is not user-named at all in the common case; it is derived from the locale. A user who changes `LANG` changes the theme cascade's base layer without editing any config file, and a user who sets `common.default_top_theme` takes that decision back.

`_find_config_files`: JSON only, all matches, error on none

The primitive under both cascades is 40 lines (`src/ported/mod.rs:104-144`) and it makes three decisions worth naming.

It appends `.json` and only `.json` (`:123`): `let with_ext = format!("{}", config_file);`. There is no YAML branch, no extension probing, no `.jsonc`. A file named `default.yaml` in a colorscheme directory is invisible.

It returns every match rather than the first (`:124-130`), which is what makes the inner loop of `load_cascade` meaningful.

And it returns `Err` — not an empty list — when nothing matched, with the search paths interpolated into the message. That error is what surfaces as `config.json not found` when the tree is empty, and it is the one place the config layer produces a diagnostic a user can act on.

`colors.json`: the layer under the colorscheme

A colorscheme never names a colour; it names a *name*. Resolving those names is `colors.json`'s whole job, and it is loaded separately from the cascade above — by `load_one`, first match only. Its census:

```
$ python3 -c "import json;d=json.load(open('src/ported/config_files/colors.json'));
              print(len(d['colors']), len(d['gradients']))"
65 10
```

Sixty-five named colours and ten named gradients. The colours are the flat part: `"black": 16, "white": 231, "green": 2, "darkestgreen": 22` — a name to a cterm index, or to a `[cterm, "RRGGBB"]` pair where a true-colour value is also supplied. This is the layer that makes a colorscheme portable between terminals: swapping `colors.json` re-maps every group in every colorscheme at once, without touching a colorscheme file.

The gradients are the shaped part, and their structure is worth stating because it explains why a gradient segment is cheap at render time. Each gradient is a two-element list — a cterm ramp and a hex ramp — and the two are *not* required to be the same length, nor are they in practice:

```
$ python3 -c "import json;d=json.load(open('src/ported/config_files/colors.json'))
[print(f'{k:26s} {len(g[0]):4d} {len(g[1]):4d}') for k,g in
  ↳ d['gradients'].items()]"
dark_GREEN_Orange_red      101  101
GREEN_Orange_red           101  100
green_yellow_red           6    101
green_yellow_orange_red    4    100
yellow_red                 5    99
yellow_orange_red          3    99
blue_red                   12    99
white_red                  7    101
dark_green_gray            101  101
light_green_gray           101  101
```

Three of the ten pairs agree; seven do not. Where a ramp holds 101 entries the number is not a coincidence — it is a precomputed lookup for the integer percentages 0 through 100 — but that shape is the exception here rather than the rule, and the disagreement between the two halves of a pair is what Chapter 45 turns into a crash. A segment that reports “82%” does not interpolate a colour; it indexes, once per ramp, with an index derived from that ramp’s own length. `pick_gradient_value` (`src/ported/colourscheme.rs:72-107`) is the function that does the indexing, and the ten ramps — `dark_GREEN_Orange_red`, `GREEN_Orange_red`, `green_yellow_red`, `green_yellow_orange_red`, `yellow_red`, `yellow_orange_red`, `blue_red`, `white_red`, `dark_green_gray`, `light_green_gray` — are the palette every load, temperature, battery, and disk segment draws from.

Two consequences follow. A user who wants a different gradient shape edits `colors.json`, not the `colourscheme`, and gets it everywhere. And a `colourscheme` group referring to a gradient name that `colors.json` does not define fails at the same place a missing colour name fails — inside `Colourscheme::new`, before any rendering — which is a different failure from the missing *group* of Chapter 36 and produces a different symptom.

The two objects that come out

`Colourscheme::new(&colourscheme_json, &colors_json)` (`render_runtime.rs:288`) takes the merged groups *and* the separately-loaded `colors.json`, because a group’s “fg”: “gray8” is a name that only `colors.json` can turn into 247. That is why `colors` is its own probe level in `collect_loaded_paths` (:440) alongside `config` — two files with no cascade of their own, each loaded by `load_one` (:89-94), which takes `matches.first()` and does not merge. Main config and colors are first-match-wins; colourschemes and themes are merge-all. The asymmetry is easy to miss and it decides whether a user’s partial override adds to the bundled file or replaces it.

The theme’s construction (:291-302) hands the merged theme JSON to `gen_segment_getter` with a resolver closure that answers one question — `adapter_id(module, name).is_some()` — and with `Some(top_theme)` so the top theme’s `segment_data` can supply per-segment presentation defaults. Then `prepare_line` (:319-345) walks `segments.left` and `segments.right`, calls the getter on each spec, and on `None` logs and drops:

```
None => {
  powerliners::extensions::diag_log::log(&format!(
    "prepare_line side={} DROP fn={} (get_segment returned None - module lookup
    ↳ failed)",
    side, fn_name
  ));
}
```

That is the second silent drop in the pipeline, and it has a different cause than the `colourscheme` one: this drop is an unresolvable *function name*, the other is an unresolvable *highlight group*. Both produce a shorter line and neither produces a message on stderr. The difference is that this one leaves a record

— `$HOME/.powerliners/powerliners.log`, written by `src/extensions/diag_log.rs`, rotated at 5 MiB with three generations kept (`:17-18`) — so a segment that vanishes because its name is wrong is diagnosable after the fact, and a segment that vanishes because its group is missing is not.

Chapter 38 – The Binding Layers As Shipped

A binding is the code that gets a statusline from powerliners into a program that did not ask for one. Appendix I gives the copy-paste lines and Appendix V covers the tmux conf files' version dispatch. This chapter asks a narrower question with a checkable answer: for each binding target, what is actually in the tree, and what happens when you run it.

The inventory

Target	Shipped artifact	Lines
tmux	<code>src/ported/bindings/tmux/mod.rs</code> + 8 <code>.conf</code> files	228 + 49
vim	<code>src/ported/bindings/vim/mod.rs</code> + <code>powerline.vim</code>	971 + 142
zsh	<code>src/ported/bindings/zsh/mod.rs</code>	1059
zsh (reactive)	<code>src/extensions/shell_hooks/reactive.zsh</code>	70
ipython / pdb / qtile / wm	modules under <code>src/ported/bindings/</code>	—
i3	<i>nothing in the build</i>	—
awesome / bar / lemonbar	<i>nothing in the build</i>	—

The last two rows are the subject of Chapter 18's table and Chapter 29's opening: four files exist on disk under `src/ported/bindings/` whose directories are not declared in `src/ported/bindings/mod.rs:8-15` and which no `[[bin]]` target builds. For a chapter about bindings as shipped, the correct entry for i3, awesome, bar, and lemonbar is a blank one. i3bar output remains reachable — `src/ported/renderers/i3bar.rs` is in the module tree (`src/ported/renderers/mod.rs:9`) and produces the protocol's JSON chunks — but there is no driver binary to feed it, so a user wiring i3 has a renderer and no client.

powerline-config, exercised

Every binding's install path runs through `powerline-config`, so the honest way to describe the binding layer is to run each action and record the exit code:

```

$ for a in "tmux setup" "tmux setenv" "tmux source" "shell command" \
    "vim source-path" "tmux bogus"; do
    out=$(eval "powerline-config $a" 2>&1); printf '%-18s rc=%-2s %s\n' "$a" "$?"
    ~ "${out%%$'\n'*}"
done
tmux setup          rc=0  error connecting to /private/tmp/tmux-501/default (No such file
~ or directory)
tmux setenv         rc=1  powerline-config: action 'setenv' requires the Powerline
~ orchestrator (deferred port)
tmux source         rc=1  powerline-config: action 'source' requires the Powerline
~ orchestrator (deferred port)
shell command      rc=0  powerline
vim source-path    rc=0  /Users/.../.cache/powerliners/vim/powerline.vim
tmux bogus         rc=2  powerline-config: unknown function 'bogus'

```

Two of the six deserve comment.

`tmux setenv` and `tmux source` are not implemented. The dispatch arm is explicit about it (`src/ported/scripts/powerline_config.rs:452-461`): both `StrFunction::Source` and `StrFunction::Setenv` fall into one branch that prints “requires the Powerline orchestrator (deferred port)” and returns 1, with the source comment recording the reasoning — “only `tmux_setup` is needed for the daemon-swap path” (:455). That is a defensible scope decision and it is also a documented capability of the upstream tool that this port does not have. `tmux setup` is implemented (:435-450, calling `tmux_setup` at :29), which is the one that matters for the install path, and it does the `setenv`-then-`source` work internally rather than by shelling out to its own subcommands.

`tmux setup` returned 0 while printing tmux’s own connection error. It calls the `tmux` binary to set environment variables and source conf files, and a failure there does not become a nonzero exit. For an install line written as `run-shell "powerline-config tmux setup"` inside a live tmux that is irrelevant, because tmux is by definition running; for anything that checks the exit status it means success is not being reported.

An off-by-one in `shell uses`

`powerline-config shell uses <component>` is the gate a shell hook uses to decide whether to draw a powerline prompt at all — upstream’s contract is exit 0 if the component is enabled, 1 if not. On this tree it returns 1 for every component:

```

$ env -i HOME=$HOME PATH=/usr/bin:/bin powerline-config shell uses tmux; echo $?
1
$ env -i HOME=$HOME PATH=/usr/bin:/bin powerline-config shell uses prompt; echo $?
1
$ env -i HOME=$HOME PATH=/usr/bin:/bin powerline-config shell uses nosuch; echo $?
1

```

All three should not agree. With no `ext.shell.components` key in the config the code falls back to `vec!["tmux", "prompt"]` (`powerline_config.rs:428`), so the first two should exit 0.

The cause is an argument index. `main` receives the full argv, and `action_name` is taken from `args.get(1)` for a `tmux/shell` prefix (:286-293) — correct. The `Uses` arm then reads the component from `args.get(1)` as well (:336), which is the action name, not the component. For `powerline-config shell uses tmux` the component under test is the literal string `uses`.

Two experiments confirm it rather than infer it. Staging a config whose components list is `["uses"]` flips the result:

```

$ POWERLINE_CONFIG_PATHS=$usescfg powerline-config shell uses tmux; echo $?
0

```

And the environment-variable gate, which builds its name as `POWERLINE_NO_{SHELL}_{COMPONENT}` (:360-364), responds to the wrong variable — against that same baseline of 0:

```
$ POWERLINE_CONFIG_PATHS=$usescfg POWERLINE_NO_SHELL_TMUX=1 \  
  powerline-config shell uses tmux; echo $?  
0  
$ POWERLINE_CONFIG_PATHS=$usescfg POWERLINE_NO_SHELL_USES=1 \  
  powerline-config shell uses tmux; echo $?  
1
```

Both invocations keep the staged config from the previous transcript, and they have to: the baseline being contradicted is the 0 that only that config produces. Run without it, both lines exit 1 — the same 1 every component produces — and the experiment shows nothing. A transcript that drops the environment which established its own baseline is a transcript that cannot be replayed, which is the failure Chapter 49 counts.

`POWERLINE_NO_SHELL_TMUX` has no effect; `POWERLINE_NO_SHELL_USES` decides the answer. The fix is one character — `args.get(2)` — and the reason the defect survived is instructive: the arm's own unit tests check the argument-parsing shape and the two `StrFunction` lookups, and no test runs the binary with a component name and asserts an exit code. A behaviour that only manifests as a process exit status needs a test that observes a process exit status.

The practical impact is bounded, because the shipped install lines in Appendix I do not gate on `shell uses`. A user who copies upstream powerline's conditional hooks — which do — gets a prompt that silently declines to draw.

The zsh side, and the one binding that is not a port

`src/ported/bindings/zsh/mod.rs` is the largest binding module in the tree at 1059 lines, and it is a faithful port of upstream's zsh integration. Beside it sits something upstream has no counterpart for: `src/extensions/shell_hooks/reactive.zsh`, 70 lines, whose job is to give the daemon somewhere to push to.

The mechanism is four zsh primitives. `_powerline_reactive_setup` makes a private directory and a FIFO inside it (`reactive.zsh:35`), exports the path as `POWERLINE_RESET_FIFO` (:38), opens it, and registers the fd with `zle -F` (:46) so ZLE calls a handler when the descriptor becomes readable. The handler's entire body is `zle reset-prompt` (:55). Teardown unregisters the fd, removes the FIFO, and unsets the three variables (:61-66).

The variable name is the whole contract between the two halves. The client exports it, the client's request frame carries the environment across the socket, and the daemon's `render_fn` reads it back out — `if let Some(fifo) = environ.get("POWERLINE_RESET_FIFO")` at `src/bin/powerline-daemon.rs:60` — and registers a watch keyed on it. A shell that never sets the variable is never watched, and nothing else changes. That is why this is the one binding whose absence costs nothing: the pull render is unmodified and unaware, which the watch module's header states as its layering rule (`src/extensions/watch.rs:24-31`).

The consequence for the binding inventory is a fifth row that does not fit the others. `tmux`, `vim`, `ipython`, `pdb`, `qtile`, and `wm` are ports of upstream bindings. The reactive zsh hook is not a port of anything — it is the client half of a capability upstream does not have, and it lives under `src/extensions/` for exactly that reason.

The vim binding, end to end

The vim path is the most complete of the three. `powerline-config vim source-path` extracts the plugin from the binary and prints where it landed:

```
$ powerline-config vim source-path
/Users/.../.cache/powerliners/vim/powerline.vim
$ ls -l ~/.cache/powerliners/vim/
-rw-r--r-- 1 ... 4865 ... powerline.vim
```

and the daemon's vim branch produces a wire format the plugin can execute directly. Rendering the `vim` ext against a two-segment theme:

```
$ POWERLINE_CONFIG_PATHS=$vimcfg powerline-render vim left
hi PL_231_None_31_None_bold ctermfg=231 guifg=NONE guibg=NONE ctermbg=31 cterm=bold
  ~ gui=bold
hi PL_31_None_236_None_NONE ctermfg=31 guifg=NONE guibg=NONE ctermbg=236 cterm=NONE
  ~ gui=NONE
hi PL_247_None_236_None_NONE ctermfg=247 guifg=NONE guibg=NONE ctermbg=236 cterm=NONE
  ~ gui=NONE
hi PL_236_None_None_None_NONE ctermfg=236 guifg=NONE guibg=NONE ctermbg=NONE cterm=NONE
  ~ gui=NONE

%#PL_231_None_31_None_bold# wizard %#PL_31_None_236_None_NONE#...
  ~ %#PL_247_None_236_None_NONE#2026-08-09 ...
```

Four `hi` commands, an empty separator line, then the `%#Group#`-marked statusline — exactly the format the plugin's header documents (`src/ported/bindings/vim/powerline.vim:16-28`), and exactly what its three end-to-end tests in `tests/vim_e2e.rs` assert: that the extracted script loads in headless vim, that it defines its load guard and refresh command, and that the daemon's `vim` response has the documented shape. The daemon holds `VimRenderer` state across requests (`render_runtime.rs:450-454`), so a group already declared is not re-declared — which is why the count of `hi` lines shrinks on subsequent renders and why the empty separator, rather than a fixed line count, is the delimiter.

The shell ext renders tmux markup

The one structural surprise in the binding layer is visible from the sweep of Chapter 36. Rendering the `shell` ext produces tmux-format highlight markup:

```
$ powerline-render shell left --jobnum 3 --last-exit-code 1 --last-pipe-status "0 1 2"
#[fg=colour220,bg=colour166,nobold,noitalics,nounderscore] 3
  ~ #[fg=colour166,bg=colour52,...]
```

`#[fg=...]` is tmux's syntax, not a shell prompt escape. The dispatch explains it: `render_once` branches on exactly one extension. `let ext_is_vim = configs.ext == "vim";` at `render_runtime.rs:2798`, and both `hl_fn (:2807-2811)` and `hlstyle_fn (:2816-2826)` select `render_hlstyle_vim` when that is true and `render_hlstyle(&tmux_...)` otherwise. `make_renderer (:2673-2681)` installs `TmuxRenderer::character_translations()` unconditionally. There is no shell branch.

This is the same fact Chapter 18 records from the tier side, seen from the runtime side. `BashPromptRenderer::do_render` and the `ShellRenderer` base it would call are the unexecuted bodies; the shell renderers exist as escape-marker constants and translation tables, correct and tested, with nothing dispatching to them. A shell binding that needs `\[/\]` or `%{/%}` non-display regions is therefore not yet wired, and the tmux markup a `shell` render emits is what a tmux status line wants rather than what a prompt wants. The constants are right, the tables are right, the parity tests on them pass, and the dispatch that would use them is a one-line branch that is not there.

Chapter 39 – The Config Format and Its Precedence Rules

Appendix B is the schema reference and Appendix N walks one merge. This chapter is about the rules that decide which file wins, demonstrated by running them: how the search path is built, what replaces it, and what the four possible outcomes of an overlap actually are.

The search path, in the order it is built

`search_paths()` ([src/bin/shared/render_runtime.rs:35-87](#)) is a short function with one early return and three appends, and the order is the whole specification.

The early return is `POWERLINE_CONFIG_PATHS`. When it is set and non-empty after colon-splitting, it is returned as the entire path and nothing else is consulted ([:42-51](#)). The comment states the intent plainly — “When `POWERLINE_CONFIG_PATHS` is set it REPLACES the default cascade entirely — no bundled fallback” — and this is the isolation mechanism every measurement in this book relies on.

Otherwise three sources are appended in order. First the live checkout’s `src/ported/config_files` when `CARGO_MANIFEST_DIR` was baked in and the directory exists ([:66-71](#)). Second the extracted copy of the bundled tree under the cache directory ([:78-84](#)), always pushed so a release-tarball install still has a base layer. Third `get_config_paths()` ([:85](#)), which is the ported XDG resolution.

`get_config_paths()` itself ([src/ported/mod.rs:278-307](#)) builds its list in an order that looks backwards until you know what consumes it:

```
let mut config_paths = vec![ PathBuf::from(format!("{}", powerline, config_home)) ];
for p in dir_paths.into_iter().rev() { config_paths.insert(0, p); }
let plugin_path = PathBuf::from("config_files");
config_paths.insert(0, plugin_path);
```

`$XDG_CONFIG_HOME/powerline` — defaulting to `~/.config/powerline` — goes in first and every `$XDG_CONFIG_DIRS` entry is inserted *before* it, then a relative `config_files` is inserted before all of them. The result is ordered least-specific-first: bundled, then system directories, then the user’s own. That is the order `load_cascade` wants, because it merges in path order and the last merge wins.

Stated as a rule: **later in the search path beats earlier**. It is the opposite of `$PATH`, and it is the single most common thing to get backwards when reading a powerline config problem.

Four outcomes of an overlap, each run

Two config roots, `A` (a full tree) and `B` (an overlay), rendered as `POWERLINE_CONFIG_PATHS=A:B` so `B` is later and therefore wins. `A`’s theme is two right-hand segments with visible ASCII dividers so the spacing is legible.

```
$ render "$A"
< wizard| 2026-08-09
```

A scalar is overwritten. `B` contains `{"spaces": 3}`:

```
$ render "$A:$B"
< wizard| 2026-08-09
```

One space became three on both sides of every divider. B's scalar replaced A's.

Order decides. The same two roots, reversed:

```
$ render "$B:$A"
< wizard| 2026-08-09
```

Back to one space. Nothing about the files changed; B is now earlier, so A's `spaces` merged over it. The precedence is positional, not by specificity, not by file size, not by which tree looks more like a user config.

A list is replaced wholesale, not merged. B contains a `segments.right` of its own with a single `hostname`:

```
$ render "$A:$B"
< code-labs-arm
```

A's two segments are gone entirely. `mergedicts` recurses only when both sides are objects (`src/ported/lib/dict.rs:121-127`); a JSON array on either side takes the else branch and overwrites. To change one segment on a side you restate the whole side.

A key can be deleted, not just overwritten. B contains the removal sentinel:

```
{"segments": {"right": {"__powerliners_remove_this_key__": true}}}
```

```
$ render "$A:$B"
```

Empty — not an empty-but-present side, but no side at all. `mergedicts` checks for the sentinel when `remove` is true and calls `dl.remove(&k)` (`dict.rs:128-136`), and every cascade in the render path passes `true`. This is the one operation overwriting cannot express, and it is why the sentinel exists.

What the format is not

`_find_config_files` appends `.json` and nothing else (`src/ported/mod.rs:123`). There is no YAML loader in the render path, no extension probing, no comment-stripping pre-pass. The `markedjson` tree under `src/ported/lint/` — with its `ScalarNode/SequenceNode/MappingNode` machinery — is the linter's parser, not the renderer's; it exists so `powerline-lint` can report a problem with a line and column, and the renderer uses `load_json_config` instead. A file that a JSON parser rejects does not produce a parse error at render time; `load_one` and `load_cascade` both take `Result` values and drop the `Err` on the floor (`:90-93`, `:106-113`), so a malformed override is indistinguishable from an absent one.

That is worth pairing with the linter's existence. The renderer is deliberately unbothered by a bad file — it renders what it could load — and the tool that will tell you the file was bad is a separate binary you have to choose to run. Appendix A1 catalogs its checks.

What a missing config produces

The one hard failure in the config layer is the main config itself:

```
$ POWERLINE_CONFIG_PATHS=$empty powerline-render tmux right
powerline-render: config error: config.json not found
$ echo $?
0
```

The message is right and the exit code is wrong. `build_configs` returns `Err("config.json not found")` (`render_runtime.rs:224`), the caller prints it, and the process exits 0 anyway. A prompt hook that tests the exit status of `powerline-render` will conclude the render succeeded and print the diagnostic into the user's

prompt. This is the same class of finding as the `shell uses` exit code in Chapter 38 — behaviour that is only observable as a process exit status, and therefore behaviour no in-process test is positioned to catch.

`-p` adds, the environment variable replaces

There are two ways to point a render at a config tree and they are not equivalent, which is worth one demonstration because it is the single easiest way to take a measurement of the wrong program. `-p PATH` reaches `search_paths()` as an *addition* to the cascade — the user's own tree is still on the path and, being later, still wins the merge. `POWERLINE_CONFIG_PATHS` returns early and replaces the cascade outright (`render_runtime.rs:42-51`).

Rendering a fixture with the flag produces the machine's installed statusline, thirteen segments long, none of which the fixture defines. Rendering the same fixture with the environment variable produces the fixture's three. Same directory, same binary, same second; the flag measured the user's config and the variable measured the fixture.

Upstream's `--config-path` replaces rather than adds, so the two tools disagree about what the flag means — which is why the daemon-parity harness passes `-p` to Python and the environment variable to Rust and lands both sides on the same tree (Chapter 40). For a reader running the transcripts in this book, the operational rule is simply that every isolated measurement here uses the environment variable, and a transcript that used `-p` would be measuring something else.

`local_themes`: the fourth theme layer

The three-level theme cascade is not the whole story for extensions that switch themes at runtime. A shell config can carry a `local_themes` map inside its `ext.shell` block:

```
"ext": { "shell": { "colorscheme": "default", "theme": "default",
                  "local_themes": { "continuation": "continuation", "select": "select" }
                } }
```

Each key is a mode name and each value is a theme name to load in that mode. This is the mechanism behind the mode machine of Appendix AF: the renderer is asked for a mode, the mode selects a local theme, and that theme is resolved through the same `themes/<ext>/<name>` level as the base one. It is a fourth axis rather than a fourth cascade level — the base theme is not replaced by the local one for every render, only for renders in that mode — and it is the reason a shell config often has more theme files than an extension with a single presentation.

The files that are watched

The last thing `build_configs` does before returning is record what it read (`render_runtime.rs:438-448`), and the level list is the union of both cascades plus the two first-match files:

```
let probe_levels: Vec<String> = vec![
    "config".to_string(),
    "colors".to_string(),
    format!("colorschemes/{}", cs_name),
    format!("colorschemes/{}/__main__", ext),
    format!("colorschemes/{}/{}", ext, cs_name),
    format!("themes/{}", top_theme),
    format!("themes/{}/__main__", ext),
    format!("themes/{}/{}", ext, theme_name),
];
```

Eight levels, and `collect_loaded_paths` (:206-220) records one `(path, mtime)` entry per match per level — every layer, not just the winner. The comment above it says why: watching `matches.first()` means watching the bundled file with its install-time mtime and missing the user's file entirely, since the user layer comes last (:432-437). The staleness check that consumes this list is Chapter 41's subject.

Chapter 40 – The Python Oracle: How the Comparison Is Actually Made

Chapter 30 measures what the parity suite does on a tree with no interpreter: 462 tests, 346 of which compare nothing. This chapter assumes the interpreter is there and asks the prior question — when the comparison does happen, what exactly is being compared? The answer is more specific than “the Python and the Rust agree,” and the specifics decide what kinds of divergence the harness can see.

Everything crosses the boundary as text

`py_eval` (`tests/parity_against_upstream.rs:22-49`) is fourteen lines of substance and the shape is the methodology:

```
let script = format!("import sys; sys.path.insert(0, '{}'); print({})", vendor.display(),
    & expr);
let out = Command::new("python3").arg("-c").arg(&script).output().ok()?;
...
let s = String::from_utf8_lossy(&out.stdout).into_owned();
Some(s.strip_suffix('\n').unwrap_or(&s).to_string())
```

A fresh `python3 -c` per call, with `sys.path` pointed at the vendored checkout, and the expression wrapped in `print()`. The oracle’s answer is therefore whatever `print()` writes — `str()` of the value — and the comparison on the Rust side is against a `String`. Nothing structured crosses; there is no pickle, no JSON envelope, no type tag.

Three consequences follow directly.

The comparison is of textual renderings, not of values. A Python `True` arrives as `"True"`, a float as its `repr`, a list as `"['a', 'b']"` with Python’s spacing and quoting. Every assertion is implicitly also an assertion about formatting, and a Rust function that computes the right value and formats it differently fails. That is stricter than value equality in some places and looser in others: two objects that print identically compare equal even if they differ in a field `__str__` does not show.

The trailing newline is stripped and nothing else is. The code says so in a comment — “Strip ONLY the trailing newline that `print()` adds — preserve any trailing whitespace that’s part of the function’s return value” (:45-46). For a project whose output is padded statusline chunks, that distinction is not pedantry; a segment that returns `" 42 "` and one that returns `" 42"` are different bugs, and a `.trim()` would have hidden one of them.

Failure is indistinguishable from absence. A non-zero Python exit prints the stderr and returns `None` (:38-44) — the same `None` the missing-vendor branch returns at :25-27. Every call site treats `None` as “skip”. So an expression that raises in Python does not fail the test; it skips it, with a diagnostic that is captured unless `--nocapture` is passed. A malformed parity expression is therefore silently costless, which is the strongest argument for the runtime check of Chapter 30 being the real signal rather than the pass count.

The shape of the expressions

Counting what the 13,734-line file actually asks Python:

Construct	Count
<code>#[test]</code> functions	462
<code>py_eval()</code> call sites	368
<code>__import__()</code> in expressions	328
<code>format!()</code> used to build an expression	198
<code>json.dumps</code> in expressions	35
<code>sorted()</code> in expressions	11
<code>repr()</code> in expressions	2
<code>assert_eq!</code> sites	738
<code>assert!</code> sites	152

`__import__(...)` appears in 328 of the 368 calls, which is the harness's uniform way of reaching a module attribute without an import statement in a `-c` one-liner:

```
py_eval("__import__('powerline.version', fromlist=['__version__']).__version__")
```

The 35 `json.dumps` sites are the escape hatch for the values `str()` renders ambiguously — dictionaries, whose Python `str` ordering and quoting are not a stable contract worth asserting against. The 11 `sorted()` sites do the same job for collections whose order is not defined. Two `repr()` sites handle values whose `str` is lossy. Everything else — 320 of the 368 — is compared as plain `str()` output, which tells you how much of upstream's surface is scalars and short strings.

The second harness compares bytes, not text

`tests/daemon_parity.rs` is a different instrument aimed at the same target. Instead of evaluating an expression, it runs the two complete programs and diffs their output streams (:153-181, :183-213):

- Python: `python3 vendor/powerline/scripts/powerline-render tmux <side> -p <fixture>` with `PYTHONPATH=vendor/powerline`, stderr discarded, non-zero exit treated as unavailable.
- Rust: spawn `powerline-daemon --foreground --socket <tmp>` with `POWERLINE_CONFIG_PATHS=<fixture>`, poll for the socket to accept for up to 15 seconds (:101-112), then write a hand-built wire request and read to EOF.

Note the asymmetry in how each side is pointed at the fixture: `-p` for Python, the environment variable for Rust. Both reach the same state because upstream's `--config-path` replaces the default cascade while the port's `-p` adds to it — the trap Chapter 32 documents, handled here by using the flag that replaces on each side.

The comparison is one line of normalization and then raw bytes:

```
let py_trim: &[u8] = py.strip_suffix(b"\n").unwrap_or(&py);
...
assert_eq!(rust, py2_trim, "byte parity failed for scenario `{}` side `{}` ...");
```

Python's `powerline-render` writes a trailing newline and the daemon's socket response does not, so exactly one byte is removed and nothing else is touched. No colour normalization, no whitespace collapsing, no token reordering. The header states what that buys: “any divergence (missing `bg=default`, dropped `nobold/noitalics/nounderscore` resets, re-ordered color tokens, off-by-one cterm index, etc.) fails the test loudly” (:12-15).

The harness speaks the wire protocol rather than running the client

One detail of `daemon_parity.rs` decides how much of the Rust side is under test. It does not run `powerline` and read its stdout; it builds the request frame itself (:115-132) and writes it to the socket:

```
out.extend(format!("{:x}", args.len()).as_bytes());
out.push(0);
for arg in args { out.extend(arg.as_bytes()); out.push(0); }
out.extend(cwd.as_bytes()); out.push(0);
for (k, v) in env { out.extend(format!("{}", k, v).as_bytes()); out.push(0); }
out.push(0); out.push(0);
```

Hex argument count, NUL, each argument NUL-terminated, the cwd, each **KEY=VALUE**, then two NULs. That is the same frame Chapter 31 walks byte by byte, restated in the test as a second, independent implementation of the encoder.

The consequence for coverage is precise and cuts both ways. The C-compatible client binary is *not* in the loop for these 45 tests — a bug in the client's frame construction would not fail any of them — but the daemon's *parser* is, and it is being fed by an encoder that was written from the specification rather than by the encoder it normally talks to. Two independent implementations agreeing on a wire format is a stronger check of the format than one implementation talking to itself. The client's own encoder is covered elsewhere, by `tests/bin_powerline_client.rs` and its 12 tests — which is a smaller number than the surface deserves, and the layering is worth stating plainly because it is easy to read “45 daemon parity tests” as covering the client. It does not. The client is checked against the specification by 12 tests; the daemon is checked against Python by 45; and the two never meet in a single assertion.

The environment is pinned, and that is the point

The frame the harness sends carries a deliberately impoverished environment (`daemon_parity.rs:142-147`):

```
conn.write_all(&build_request(argv, "/tmp", &[("HOME", "/tmp"), ("PWD", "/tmp")]))
```

Two variables and a cwd of `/tmp`. Python's side gets whatever the test process has, but the fixtures are written so that nothing in them reads the environment — the scenarios use `date`, `hostname`, string segments, and fixed-width cases rather than `virtualenv`, `cwd`, or `branch`.

That is a methodological choice with a cost. Byte-for-byte comparison of two programs is only possible when every input they read is equal, and the cheapest way to guarantee that is to choose fixtures whose segments read almost nothing. The 45 scenarios therefore cover composition — divider placement, highlight resolution across adjacent segments, truncation, escaping, mode switching — and cover almost none of the segment implementations. The segments are checked one at a time by `parity_against_upstream.rs`, against `str()` output, which is the other harness's job. Neither harness checks a machine-dependent segment against Python, and neither claims to.

The one deliberate looseness

There is a single retry in the whole harness (:197-212). When the first comparison fails, both sides are re-rendered and the assertion is made against the second pair. The reason is named in the file header: the fixtures include `date` and `time` segments formatted to `%Y-%m-%d` and `%H:%M`, and two separately-spawned processes can land on opposite sides of a minute boundary (:20-25).

This is worth being precise about, because a retry-on-failure is exactly the shape of a test that hides a real

bug. What makes it safe here is that the retry compares a *fresh* Python render against a *fresh* Rust render rather than re-running only one side: a genuine divergence reproduces in the second pair and fails, while a minute-boundary race does not reproduce twice in a row with any meaningful probability. A retry that re-ran only the Rust side against the cached Python output would have the failure mode; this one does not.

The width variant (:219-242) has no retry at all, because it passes `-w N` to both sides and the truncation path is deterministic.

What the two harnesses cover between them

Stated as a division of labour: `parity_against_upstream.rs` compares *functions* through their textual renderings, one call at a time, and covers breadth — 368 call sites across the module tree. `daemon_parity.rs` compares *whole rendered lines* byte for byte, and covers composition — 45 tests over 42 distinct fixture directories, which is the only place the interaction between segments, dividers, highlight resolution, and truncation is checked against upstream at once.

Neither covers the third thing. A segment whose value depends on the machine — `battery`, `external_ip`, `git_status` — cannot be compared against upstream by either harness, because the two implementations sample different moments and, for the extension segments, upstream has no counterpart to sample. Chapter 36's sweep is the closest thing to a check on those, and it checks only that they produce something. The parity claim covers the port's ported surface, which is what it says on the label, and the extension tier is verified by its own unit tests and by nothing that compares it to Python — because there is nothing to compare it to.

Chapter 41 – The Daemon’s Lifecycle and Its Failure Modes

Appendix E documents the daemon’s protocol and Appendix AG its signal handling. This chapter runs the daemon and watches it start, refuse to start, reload, and stop, on a probe socket that is not the user’s. Every figure below is from a transcript.

Starting

`powerline-daemon -s <socket>` returns immediately and leaves a process behind:

```
$ powerline-daemon -s /private/tmp/probe.sock
$ ls -l /private/tmp/probe.sock*
srwxrwxrwx 1 ... 0 ... /private/tmp/probe.sock
-rw-r--r-- 1 ... 4 ... /private/tmp/probe.sock.pid
$ ps -o pid,ppid,stat,command -p $(cat /private/tmp/probe.sock.pid)
  PID  PPID  STAT  COMMAND
  4958    1   S    .../powerline-daemon -s /private/tmp/probe.sock
```

PPID 1 is the tell: the process was double-forked away from the shell. `daemonize()` (`src/ported/scripts/powerline_daemon.rs:925-978`) forks, exits the parent, `chdir("/")`, `setsid()`, `umask(0)`, forks again, exits again, then redirects fds 0/1/2 to `/dev/null (:974, :980-1010)`. Two artifacts remain on disk: the socket, and a `.pid` file at `<address>.pid (:1250-1254)` holding the pid, locked with `fcntl(F_SETLK, F_WRLCK)` for as long as the daemon lives (`:1125-1142`).

The ordering of the startup steps is where the interesting failure mode lives. `main` does them in this order (`:1291-1324`): `daemonize`, then `lockpidfile`, then `check_existing`.

Refusing to start, invisibly

Starting a second daemon on a busy socket in the foreground says so and exits 1:

```
$ powerline-daemon -f -s /private/tmp/probe.sock
The daemon is already running. Use powerline-daemon -k to kill it.
$ echo $?
1
```

Starting it the normal way does neither:

```
$ powerline-daemon -s /private/tmp/probe.sock
$ echo $?
0
```

Nothing printed, exit 0, and no new daemon. The message is not missing — it is written to a stderr that `daemonize()` already pointed at `/dev/null` two steps earlier, and the exit code belongs to the *parent*, which `process::exit(0)`s at `:942` before the child ever reaches `lockpidfile`. The child’s `return 1` at `:1304` has nowhere to go.

The running daemon is unharmed, which is the important half:

```
$ cat /private/tmp/probe.sock.pid
16378      (unchanged)
```

`lockpidfile` fails before `check_existing` runs, and `check_existing` is the function that unlinks the socket path before binding (:1026-1028). Had the order been reversed, a second start would have unlinked a live daemon's socket and then failed to bind, orphaning the first daemon behind an address nothing could reach. The ordering is load-bearing and it is not obvious from reading either function alone.

The practical consequence for an install line: `powerline-daemon -q` in a shell rc file is safe to run on every shell start, and it is also incapable of telling you it did nothing.

Replacing

`--replace` is a loop, not a flag (:1280-1289): kill, sleep two seconds, kill again, until `kill_daemon` reports no one answering. Measured:

```
$ powerline-daemon -r -s /private/tmp/probe.sock -q
$ cat /private/tmp/probe.sock.pid
16883      (was 16378)
```

New pid, same socket, and the first client through afterwards renders normally.

Stopping, and the two seconds

`--kill` is not a signal. `kill_daemon` (:1039-1050) connects to the socket and writes the five-byte sentinel `EOF\0\0` (:73); the daemon's read loop compares the request to it and returns from `do_one` with an exit code (:759-760). The `--kill` invocation itself returns as soon as the write lands, so its exit status says the message was delivered, not that the daemon is gone:

```
$ powerline-daemon -k -s /private/tmp/probe.sock
Kill command sent to daemon, if it does not die in a couple of seconds use kill to kill it
```

"A couple of seconds" is exact. Measured three times, spawning a fresh daemon each round and polling `ps` every 20 ms from the moment `-k` returned:

```
run 1: pid=20351 exit latency 2.012s
run 2: pid=20568 exit latency 2.049s
run 3: pid=20821 exit latency 2.053s
```

The two seconds are `shutdown()`'s budget: `let total = Duration::from_secs(2);` at `powerline_daemon.rs:807`, ported from upstream's `total_wait_time = 2` (sh:275). It is the window in which in-flight connections are allowed to drain. A caller that kills and immediately restarts on the same socket will collide with a daemon that has not finished draining, which is precisely why `--replace` sleeps two seconds per iteration rather than spinning.

`--kill` against nothing is a clean 1:

```
$ powerline-daemon -k -s /private/tmp/probe.sock
No running daemon found
$ echo $?
1
```

and mutually exclusive flags are rejected before anything happens, with the one exit code in this family that is unambiguous:

```
$ powerline-daemon -k -f -s /private/tmp/probe.sock
powerline-daemon: --kill and --foreground/--replace cannot be used together
$ echo $?
2
```

What is left on disk

After the daemon exits, the pidfile is gone — the `PidLock` guard unlinks it on drop — and the socket inode is not:

```
$ ls probe.sock probe.sock.pid
ls: probe.sock.pid: No such file or directory
probe.sock
```

A stale socket file is not a problem for the next start, because `check_existing` unlinks any non-abstract address before binding (`:1026-1028`). It is a problem for anything that treats the file's existence as “a daemon is running” — the pidfile is the artifact that answers that question, and it is the one that gets cleaned up.

The daemon's environment is the daemon's, not the client's

The client sends its environment across the wire; the daemon renders with it. But the *config search path* is read from the daemon's own process environment, because `search_paths()` calls `std::env::var("POWERLINE_CONFIG_PATHS")` (`render_runtime.rs:42`) inside the daemon. Two trees, two different themes, one experiment:

```
$ POWERLINE_CONFIG_PATHS=$treeA powerline-daemon -s $sock      # treeA's theme renders
~ `date`
$ POWERLINE_CONFIG_PATHS=$treeB powerline --socket $sock tmux right
2026-08-09
$ POWERLINE_CONFIG_PATHS=$treeB powerline-render tmux right
wizard
```

Same environment variable on both client invocations, two different answers. The one-shot renderer honours it because it resolves the path in its own process; the client cannot, because resolution happens on the far side of the socket. For a user this is invisible — the daemon and the shell usually share an environment — and for anyone testing, it is the difference between a measurement and a misreading. Chapter 40's daemon-parity harness handles it by spawning a daemon per scenario with the fixture in *its* environment (`daemon_parity.rs:96`).

Reloading without restarting

The daemon re-reads its config when a watched file's mtime moves. Editing the theme under a running daemon, with no restart and no signal:

```
$ powerline --socket $sock tmux right
2026-08-09
$ printf '%s' '{"segments":{"right":[{"function":"...env.user"}]}}' >
~ $tree/themes/tmux/default.json
$ powerline --socket $sock tmux right
wizard
```

The check is `Configs::is_stale()` (`render_runtime.rs:167`), stat-ing every `(path, mtime)` pair recorded at build time, run on each request by the daemon's `render_fn` (`src/bin/powerline-`

`daemon.rs:72-80`). Stale means the cache entry is dropped and `build_configs` re-runs; the reload is logged to the diagnostic log, not to the client.

Setting `common.reload_config` to `false` pins it:

```
$ # reload_config: false
$ powerline --socket $sock tmux right
2026-08-09
$ # ...theme edited to `user`...
$ powerline --socket $sock tmux right
2026-08-09
$ # control: reload_config default (true), fresh daemon, same edit
2026-08-09 → wizard
```

Two runs, one variable changed, opposite outcomes. The default is `true` (`render_runtime.rs:426-430`), so the polling cost is paid on every render unless the user opts out — a `stat` per loaded config file, which for the bundled cascade plus a user tree is a small handful of syscalls, and which is the price of a config that takes effect without a restart.

The three timeouts

A long-lived process that serves a prompt has to be wrong about the network in bounded time, and the daemon has exactly three numbers for that, at three different scopes.

The **event loop** blocks in `poll` for 60 seconds: `libc::poll(pfds.as_mut_ptr(), pfds.len() as libc::nfds_t, 60_000)` (`powerline_daemon.rs:690`). Nothing happens on expiry — the loop iterates and polls again — so this is not a deadline, it is a wakeup granularity. Its practical effect is that an idle daemon wakes once a minute and does nothing, which is what makes an idle daemon invisible in a process listing sorted by CPU.

A **single request read** gets 2 seconds: `do_read(&mut conn.stream, Duration::from_secs_f64(2.0))` (`:757`). A client that opens a connection and stalls mid-frame is dropped rather than blocking the loop. This is the timeout that keeps one wedged client from freezing every other shell on the machine, and it is per-connection rather than global.

Shutdown gets 2 seconds, the **total** of `shutdown()` (`:807`), measured in the transcript above at 2.012 / 2.049 / 2.053 seconds. It is a drain window, not a kill timer: connections that finish inside it are served.

The harness adds a fourth number that is not the daemon's. `daemon_parity.rs:102` gives a freshly spawned daemon 15 seconds to accept on its socket before declaring failure, polling every 25 ms. That budget is about cold-start cost — building the config cascade from JSON on first request — and it belongs to the test rather than to the program, which is the correct place for it.

The failure modes, collected

Situation	Observable	Exit code
second start, foreground	message on stderr	1
second start, backgrounded	nothing	0
<code>--kill</code> with no daemon	"No running daemon found"	1
<code>--kill</code> with a daemon	message; process exits ~2.0 s later	0
<code>--kill</code> with <code>--foreground/--replace</code>	usage error	2

Situation	Observable	Exit code
config error inside the daemon	error text returned to the client as the statusline	—
client with no daemon	correct markup via the direct renderer	0
client whose peer hangs up mid-write	<code>powerline: write() to daemon failed</code> on stderr	1

The last three rows are the ones a prompt hook feels, and an earlier printing of this table merged the final two into one row that was wrong in both halves. A client with no daemon does not print a write failure and does not fail: it execs the direct renderer and the prompt is correct, only slower, which is the whole point of the fallback. The write failure belongs to a narrower case — a peer that accepts the connection and then hangs up, which is what a daemon in the middle of shutting down does — and that case exits **1**, not 0, with empty stdout. Chapter 47 measures the exit code of every path in the suite; the short version is that this row is one of only three places in the whole CLI where a failure is reported by status rather than by text on stdout.

A daemon that cannot build its config returns the error text *as the rendered line* (`powerline-daemon.rs:97`) — the error appears in the prompt rather than on stderr, which is the defensible choice given that the client's stdout *is* the prompt. The direct renderer does the same thing and adds an exit code of 0 to it (Chapter 39, Chapter 47), which is the part a hook cannot defend against.

Chapter 42 – The Watcher, and Why Its “Debounce” Is Not a Timer

The daemon has two ways of noticing that the world changed. One polls file mtimes on each request and rebuilds the config — Chapter 41’s `is_stale`. The other watches the filesystem and pushes a redraw at the shell without waiting to be asked. The second is `src/extensions/watch.rs`, 436 lines, and it is not a port: upstream powerline has no equivalent, because its own tree watcher feeds the pull render and never signals back across a process boundary.

The thing most readers will look for in a watcher is a debounce interval. There is not one, and the absence is the design rather than an omission.

The mechanism, in four moves

Register. The daemon’s `render_fn` calls `watch::register(fifo, cwd)` on every render, when and only when the client advertised a FIFO in `POWERLINE_RESET_FIFO` (`src/bin/powerline-daemon.rs:60-64`). The FIFO path is the registry key, so one entry per shell.

Fingerprint. `Fingerprint::compute(cwd)` (`watch.rs:79`) snapshots the inputs that decide what the prompt shows: the cwd itself, the discovered git directory, the contents of `HEAD`, the mtime of the index, and the resolved sha of the branch ref. `watched_paths()` (:107) turns that into the set of directories to watch — the cwd, the git dir, and `refs/heads` — deliberately excluding the object store, with the reason in the source: “we watch specific dirs ... never the object store, so a busy repo does not drown us in irrelevant events” (:219-221).

Watch. `watch_paths` (:205-225) builds a `notify::recommended_watcher` — `kqueue/FSEvents` on macOS, `inotify` on Linux — and registers each path `NonRecursive`. The callback discards the event entirely: “We do not care *what* changed, only that something did — the caller re-derives truth from the filesystem” (:210-212).

Compare and wake. On each event the closure recomputes the fingerprint and compares it to the stored baseline under a mutex (:303-319). If they differ it advances the baseline and calls `wake`, which opens the FIFO `O_WRONLY | O_NONBLOCK` and writes one byte `\x01` (:233-241). The shell’s `zle -F` handler sees the byte and redraws in place.

Where the de-duplication actually lives

Three separate mechanisms suppress redundant wakes, and none of them is a timer.

The OS is edge-triggered. The backend delivers an event when something changes, not on an interval. A quiet repository produces zero callbacks and therefore zero cost, which is a different property from a debounced poller — a debounced poller still polls.

The watched set is narrow. Three directories, non-recursive, chosen so that the high-frequency churn of a git repository — objects being written during a fetch, packfiles being rewritten — does not reach the callback at all. This is the mechanism that would otherwise need a debounce, handled by not subscribing rather than by discarding.

The fingerprint is the filter. This is the one people mean when they say debounce, and it is content-based, not time-based:

```
let changed = match cb_baseline.lock() {
  Ok(mut base) => { if *base != current { *base = current; true } else { false } }
  Err(_) => false,
};
if changed { wake(&cb_fifo); }
```

Ten events that leave the fingerprint identical produce zero wakes. One event that changes it produces exactly one, and the baseline advances inside the same lock so a second event carrying the same change cannot produce a second wake. A time-based debounce would answer “how often may I be woken”; this answers “may this event change what is displayed”, which is the question that actually determines whether a redraw is useful.

The difference is not academic. A 200 ms debounce over a `git checkout` would coalesce the `HEAD` write and the index write into one wake — and it would also delay it by up to 200 ms, and it would still fire for a change that turned out to be a no-op. The fingerprint fires immediately for a real change and never for a spurious one. What it gives up is the ability to rate-limit a genuinely oscillating input: a script flipping branches in a loop produces a wake per flip. That is the correct trade for a prompt, where every flip is a real change the user asked to see.

The baseline advance on re-render

`register` is called on every render, not only the first, and the same-cwd path is the interesting one (:285-295):

```
if let Some(state) = reg.get(&wake_fifo.to_string()) {
  if let Ok(mut base) = state.baseline.lock() {
    if base.cwd == now.cwd { *base = now; return; }
  }
}
```

Same client, same directory: advance the baseline to the freshly-rendered state and return without rebuilding the OS watch. This closes a loop that would otherwise never settle. The prompt the user is looking at *is* the state described by the new fingerprint, so a subsequent event that recomputes to the same value correctly produces no wake. Without the advance, every event after the first change would keep differing from a stale baseline and every one would wake.

A changed cwd falls through and rebuilds the watch, because the set of paths worth watching moved.

The failure modes are all silent by design

Every error path in this module logs and continues. A watcher that cannot be created logs `reactive WATCH-FAIL ... (falling back to pull-only)` and returns (:337-345). A poisoned registry mutex returns without watching (:280-283). A poisoned baseline mutex reports “not changed”, so the worst case is a missed redraw rather than a storm. `wake` opens the FIFO non-blocking so a shell that has exited yields `ENXIO` or `EWOULDBLOCK` instead of hanging the daemon, and swallows the error (:229-232).

That uniformity is the point, and the module’s header states the contract: “When no backend can be created (or `notify` errors), `register` logs and returns without watching; the prompt then simply behaves as it always has (pull-only on each `precmd`)” (:41-44). The push path is strictly additive. Everything it can fail at degrades to the behaviour of a shell that never had it, and the record of the degradation is in `$HOME/.powerliners/powerliners.log` rather than in the user’s prompt.

What is tested and what is not

Three tests cover the module (`watch.rs:355-436`), and they test the parts that can be tested without a shell:

- `watch_paths_fires_callback_on_real_change` creates a temp directory, watches it, touches a file, and polls up to five seconds for the callback — an integration test against the real OS backend, not a mock.
- `fingerprint_changes_on_head_checkout` writes two different `HEAD` contents and asserts the fingerprints differ.
- `fingerprint_changes_on_cwd_change` asserts the same for two directories.

What no test covers is the wake itself reaching a shell, because that needs a live `zle -F` handler on the other end of a FIFO. The seam between the daemon and `reactive.zsh` is the one part of the reactive path verified by using it rather than by asserting on it — which is worth saying plainly in a book that spends four chapters insisting the difference between those two matters.

The relationship to the other watcher

`src/ported/lib/watcher/` is a faithful port of upstream's own tree watcher — the `inotify`, `uv`, `dummy`, and `stat` backends selected by the `watcher` config key, whose default of `"auto"` is set in `src/ported/mod.rs:344`. It answers a different question (“did this tree change since I last polled it”) for VCS segments inside a pull render, and it is not wired to the push path at all. `Configs::is_stale` does not use it either; the config staleness check is a hand-rolled mtime walk (`render_runtime.rs:163-190`) whose doccomment says the ported watcher backends “are ready but” unused for this purpose (`:150-151`).

So the daemon carries three change-detection mechanisms with three different owners: a ported tree-watcher for segments, an mtime poll for config reloads, and an OS-event watcher for reactive redraws. They do not share code and they do not need to. Naming all three is the useful thing, because “the watcher” is ambiguous in this codebase in a way that a reader tracking down a stale prompt would otherwise have to discover the hard way.

The three also fail in three different ways, which is the practical reason to keep them apart when diagnosing. A stale *config* is an mtime that did not move, or `reload_config` set to `false` — a condition Chapter 41 reproduces in two commands. A stale *segment value* is the ported tree-watcher answering “unchanged” for a tree it is watching with the `stat` backend rather than an event backend. A prompt that does not redraw when the branch changes is the push path: either no FIFO was advertised, or `register` logged a `WATCH-FAIL` and fell back to pull-only, or the fingerprint genuinely did not move. Only the third leaves a record, and it leaves it in the log file rather than anywhere the user is looking — which makes `$HOME/.powerliners/powerliners.log` the first place to read and the last place anyone thinks of.

Chapter 43 – The Left Side Renders Multi-Chunk Segments Backwards

Chapter 36 rendered every row of the registry one at a time and reported what came back. It rendered all fifty-nine of them into `tmux right`. This chapter renders them into `tmux left` as well and diffs the two, which is one extra loop over the same harness and which turns up a defect the whole rest of the book's method was positioned to miss.

The shortest demonstration

A segment may return several chunks from one theme entry. To take sampling out of the picture entirely, the probe is an `exec` segment whose script prints three fixed chunks and nothing else:

```
$ cat /tmp/three.sh
#!/bin/sh
printf '{"contents":"A","highlight_groups":["exec"]},
        {"contents":"B","highlight_groups":["exec"]},
        {"contents":"C","highlight_groups":["exec"]}]'
```

Rendered into each side of a line, with the markup stripped so only the visible cells remain:

```
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux left | perl -pe 's/#\[^\]*\\//g'
C B A
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux right | perl -pe 's/#\[^\]*\\//g'
A B C
```

Same adapter, same script, same second. The right side prints `A B C`. The left side prints `C B A`.

What it does to real segments

The probe is synthetic; the segments it stands for are not. Sweeping all fifty-nine registry rows through both sides and comparing the chunk sequences finds five adapters that can emit more than one chunk — `cwd`, `system_load`, `network_load`, `weather`, and `last_pipe_status` — and every one of them is affected on the left. Any `exec` segment whose script prints a JSON array of more than one entry joins them.

The load segment is the sharpest case because its chunks are not interchangeable — they are the one-, five-, and fifteen-minute averages, in that order:

```
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux left | perl -pe 's/#\[^\]*\\//g'
77.8 59.3 30.3
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux right | perl -pe 's/#\[^\]*\\//g'
30.3 59.3 77.8
$ uptime
... load averages: 30.26 59.31 77.77
```

`uptime` is the arbiter. The right side agrees with it. The left side prints the fifteen-minute average in the position where a reader expects the one-minute average, and vice versa. Nothing about the line looks wrong — the numbers are real, the colours are right, the dividers meet — and the reading is inverted.

The working directory is the case most users would meet first, because `cwd` emits one chunk per path component and the bundled shell theme puts it on the left:

```
$ cd /private/tmp/plbook/a/b/c
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux left | perl -pe 's/#\[^\]*\\//g'
c b a plbook tmp private /
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux right | perl -pe 's/#\[^\]*\\//g'
/ private tmp plbook a b c
```

And the pipeline status, whose whole purpose is to tell you *which* stage failed:

```
$ ... --last-pipe-status "0 0 7" (left) -> 7 0 0
$ ... --last-pipe-status "0 0 7" (right) -> 0 0 7
```

A user reading the left-side form concludes the first stage of the pipeline failed. It was the third.

The line

The reversal is one statement. `process_segment` walks the returned chunk list and, for the left side, walks it *reversed* — which is correct, and is a faithful transcription of upstream:

```
// py:208 for subsegment in (contents if side == 'right' else reversed(contents)):
let iter: Box<dyn Iterator<Item = Value>> = if side == "right" {
    Box::new(arr.into_iter())
} else {
    Box::new(arr.into_iter().rev())
};
```

Upstream pairs that reversed walk with an *insert at a fixed index*, and the two together restore the original order. The port's own comment says so, names the upstream line, and then does something else (`src/ported/segment.rs`):

```
} else {
    // py:206 append = lambda item: parsed_segments.insert(pslen, item)
    // `pslen` is captured at the start of the loop in Python so
    // inserts go in iteration-reversed order from a fixed index.
    // Rust mirrors by always inserting at the same `pslen`, which
    // resolves to the current length here (the loop appends below).
    let _pslen = parsed_segments.len();
    parsed_segments.push(Value::Object(segment_copy));
}
```

`_pslen` is computed and discarded. `push` appends. Reversed iteration plus append is reversal; reversed iteration plus `insert(pslen, ...)` is identity. Simulating upstream on three chunks with one segment already in the list makes the difference arithmetic rather than argument: `pslen` is 1, and `insert(1,C)`, `insert(1,B)`, `insert(1,A)` leave `[X,A,B,C]`, while three pushes leave `[X,C,B,A]`.

The leading underscore is why the compiler never mentioned it. An unused binding named `pslen` is a warning; an unused binding named `_pslen` is a convention meaning “deliberately unused,” and the code says that about the one value it needed.

Why none of the four verification mechanisms saw it

This is the useful part, because the bug is not exotic and the port's verification is not thin.

The citation rule did not catch it: the citation is *present*, it names the right upstream line, and it describes the right behaviour. What is missing is the behaviour, not the attribution. Chapter 10 argues that citation density is evidence the branch was ported deliberately; here it is evidence the branch was *read* deliberately and then not

implemented, which the metric cannot distinguish.

The tier classifier did not catch it, for the same reason and by construction: it counts `// py:` comments and `/// Port of` doccomments in the file's text. This function is dense in both.

The drift gate did not catch it. The gate matches function *names* against the upstream manifest. `process_segment` exists upstream and the name is spelled correctly.

The parity harness would have caught it and did not run. `daemon_parity.rs` diffs whole rendered lines against upstream, which is exactly the instrument for a chunk-ordering defect — and Chapter 30 establishes that all forty-five of its scenarios skip without a vendored interpreter, reporting `45 passed` while comparing nothing. The fixtures compound it: of the forty-four scenario directories, the ones that exercise multi-chunk segments render into `tmux right`, the side that is correct.

So the bug sits precisely in the intersection of every blind spot Chapter 10 enumerates — a behaviour on an unexercised input, in a ported function, checked by an oracle that is absent — and it is a reminder that the four mechanisms are complementary rather than redundant. Three of them were green about this function the entire time.

Chapter 44 – Two Segments the Bundled Theme Names and the Registry Does Not

Chapter 33 measures what the registry tests certify and states the limit precisely: they certify reachability, not output. This chapter asks the complementary question about the shipped configuration — whether every segment the bundled themes *name* is reachable at all — and the answer is no.

The bundled shell theme, segment by segment

`themes/shell/default.json` names six segments on its left side. Rendered one at a time, from a directory three levels deep, with a job count supplied:

```
$ for f in ..mode ..hostname ..user ..virtualenv ..cwd ..jobnum; do ... done
powerline.segments.shell.mode          -> []
powerline.segments.common.net.hostname -> []
powerline.segments.common.env.user     -> [ wizard ]
powerline.segments.common.env.virtualenv -> []
powerline.segments.shell.cwd           -> []
powerline.segments.shell.jobnum        -> [ 2 ]
```

Two of the four empties are correct and uninteresting. `virtualenv` is silent because no environment is active. `hostname` is silent because `themes/shell/__main__.json` supplies `{"hostname": {"args": {"only_if_ssh": true}}}` through `segment_data`, and this is not an ssh session — a good demonstration that short-name `segment_data` reaches a fully-qualified segment, and the behaviour Appendix AO.5 recommends.

The other two are not conditional absence. The diagnostic log says what happened:

```
$ tail -1 ~/.powerliners/powerliners.log
prepare_line side=left DROP fn=powerline.segments.shell.cwd
(get_segment returned None - module lookup failed)
```

Module lookup failed. The names are not in the table:

```
$ perl -ne 'next unless $.>=2442 && $.<=2548; next if m{^\s*//};
           print "$1\n" while /"([^\"]+)"/g' src/bin/shared/render_runtime.rs \
  | grep '^powerline.segments.shell'
powerline.segments.shell.jobnum
powerline.segments.shell.last_status
powerline.segments.shell.last_pipe_status
```

Three `shell.*` keys are registered. The bundled theme names five, and `shell.mode` and `shell.cwd` are the two that are not among them. The four-rung resolution cascade of Chapter 33 runs to exhaustion on both: the dotted path misses, the bare name (`mode`, `cwd`) is not itself a key, and the two filesystem rungs find no script.

What the default prompt actually contains

The consequence is not subtle. On a fresh install, with no user configuration, a shell prompt from powerliners has no working-directory segment and no mode indicator. What survives from the six is the user name, and the job count when there are jobs.

That is worth putting beside three passages written on the assumption that both render. Appendix AA reads the same file and concludes that “a plain interactive shell collapses this six-segment left side to just the mode, host, and cwd” — the three that in fact do not appear, in place of the one that does. Appendix AF.4 explains that `mode` carries no priority so the truncation pass can never drop it, which is true and is not the reason it is missing. Chapter 15’s worked line opens with the mode indicator at highest priority.

None of those passages is wrong about the *theme*. They are wrong about the render, and the distance between the two is one registry lookup.

Why the registry tests are green

`adapter_id_extension_segments_register` walks a hand-maintained list of twenty-five (`module`, `name`) pairs and asserts each resolves. Chapter 33 calls that a floor rather than a lock, correctly, and notes that adding a row does not fail it. The case here is the third possibility: a name that no test lists, that no theme test renders, and that the bundled theme uses. The registry is verified against a list someone wrote; it is not verified against the configuration the project ships.

That check is one loop and it is worth stating as the chapter’s residue, because it costs nothing and it is the exact test whose absence this chapter is:

```
$ python3 - <<'PY'
import json,glob,re
keys=set(); lines=open('src/bin/shared/render_runtime.rs').read().split('\n')
for l in lines[2441:2548]:
    if not l.strip().startswith('//'): keys|=set(re.findall(r'"([^\"]+)"',l))
named=set()
for p in glob.glob('src/ported/config_files/themes/**/*.json',recursive=True):
    d=json.load(open(p))
    for side in d.get('segments',{}).values():
        if isinstance(side,list):
            named|={s['function'] for s in side if 'function' in s}
print(sorted(named-keys))
PY
['powerline.segments.shell.cwd', 'powerline.segments.shell.mode']
```

Two names, derived by command, in a tree whose test suite reports 2497 passing library tests.

Chapter 45 – The Gradient Crash Under the Shipped Configuration

Chapter 32 records a panic in `pick_gradient_value` seen once on a loaded machine and reasons that the bundled palette cannot be the source. The reasoning was sound and the premise was wrong. This chapter reproduces the crash on demand, names every link in the chain, and gives the threshold.

The crash

```
$ powerline-render tmux right
thread 'main' panicked at src/ported/colorscheme.rs:76:5:
index out of bounds: the len is 100 but the index is 100
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
$ echo $?
101
```

101 is the Rust panic exit status, and it is the only nonzero exit `powerline-render` produces on this machine other than a usage error. The frequency depends on load. Idle, sixty consecutive renders produced three panics; under eight spinning processes the first render panicked.

```
$ for i in $(seq 1 60); do powerline-render tmux right >/dev/null 2>61 || bad=$((bad+1));
~ done
ok=57 panicked=3
```

The chain, link by link

The palette. A gradient in `colors.json` is a pair of parallel ramps, and seven of the ten bundled pairs have sub-lists of different lengths (Chapter 37 tabulates all ten). One of them is the ramp the load and CPU groups use:

```
$ python3 -c "...
green_yellow_orange_red          4 100
```

Four cterm entries, one hundred hex entries.

The lookup. `get_gradient` maps `pick_gradient_value` over each ramp *independently*, so each index is computed from that ramp's own length:

```
let raw = gradient_level * (grad_list.len() as f64 - 1.0) / 100.0;
let idx = py_round(raw) as usize;
grad_list[idx]                                // colorscheme.rs:76 - unchecked
```

For the hundred-entry hex ramp that is `round(level * 99 / 100)`, and the index leaves the list when `level ≥ 100.505`.

The feed. Chapter 32 already establishes the asymmetry between the two producers, and it is correct: `system_load` clamps its level to `100.0` in the else arm of a three-way branch, while `cpu_load_percent`

forwards the sampler's reading unmodified. On macOS that sampler is the user-plus-system total from `top`, and it exceeds one hundred:

```
$ ... cpu_load_percent with format "{0:.4f}%" (40 samples under load, top five)
99.9900
100.1700
100.2600
100.3500
100.3500
```

The theme. The bundled `themes/tmux/default.json` does not name `cpu_load_percent`, which is why the isolated fixture never panics:

```
$ POWERLINE_CONFIG_PATHS=$W powerline-render tmux right # bundled theme, isolated
panics=0/30
$ powerline-render tmux right # full cascade
panics=4/30
```

The difference is the last root on the search path. This machine's `~/.config/powerline` carries a thirteen-segment tmux theme, `cpu_load_percent` among them, and — per Chapter 39's rule that later beats earlier — it wins. Which is the second finding here and is not about gradients at all: **a bare `powerline-render` measures the machine's configuration, not the repository's**. Every transcript in this book that omits `POWERLINE_CONFIG_PATHS` measured something that is not in the tree.

The `backtrace` closes it:

```
3: powerliners::ported::colorscheme::pick_gradient_value
   at ./src/ported/colorscheme.rs:76:5
4: <...::Colorscheme>::get_gradient::{closure#0}
   at ./src/ported/colorscheme.rs:223:41
15: <...::Colorscheme>::get_gradient
   at ./src/ported/colorscheme.rs:226:22
16: <...::Colorscheme>::get_highlighting::{closure#3}
   at ./src/ported/colorscheme.rs:328:37
```

The threshold, on demand

Waiting for load is unnecessary. A gradient of known length plus an `exec` script that reports a chosen level turns the edge into a table. Against a hundred-entry ramp:

```
level=0      rc=0
level=50     rc=0
level=100    rc=0
level=100.5  rc=0
level=100.51 rc=101  index out of bounds: the len is 100 but the index is 100
level=101    rc=101
level=150    rc=101
```

`100.5` survives because `py_round` is banker's rounding and `round(99.495)` is `99`. The first level that does not survive is `100.51`. Against a hundred-and-one-entry ramp the threshold is the same to two decimal places, because the extra entry moves the multiplier and not the boundary.

What the port owes here, and what it does not

Chapter 32 states the rule this produces — a `gradient_level` is a contract, not a hint, and clamping is the segment's job — and that rule is right and is upstream's position too: the doccomment on `pick_gradient_value` says the level “is not checked for being inside `[0, 100]` interval (matches Python behaviour),” and declining to extend a ported function's domain is the correct call for a port.

What is not inherited is the rest of the chain. The mismatched ramp lengths are a property of the bundled `colors.json`, the unclamped forward is a property of an adapter, and the sampler that returns 100.35 is a property of this port's macOS probe rather than of `psutil`. Three port-side facts meet at an upstream-faithful unchecked index, and the result is that the shipped default configuration terminates the renderer under load. A faithful port of an unguarded function is a defensible decision; shipping a palette that walks off the end of it is a separate one.

Chapter 46 — What time powerline Cannot Show

Chapter 1 opens on a latency claim and Appendix G gives a method for confirming it. Neither runs the measurement. This chapter runs it, and the result is that the claim is true and the demonstration Chapter 1 offered could never have shown it.

Three numbers

All medians, taken on the machine and tree named in the front matter, against the `scenario_mode_xlate` fixture so the render is deterministic.

what is being timed	median	n
daemon round trip, no process	0.452 ms	400
<code>/usr/bin/true</code> (spawn floor)	7.597 ms	21
<code>powerline</code> client, warm daemon	15.707 ms	21

The first row is the render plus the socket. It is measured by speaking the wire protocol from a script that is already running — connect, write the frame of Chapter 31, read to EOF, close — so no process is created and nothing but the daemon's work is inside the interval:

```
$ python3 ... # 400 iterations of connect/sendall/rcv/close
daemon round-trip, no process spawn (n=400)
median 0.452 ms    p90 4.452 ms    min 0.228 ms    max 135.001 ms
reply bytes: 119
```

Sub-millisecond, as Chapter 5 says, with a long tail that is scheduler noise rather than render cost.

The second row is the floor. Nothing on this machine that is invoked as a process can beat it, because it is the cost of being a process.

The third row is what `time powerline shell left` measures. It is twice the floor and thirty-five times the render.

The arithmetic

`time` on the client cannot expose a 0.452 ms render because 0.452 ms is under three percent of what `time` is measuring. Reduce the render to zero and the client still costs about fifteen milliseconds. The demonstration is insensitive to the thing it is demonstrating.

This does not soften the argument of Chapter 1; it relocates it. The comparison that matters is between a warm daemon's 0.452 ms of render and a Python interpreter's boot, and both of those are *inside* a process whose own cost is a constant the two share. Against the client's fifteen milliseconds the Python path adds its startup on top; against the render the daemon has deleted it. The win is real at the layer the daemon operates on, and the layer above it is a tax everyone pays.

It also gives the fallback its correct price:

```
client, no daemon -> exec fallback 35.085 ms n=21
```

Two process creations rather than one — the client, then the `powerline-render` it execs — and the measured cost is about double the client alone, which is what that structure predicts. The fallback is a latency event of roughly one extra process, not of an order of magnitude.

What to measure instead

Appendix G's loop times the client, which measures the spawn floor plus a render. That is a legitimate quantity — it is what a shell actually pays per prompt — provided it is labelled as such. What it is not is a measurement of the daemon.

The two are separable and both are worth knowing:

- **What a prompt costs the user.** Time the client. Expect the spawn floor plus the render, and expect the floor to dominate.
- **What the architecture bought.** Speak the protocol from an already-running process. Expect sub-millisecond, and expect it to stay sub-millisecond as segments are added until a class-three segment lands on the line.

Quoting the second number for the first measurement is the error this chapter corrects, and it is an easy one to make, because the second number is the interesting one and the first is the one a shell prompt has to live with.

Chapter 47 – Exit Codes, Surveyed

A renderer that prints the right answer with the wrong status is broken in a pipeline, and a prompt hook is a pipeline. The book asserts exit codes in three places and measures them in one. This chapter runs every path in the suite that can fail and records what a caller sees.

The survey

Each row is the process's exit status, whether stdout was empty, and the first line of stderr.

invocation	rc	stdout	stderr
<code>render tmux right</code> (valid)	0	markup	—
<code>render tmux right</code> (no <code>config.json</code>)	0	error text	—
<code>render bogusext right</code>	0	error text	—
<code>render tmux sideways</code>	0	empty	—
<code>render</code> (no args)	2	empty	<code>--ext is required</code>
<code>render tmux right -w notanumber</code>	0	markup	—
<code>powerline</code> , no daemon	0	markup	—
<code>powerline</code> , no args	1	usage	—
<code>powerline</code> , peer hangs up	1	empty	<code>write() to daemon failed</code>
<code>powerline-lint</code> on empty tree	0	empty	—
<code>powerline-lint</code> on broken theme	0	empty	—
<code>powerline-config tmux bogus</code>	2	empty	unknown function
<code>powerline-daemon -k</code> , none running	1	message	—

The three that matter

Config errors exit 0 and print to stdout. Chapter 39 records the exit code and stops there. The stream is the other half:

```
$ POWERLINE_CONFIG_PATHS=$empty powerline-render tmux right 2>/dev/null
powerline-render: config error: config.json not found
$ POWERLINE_CONFIG_PATHS=$empty powerline-render tmux right 1>/dev/null
$
```

Redirecting stderr changes nothing; redirecting stdout silences it. So a prompt hook written the obvious way — `PS1="$(powerline-render shell left)"` — assigns the diagnostic to `PS1` and gets a success status while doing it. There is no arrangement of redirections and status checks that distinguishes this from a successful render. Chapter 4 argues that errors in the prompt are a feature because they get noticed, and that argument holds for the *daemon*, whose reply channel genuinely is the prompt. It does not extend to a binary that has a stderr and an exit status available and uses neither.

The same shape covers an unknown extension:

```
$ POWERLINE_CONFIG_PATHS=$F powerline-render bogusext right; echo $?
powerline-render: config error: no colorscheme for bogusext
0
```

A malformed width is discarded silently. `-w notanumber` neither errors nor truncates; the line renders at full width and exits 0. For a binding that computes a width and passes it through, a typo in the computation is indistinguishable from the “no width supplied” case that Appendix AN.4 describes, and both present as “truncation never fires.”

The linter is not a gate. Appendix AI.4 says a lint run “either prints nothing and exits 0 — the config is clean — or prints one diagnostic per problem ... and exits non-zero,” and Appendix M puts it first in the migration runbook for that reason. Chapter 50 measures it.

Reading the table

Three commands in the suite report failure by status: `powerline-render` with no arguments, `powerline-config` with an unknown function, and `powerline-daemon -k` with no daemon. Everything else that goes wrong reports it as text on standard output with a status of 0. That is a coherent design for the daemon reply channel and an incoherent one for four command-line tools, and the practical consequence is short: **in this suite, a zero exit status means the process ran, not that the render succeeded.**

Chapter 48 – Degenerate Inputs, Executed

Chapter 31 feeds the daemon four malformed frames and tabulates the replies. Appendix AN reasons about degenerate geometry from the algorithms. This chapter widens both: eleven inputs chosen to break something, sent to a live daemon, with the wall-clock cost of each.

The frames

frame	ms	reply
empty (<code>\0\0</code>)	145.1	malformed request
no terminator, then stall	2001.9	(none)
non-hex argc	0.2	malformed request
argc far too large	0.1	malformed request
argc=1, unknown arg	0.1	expected one argument
4 KiB of NULs	0.2	malformed request
invalid UTF-8 in cwd	3.6	correct markup
invalid UTF-8 in an env value	10.4	correct markup
newline and space in cwd	0.4	correct markup
10 ⁶ env entries (9.4 MiB)	16523.2	correct markup
valid control	0.4	correct markup

The daemon was alive and serving at the end of the sequence.

The first six confirm Chapter 31's table and add the number that chapter's prose asserts: the unterminated frame is answered by the read timeout at **2001.9 ms**, which is the two seconds `do_read` is given, measured rather than quoted.

Bytes that are not text

Three rows are the interesting ones for Chapter 21, which argues that the wire is byte-transparent because NUL is the one byte the payload cannot contain, and that the transparency is load-bearing for paths and environment values that are not valid UTF-8. The argument is correct and had not been exercised:

- a cwd of `/tmp/\xff\xfe` renders normally;
- an environment value of `\xff\xfe\x80` renders normally;
- a cwd containing a space and a literal newline renders normally.

No error, no replacement character in the markup, no truncation at the offending byte. A path that survives `execve` survives the render, which is the property Chapter 21 predicts and the reason the protocol needs no escaping layer.

The frame that is not malformed and is still a problem

The last row is the one that changes a conclusion. A frame with a million environment entries is *valid*. Every field is well-formed, the terminator is correct, and the daemon answers it correctly — after sixteen and a half seconds:

env entries	frame size	ms
0	18 B	17.4
1 000	6.7 KiB	2.0
10 000	77 KiB	19.0
100 000	868 KiB	804.7
1 000 000	9.4 MiB	16523.2

The growth is superlinear: a tenfold increase from 10^4 to 10^5 entries costs forty-two times the time, and the next tenfold costs twenty. And because Chapter 12 establishes that renders are serialized, that cost is not paid by the sender alone. A second client issuing an ordinary request during the large one waits behind it; measured, its request timed out at six seconds while the daemon was still demarshalling. Afterwards the daemon recovered and served a small request in 74.7 ms.

Chapter 12 anticipates the shape of this — it names the forwarded environment as the one slice of the request budget that grows with the caller, and observes that a shell with a pathologically large environment makes even the warm path slower. What it does not say, and what the measurement adds, is that the two-second read timeout does not bound it. The timeout fires on a peer that *stops writing*; a peer that keeps writing keeps making progress and is never timed out, so the only limit on how long one client can hold the shared daemon is how fast it can produce bytes. Chapter 12's read timeout is described there as a backpressure valve, and against this input it is not one.

The severity is bounded by Chapter 14's trust boundary rather than by any mechanism in the loop: whoever can reach the socket can do this, and on a single-user machine that is the user. It is worth naming anyway, because the mechanism that was expected to bound it is documented as bounding it.

Chapter 49 – This Book’s Own Transcripts, Replayed

The book asserts, repeatedly, that a figure is worth what the command that produced it is worth. This chapter applies that to the book. Every command in it was extracted mechanically and run against the tree named in the front matter, and the reproduction rate is reported the same way any other measurement in these chapters is.

The corpus

```
$ python3 ... # fenced blocks, then lines beginning "$ "
total '$ ' command lines: 96
pseudo/comment lines : 5
elided/parameterised : 28
verbatim-runnable    : 63
```

Twenty-eight lines carry an ellipsis, a shell variable defined in the surrounding prose (`$WORK`, `$A`, `$sock`), or a helper function the book describes but does not define. Those are not defects — a transcript may reasonably elide a path — but they cannot be replayed as printed, so they are scored separately. Five are comments or shell no-ops. Sixty-three remain.

The result

Of the 63 verbatim-runnable command lines, 52 reproduced their printed output exactly and 11 did not. The eleven, and what each turned out to be:

where	what failed
Ch. 1, ×2	<code>time powerline</code> — no output printed; claim unmeasurable by that command
Ch. 31, ×3	fallback renders — <code>POWERLINE_CONFIG_PATHS</code> omitted
Ch. 31, ×1	write-failure — described as observed once, not reproducible as printed
Ch. 35, ×1	<code>find</code> output in an order <code>find</code> does not produce
Ch. 36, ×1	<code>shell left</code> — theme not shown; printed order and count both wrong
Ch. 38, ×2	<code>POWERLINE_NO_*</code> — baseline config omitted from both lines
Ch. 38, ×1	<code>shell left</code> raw markup — colours from a theme not shown

Every one is now corrected in place, and the corrections are cited in the chapters they belong to rather than collected here.

What the failures have in common

Nine of the eleven are one mistake made repeatedly: **an environment or a file that the measurement depended on was not printed with it**. The commands were run; their outputs are real; what was dropped is the state that makes them reproducible. That is a different failure from a fabricated transcript and it is worth distinguishing, because it is much easier to make and just as fatal to replay.

The remaining two are the ones with content. Chapter 1's **time** fence is an instrument that cannot resolve the quantity it is pointed at (Chapter 46). Chapter 36's shell render printed a chunk order that the current tree does not produce and that Chapter 43 explains.

What reproduced

The fifty-two are worth naming in aggregate because they are the load-bearing ones. Every test count reproduced exactly: **2497** library tests, **462** parity tests, **45** daemon-parity tests, **26** per binary, **59** registry keys, **0** matches for the unreachable bindings' test names. The wire frame of Chapter 31 reproduced **byte for byte**, all sixty-nine of them. Every daemon lifecycle transcript reproduced, including the pidfile behaviour, the three exit codes, and the two-second kill latency (measured 2.053 / 2.053 / 2.079 s against a printed 2.012 / 2.049 / 2.053 s). Every fixture render in Chapters 32 and 35 reproduced exactly, including the truncation ladder and the truecolor encoding. The **powerline-config** action matrix reproduced all six rows. The gradient panic of Chapter 32 reproduced, and its root cause turned out to be different from the one printed beside it.

A book that measured nothing would not have a rate to report. Eighty-three percent is what this one turned out to be worth on replay, and the residue is documented rather than removed.

Chapter 50 – The Linter That Passes Everything

Appendix A1 catalogues four kinds of reference check `powerline-lint` performs, Appendix M puts it first in the migration runbook, Chapter 9 leans on it for “fidelity in failure,” and Chapter 17 recommends it as the tool that converts a render-time mystery into an authoring error with a line number. None of those passages runs it against a broken config. This chapter does.

An empty tree

```
$ powerline-lint -p /private/tmp/plbook/empty
$ echo $?
0
```

No output, exit 0, on a directory containing no configuration at all. The renderer pointed at the same directory reports `config.json not found`.

A theme with two errors in it

The config below names a segment that no adapter provides and requests a highlight group that no colorscheme defines — the first two entries in Appendix A1.2’s catalogue:

```
$ cat $BAD/themes/tmux/default.json
{"segments":{"right":[
  {"function":"totally.nonexistent.segment"},
  {"function":"powerline.segments.common.time.date",
   "highlight_groups":["no_such_group"]}]}}}

$ powerline-lint -p $BAD; echo "lint rc=$?"
lint rc=0

$ POWERLINE_CONFIG_PATHS=$BAD powerline-render tmux right; echo "render rc=$?"
render rc=0
```

Both tools accept it. The render produces an empty line, because the first segment fails module lookup and drops (Chapter 37’s logged drop) and the second requests a group that does not resolve (Chapter 36’s silent drop). The linter, whose entire purpose is to catch both of these before a render, reports nothing.

What this costs the surrounding claims

Appendix A1.4’s sentence — “The non-zero exit is what makes the linter scriptable: the migration runbook runs `powerline-lint` as a gate before swapping daemons, so a config with a dangling reference is caught by the exit code before it can become a broken prompt” — describes a gate that does not close. The runbook step is not harmful; it is a no-op.

The larger point is the one Chapter 33 makes about the registry and Chapter 29 makes about the tier classifier, in a third instance: a tool that is documented by its intended checks rather than by its observed output will be believed to perform them. The catalogue in Appendix A1 is a faithful description of upstream's linter and of the pipeline the port carries under `src/ported/lint/`; what it does not establish is that the pipeline is reached from the binary's `main`. Two commands settle that, and neither had been run.

The check that is available today

Until the linter is wired, the render *is* the check, provided you read the log rather than the prompt. A name that fails module lookup leaves a record; a group that fails to resolve does not:

```
$ POWERLINE_CONFIG_PATHS=$BAD powerline-render tmux right >/dev/null
$ tail -1 ~/.powerliners/powerliners.log
prepare_line side=right DROP fn=totally.nonexistent.segment
(get_segment returned None - module lookup failed)
```

That asymmetry is Chapter 37's, and it is the whole of the port's authoring diagnostics as shipped: one of the two error classes is discoverable after the fact, in a file, and the other is not discoverable at all.

Chapter 51 – Which Numbers in This Book Are Stable

Chapter 34 states the practice: publish the command beside the number. Replaying the whole book gives the data to say which numbers actually needed it.

Counts held; timings did not

Every count in the book reproduced exactly. Not approximately — exactly, on a tree one commit behind the version the figures were taken at.

figure	printed	replayed
library tests	2497	2497
parity tests	462	462
daemon-parity scenarios	45	45
tests per binary	26	26
registry keys	59	59
bundled colours / gradients	65 / 10	65 / 10
top-theme <code>segment_data</code>	19	19
wire frame, Chapter 31	69 bytes	69 bytes

Every wall-clock figure did not.

figure	printed	replayed
<code>cargo test --lib</code>	11.19 s	3.12 s
<code>cargo test --test parity...</code>	1.14 s	2.00 s
daemon-parity suite	0.03 s	0.00 s
daemon kill latency	≈2.01–2.05 s	≈2.05–2.08 s

The library suite moved by a factor of 3.6 in one direction and the parity suite by 1.8 in the other, on the same machine, from the same tree. The library figure moved again within a single session: the first run of it during this replay took 65.84 s because another `cargo test` was running concurrently, and the two runs after it took 3.17 s and 3.12 s. A timing is a measurement of a machine at a moment, and the moment includes whatever else was running.

The one timing that held is the kill latency, and it held because it is not a performance number — it is a constant in the source (`let total = Duration::from_secs(2)`) observed through a stopwatch. Chapter 41 is right to present it as an exact figure.

The rule this produces

Chapter 34 recommends publishing the command beside every number. The replay suggests a sharper form, because the two classes behave differently and warrant different treatment:

A count is a property of the tree. It is reproducible by anyone with the commit, it is worth printing as a bare number, and the command beside it is a convenience. Every count in this book survived a commit's worth of drift.

A timing is a property of a machine at a moment. It is not reproducible in principle, only in order of magnitude, and printing it as a bare number invites a reader to treat it as a count. The honest form carries the conditions — the build profile, the concurrency, the iteration count, and the statistic. Chapter 46's table does this and Chapter 34's [11.19s](#) did not, which is why one of them replayed and the other did not.

The distinction is the same one Appendix AP draws between structurally reproducible output and live-sampled output, applied to the book's own figures rather than to the program's. A test count is a pure function of the source. A wall-clock time is a measurement of a moving quantity by a process that is itself part of the load. Neither is more honest than the other; they need different labels, and the failures in this book's numbers were entirely in the class that got the wrong one.

Appendix A – The Full Segment Catalog

The chapters name segments as they come up; this appendix names all of them, in one place, at the level of detail a theme author needs to *use* one. Everything here is grounded in the `ADAPTERS` registry (`src/bin/shared/render_runtime.rs`, the `pub const ADAPTERS: &[(&str, AdapterFn)]` table around lines 2380–2482) and the adapter implementations under `src/ported/segments/` and `src/extensions/`. For each segment the entry gives its registry key (the dotted path a theme’s `function` field uses), what it reads, the `args` it accepts with their defaults, what it emits, the highlight groups it requests, and the condition under which it returns nothing.

Three conventions run through the whole catalog and are stated once here rather than repeated in every entry. **info keys:** adapters read the render environment (`segment_info`) through a small set of keys — `environ` (the forwarded environment map), `getcwd` (the forwarded working directory), and the parsed client arguments `args.jobnum`, `args.last_exit_code`, and `args.last_pipe_status`. **Icons:** a leading `{...-glyph}` or `{icon}` token in a default `format` resolves through `src/extensions/icons.rs`, whose three-tier registry (Nerd Font → Unicode pictogram → ASCII) is selected by `POWERLINERS_ICONS=nerdfont|unicode|ascii`. **Dividers:** most extension segments set `"divider_highlight_group": "background:divider"` so their soft dividers paint in the theme’s divider color. Where an adapter “never returns `None`,” it means the sampler falls back to a zeroed reading rather than vanishing; where it returns `None`, the segment is simply absent from the line.

A.1 CPU, load, and uptime

`powerline.segments.common.sys.cpu_load_percent` — CPU utilization as a percentage, the canonical gradient segment. Reads `top -l 1 -s 0 -n 0` on macOS (parsing the `CPU usage:` user+sys fields) and `/proc/stat` on Linux (a 100 ms delta), cached about 1.9 seconds in-process so a burst of prompt redraws does not re-sample. `args: format` (default `"{cpu-glyph} {0:.0f}%"`). Emits one chunk with the percentage substituted into the `{0:.0f} / {0:.1f}` slot. Highlight groups `["cpu_load_percent_gradient", "cpu_load_percent"]` with `gradient_level` equal to the load, so the color shades from calm to hot. Never returns `None` (defaults to `0.0` if sampling fails).

`powerline.segments.common.sys.system_load` — the classic load-average segment, reading `getloadavg`. `args: format` (`"{avg:.1f}"`), `threshold_good` (`1.0`), `threshold_bad` (`2.0`), `track_cpu_count` (`false`), `short` (`false`). Emits one chunk when `short`, otherwise three (the 1/5/15-minute averages). Highlight `["system_load_gradient", "system_load"]`, with the gradient level computed from the thresholds: 0 below `threshold_good`, linearly scaled between, 100 above `threshold_bad`. Returns `None` only if the underlying load read fails.

`powerline.segments.common.sys.uptime` — system uptime, formatted from four part-templates. `args: days_format` (`"{days:d}d "`), `hours_format` (`"{hours:d}h "`), `minutes_format` (`"{minutes:d}m "`), `seconds_format` (`"{seconds:d}s"`), `shorten_len` (`3`, the number of leading nonzero parts to keep). Emits one string; returns `None` when uptime is unavailable.

A.2 Memory and swap

The memory family lives in `src/extensions/mem_usage.rs` under the third-party `powerlinemem.*` namespace (it is a 1:1 port of `mKaloer/powerline_mem_segment`), reading `vm_stat + sysctl -n hw.memsize` on macOS and `/proc/meminfo` on Linux. All four share the highlight groups `["mem_usage_gradient", "mem_usage"]` with `gradient_level = used/total*100`, and none of them returns `None` (a failed read yields a zeroed `MemStats`).

`powerlinemem.mem_usage.mem_usage` — RAM as `USED/TOTAL`. `args: format("{mem-glyph} %s/%s")`, `mem_type` ("used"; one of `used/free/available/total`), `short` (false, toggling Ki/Mi/Gi vs short K/M/G). `.mem_usage_percent` — the percentage variant, `format` default `"{mem-glyph} %d%%"`. `.mem_swap` — swap as `USED/TOTAL`, reading `sysctl -n vm.swapusage` (macOS) or `/proc/meminfo`'s `SwapTotal/SwapFree` (Linux), `format "{swap-glyph} %s/%s"`. `.mem_swap_percentage` — swap percentage, `format "{swap-glyph} %d%%"`.

A.3 Disk and GPU

`powerliners.disk.disk_usage` (`src/extensions/disk.rs`) — a filesystem's used space, reading `df -k -P <mount>` (on macOS `/` is redirected to `/System/Volumes/Data` to read the writable volume). `args: mount ("/")`, `format "{disk-glyph} %s/%s"`, `short` (false). Highlight `["disk_usage_gradient", "disk_usage", "mem_usage_gradient", "mem_usage"]`, `gradient` `used/(used+free)*100`. `.disk_usage_percent` — percentage form, `format "{disk-glyph} {0:d}%"`. `.disk_io` — live read/write throughput, reading `/proc/diskstats` (two 500 ms samples) on Linux and `ioreg -c IOBlockStorageDriver -r` (a cross-call delta) on macOS. `args: device("auto")`, `format "{disk-glyph} {io-glyph} R %s W %s"`, `short` (default true here), `recv_max/sent_max` (`104857600.0` = 100 MiB/s each). Highlight `["disk_io_gradient", "disk_io", "network_load_gradient", "network_load"]`.

`powerliners.gpu.gpu_usage_percent` (`src/extensions/gpu.rs`) — GPU compute percent through a vendor-dispatch chain: `nvidia-smi --query-gpu=utilization.gpu,memory.used, memory.total --format=csv,noheader,nounits`, then `rocm-smi` (Linux), then `intel_gpu_top -j` (Linux), then `ioreg ... -c IOAccelerator` (macOS Apple Silicon). `args: format("{gpu-glyph} {0:d}%"`). Highlight `["gpu_load_gradient", "gpu_load", "cpu_load_percent_gradient", "cpu_load_percent"]` (the CPU groups are a fallback so the segment colors even in a colorscheme that never defined GPU groups), `gradient_level = util`. `.gpu_vram` — VRAM `USED/TOTAL` via the same probe chain, `format "{gpu-glyph}{mem-glyph} %s/%s"`, highlight `["gpu_vram_gradient", "gpu_vram", "mem_usage_gradient", "mem_usage"]`. Neither returns `None`.

A.4 Thermal and processes

`powerliners.thermal.thermal` (`src/extensions/thermal.rs`) — CPU/GPU temperature and fan RPM. On Linux it reads `/sys/class/hwmon/hwmon*/temp*_input` and `fan*_input`, name-filtering the hwmon by family (`coretemp/k10temp/zenpower/cpu_thermal` for CPU, `amdgpu/nouveau/radeon/i915` for GPU); on macOS it reads `sysctl -n machdep.xcpm.cpu_thermal_level` on Intel and derives a chip temperature from `ioreg -r -n AppleSmartBattery` on Apple Silicon (where fan RPM is always unavailable). `args: family("cpu"; cpu/gpu/all)`, `format "{cpu-glyph} %s°C %sRPM"`, or `"{gpu-glyph} %s°C"` for the gpu family), `temp_max` (`95.0`). When the fan reading is absent, the RPM tail (from the second `%s` on) is dropped and the trailing space trimmed. Highlight `["thermal_gradient", "thermal", "system_load_gradient", "system_load"]`, `gradient` `temp/temp_max*100`.

`powerliners.proc.process_count` (`src/extensions/proc_count.rs`) — a POSIX process tally from `ps -eo stat=`, bucketed by the first state character. `args: format("{proc-glyph} {total}")`, `warn_zombie` (default true). Tokens `{total} {running} {sleeping} {zombie} {dwait} {stopped}`.

Highlight `["process_count", "proc"]`, with `"process_count_zombie"` prepended when `warn_zombie` and there are defunct processes — a standing warning in the prompt. Returns `None` only if `ps` fails.

A.5 Version control

`powerliners.vcs.git_status (src/extensions/git_status.rs)` — the richer, MenkeTechnologies-native VCS segment, p10k-style. From the forwarded `cwd` it runs `git status --porcelain=v2 --branch` (one fork for the whole state), plus `git describe --tags --exact-match`, `git rev-parse --short HEAD` (detached HEAD), a `git rev-parse --git-dir` and stat of `MERGE_HEAD/rebase-merge/rebase-apply/CHERRY_PICK_HEAD/REVERT_HEAD/BISECT_LOG` (to detect an in-progress operation), `git stash list`, and `git config --get remote.origin.url` (to pick the GitHub glyph). `args` are all icon overrides: `branch_icon`, `github_icon`, `tag_icon`, `unstaged_icon ("!")`, `untracked_icon ("?")`, `staged_icon ("+")`, `conflict_icon ("~")`, `ahead_icon ("↑")`, `behind_icon ("↓")`, `stash_icon ("*")`, plus `status_colors (false)`. Emits a single chunk: a header (GitHub glyph → branch, optionally `tag` → tag → `@shortsha`) followed by count badges in the order action, behind, ahead, stash, conflicts, staged, unstaged, untracked, each shown only when nonzero. Highlight: with `status_colors`, clean → `["git_status_clean", "git_status", "branch_clean", "branch"]` and dirty → the `_dirty` equivalents; without it, `["git_status", "branch"]`. Returns `None` outside a git work tree or when there is nothing to show.

`powerline.segments.common.vcs.branch` — the faithful upstream port, a single chunk holding the branch name. Reads git (`git rev-parse --abbrev-ref HEAD + git status --porcelain`), falling through to Mercurial (`hg branch + hg status`) then Bazaar (`bzr nick + bzr status`). `args`: `status_colors (false)`, `ignore_statuses` (default `[]`, status codes to treat as clean). Highlight `["branch"]`, with `"branch_dirty"` or `"branch_clean"` prepended when `status_colors` is on. Returns `None` outside any VCS repo. `.stash` — the stash count from `git stash list`, highlight `["stash"]`, `None` when there are zero stashes.

A.6 Shell status

These three live in `src/ported/segments/shell.rs` and are the reason the wire protocol forwards the client's parsed arguments. `powerline.segments.shell.jobnum` — the background-job count from `info.args.jobnum`; `args`: `show_zero (false)`; `None` when absent or (zero and not `show_zero`). `.last_status` — the previous command's exit code from `info.args.last_exit_code`; `args`: `signal_names (true`, so exit code `130` renders as `SIGINT`); highlight `["exit_fail"]`; `None` when the code is absent or `0`. `.last_pipe_status` — one chunk per pipeline stage from the `info.args.last_pipe_status` array, each highlighted `["exit_fail"]` (nonzero) or `["exit_success"]` (zero), so a mid-pipeline failure is visible even when the last stage succeeded.

A.7 Identity and environment

From `src/ported/segments/common/env.rs`: `.cwd` — the working directory as one chunk per path component (only the last carries `["cwd:current_folder", "cwd"]`; a `divider_highlight` is set on all); `args`: `dir_shorten_len`, `dir_limit_depth`, `use_path_separator (false)`, `ellipsis`. `.user` — the username, reading `info.envIRON` and `libc:geteuid()`; `args`: `hide_user`, `hide_domain (false)`; highlight `["superuser", "user"]` when `euclid` is `0` (the red-root warning), else `["user"]`. `.environment` — the value of a named variable; `args`: `variable (required)`; `None` when the variable is unset. `.virtualenv` — the active `VIRTUAL_ENV` or `CONDA_DEFAULT_ENV` leaf; `args`: `ignore_venv/ignore_conda (false)`, `ignored_names (["venv", ".venv"])`; `None` when no environment is active.

From `src/ported/segments/common/net.rs`: `.hostname` — the host name from `info.envIRON` (fallback subprocess `hostname`); `args`: `only_if_ssh (false)`, `exclude_domain (false)`. `.internal_ip` — a local interface address, resolving the default-route interface via `netstat -rn -f inet` (macOS) or `ip route show default` (Linux) then parsing `ifconfig`; `args`: `interface ("auto")`, `ip (4)`. `.external_ip` —

the public address via `curl -s https://ipv4.icanhazip.com` (a network round-trip — keep it off the per-keystroke path); no `args`; `None` when the query fails. `.network_load` — DL/UL throughput as two chunks, reading `netstat -ibn` (macOS) or `/sys/class/net/<iface>/statistics/{rx,tx}_bytes` (Linux) across a 500 ms sample; `args`: `interface` ("auto"), `recv_format` ("DL {value:>8}"), `sent_format` ("UL {value:>8}"), `suffix` ("B/s"), `si_prefix` (false), `recv_max/sent_max` (1_000_000.0); highlight per chunk `["network_load_<dir>_gradient", "network_load_gradient", "network_load_<dir>", "network_load"]`.

A.8 Time and battery

`powerline.segments.common.time.date` — the clock, strftime-formatted; `args`: `format` ("%Y-%m-%d"), `istime` (false), `timezone`; highlight `["time", "date"]` when `istime` else `["date"]`; never `None`. `.fuzzy_time` — the human phrasing ("twenty to three"); `args`: `format`, `unicode_text` (false), `timezone`, and `hour_str/minute_str` overrides for the word tables. Both are pure-clock reads and safe on a per-keystroke prompt.

`powerline.segments.common.bat.battery` (`src/ported/segments/common/bat.rs`) — battery charge from `pmset -g batt`. `args`: `format` ("{ac_state} {capacity:3.0%}"), `steps` (5), `gamify` (false, the heart-meter mode), `full_heart` / `empty_heart` ("0"), `online` ("C"), `offline` (" "). Non-gamify highlight `["battery_gradient", "battery"]` with `gradient_level` = `100 - capacity` (so it reddens as it drains); the gamify path emits `battery_full/battery_empty` and `battery_online/battery_offline` variants. `None` when `pmset` fails or there is no battery.

A.9 Media players

The player family (`src/ported/segments/common/players.rs`) mirrors upstream's class hierarchy: each adapter shares `args format` (default `"{state_symbol} {artist} - {title} ({total})"`) and `state_symbols` (override, default `>/-/X/'` for playing/paused/stopped/none), emits a single chunk, requests `["player_<state>", "player"]`, and returns `None` when its player is not running. The members: `.mpd` (reads `mpc`; `args` add `host/port/password`), `.spotify` (macOS `osascript`; Linux returns `None` — no D-Bus path here), `.cmus` (`cmus-remote -Q`), `.mocp` (`mocp -i`), `.rhythmbox` (`rhythmbox-client --print-playing-format`), `.itunes` (macOS `osascript`), `.clementine` (qdbus MPRIS call), and `.dbus_player` (the generic MPRIS path via qdbus; `args`: `player_name` required). `powerline.segments.common.mail.email_imap_alert` — registered with `args username` (required), highlight `["email_alert"]`; note honestly that this is currently a *placeholder* — it emits `"<username>: 0"` rather than performing a real authenticated IMAP probe (the TLS dependency is not wired in), so it is present as an API-compatible stub, not a working mail checker.

A.10 Weather

`powerline.segments.common.wthr.weather` (`src/ported/segments/common/wthr.rs` and `src/extensions/wthr_extensions.rs`) — current conditions and temperature from the OpenWeatherMap API (<https://api.openweathermap.org/data/2.5/weather>), disk-cached, with the location auto-detected or set by query. `args`: `location_query`, `weather_api_key`, `unit` ("C"; C/F/K), `temp_format` (default `"{temp:.0f}<unit>"`), `temp_coldest/temp_hottest` (unit-dependent bounds for the gradient), and an `icons` override map. Emits two chunks — a condition-icon chunk (highlight `["weather_condition_<name>", "weather_conditions", "weather"]`) and a temperature chunk (`["weather_temp_gradient", "weather_temp", "weather"]`, gradient scaled between coldest and hottest). `None` when no weather data is available. Like `external_ip`, this is a network segment for the timer-refreshed statusline, not the keystroke path.

A.11 Cloud and orchestration context

`powerliners.k8s.kubecontext` (`src/extensions/k8s.rs`) — the current `kubectl` context and namespace, via `kubectl config current-context` and `kubectl config view --minify -o jsonpath={..namespace}`, honoring `$KUBECONFIG`. `args: format ("{k8s-glyph} {context}:{namespace}")`, `cli ("kubectl")`, `default_namespace ("default")`, `hide_default (true)`. When `hide_default` and the namespace equals the default, the namespace and one adjacent separator (`://@`) are dropped. Highlight `["kubecontext", "k8s"]`; `None` when `kubectl` is missing or there is no current context.

`powerliners.docker.containers` (`src/extensions/docker.rs`) — container counts via three forks (`docker ps -q`, `ps -aq`, `images -q`). `args: format ("{docker-glyph} {running}/{total}")`, `cli ("docker")`, `show_when_zero (false)`. Tokens `{running}` `{total}` `{images}` `{stopped}`. Highlight `["docker_containers_gradient", "docker_containers", "docker"]`. `None` when the CLI is missing/daemon unreachable, or (unless `show_when_zero`) when there is nothing running and no images.

`powerliners.github.ci_status` (`src/extensions/github_ci.rs`) — the HEAD commit's check-runs. From the cwd it derives owner/repo from `git config --get remote.origin.url` (GitHub only) and the SHA from `git rev-parse HEAD`, then serves from a per-SHA disk cache under `$XDG_CACHE_HOME/powerliners/github_ci/`, falling back to `gh api repos/<o>/<r>/commits/<sha>/check-runs`. `args: format ("{github} {icon} {passed}/{total}")`, `cli ("gh")`, `tty_secs (30)`, and `ok_icon/fail_icon/run_icon`. Tokens `{icon}` `{state}` `{passed}` `{failed}` `{running}` `{total}`. Highlight splits into `github_ci_success/github_ci_failure/github_ci_pending` (with `github_ci` and `information:regular` fallbacks). `None` outside a GitHub tree, when `gh` is missing and no fresh cache exists, or when there are no checks.

`powerliners.aws.context` (`src/extensions/aws_ctx.rs`) — the active AWS profile and region, a pure filesystem probe (no subprocess): environment `AWS_PROFILE/AWS_REGION/ AWS_DEFAULT_REGION`, then `~/.aws/config` (honoring the `[profile NAME]` section quirk). `args: format ("{aws-glyph} {profile}@{region}")`, `hide_default_profile (false)`, `icon`. Tokens `{profile}` `{region}` `{icon}`. Highlight `["aws_context", "aws"]`; `None` in the never-configured case. `powerliners.gcp.context` (`src/extensions/gcp_ctx.rs`) — the active gcloud configuration's project and account, likewise pure-fs: environment `CLOUDSDK_ACTIVE_CONFIG_NAME/CLOUDSDK_CORE_PROJECT/CLOUDSDK_CORE_ACCOUNT`, then `~/.config/gcloud/active_config` and `configurations/config_<NAME>` (`[core]` section). `args: format ("{gcp-glyph} {project}:{account}")`, `hide_account (false)`, `icon`. Tokens `{project}` `{account}` `{config}` `{icon}`. Highlight `["gcp_context", "gcp"]`.

A.12 The MenkeTechnologies stack segments

These tie the prompt to the sibling tools and are pure filesystem probes (no subprocess, no daemon RPC). The four **cache-size** segments share a `{size}/{bytes}/{logical_size}/ {logical_bytes}` token surface (disk-block `du -sh`-style sizes versus stat-style content sums) and a `<segment>_rkyv_cache` → `<segment>` → `information:regular` highlight chain, returning `None` on a missing archive unless `show_when_empty` is set. `.fusevm.jit_cache` (`fusevm_jit.rs`) recursively walks the Cranelift JIT cache root (`$FUSEVM_JIT_CACHE` → `$XDG_CACHE_HOME/fusevm-jit` → `~/.cache/fusevm-jit` → `/tmp/fusevm-jit`), also tracking an `{entries}` count (symlinks counted once, never followed); `format "{fusevm} {count} {entries} {harddisk} {size}"`. `.zshrs.rkyv_cache`, `.stryke.rkyv_cache`, `.awkr.rkyv_cache`, `.vimlr.rkyv_cache` and `.elisp.rkyv_cache` each stat a single `scripts.rkyv` archive through the four-level `$<NAME>_RKYV_CACHE` → `$<NAME>_HOME/scripts.rkyv` → `$XDG_DATA_HOME/<name>/scripts.rkyv` → `~/.<name>/scripts.rkyv` chain. The three **version** segments (`.zshrs.version/.stryke.version/.awkr.version`) parse `<bin> --version`, cache the result in-process for `tty_secs` (default `300`) so the daemon does not fork on every tick, expose `{version}`, and use the `<tool>_version` → `<tool>` → `information:regular` chain.

A.13 The `exec` escape hatch

`powerliners.exec.exec` (and the bare alias `exec`, `src/extensions/exec_segment.rs`) — the user-extensibility segment. It spawns `args.command` with the positional list `args.args` (no shell), and auto-detects the output: stdout beginning with `[` is parsed as a JSON array of segment dicts and used verbatim (full control over `highlight_groups`, `gradient_level`, `divider_highlight_group`, and multi-chunk output); anything else is wrapped as a single plain-text chunk with the `format` template applied (`%s` → stdout, `%%` → literal `%`) and `highlight_groups` attached. `args`: `command` (**required**), `args` (string array), `format`, `highlight_groups`. `None` when `command` is absent, the process exits nonzero, or stdout is not UTF-8. The same code path backs the *dotted-path filesystem dispatch* fallback in `invoke_adapter`: any unresolved dotted `function` name is resolved to `<config_path>/segments/<path>.{sh,py,rb,pl,lua,js,<none>}` — the `SCRIPT_EXTENSIONS` order at `exec_segment.rs:31` — with the first match winning, and `args/format/highlight_groups` passed through exactly as for the explicit adapter.

A.14 Reading the catalog as a whole

Two patterns are visible once the whole table is in front of you, and both are design, not accident. The first is the **cost gradient**: identity and time segments read only in-process state (microseconds); the system-metrics segments fork a probe or read `/proc` (sub-millisecond to a few milliseconds); and a handful — `external_ip`, `weather`, the network players — make network or authenticated calls that can block. The catalog is meant to be read with placement in mind, keeping the third class off the per-keystroke prompt and on the timer-refreshed statusline. The second is the **graceful-absence convention**: nearly every segment has a documented `None` condition, and a segment that returns `None` costs almost nothing, which is what lets a theme list two dozen segments and pay only for the handful that have something to say in the current context. The library is broad *because* absence is cheap.

Appendix B – The Config Schema Reference

Chapter 6 walks the *cascade*; this appendix is the *schema* — the shape of each JSON file the cascade resolves, the keys each level reads, and the exact merge and search rules, so a theme author can read a config tree with the same model the loader uses. Everything here is grounded in `src/ported/mod.rs` (search path), `src/ported/lib/dict.rs` (merge), `src/ported/lib/config.rs` (load), `src/ported/theme.rs` (theme grammar), `src/ported/colourscheme.rs` (colourscheme grammar), and the bundled tree under `src/ported/config_files/`.

B.1 The search path, exactly

`get_config_paths` (`src/ported/mod.rs:278`) builds the ordered list of roots the cascade walks. Reading it from the code, the effective order (index 0 searched first) is:

1. `config_files` — the plugin/bundled-defaults path, inserted first (`mod.rs:304`). This is where `bundled_config.rs` extracts the shipped tree, so the defaults are always on the path.
2. **Reversed** `$XDG_CONFIG_DIRS` — each entry as `<dir>/powerline`, colon-split, empties filtered, inserted front-to-back in reverse (`mod.rs:300`). This is the system-wide tier; `DEFAULT_SYSTEM_CONFIG_DIR` is `None` by default (`src/ported/config.rs:148`), so this tier is empty unless `$XDG_CONFIG_DIRS` is set.
3. `$XDG_CONFIG_HOME/powerline` (default `$HOME/.config/powerline`) — the user tree, the initial seed of the vector (`mod.rs:283`).

Within a root, `_find_config_files` (`mod.rs:104`) appends `.json` to the requested config name and collects every root where that file exists — not the first, all of them — so a `themes/tmux/default.json` present in both the user tree and the bundled defaults yields two paths, later merged rather than one shadowing the other. When no root has the file it raises "Config file not found in search paths (...): `<file>.json`", the same loud failure upstream produces. `load_json_config` (`src/ported/lib/config.rs:48`) reads and parses a single file into a `serde_json::Value`, erroring with "open_file: `<e>`" or "json parse: `<e>`".

B.2 The merge primitives

`src/ported/lib/dict.rs` supplies three merge flavors used at different phases of config assembly. `mergedicts(d1, d2, remove)` (`dict.rs:117`) is the deep merge that makes the cascade a single effective config: for each key in `d2`, if both sides are objects it recurses; if `remove` is on and `d2[k]` is the removal sentinel it deletes the key; otherwise `d2` wins on the scalar. `mergedefaults(d1, d2)` (`dict.rs:153`) is the gap-filler — recurse on object collisions, otherwise `d1` wins (`setdefault` semantics) — used to lay defaults under a config without overriding it. `mergedicts_copy(d1, d2)` (`dict.rs:192`) is the non-mutating variant (clones `d1`, `d2` wins, no sentinel handling).

The **removal sentinel** is worth stating exactly, because it is the port's spelling of upstream's `REMOVE` marker. `REMOVE_THIS_KEY()` (`dict.rs:34`) returns the JSON value `{"__powerliners_remove_this_key__": true}`, and a key in `d2` whose value is that object is *deleted* from `d1` during a `mergedicts` with `remove`.

enabled. This is how a user config *removes* an inherited key rather than only overriding it — for example, deleting a segment the bundled default placed on the right side:

```
{ "segments": { "right": { "__powerliners_remove_this_key__": true } } }
```

B.3 config.json – the main config

The root file. Its keys, from the bundled `config_files/config.json`:

- **common** — cross-extension settings, e.g. `{ "term_truecolor": false }`.
- **ext** — a per-extension map. Each of `shell`, `tmux`, `vim`, `ipython`, `pdb`, `wm` carries a `colorscheme` (name), a `theme` (name), and optionally `local_themes` (a `{matcher_or_key → theme_name}` map, used for the window-lister and per-buffer themes) and an `update_interval`.

A minimal user override that only swaps the shell colorscheme need restate nothing else, because `mergedicts` merges it into the bundled `config.json` key by key:

```
{ "ext": { "shell": { "colorscheme": "solarized" } } }
```

B.4 colors.json – the palette

Two keys, read by `Colorscheme::new`:

- **colors** — `{name → cterm}` or `{name → [cterm, "hexstr"]}`. A bare integer means “derive the hex from the `cterm_to_hex` table”; a two-element array supplies both. This is the indirection that lets a colorscheme name “red” and a palette edit restyle every group that uses it.
- **gradients** — `{name → [cterm_list, hex_str_list]}`, two parallel lists so a gradient exists in both 256-color and truecolor encodings (Appendix C).

B.5 colorschemes/*.json – highlight groups

A colorscheme maps highlight-group names to appearance. Keys:

- **name** — a display label (“Default”).
- **groups** — `{group_name → {fg, bg, attrs}}` for a concrete group, or `{group_name → "other_group"}` for a string alias (chased at resolution, Chapter 6). `attrs` is an array drawn from “bold”, “italic”, “underline”.
- **mode_translations** — `{mode → {"groups": {...}, "colors": {old → new}}}`, the per-mode overrides that let, say, vim’s insert mode recolor the whole line. Read into `Colorscheme.translations` and applied by `get_group_props`.

The cascade assembles a colorscheme from three levels (`load_colorscheme_config_levels`, `mod.rs:935`): `colorschemes/<name>`, `colorschemes/<ext>/__main__`, and `colorschemes/<ext>/<name>`, with level 1 (`__main__`) on the ignore list for the top-level pass — so an extension’s `__main__` supplies shared groups that the named scheme extends.

B.6 themes/*.json – the line grammar

A theme describes the segments and their layout. Keys read by `Theme` (`src/ported/theme.rs:204`):

- **segments** — `{ "left": [ ... ], "right": [ ... ] }`, each entry a segment spec with `function` (the dotted adapter name), `type` (“function”/“string”/“segment_list”, defaulting to function), op-

tional `priority` (an integer for truncation ordering), `args`, `before/after` (strings wrapped around the contents), `width/align/expand` (for spacers), and `name`.

- `segment_data` — `{segment_name → {before, after, contents, args}}`, a table of per-segment overrides keyed by short name ("branch") or fully-qualified path. This is how a theme tweaks one segment's arguments without inlining them at the use site.
- `dividers` — `{"left": {"hard", "soft"}, "right": {"hard", "soft"}}`, the divider glyphs per side and kind, read by `get_divider` (`theme.rs:307`).
- `spaces` (int, read by `get_spaces`), `outer_padding` (int, default 1), `cursor_space/cursor_columns` (optional).

One honest note for authors coming from upstream: the top-theme `extends` inheritance key is *not* implemented in the port's `Theme` — none of the bundled themes use it, and `src/ported/theme.rs` has no `extends` handling. Theme composition in the bundled tree is done through the cascade's level-merging (`load_theme_config_levels`, `mod.rs:956`, appends `themes/<ext>/<name>` with levels 0 and 1 ignored) rather than an in-file `extends`. If a migrated theme relies on `extends`, that is a place to verify with `powerline-lint` and the parity check of Chapter 11 before committing to the swap.

B.7 The bundled tree

`src/extensions/bundled_config.rs` embeds the whole default config tree at compile time — `const BUNDLED: &[(&str, &str)]` (line 29) is a table of 46 `(relative_path, include_str!(...))` pairs, pinned to exactly 46 by the `all_46_files_extracted` test — and `extract()` writes them to `$XDG_CACHE_HOME/powerliners/config_files/` on first use, overwriting each run so a version bump refreshes the defaults. This is the same install-method-agnostic embedding used for the tmux conf files and the vim plugin (Chapter 8): the defaults travel *inside* the binary, so there is no compile-time path for a Homebrew or manual install to get wrong.

Appendix C – The Color and Highlight-Group Reference

Color resolution is the part of powerline most people never look inside, and it is worth a reference of its own because so much of the “drop-in” guarantee lives here: two implementations resolve a highlight group to the same bytes only if they carry the same tables and apply them in the same order. This appendix documents the constants, the `cterm_to_hex` table, the attribute packing, the gradient math, and the standard highlight-group vocabulary — all grounded in `src/ported/colourscheme.rs`.

C.1 The two-value color model

Every color in powerline is *two* values: a 256-color terminal index (`cterm`) and a 24-bit hex value, carried together as a `[cterm, hex]` pair through the whole pipeline. The reason is that the same colorscheme must render on both a truecolor terminal and a 256-color one, and the choice between the two encodings is deferred to the very last step — the renderer’s `term_truecolor` flag (Appendix D) picks the hex form when truecolor is on and the `cterm` index when it is off. `Colourscheme::new(colourscheme.rs:109)` accepts a color as either `[cterm, "hexstr"]` (both given) or a bare `cterm` integer (*hex derived*), so an author can maintain a 256-color-only palette and still render on a truecolor terminal, or vice versa.

The **falsy-hex fallback** is the defined edge case: when a color’s hex value is missing or zero, the renderer falls back to the `cterm` index even on a truecolor terminal. This is why the README lists “`cterm/truecolor` encoding with falsy-hex fallback” among the byte-identical behaviors — the degenerate case has a defined answer, and the port reproduces it.

C.2 The `cterm_to_hex` table

`cterm_to_hex(colourscheme.rs:358)` is a 256-entry `const [u64; 256]` mapping an 8-bit terminal color index to its 24-bit RGB value, a line-for-line port of upstream’s module-level table. Its structure is the standard xterm 256-color layout: indices 0–15 are the system colors, 16–231 are a 6×6×6 RGB cube, and 232–255 are a 24-step grayscale ramp that begins at `0x080808` (index 232) and ends at `0xffffffff` (index 255). Carrying this table verbatim is what lets the constructor derive a hex from a bare `cterm` and lets the renderer resolve a 256-color-only theme on a truecolor display; get one entry wrong and a palette shifts by a shade in one encoding but not the other, which a parity diff would catch immediately.

C.3 Attribute packing

Text attributes are a bitfield. The constants (`colourscheme.rs:30,34,38`) are `ATTR_BOLD = 1`, `ATTR_ITALIC = 2`, `ATTR_UNDERLINE = 4`, and `get_attrs_flag(colourscheme.rs:43)` bit-ORs them by scanning a group’s `attrs` array for the strings “bold”, “italic”, “underline”. So `["bold", "underline"]` packs to `1 | 4 = 5`, and an empty array packs to `0`. The renderers unpack this integer back into their target’s attribute syntax (Appendix D) — tmux’s `bold/nobold` pairs, vim’s `cterm=bold`, the shell’s SGR codes — so the *same* packed flag drives every target consistently.

C.4 Gradients and the rounding rule

A gradient is a list of colors selected by a 0-to-100 level. `pick_gradient_value` (`colorscheme.rs:72`) computes the index as `grad_list[round(gradient_level * (len - 1) / 100)]` — the level is *not* bounds-checked, matching upstream. The load-bearing detail is the rounding: the port's `py_round` (`colorscheme.rs:82`) wraps Rust's `round_ties_even` so that `round(2.5)` is 2, not 3, matching Python 3's banker's rounding exactly. This is precisely the kind of edge that a naive port gets wrong — Rust's default `f64::round` rounds half *away from zero* — and it is why the port carries a dedicated helper rather than calling the standard method. Because a gradient is built as two parallel lists (a cterm list and a hex list, from the constructor's `[cterm_list, hex_str_list]` handling), the same computed index picks a matching color in whichever encoding the terminal needs. `get_gradient` (`colorscheme.rs:208`) maps `pick_gradient_value` over both sub-lists to produce the final `[cterm, hex]` pair, falling back to a flat color of the same name if the gradient is undefined.

C.5 Resolution order in `get_highlighting`

`get_highlighting` (`colorscheme.rs:292`) is the function every colored segment calls. Its order is: (1) look up the mode's `translations` (default empty); (2) walk the segment's `groups` in priority order, calling `get_group_props` for each and stopping at the first that resolves; (3) if none resolves, error with "Highlighting groups not found in colorscheme: <joined>"; (4) extract `fg` and `bg` — from `self.colors` when there is no gradient level, from `get_gradient` when there is — and pack `attrs` via `get_attrs_flag`; (5) return `{fg, bg, attrs}`. `get_group_props` (`colorscheme.rs:242`) is the recursive resolver: a string group value is an alias (recurse), an object with `translate_colors` applies the mode's `colors` remap to `fg/bg`, and a missing group returns `None` so the next group in the segment's list gets its turn. The first-match-wins walk is exactly the graceful-degradation mechanism of Chapter 2: a segment offering `["branch_dirty", "branch"]` still colors under a scheme that defined only `branch`.

C.6 The standard highlight-group vocabulary

Themes and colorschemes agree on group *names*, and a working knowledge of the common ones makes a colorscheme readable at a glance. The names below are drawn from the highlight groups the catalogued adapters actually request (Appendix A) and the bundled colorschemes.

- **Segment-specific groups:** `cpu_load_percent` / `cpu_load_percent_gradient`, `system_load` / `system_load_gradient`, `mem_usage` / `mem_usage_gradient`, `disk_usage`, `disk_io`, `gpu_load`, `gpu_vram`, `thermal` / `thermal_gradient`, `network_load` (with `network_load_recv` / `network_load_sent` and their `_gradient` variants), `battery` / `battery_gradient` (plus `battery_full`, `battery_empty`, `battery_online`, `battery_offline`, `battery_ac_state`), `branch` (with `branch_clean` / `branch_dirty`), `git_status` (with `git_status_clean` / `git_status_dirty`), `stash`, `cwd` (with `cwd:current_folder`), `user` / `superuser`, `hostname`, `date` / `time`, `player` (with `player_playing` / `player_paused` / `player_stopped` / `player_fallback`), `weather` (with `weather_conditions`, `weather_temp` / `weather_temp_gradient`).
- **Shell-status groups:** `exit_fail` / `exit_success` (the `last_status` / `last_pipe_status` pair), `email_alert`.
- **Stack / cloud groups:** `kubecontext`, `docker_containers`, `github_ci` (with `github_ci_success` / `github_ci_failure` / `github_ci_pending`), `aws_context`, `gcp_context`, and the `<tool>_rkyv_cache` / `<tool>_version` families for the MenkeTechnologies segments.
- **Structural / neutral groups:** `information:regular` (the neutral fallback several stack segments end their group chain on, so a chunk renders in *any* colorscheme), `background`, `background:divider` (the soft-divider color), and the `:divider`-suffixed variants that the renderer uses for `divider_highlight`.

The `<name>` / `<name>_gradient` pairing is the pattern to internalize: a gradient segment lists the `_gradient` group *first* and the flat group second, so a colorscheme with a defined gradient gets the shading and one

without it degrades to a flat color rather than erroring. Naming a colorscheme's groups to cover the flat names is the minimum; adding the `_gradient` variants is what turns a static prompt into a shaded one.

Appendix D – The Renderer Reference

Chapter 7 covers the render *loop*; this appendix is the per-target *encoding* reference — the exact bytes each renderer emits for a resolved `{fg, bg, attrs}` — because the encoding is where “byte-identical to upstream” is won or lost one target at a time. Every renderer under `src/ported/renderers/` shares the layout core (`do_render`, `_render_length`, `_render_segments`) in `src/ported/renderer.rs` and differs only in its `hlstyle/hl` translation. The attribute constants are shared with the colorscheme: `ATTR_BOLD = 1`, `ATTR_ITALIC = 2`, `ATTR_UNDERLINE = 4`.

D.1 The shared layout core

`do_render` (`renderer.rs:792`) is the layout algorithm, and it is worth stating as steps. It fetches the theme's segments, prepares them (`_prepare_segments`, which runs `translate_np` over contents and precomputes per-segment lengths), and — when there is no width constraint — measures once and returns. When a width is set, it computes divider widths (`compute_divider_widths`, `renderer.rs:314`), orders segments by priority (`priority` non-null, sorted *descending*, then the null-priority segments), measures with `_render_length`, and if the line is too wide runs the truncation loop: drop segments in priority order, re-measuring after each removal, until the line fits (a fast path handles lines of more than 100 segments in bulk). Finally it distributes any spacer expansion (`expand`-marked segments absorb the leftover width via the `l/r/c` align functions) and joins the rendered highlight fragments. `_render_length` (`renderer.rs:1075`) is the measurement half; `_render_segments` (`renderer.rs:1237`) is the emit half, and both decide each boundary's divider by the same rule — `divider_type = "soft" if compare_bg == seg_bg else "hard"` (`renderer.rs:1186 / 1358`). Non-printing bytes are handled by `translate_np` (`renderer.rs:158`), which maps control characters `0x00-0x1F` to caret forms (`^@...^_`) and invalid bytes to `<hh>` forms — the same `translate_np` the README names among the byte-identical behaviors.

D.2 tmux – renderers/tmux.rs

The most-used emitter. `hlstyle` (`tmux.rs:114`) builds `#[<comma-joined attrs>]` (line 158), returning `""` when the segment has no fg, bg, or attrs. Color encoding is `color_spec` (`tmux.rs:265`): the default sentinel emits `<channel>=default`; a truecolor value with `term_truecolor` on emits `<channel>=#{rgb:06x}`; otherwise `<channel>=colour<cterm>`. Attributes (`attrs_to_tmux_attrs`, `tmux.rs:25`) always emit all three slots: `attrs == None` gives `nobold,noitalics,nounderscore`, and a set flag emits the *on* form (`bold/italics/underscore`) with the other slots *off* — the explicit `no*` forms are mandatory because tmux markup is stateful and a missing `nobold` bleeds bold forward. Character translation (`tmux.rs:104`) maps `#` → `##[]`, since `#` is tmux's own escape lead-in. Example output: `#[fg=#ffffff,bg=#d78700,bold,noitalics,nounderscore]71%`.

D.3 vim – renderers/vim.rs

vim references named highlight groups rather than inlining colors. `hlstyle` (`vim.rs:442`) returns a `%#<GroupName>#` reference (line 527) and, on a cache miss, pushes a `:hi <name> ctermfg=... guifg=... ctermbg=... guibg=... cterm=... gui=...` command into a command list the plugin executes. The group name (`build_group_name`, `vim.rs:395`) is `Pl_<ctermfg>_<guifg>_<ctermbg>_<guibg>_<attrs>` with the gui values as *decimal* RGB ints or `None`. It dedups two ways: it returns `""` when the requested highlight equals

the previous one (upstream's E541 workaround) and when there is nothing to set. Attributes (`attrs_to_highlight`, `vim.rs:417`) emit the plain names `bold/italic/underline` (no `no*` forms — vim's default is `NONE`), rendered into both `cterm=` and `gui=`. Character translation maps `%` → `%%` (vim format strings use `%`).

0.4 Shell — `renderers/shell/`

The shell base (`shell/mod.rs`) emits ANSI SGR sequences. `hlstyle` (`shell/mod.rs:339`) starts with a reset `[0]`, then appends foreground (`[38, 2, r, g, b]` truecolor or `[38, 5, cterm]`), background (`[48, ...]`), and *one* attribute code (bold `1`, else italic `3`, else underline `4` — a mutually exclusive chain, matching upstream's `elif`). The default assembly is `\x1b[<codes joined by >;m`; an `fbterm` terminal gets a different assembly (`\x1b[1;<n>] / \x1b[2;<n>]`), and `tmux/screen` passthrough wraps the sequence in `\x1bPtmux;...\x1b\\ / \x1bP...\x1b\\` when those flags are set. This is the one renderer whose `hlstyle` emits an explicit reset — the others do all their coloring inside `hl`.

The per-flavor renderers differ only in the *escape markers* wrapped around each SGR sequence, so the line editor can measure the visible prompt width. From the source (each file below sits under `renderers/shell/`):

Flavor	File	<code>escape_hl_start</code>	<code>escape_hl_end</code>	Char translations
<code>zsh</code>	<code>zsh.rs</code>	<code>%{</code>	<code>%}</code>	<code>%</code> → <code>%%</code>
<code>bash</code>	<code>bash.rs</code>	<code>\[</code>	<code>\]</code>	<code>\$</code> → <code>\\$</code> , <code>`</code> → <code>\`</code> , <code>\</code> → <code>\\</code>
<code>readline</code>	<code>readline.rs</code>	<code>\x01</code>	<code>\x02</code>	none
<code>ksh</code>	<code>ksh.rs</code>	<code>\x01</code>	<code>\x01</code>	prepends <code>\x01\r</code>
<code>tcsh</code>	<code>tcsh.rs</code>	<code>%{</code>	<code>%}</code>	<code>%</code> → <code>%%</code> , <code>\</code> → <code>\\</code> , <code>^</code> → <code>\^</code> , <code>!</code> → <code>!\</code>
<code>rcsh</code>	<code>rcsh.rs</code>	<code>\x01</code>	<code>\x02</code>	inherits <code>readline</code>

The base `\x01\x02` pair (SOH/STX) is what `zsh`'s `%{...%}` and `bash`'s `\[...\]` stand in for: each tells that shell's line editor "the bytes between these markers are non-printing," so a prompt full of color escapes still measures at its visible width and long command lines wrap correctly. The `readline.py` / `zsh.py` files being tagged NEAR (Chapter 18) is a classifier artifact of these being constant-only shells; the constants themselves are present and correct in every flavor above.

0.5 The bar and markup renderers

The remaining renderers do all coloring in `hl` (their `hlstyle` returns `""`):

- lemonbar** (`renderers/lemonbar.rs`): `%{F#ff<rgb>}` foreground, `%{B#ff<rgb>}` background (note the doubled `ff` alpha prefix), `%{+u}` for underline, terminated by `{F-B--u}`; `render` wraps sides with `{l}/{r}` position markers; `%` → `%%`.
- i3bar** (`renderers/i3bar.rs`): a JSON object per segment — `{"full_text": ..., "separator": false, "separator_block_width": 0, "color": "#<rgb>", "background": "#<rgb>"}` — with a trailing comma, the shape `i3bar`'s streaming array wants. No attribute support in the protocol.
- pango_markup** (`renderers/pango_markup.rs`): `<span foreground="#<rgb>" background="#<rgb>" font_weight="bold" font_style="italic" underline="single">...</span>`, attributes as independent flags; `escape` does XML entity replacement (`&` → `&`; , then `<`, `>`).

- `pdb` (`renderers/pdb.rs`): a `ReadlineRenderer` subclass (`\x01\x02`) whose `do_render` ASCII-folds the output (non-ASCII $\rightarrow$ `?`), mirroring Python's `.encode('ascii', 'replace')`.
- `ipython` (`renderers/ipython/`): the `prompt_toolkit` token renderers, emitting pygments token names (`Pl_...` on `prompt_toolkit` 1.x, `pl_...` on 3.x) and registering style rules rather than raw escapes — the one target where powerline hands *tokens* to the host rather than bytes to a terminal.

0.6 Why the encodings are a fidelity surface, not a formality

It is tempting to treat the encoders as trivia — every one of them is a small function — but they are exactly where a “port” and an “inspired-by” diverge invisibly. Forget tmux's `#` $\rightarrow$ `##[ ]` and any prompt with a hash in it mangles; forget the `no*` attribute forms and everything after a bold segment stays bold; forget the shell escape markers and long command lines wrap into the prompt; use Rust's default rounding instead of banker's rounding and a gradient shifts one color at the midpoint. None of these is visible on a simple prompt, and all of them are visible on some real one — which is why the parity harness (Appendix F) diffs the actual bytes for each target against the live interpreter rather than eyeballing a screenshot. The encoders are short, and they are the port.

Appendix E – The Daemon Protocol and Lifecycle Reference

Chapter 4 tells the daemon’s story; this appendix is its specification — the wire format, the flags, the socket paths, the cache key, the lifecycle steps, and the signal behavior — at the level of detail someone reimplementing a compatible client or debugging a live daemon needs. Everything is grounded in `src/ported/scripts/powerline_daemon.rs` (line numbers below refer to it) and the client at `src/bin/powerline.rs`.

E.1 The wire format

Request layout (documented at `powerline_daemon.rs:252`):

```
<numargs_hex> \0 <arg> \0 <arg> ... \0 <cwd> \0 <env=val> \0 ... \0\0
```

The first field is the argument count *minus one* in hexadecimal text; then that many NUL-terminated arguments; then the NUL-terminated cwd; then the environment as NUL-terminated `KEY=VAL` entries; then a terminating double NUL. `parse_args` (`powerline_daemon.rs:249`) splits on NUL, drops empty fragments, reads `usize::from_str_radix(parts[0], 16)` for the count, slices the args, takes the cwd, and splits each remaining entry at its `first =`. Two guards: an empty request returns `None` (the daemon replies `"malformed request"`), and an empty cwd falls back to `environ["PWD"]` then the literal `"/"`. The reader, `do_read` (`:296`), accumulates until the buffer `ends_with(b"\0\0")` or a two-second timeout; a zero-length read (peer EOF) breaks the loop. The client’s marshaller (`powerline.rs:88-130`) writes the mirror image: hex count, args, cwd, `KEY=VAL` env pairs, double-NUL terminator (the byte-by-byte dump is in Chapter 4).

E.2 The EOF sentinel

`EOF = b"EOF\0\0"` (`powerline_daemon.rs:73`) — the exact bytes `E`, `O`, `F`, NUL, NUL. A client that sends exactly this triggers `do_one` to return `Some(0)`, a clean exit. This is how `powerline-daemon -k` stops a running daemon (`kill_daemon`, `:1044`), and it is a faithful port down to the byte so a Python client can stop the Rust daemon and vice versa.

E.3 Socket paths and the two namespaces

`USE_FILESYSTEM()` (`:63`) is `!cfg!(target_os = "linux")`. On non-Linux the socket is a filesystem path `/tmp/powerline-ipc-<uid>` (`:1242`); on Linux it is the abstract-namespace socket `\0powerline-ipc-<uid>` (`:1245`), which has no filesystem entry and so cannot be orphaned by a crash or swept by a `/tmp` cleaner. The pidfile `/tmp/powerline-ipc-<uid>.pid` exists only on the filesystem platforms (`:1250`); Linux skips pidfile locking. An explicit `--socket PATH` is honored *literally on every platform* (`:1234`) — a deliberate, documented divergence from upstream, which prepends `\0` on Linux even for explicit paths; the port does not, so a caller passing a real path gets a real filesystem socket. `check_existing` (`:1018`) removes a stale filesystem socket before binding and returns `Ok(None)` on `AddrInUse` (the “already running” signal).

E.4 The daemon flags

`parse_daemon_argv (:1190)` recognizes exactly five flags into `DaemonArgs { quiet, socket, kill, foreground, replace }`:

- `-q / --quiet` — suppress diagnostic prints; the flag `.zshrc/.tmux.conf` use.
- `-s / --socket <PATH>` — target a non-default socket (honored literally, E.3).
- `-k / --kill` — send the EOF sentinel to stop a running daemon; returns 0 if one was killed, 1 if none was running; errors (exit 2) if combined with `--foreground/--replace`.
- `-f / --foreground` — skip `daemonize()`, staying attached to the terminal (systemd `Type=simple` or interactive debugging).
- `-r / --replace` — loop `kill_daemon` + a 2-second sleep until the old daemon is gone, then start fresh. This is the “reload config with a full restart” path in the absence of a `SIGHUP` handler.

Unknown flags are ignored. Two upstream affordances are *absent* and named as such: there is no `-a` flag and no `SIGHUP` re-exec in the port (Chapter 4).

E.5 Startup: lock, bind, daemonize

The startup sequence, ported from upstream’s `scripts/powerline-daemon`: on filesystem platforms, take the **pidfile lock** — `lockpidfile (:1106)` opens the pidfile `O_WRONLY|O_CREAT`, takes a non-blocking `fcntl(F_SETLK, F_WRLCK)`, truncates and writes the pid, and installs the SIGTERM handler; the returned `PidLock (:1085)` is an RAI guard whose `Drop` cleans up the lockfile and closes the fd. **Bind** via `check_existing`. Then, unless `--foreground`, **daemonize** — `daemonize() (:925)`: `fork #1` (parent exits), `chdir("/")`, `setsid`, `umask(0)`, `fork #2` (parent exits), and `dup2 /dev/null` onto fds 0/1/2. After it the process is a session-leaderless background server with no controlling terminal and no stray descriptors.

E.6 The accept loop

`main_loop (:854)` sets the listener non-blocking, keeps a `HashMap<RawFd, Conn>`, installs a SIGINT flag, and loops `do_one (:652)` until it returns an exit code. `do_one` builds a `pollfd` vector — the listener always watched for `POLLIN`, each connection watched for `POLLOUT` if it has a queued response else `POLLIN` — and calls `libc::poll` with a 60-second timeout. A readable listener accepts a new connection through `eintr_retry_call (:222)`, which retries on `EINTR`; a readable connection is read to the double-NUL, and if the request is `EOF` the loop returns `Some(0)`, otherwise `get_answer` renders and the response is queued for the writable pass. `poll_readable (:338)` folds `POLLHUP` into “readable” so a peer hangup reads as `EOF`, exactly as upstream’s `select` reported it — the behavior-neutral reason the port could swap `select` for `poll`.

E.7 The Powerline cache key

`PowerlineKey (:117)` is the tuple that decides which warm orchestrator serves a request, a port of upstream’s cache key. Its fields are the extension (`args.ext[0]`), the renderer module, the config/theme/path overrides (`config_override`, `theme_override`, `config_path`), and the three environment values `POWERLINE_THEME_OVERRIDES`, `POWERLINE_CONFIG_OVERRIDES`, and `POWERLINE_CONFIG_PATHS`. It derives `Hash/Eq` and keys the daemon’s orchestrator map, so two renders that share those keys share a warm `Powerline` instance and a render that differs in any of them gets its own, built once and cached. This is why the *second* render of a given kind is even cheaper than the first (Chapter 5): the first built and cached the orchestrator keyed by this tuple.

E.8 Signals and shutdown

`SIGTERM (:1067)`: `sigterm_handler (:1069)` unlinks the pidfile (its path held in a static `Mutex<Option<String>>`) and calls `libc::_exit(1)` — async-signal-safe, no allocation in the handler. `SIGINT (:898)`: sets an `AtomicBool` the main loop polls, breaking with exit code 0 (upstream's `KeyboardInterrupt → SystemExit(0)`). There is deliberately **no** `SIGHUP` handler. Config changes are picked up instead by the per-extension mtime-staleness reload in `src/bin/powerline-daemon.rs`, which rebuilds the config cache when the underlying files change, and by `--replace` for a full restart.

E.9 The client and its fallback

`src/bin/powerline.rs` parses only a leading `--socket PATH` (consuming `argv[1..=2]`), requires at least one remaining argument (else prints `"Must provide at least one argument."` and exits 1), marshals the wire request, connects to the socket (abstract-name connect on Linux), and copies the response to stdout. On a connect failure it does not error — it `execvps powerline-render` with `argv[0] = "powerline-render"` and the client's own arguments (`exec_render_fallback, powerline.rs:155`), so a dead daemon degrades to a slower but working direct render rather than a broken prompt. That transparent fallback is the resilience property that makes the daemon safe to depend on.

Appendix F – The Parity Harness, the Tiers, and the Bug Log

Chapter 10 argues that “faithful port” is only worth its verification; this appendix documents that verification as machinery — how the harness is built, how the tier classifier scores a file, how the drift gate works, and what *classes* of bug the harness has caught — so the claim can be inspected rather than trusted. It is grounded in `tests/parity_against_upstream.rs`, `tests/daemon_parity.rs`, `tests/ported_fn_names_match_py.rs`, and the `scripts/gen_port_*.py` classifiers.

F.1 How a parity assertion works

The harness does not compare against captured golden files; it runs the *live* Python interpreter on the vendored upstream source and diffs. The mechanism is `py_eval` (`tests/parity_against_upstream.rs:22`): it shells out to `python3`, evaluates an upstream expression (importing the real `powerline` package from `vendor/powerline`), captures stdout, and strips exactly the trailing newline `print()` adds while preserving any trailing whitespace that is part of the value. `python_available (:51)` lets an assertion *skip* rather than fail when Python is absent, so the suite degrades gracefully on a machine without the interpreter while still running fully in CI where Python is present. Each assertion computes the same input two ways — once through the Rust port, once through `py_eval` on upstream — and asserts equality.

The file is organized by upstream module, section by section: `version.py`, `lib/humanize_bytes.py`, `colorscheme.py` (the `ATTR_*` constants, `get_attrs_flag`, `pick_gradient_value`, the full 256-entry `cterm_to_hex` table), `lib/dict.py` (`mergedicts/mergedefaults`), `config.py` layout invariants, `lib/path.py`, and so on down through the renderers, segments, and bindings. The 462 assertions accreted in numbered “rounds” (the git log shows `round-5` through `round-19` and beyond), each round adding a batch targeting a specific module’s edge cases — which is the visible fossil record of the port being pinned one behavior at a time.

F.2 The daemon parity suite

`tests/daemon_parity.rs` is the sharpest test because it exercises the whole pipeline end-to-end: it sends a request over the wire protocol to the Rust daemon and diffs the rendered markup against what upstream’s `powerline-render` produces for the same config and request. It covers 45 byte-for-byte scenarios — representative configs and requests across the `tmux`, `shell`, and `vim` extensions — and it is exactly the test the migration guide’s Step 2 has a user run by hand: spin up the Rust daemon on a probe socket, send a wire request, diff against `powerline-render tmux right`. Alongside it, `daemon_e2e.rs`, `daemon_fuzz.rs`, and `daemon_stress.rs` exercise the socket lifecycle, malformed-request handling, and concurrency, so the protocol is checked for robustness as well as for byte-fidelity.

F.3 The tier classifier

`scripts/gen_port_checklist.py` assigns each upstream Python file a tier, and `DONE` — the top tier — requires two things of the corresponding Rust: a `// py:NNN` citation density of at least 0.5 (half the function body’s lines pinned to an upstream source line) and a `/// Port of <py_fn>()` doccomment naming the

ported function. Both must hold. The current output is 134 DONE / 3 NEAR / 0 PARTIAL / 0 SPARSE / 0 STUB-HEAVY — two of the three NEAR files being the constant-only shell classes of Chapter 18 and the third a mirror placeholder, all three tagged by the classifier’s class-only branch rather than by a ratio (Chapter 29 reads that branch). The classifier is re-runnable (`python3 scripts/gen_port_checklist.py > docs/PORT_CHECKLIST.md`) against a tree that has the vendored upstream in place, so the badge numbers are outputs of a tool over real source rather than hand-maintained claims; on a tree without the vendor checkout the script emits zeros and the checklist is a frozen artifact (Chapter 29).

Two properties of that output are not visible in it and must be carried alongside it. First, both of the tier’s inputs are text properties of the Rust file — `count_rs` at `scripts/gen_port_checklist.py:79-99` counts `// py:` and `/// Port of` occurrences and nothing else — so DONE certifies transcription and attribution, not execution; `src/ported/renderers/shell/bash.rs` is DONE with 80 citations and a `do_render` returning `String::new()` at `:123` and `:135` under its own “Status: stub” doccomment at `:56`, while the completely-ported `zsh.rs` sits at NEAR. Second, `main()` at `:127` pairs upstream files to Rust files by walking the filesystem, so four rows in the 137 are scored from `src/ported/bindings/{i3,bar,lemonbar,awesome}/powerline-*.rs`, none of which is declared in `src/ported/bindings/mod.rs` or built as a `[[bin]]`; three of those four are inside the 134 DONE. Chapter 18 tabulates them and Chapter 36 shows the same filesystem-versus-build-graph gap from the segment side.

F.4 The drift gate

`tests/ported_fn_names_match.py.rs` guards a failure mode the parity harness cannot see: the port claiming to have ported a function that upstream renamed or that never existed. It asserts that every `/// Port of <py_fn>()` doccomment names a real entry in `docs/powerline_py_functions.txt`, the canonical list of upstream functions. A misspelled or stale doccomment fails the gate. Where the parity harness checks *behavior* for the inputs it exercises, the drift gate checks the *declared correspondence* against the authoritative function set — the two catch disjoint classes of error, which is why the port runs both.

F.5 What classes of bug the harness has caught

The project counts 11 port bugs surfaced by the parity harness and corrected (the README points to the git log for the running record rather than freezing a canonical list). Reading the log and the edge-case tests, they fall into recognizable classes, each a place where the Rust *looked* right and produced different bytes — exactly the kind a self-authored unit test would have missed because it would have encoded the same wrong assumption:

- **Rounding:** the `pick_gradient_value` midpoint. Rust’s default `f64::round` rounds half away from zero; Python 3 rounds half to even. A gradient at exactly 50% picked the wrong color until the port adopted `round_ties_even` (`py_round`).
- **Width fallback:** the `strwidth` N/Na precedence and char-vs-byte width (`987cd2437d` in the log) — measuring the line by the wrong unit throws off truncation.
- **Escaping:** the tmux `#` → `##[]`, bash `\[...\]`, and tcsh `%/\^/!` translations, each pinned by its own parity round (`round-10`, `round-11`) — a forgotten translation mangles exactly the prompts that contain the escape character.
- **Merge semantics:** the top-level `REMOVE` sentinel in `mergedicts` and the empty-overlay case in `mergedicts_copy` (contract tests) — a deep-merge that handled the sentinel one level too shallow would fail to delete an inherited key.
- **Size accounting:** the fusevm cache `{size}/{bytes}` reporting disk allocation (`du -sh`) versus logical sum (`1a709a30dc`) — a real divergence in an extension segment, caught the same way.
- **Glyph handling:** a glyph bug (`ae3c3a4c99`) in icon substitution.

The pattern across all of them is the argument for testing against the live interpreter: a port checked only by reading code and running its own unit tests would have shipped these, because each is a place where the code looked plausible and behaved wrong, and only a diff against the actual Python — which does not care

what the porter *thought* upstream did — distinguishes the two. The eleven are eleven divergences that no longer exist, and the harness that found them runs on every push, so the twelfth gets caught the same way.

Appendix G – Performance Measurement Methodology

Chapter 5 states the performance numbers; this appendix is about *how to measure them yourself*, because a performance claim you cannot reproduce is a marketing claim. The point is not to re-derive the headline figures but to give a reader the method to confirm them on their own machine, and to separate the three distinct wins so a measurement attributes each to the right cause.

G.1 What to measure, and against what

The comparison that matters is same-command, same-config, three ways: Python `powerline` cold, `powerliners` cold (`powerline-render`, no daemon), and `powerliners` warm (the client against a running daemon). Holding the command and config fixed is what isolates *startup* as the variable — the render itself is identical work in all three, so any difference is overhead around the render, which is the whole thesis. A useful harness is a loop that runs the same `powerline shell left` invocation many times and reports the distribution, not a single timing, because the interesting quantity for an interactive prompt is the *per-invocation* cost under repetition, and a single run is dominated by noise.

```
# Warm daemon path - start the daemon, then time N client round-trips
powerline-daemon --replace -q
python3 - <<'PY'
import subprocess, time
N = 200
t0 = time.perf_counter()
for _ in range(N):
    subprocess.run(["powerline", "shell", "left", "--last-exit-code=0"],
                  stdout=subprocess.DEVNULL)
dt = (time.perf_counter() - t0) / N * 1000
print(f"warm client: {dt:.3f} ms/render")
PY
```

Swapping the invoked binary — the Python `powerline` client and daemon, then the Rust ones — while keeping this loop identical is what makes the numbers comparable. The same loop against `powerline-render` (bypassing the daemon) measures the cold native path; against the Python `powerline-render` script it measures the cold Python path.

G.2 Reading the three tiers honestly

The measurement should reproduce three tiers, and Chapter 5's decomposition says which cause owns each. **Python cold, ~50–150 ms**: essentially all interpreter startup and import, paid in full every invocation because a fresh process cannot amortize its own startup. **powerliners cold, ~30 ms**: a native binary's cold start — the kernel mapping a static executable plus the config parse — with no interpreter to boot, which is why it is several times faster than Python cold even with no warm state. **powerliners warm, sub-millisecond**: a UNIX-socket round-trip plus the microseconds of the actual render, with the config parse, adapter setup, and orchestrator all already paid once at daemon start and cached under the `PowerlineKey` (Appendix E.7).

The honest framing is that these are three separate wins, and a measurement that blurs them misattributes

the cause. The daemon deletes per-render startup — the big win, available in principle to any language. The native binary makes the client and the cold fallback cheap — the Rust win. The single static binary makes deployment trivial — the packaging win. A measurement that only compares “Python” to “Rust” collapses all three into one number and loses the ability to say which lever did what; measuring the cold and warm native paths *separately* keeps them distinct.

G.3 The caveats that keep a benchmark honest

Two things make a naive benchmark lie, and both are worth controlling for. First, the **page cache**: Python’s import cost is dominated by filesystem reads, so a cold-cache run (first invocation after boot, or on a network filesystem) is dramatically slower than a warm-cache run, and quoting one without saying which overstates or understates the gap. The ~50–150 ms range is wide precisely because it spans warm-cache to cold-cache and fast-disk to slow-disk. Second, the **live-data segments**: a config with `external_ip` or `weather` on the measured line will show latency spikes that are network round-trips, not render cost, and on powerliners specifically the on-demand sampling model (versus Python’s threaded caching, Chapter 9) means those spikes land on the render that samples rather than being hidden behind a background thread. A performance measurement should use a config of *cheap* segments (identity, time, git) to measure render overhead, and treat the expensive samplers as a separate, placement-dependent concern — which is the same advice the segment catalog gives for building a prompt in the first place.

Appendix H – A Complete Worked Example: From Config to Bytes

The chapters trace pieces; this appendix traces the *whole* path once, end to end, so the reader can see every stage of the pipeline act on a single concrete input. It reuses the worked examples of Chapters 2, 4, 6, and 7 and stitches them into one continuous render.

H.1 The inputs

A user's `~/.config/powerline` tree, trimmed to the essentials. `config.json`:

```
{ "common": { "term_truecolor": true },
  "ext": { "shell": { "colorscheme": "demo", "theme": "demo" } } }
```

`themes/shell/demo.json`:

```
{ "segments": {
  "left": [
    { "function": "powerline.segments.common.net.hostname", "priority": 10 },
    { "function": "powerline.segments.common.sys.cpu_load_percent",
      "args": { "format": "{0:.0f}%" }, "priority": 30 }
  ],
  "right": [] },
  "dividers": { "left": { "hard": " ", "soft": " " },
    "right": { "hard": " ", "soft": "f" } },
  "spaces": 1 }
```

`colorschemes/shell/demo.json` and `colors.json` are the two files from Chapter 6's worked example — `gray0/white/orange` in the palette, the `cpu_load_percent_gradient` gradient, and the `hostname`, `cpu_load_percent_gradient`, and `cpu_load_percent` groups.

H.2 Stage 1 – the request arrives

The shell's `precmd` runs `powerline shell left --last-exit-code=0`. The client marshals `argv + cwd + env` into the NUL-delimited wire blob (Appendix E.1) and writes it to the per-UID socket. The daemon's `do_read` accumulates to the double NUL, `parse_args` reconstructs `(["shell", "left", "--last-exit-code=0"], {...env...}, "/home/j")`, and `get_answer` builds the `PowerlineKey` — `ext = "shell"`, the overrides empty — and finds (or builds and caches) the warm orchestrator for that key.

H.3 Stage 2 – config resolution

The orchestrator was built once. `get_config_paths` produced the search roots (user tree, then the bundled defaults under `config_files`); `_find_config_files` located `config.json`, `themes/shell/demo.json`, `colorschemes/shell/demo.json`, and `colors.json` across them; `load_json_config` parsed each; and `mergedicts` deep-merged the user files over the bundled defaults. `term_truecolor` resolved to `true`. The `Colorscheme` was constructed from the merged colorscheme and palette, deriving `gray0`'s hex from `cterm_`

`to_hex` and loading the gradient's two parallel lists. The `Theme` holds the two left segments, the divider glyphs, and `spaces = 1`. None of this re-runs on this render — it is warm state.

H.4 Stage 3 — segment resolution

`Theme::get_segments` iterates the left side. For each segment, `process_segment` runs the adapter via the registry. `hostname` reads the environment and returns `{contents: "jbox", highlight_groups: ["hostname"]}`. `cpu_load_percent` samples CPU (say 71%), formats "71%" through the `{0:.0f}%` arg, and returns `{contents: "71%", highlight_groups: ["cpu_load_percent_gradient", "cpu_load_percent"], gradient_level: 71.0}`. Each segment's groups go to `get_highlighting`: `hostname` resolves to its flat colors (`white` on `gray0`); `cpu_load_percent_gradient` resolves through the gradient at level 71 to `[136, 0xd78700]` for both fg and bg (Chapter 6's trace). Both segments are now `{contents, highlight: {fg, bg, attrs}}` records in `parsed_segments`, carrying their `priority` values (10 and 30).

H.5 Stage 4 — layout

`do_render` measures the two-segment line with `_render_length`. There is no width constraint from a shell prompt, so no truncation runs — but the divider decision still fires per boundary: the two backgrounds differ (`gray0` vs the gradient orange), so the boundary is a **hard** divider. `_render_segments` emits each segment's contents wrapped in one space of padding (`spaces = 1`), with the hard chevron between them drawn `fg = gray0's bg` on `bg = orange`.

H.6 Stage 5 — encoding and reply

The shell renderer's `hlstyle` turns each resolved highlight into an ANSI SGR sequence wrapped in the flavor's escape markers (`%{...%}` for zsh). With `term_truecolor` true, colors emit as `[38;2;r;g;b] / [48;2;r;g;b]`; the bold attribute on the gradient segment emits `1`. The finished byte string — non-printing escapes wrapped so `zle` measures the prompt correctly — is written back over the socket. The client copies it to stdout, and zsh assigns it to `PS1`. Total time on the warm path: a socket round-trip plus microseconds of render. The same two lines of theme JSON that a Python powerline user wrote years ago produced the same bytes, in under a millisecond, from a process that never paid a startup cost to do it. That is the whole of powerliners, traced once from keystroke to cell.

Appendix I – The Bindings Cookbook

Chapter 8 explains *how* each binding works; this appendix is the copy-paste reference — the exact lines that wire each target to the daemon, what each invocation carries, and the troubleshooting table for when a prompt misbehaves. Because the wire protocol is upstream's, these are the same lines a Python powerline user already has; the only thing that changes in a migration is which `powerline/powerline-daemon` resolves in `$PATH`.

I.1 tmux

Two lines in `~/.tmux.conf`:

```
run-shell "powerline-config tmux setup"
run-shell -b "powerline-daemon -q &>/dev/null || exit 0"
```

The first invokes `powerline-config tmux setup`, which is `setenv` then `source` (`src/ported/commands/config.rs`): it sets the `_POWERLINE_*` tmux environment variables and sources the version-appropriate conf, pointing tmux's `status-left`, `status-right`, and window-status formats at the `powerline` client. The conf files are embedded in the binary — `powerline-base.conf` plus seven version-specific variants — and extracted to `$XDG_CACHE_HOME/powerliners/tmux/` on first call, so the command works identically however the binary was installed. The second line starts the daemon in the background so the client's per-tick, per-window invocations hit a warm socket. Confirm the swap took by checking that the running daemon is the Rust binary: `ps -p $(cat /tmp/powerline-ipc-$(UID).pid)` on a filesystem platform, or `ss -x | grep powerline-ipc` on Linux (the abstract socket has no file).

I.2 zsh

A `precmd` hook rebuilds the prompt before each render:

```
powerline-daemon -q          # start the warm daemon once, at login

function powerline_precmd() {
    PS1="$(powerline shell left -r zsh_prompt --last-exit-code=$?
    ~ --last-pipe-status="$pipestatus" --jobnum=%j)"
    RPROMPT="$(powerline shell right -r zsh_prompt --last-exit-code=$?)"
}
function powerline_setup() {
    autoload -Uz add-zsh-hook
    add-zsh-hook precmd powerline_precmd
}
powerline_setup
```

The invocation carries the shell-local facts the daemon cannot see: `--last-exit-code=$?` feeds `last_status`, `--last-pipe-status="$pipestatus"` feeds `last_pipe_status`, and `--jobnum` feeds `jobnum`. The renderer flavor `-r zsh_prompt` selects the zsh shell renderer, whose `%{...%}` escape markers let zle measure the prompt width correctly. `RPROMPT` gets the same treatment; because zsh redraws the right prompt on interactive events, keeping the daemon warm is what keeps those redraws from each booting an interpreter.

I.3 bash

The same idea through `PROMPT_COMMAND`:

```
powerline-daemon -q
_update_ps1() { PS1="$(powerline shell left -r bash_prompt --last-exit-code=$?)"; }
if [ "$TERM" != "linux" ]; then PROMPT_COMMAND="_update_ps1; $PROMPT_COMMAND"; fi
```

The bash-specific wrinkle is the readline escape markers: the `bash_prompt` renderer wraps non-printing bytes in `\[...\]` (Appendix D.4) so readline computes the visible width, which is what keeps long command lines from wrapping into the prompt or corrupting history navigation. `powerline-config shell command` prints the preferred binding command for the current shell, and `powerline-config shell uses tmux / uses prompt` exit 0 when those components are enabled — the hooks a more elaborate rc file can branch on.

I.4 vim / neovim

One line in `.vimrc`:

```
execute 'source' trim(system('powerline-config vim source-path'))
```

`powerline-config vim source-path` extracts the embedded `powerline.vim` to `~/.cache/powerliners/vim/powerline.vim` and prints its path; there is no `+python3` requirement, and the plugin uses vim8-compatible syntax so it runs on vim 7.4+, vim 9, and neovim. The plugin sets `laststatus=2` and wires an `augroup powerliners` of autocmds — `VimEnter` for the first render, `WinEnter/BufWinEnter/BufEnter/TabEnter` for context switches, `ModeChanged` on modern vim, and a legacy `CursorMoved/CursorMovedI` fallback for ancient vim. Each fires `system('powerline vim left ...')`, which is a microsecond socket round-trip *with the daemon running* and a ~30 ms cold `powerline-render` without it — hence the recommendation to pre-start the daemon for vim. The per-request keys match upstream's `powerline.bindings.vim: mode (n/i/v/R)`, `bufnr (bufnr('%'))`, `winnr (winnr())`, and `buf_name (expand('%:p'))`, omitted when empty). The renderer emits vim's `##HighlightGroup#` markup and the plugin assigns it to `&statusline`.

I.5 ipython / REPL

The REPL binding is a deliberate non-port (Chapter 8): rather than embedding in the interpreter as upstream's Python provider does, powerliners renders REPL prompts through the external client hook, the same mechanism as a shell prompt. The result is the same colored prompt; the mechanism is external, which keeps the single-static-binary discipline intact.

I.6 Troubleshooting

Distilled from the README's troubleshooting table:

Symptom	Cause / fix
empty statusline in vim	<code>laststatus=2</code> was stomped later in <code>.vimrc</code> ; re-set it after the source line
garbled escape codes	terminal lacks truecolor; ensure <code>\$TERM=xterm-256color</code> so the cterm fallback path is used

Symptom	Cause / fix
colors don't match terminal	colorscheme JSON missing your palette; copy the relevant <code>colorschemes/<ext>/default.json</code> into your user tree
<code>write() to daemon failed</code>	daemon isn't running; <code>powerline-daemon -q</code> (the client would otherwise fall back to <code>powerline-render</code>)
refresh stutters on every keystroke in vim	the legacy <code>CursorMoved</code> fallback is firing (vim < 8.2.2871); upgrade vim or disable that autocmd
exit code never shows	the binding isn't forwarding <code>--last-exit-code=\$?</code> , so <code>last_status</code> has nothing to read

Appendix J – The Custom-Segment Cookbook

Chapter 3's `exec` section states the mechanism; this appendix is the working reference for building custom segments, with complete scripts and the exact output protocol, grounded in `src/extensions/exec_segment.rs`.

J.1 Pattern A – the explicit `exec` adapter

Spell the command out inline in the theme:

```
{ "function": "exec",
  "args": {
    "command": "/usr/local/bin/cpu_temp.sh",
    "args": ["--unit", "C"],
    "format": "%s°C",
    "highlight_groups": ["exec"]
  } }
```

The daemon spawns `command` with the positional `args` (no shell), reads stdout, and wraps the result. Because of the highlight-group caveat (J.4), name the colorscheme group `"exec"` — the function name — unless you take control of the groups via JSON-array output.

J.2 Pattern B – dotted-path filesystem dispatch

Reference the segment by dotted path and drop an executable under the config tree:

```
{ "function": "myseg.cpu_temp" }
```

```
mkdir -p ~/.config/powerline/segments/myseg
cat > ~/.config/powerline/segments/myseg/cpu_temp.sh <<'EOF'
#!/bin/sh
echo "${osx-cpu-temp | sed 's/°C//'}"
EOF
chmod +x ~/.config/powerline/segments/myseg/cpu_temp.sh
```

`resolve_dotted_path` (`exec_segment.rs:45`) turns `myseg.cpu_temp` into `<config_path>/segments/myseg/cpu_temp.{sh,py,rb,pl,lua,js,<none>}` — the `SCRIPT_EXTENSIONS` order at `exec_segment.rs:31`, first match wins, `sh` beating `py`, the empty extension matching a bare executable — searched across the *full config-path cascade*, so a script in any config root on the cascade becomes available. Pattern B is the tidier of the two because its natural function name (`cpu_temp`) is also a sensible colorscheme group name.

J.3 The output protocol

Both patterns inspect stdout at the first non-whitespace byte. If it is `[`, stdout is parsed as a JSON array of segment dicts and used verbatim — full control over `highlight_groups`, `gradient_level`, `divider_highlight_group`, and multi-chunk output. Anything else is treated as plain text and wrapped as one chunk with the `format` template applied (`%s` → trimmed stdout, `%%` → literal `%`). So a two-line shell script becomes a segment, and a richer script that prints JSON becomes a fully-styled multi-chunk one:

```
#!/usr/bin/env python3
import json, psutil
print(json.dumps([
    {"contents": "[] ", "highlight_groups": ["thermal_icon"]},
    {"contents": f"{psutil.cpu_percent():.0f}%",
      "highlight_groups": ["thermal_gradient", "background"],
      "gradient_level": psutil.cpu_percent()}
]))
```

The JSON path is how a script drives its own gradient and multi-chunk coloring — the plain-text path is for the common “echo a number” case.

J.4 The highlight-group caveat

A documented Rust-port divergence: `gen_segment_getter` (`src/ported/segment.rs:1011`) uses a function segment’s resolved `function_name` as its *only* highlight group, ignoring a theme’s `highlight_groups` override (upstream respects it). The practical consequences: for Pattern A, define a colorscheme group literally named `"exec"` (or take control via JSON-array stdout that emits explicit `highlight_groups`); for Pattern B, the dotted path’s trailing component is the group name, so `myseg.cpu_temp` colors under a `"cpu_temp"` group. This is why Pattern B is the recommended shape for anything that needs custom coloring — its function name and its group name are the same string, so the caveat simply does not bite.

J.5 When to reach for a native adapter instead

The `exec` hatches cover essentially everything upstream’s `__import__`-based custom segments did, at the cost of a subprocess per render. For a segment you will use everywhere and want at full speed, the alternative is a native adapter under `src/extensions/` — a Rust closure of the `fn(args, info) -> Option<Value>` shape wired into the `ADAPTERS` table, held to the same test discipline but carved out of the port’s fn-name rules by `docs/PORT.md`. That is how the whole stack-aware and system-metrics families were added; adding another is the same pattern rather than a special case, and it is the path for a segment that a subprocess fork would make too slow for the per-keystroke prompt.

Appendix K – Listers and `segment_list` Expansion

Chapter 2 introduces the third segment kind — the `segment_list`, or *lister* — and this appendix documents the mechanism, because it is the part of the model most themes never touch directly but that powers the “one segment per window / per workspace” displays. It is grounded in `process_segment_lister` (`src/ported/segment.rs:578`) and the listers under `src/ported/listers/`.

K.1 The fan-out mechanism

A *lister* segment expands into a variable number of sub-segments discovered at render time. `process_segment_lister` takes a `lister` closure (which enumerates the things to render one segment each), a slice of `subsegments` (the template to instantiate per item), a `display_condition` predicate, and a `contents_func`. For each item the *lister* yields, it filters by `display_condition` and processes each subsegment against that item’s *patched* environment — the item’s data folded into a copy of `segment_info`. So a window *lister* yields one entry per tmux window with the window’s number and name folded in, and the subsegment template renders once per window with those values available. The count is never in the theme; it is discovered when the *lister* runs, which is exactly why the theme can say “render one segment per X” without knowing how many Xs there will be.

K.2 The i3 workspace and output listers

`src/ported/listers/i3wm.rs` ports the i3 listers. `output_lister` (`i3wm.rs:41`) yields one subsegment per xrandr output; `workspace_lister` (`i3wm.rs:88`) yields one per i3 workspace, setting the `workspace` and `output` keys in each item’s `segment_info` and honoring `only_show` (which workspaces to list) and `output` (restrict to one output). One honest boundary: the live i3 connection (`get_i3_connection().get_workspaces()`) depends on the i3-ipc Python bindings, which have no live Rust equivalent wired in, so the *lister*’s real data source returns nothing until that is connected — but the fan-out logic itself is ported and tested against a synthetic workspace list through a `workspace_lister_for` seam, so the *mechanism* is faithful even where the *data source* is stubbed. This is the port being explicit about a boundary rather than pretending a native i3-ipc client exists.

K.3 The vim tabpage and buffer listers

`src/ported/listers/vim.rs` ports the vim listers used by the *tabline*: `tablister` yields one subsegment per tabpage and `bufferlister` one per buffer. `tabpage_updated_segment_info` (`vim.rs:26`) is the environment-patching function — it returns a copy of `segment_info` with the tabpage-local `tabpage/tabnr/window/winnr/window_id/buffer/bufnr` keys set — so each tab’s subsegment renders with that tab’s window and buffer in scope. As with i3, the fan-out is ported and tested against synthetic tabpage/buffer lists; the live vim objects come from the vim bindings.

K.4 Why the machinery is more than most themes need

A single segment does not need any of this — `gen_segment_getter` resolves it once and `process_segment` runs it — which is why the `lister` path is a separate function most themes never invoke directly. But it is the only way to render a count that is unknown until render time, and it is a faithful port down to the generic closures and the per-item environment patching. When you see a `tmux` statusline with one colored block per window, or an `i3` bar with one block per workspace, that is `process_segment_lister` fanning a single theme entry across a runtime-discovered list — the segment model's most general case, carried in full even where its upstream data sources are native dependencies the port names rather than fakes.

Appendix L – The Five Binaries in Detail

Chapter 4 names the five binaries; this appendix documents each one's command surface — the `argparse` it implements, the subcommands it dispatches, and the upstream file it mirrors — so a reader can drive the whole suite from the shell. The manifest declares all five explicitly (`[[bin]]` entries in `Cargo.toml`) so `cargo install powerliners` ships the complete upstream-equivalent CLI.

L.1 `powerline` – the client

Mirrors `client/powerline.c`. Surface: an optional leading `--socket PATH` (which must come *before* the extension name, per the C-client convention), then the extension arguments passed straight through (`shell left, tmux right, vim left, ...`). It marshals `argv + cwd + env` to the socket and copies the response to stdout; on connect failure it `execvps powerline-renderer` with the same arguments (Appendix E.9). It is the binary shells, tmux, and vim actually invoke through `$POWERLINE_COMMAND`. It has no rendering logic of its own — every flag it does not consume is forwarded to whichever renderer serves the request.

L.2 `powerline-render` – the direct renderer

Mirrors `scripts/powerline-render`, and its `argparse` (`src/ported/commands/main.rs`) is the richest in the suite because it is the full render surface the client forwards to. Positional: `ext` (`nargs=1`, e.g. `shell`, `tmux`, or a `wm.*` extension) and `side` (`nargs='?'`, choices `left/right/above/aboveleft`). Flags:

- `-r, --renderer-module MODULE` — the renderer flavor (`.bash`, `.zsh`, `.tmux`, ...).
- `-w, --width INT` — the maximum width, which triggers the priority truncation of Appendix D.1.
- `--last-exit-code INT` — the previous exit code (an `int_or_sig`, so a signal name is accepted), feeding `last_status`.
- `--last-pipe-status LIST` — the space-separated pipeline exit codes, feeding `last_pipe_status`.
- `--jobnum INT` — the background-job count, feeding `jobnum`.
- `-c, --config-override KEY.KEY=VALUE` (append) — overrides into `config.json`.
- `-t, --theme-override THEME.KEY.KEY=VALUE` (append) — theme-specific overrides.
- `-p, --config-path PATH` (append) — additional config roots, prepended to the cascade.
- `-R, --renderer-arg KEY=VAL` (append) — renderer arguments (e.g. `client_id`, and the vim per-request keys), the general-purpose channel for values the segments read out of `segment_info`.

`powerline-render` runs the whole pipeline in one process — `argparse`, config resolution, segment run, encode — and is both a standalone tool and the client's cold fallback.

L.3 `powerline-daemon` – the server

Mirrors `scripts/powerline-daemon`; its five flags (`-q/-s/-k/-f/-r`) and full lifecycle are Appendix E. It is the warm process that holds the parsed config, the adapters, and the color palettes, keyed by `PowerlineKey`, and answers wire requests. In a login rc it is one line — `powerline-daemon -q` — and it then serves every shell, tmux, and vim for the session.

L.4 powerline-config — setup and dispatch

Mirrors `scripts/powerline-config`. It is the tool the tmux and vim bindings invoke. Its subcommands (`src/ported/commands/config.rs`):

- `tmux source` — source the version-specific tmux confs.
- `tmux setenv` — set the `_POWERLINE_*` tmux environment variables.
- `tmux setup` — `setenv` then `source` (what `~/.tmux.conf` calls).
- `shell command` — print the preferred shell binding command for the current shell.
- `shell uses tmux / shell uses prompt` — exit 0 if that component is enabled (a branchable predicate for rc files).
- `vim source-path` — extract the embedded `powerline.vim` and print its path.

It handles the same tmux/shell known-function dispatch upstream did in its config script, with the conf files and the vim plugin embedded in the binary (Chapter 8).

L.5 powerline-lint — the validator

Mirrors `scripts/powerline-lint`. Its argparse (`src/ported/commands/lint.rs`) is small: `-p/--config-path PATH` (append; the roots to check, in order) and `-d/--debug`. Behind it sits the full check pipeline under `src/ported/lint/` — a `markedjson` loader that preserves source positions, the `spec.rs` structural checks, `checks.rs`, `selfcheck.rs`, `inspect.rs`, and `context.rs` — which flags unknown segment names, missing color references, and malformed `before/after` strings *statically*, with source positions, before they become a broken prompt. It is the tool a migrating user runs first (Appendix M, step 2) to catch the errors that `get_highlighting`'s “groups not found” failure would otherwise raise at render time.

Appendix M – The Migration and Roll-back Runbook

Chapter 11 argues the migration is reversible; this appendix is the runbook — the exact sequence, with the parity verification against your *own* config that makes the swap safe. The governing property is that the migration touches only which binary resolves in `$PATH`: no config edits, no `.tmux.conf` edits, no shell-rc edits, because the wire protocol, socket path, and config tree are all unchanged.

M.1 Install

```
brew tap MenkeTechnologies/menketech && brew install powerliners
# or: cargo install powerliners
# or, from source:
cargo build --release --locked \
  --bin powerline --bin powerline-daemon \
  --bin powerline-config --bin powerline-render --bin powerline-lint
```

All three paths produce the same five binaries; the `--locked` flag on the source build pins the exact `Cargo.lock` dependency versions, the reproducibility discipline for a build you intend to trust in place of a working tool.

M.2 Verify parity against your own config – before changing anything

This is the step that makes the migration safe rather than a leap of faith. Run the Rust daemon on a *throwaway* socket and diff its output against the tool you are replacing, for your actual config:

```
# lint your config first – catch authoring errors statically
powerline-lint -p ~/.config/powerline

# spin up the Rust daemon on a probe socket that does not touch the real one
powerline-daemon --replace -q --socket /tmp/powerliners-probe

# render the same request both ways and diff the bytes
diff <(powerline --socket /tmp/powerliners-probe tmux right) \
  <(powerline-render tmux right)
```

If they match for your segments, the port is faithful to *your* config — the same check the `daemon_parity` suite runs on 45 representative configs continuously (Appendix F.2). The deterministic parts (markup, colors, layout, config resolution) will match byte-for-byte; the only legitimate differences are the live-sampled numeric values of Chapter 9 (a `cpu_load_percent` sampled a moment apart), which is why a diff is best run on a config of cheap, deterministic segments first.

M.3 Swap

```
powerline-daemon -k                # stop the Python daemon (sends the EOF
^ sentinel)
ln -sf "$(command -v powerline-daemon)" ~/bin/  # put the Rust daemon ahead in $PATH
# restart tmux so it re-runs its setup and starts the new daemon
tmux kill-server; tmux new-session
# confirm the running daemon is the Rust binary
ps -p "$(cat /tmp/powerline-ipc-$$ID.pid)"      # or: ss -x | grep powerline-ipc (Linux)
```

Because the Python C client speaks the same wire protocol, you can even leave it in place and swap only the daemon — the client does not know or care which language the daemon behind the socket is written in.

M.4 Rollback

Symmetric and complete:

```
powerline-daemon -k                # stop the Rust daemon
rm ~/bin/powerline-daemon          # the symlink; powerline-daemon re-resolves to the Python
^ script
powerline-daemon -q                # start Python again
```

Nothing but the symlink changed, so there is nothing else to undo — the config, the tmux config, and the shell rc are all untouched, and the Python daemon comes back to exactly the state it was in. A migration you can try in a minute and reverse in another is one people will actually attempt, and the reversibility is the direct consequence of the port changing the implementation while preserving every interface.

Appendix N — The Config Cascade, Worked

Appendix B gives the cascade's rules; this appendix runs them once on a concrete three-file overlap, because deep-merge semantics are far clearer traced than described. Suppose two roots are on the search path: the bundled defaults (searched first, via `config_files`) and the user tree (searched last).

N.1 The two files

Bundled `themes/tmux/default.json` (trimmed):

```
{ "segments": {
  "left": [ { "function": "hostname" }, { "function": "branch" } ],
  "right": [ { "function": "date", "args": { "format": "%H:%M" } } ],
  "spaces": 1,
  "dividers": { "left": { "hard": "", "soft": "" } } }
```

User `themes/tmux/default.json`:

```
{ "segments": {
  "right": [ { "function": "date", "args": { "format": "%H:%M:%S" } } ],
  "spaces": 2 }
```

N.2 The merge, key by key

`_find_config_files` collects *both* files (not the first only), and `mergedicts` merges the user file *over* the bundled one, recursing on object collisions:

- `segments` — both sides are objects, so recurse. `segments.left` exists only in the bundled file, so it survives unchanged (`hostname`, `branch`). `segments.right` exists on both; it is a *list*, not an object, so the merge does not recurse into it — the user's list *replaces* the bundled list wholesale. The right side is now the user's single `date` segment with format `%H:%M:%S`.
- `spaces` — a scalar on both; the user's `2` wins.
- `dividers` — exists only in the bundled file, so it survives (`" "/" "`).

The effective theme is: left side `hostname` + `branch` (from defaults), right side `date %H:%M:%S` (from the user), `spaces` `2` (user), dividers from defaults. Note the two different behaviors on `segments.right` versus `spaces`: because `right` holds a *list*, the merge could not recurse into it and the user's list replaced the default's; because `spaces` is a scalar, the user's value simply overwrote. This list-vs-object distinction is the single most common surprise when hand-editing a config, and it is faithful to upstream: to *change one segment* on a side you must restate the whole side's list, whereas to change a nested `args` field you can override just that field.

N.3 Removing rather than overriding

To *delete* the bundled right side rather than replace it, the user file uses the REMOVE sentinel (Appendix B.2):

```
{ "segments": { "right": { "__powerliners_remove_this_key__": true } } }
```

During the merge with `remove` enabled, `mergedicts` sees the sentinel value on `segments.right` and *deletes* the key from the merged result, so the effective theme has no right side at all — a thing an overwrite cannot express, since overwriting with `[]` would leave an empty-but-present side. This is the mechanism that lets a user subtract from the bundled defaults, not only add to or replace them.

Appendix 0 – Glossary

A compact reference for the terms this book uses in their powerliners-specific sense.

- **Adapter** — a Rust closure of shape `fn(args, info) -> Option<Value>` that produces a segment's contents, looked up by name in the `ADAPTERS` registry. Returning `None` renders nothing.
- **ADAPTERS** — the static registry (`src/bin/shared/render_runtime.rs`) mapping a segment's dotted name to its adapter, the port's replacement for upstream's dynamic `__import__`.
- **Cascade** — the ordered set of config roots (`config_files`, system dirs, user tree) that `_find_config_files` searches and `mergedicts` deep-merges into one effective config.
- **cterm** — a 256-color terminal color index (0–255), one half of every resolved color; the other half is the 24-bit hex value.
- **Divider** — the separator between two segments: *hard* (a solid chevron, drawn when backgrounds differ) or *soft* (a thin muted separator, drawn when they match). Chosen per boundary by comparing backgrounds, never placed by the author.
- **Gradient** — a color list selected by a 0–100 level via `pick_gradient_value`, used to shade a segment (e.g. `cpu_load_percent`) from calm to hot.
- **Highlight group** — a name both a segment and the colorscheme agree on (e.g. `branch_dirty`), resolved by `get_highlighting` into `{fg, bg, attrs}`.
- **Lister / segment_list** — a segment that expands into a runtime-discovered number of sub-segments (one per window, workspace, tabpage) via `process_segment_lister`.
- **PowerlineKey** — the daemon's cache key (extension + renderer + overrides + three env vars) that selects which warm orchestrator serves a request.
- **REMOVE sentinel** — `{"__powerliners_remove_this_key__": true}`, the value that *deletes* an inherited key during a merge rather than overriding it.
- **segment_info** — the render environment (forwarded environ, cwd, and parsed client args) the daemon reconstructs from the wire request and hands to each adapter.
- **term_truecolor** — the renderer flag that chooses the hex encoding (truecolor) or the cterm index (256-color) at the last step of encoding a resolved color.
- **Tier (DONE / NEAR)** — the port classifier's grade for a file: DONE requires ≥ 0.5 citation density plus a `/// Port of ...` doccomment per function; NEAR is the constant-only-file case that cannot promote a 0/0 function ratio.
- **Wire format** — the NUL-delimited `hex(argc-1) argv cwd env \0\0` request the client sends and `parse_args` demarshals, copied exactly from upstream's C client.

Appendix P – The Truncation Algorithm, Worked

Appendix D.1 states the truncation loop in the abstract; this appendix runs it once on a narrow tmux pane, because priority truncation is the render feature most authors under-use and most misunderstand. It is grounded in `do_render` ([src/ported/renderer.rs:792](#)) and `_render_length` ([renderer.rs:1075](#)).

P.1 The setup

A tmux `status-right` with five segments, each with a `priority` the theme set (lower number = more important = dropped last), and their rendered widths including padding and dividers:

Segment	priority	width (cells)
<code>branch</code>	10	12
<code>cpu_load_percent</code>	30	8
<code>mem_usage</code>	40	14
<code>date</code>	20	7
<code>hostname</code>	50	10

The full line measures $12 + 8 + 14 + 7 + 10 = 51$ cells. Now the pane is resized so the right side has only 36 cells. `do_render` is called with `width = 36`.

P.2 The loop

`do_render` first orders the segments for dropping. It collects the segments that *have* a priority, sorted *descending* by priority value (so the least-important goes first): `hostname` (50), `mem_usage` (40), `cpu_load_percent` (30), `date` (20), `branch` (10). Segments with a null priority (none here) are never dropped. It measures the line at 51, sees $51 > 36$, and enters the drop loop, removing one segment at a time and re-measuring with `_render_length`:

1. Drop `hostname` (priority 50): width $51 - 10 = 41$. Still > 36 .
2. Drop `mem_usage` (priority 40): width $41 - 14 = 27$. Now $27 \leq 36$ — stop.

The rendered right side is `branch`, `cpu_load_percent`, `date` — the three lowest-priority-number (most important) segments — and the two least important dropped entirely. Crucially, dividers re-flow around the survivors: because the drops happen before `_render_segments` emits bytes, the boundary comparisons run over the *surviving* list, so the chevrons connect the segments that remain, not the ones that were there before. The line does not wrap, overflow, or show a truncated segment; it shows fewer segments, cleanly.

P.3 Why priority is a design tool, not a fallback

The lesson for a theme author is that `priority` is how you tell the renderer what to sacrifice when space runs short — and that a segment with *no* priority is never dropped, so the essential segments (a branch, an

exit code) can be left priority-less to pin them, while the decorative ones (a hostname, a memory readout) carry high priority numbers so they are the first to go on a narrow pane. The alternative renderers behave the same: the truncation is in the shared `do_render` core, so a narrow vim statusline or a short lemonbar drops the same segments in the same order. One honest note carried from Chapter 7: a segment's `truncate callable` (upstream's per-segment shortening hook) is not invoked by the port — it cannot call a callable off a `serde Value` — so the port drops whole segments rather than shortening them in place, a documented gap that affects only themes that relied on in-place truncation rather than whole-segment dropping.

Appendix Q – Platform Notes: macOS, Linux, and the BSDs

powerliners runs the same config and renders the same markup on every platform, but *how* it samples live data and where it binds its socket differ by OS, because the underlying facilities do. This appendix consolidates the platform-specific behavior scattered through the catalog, so a user debugging a segment on a specific OS has it in one place. All of it is grounded in the adapter sources under `src/extensions/` and `src/ported/segments/`.

Q.1 The socket namespace

The single most visible platform split. `USE_FILESYSTEM()` is `!cfg!(target_os = "linux")`, so on **Linux** the daemon binds an abstract-namespaced socket `\0powerline-ipc-<uid>` with no filesystem entry (visible only in `ss -x / lsof -U`), and skips pidfile locking; on **macOS and the BSDs** it binds a real file `/tmp/powerline-ipc-<uid>` with a matching pidfile `/tmp/powerline-ipc-<uid>.pid`. The abstract socket cannot be orphaned by a crash or swept by a `/tmp` cleaner — it vanishes exactly when the daemon exits — which is why Linux gets it by default; the filesystem platforms unlink a stale socket before binding to get the equivalent robustness.

Q.2 CPU, memory, and load

- **cpu_load_percent**: macOS parses `top -l 1 -s 0 -n 0`; Linux reads `/proc/stat` with a 100 ms delta.
- **mem_usage / mem_swap**: macOS uses `vm_stat + sysctl -n hw.memsize` (and `sysctl -n vm.swapusage` for swap); Linux reads `/proc/meminfo`.
- **system_load / uptime**: `getloadavg` and the platform uptime source, cross-platform.

The README's third documented divergence is the reason these use OS probes rather than `psutil`: Python upstream errors and skips the affected segment when `psutil` is missing, whereas powerliners resolves the same data through `top/vm_stat/netstat/pmset/uptime` and renders — so the port is *more* robust in the no-`psutil` case, a divergence in the user's favor.

Q.3 Disk, network, GPU, thermal

- **disk_usage**: `df -k -P <mount>` on both, with macOS redirecting `/` to `/System/Volumes/Data` to read the writable volume. **disk_io**: Linux `/proc/diskstats` (two 500 ms samples); macOS `ioreg -c IOBlockStorageDriver -r` with a cross-call delta.
- **network_load / internal_ip**: macOS resolves the default-route interface via `netstat (-rn -f inet, -ibn for counters)`; Linux via `ip route show default` and `/sys/class/net/<iface>/statistics/{rx,tx}_bytes`.
- **gpu_usage_percent / gpu_vram**: a vendor-dispatch chain — `nvidia-smi`, then (Linux) `rocm-smi`, then (Linux) `intel_gpu_top`, then (macOS Apple Silicon) `ioreg ... -c IOAccelerator` — so one segment works regardless of GPU vendor.

- **thermal**: Linux reads name-filtered `/sys/class/hwmon/hwmon*/temp*_input` and `fan*_input`; macOS reads `sysctl -n machdep.xcpm.cpu_thermal_level` on Intel and derives a chip temperature from `ioreg -r -n AppleSmartBattery` on Apple Silicon (where fan RPM is always unavailable, so the RPM tail of the format is dropped).

Q.4 The consequence for parity

This is the concrete source of Chapter 9's “where output may legitimately differ”: two different samplers reading at two different instants read two different numbers, so a `cpu_load_percent` on macOS `top` and one on Python's `psutil` can differ by a sampling window's worth of data. The markup framing is byte-identical — same format, same gradient, same colors — and only the number inside can differ, and only by the sampling window. Everything deterministic is identical across platforms and against upstream; the live-sampled numbers are the one place where “identical” was never physically possible, and the platform matrix above is *why*.

Appendix R — The Extension Provenance Map

`docs/PORT.md` carves `src/extensions/` out of the port's fn-name rules, and this appendix maps the provenance so the boundary between faithful port and net-new work is unambiguous at the file level. Everything under `src/ported/` is graded by the DONE tier and the parity harness against a Python original; everything under `src/extensions/` is new code held to the same test discipline but with no Python original to measure against.

R.1 Ports of third-party powerline segments

- `src/extensions/mem_usage.rs` — a 1:1 port of `mKaloer/powerline_mem_segment`, which is why its dotted keys use the third-party `powerlinemem.*` namespace. It carries `humanize_bytes` (the same K/M/G rounding as powerline's own `lib/humanize_bytes.py`) so a memory readout matches upstream byte-for-byte.

R.2 Richer or more-portable takes on upstream segments

- `git_status.rs` — a p10k-style single-chunk VCS segment, richer than the ported `branch` (which lives in `render_runtime.rs`). Both are registered; a theme picks whichever fits.
- `disk.rs`, `gpu.rs`, `thermal.rs`, `proc_count.rs` — system-metrics segments sampled through OS probes so they work without Python's `psutil` dependency (Appendix Q).

R.3 Net-new cloud and stack segments

- `k8s.rs`, `docker.rs`, `github_ci.rs`, `aws_ctx.rs`, `gcp_ctx.rs` — orchestration and cloud context, mostly pure filesystem probes of the tools' own config files with silent fall-through.
- `fusevm_jit.rs`, `zshrs_rkyv.rs`, `stryke_rkyv.rs`, `awkrs_rkyv.rs` — cache-stat segments for the sibling MenkeTechnologies runtimes, sharing a resolution chain and token surface.
- `zshrs_version.rs`, `stryke_version.rs`, `awkrs_version.rs` (with `bin_version.rs`) — version readouts with a TTL cache so the daemon does not fork per tick.
- `exec_segment.rs` — the two subprocess escape hatches that replace upstream's `__import__` custom segments; `icons.rs` — the three-tier glyph registry; `bundled_config.rs` — the compile-time embed of the default config tree; `diag_log.rs`, `bin_version.rs`, `wthr_extensions.rs` — supporting glue.

R.4 The test seam pattern

The extension segments are unit-tested through pure-functional seams that pin behavior without touching the environment — for example the cache segments' `default_path_with(get_env, path_exists)` and `default_root_with(get_env)` seams (Chapter 18), which let every precedence level of the resolution

chain be asserted with no env-var mutation and no thread-safety hazard. This is the discipline that lets net-new code be held to the same bar as the ported tree even though there is no Python original to diff against: where the ported code is verified by parity, the extensions are verified by seams that make their logic deterministic and exhaustively testable.

Appendix S — The Test Suite Map

The `tests/` directory is the executable form of every claim in this book, and this appendix maps it so a reader can find the test behind a claim. The library's own 2497 unit tests live *inside* `src/` alongside the code they test (in-module `#[cfg(test)]` blocks); the integration and parity tests live under `tests/`.

- `parity_against_upstream.rs` — the 462-test parity harness (Appendix F.1, Chapter 30), organized by upstream module, diffing the port against the live Python interpreter.
- `daemon_parity.rs` — the 45 byte-for-byte end-to-end daemon scenarios (Appendix F.2).
- `daemon_e2e.rs`, `daemon_fuzz.rs`, `daemon_stress.rs` — the socket lifecycle, malformed-request handling, and concurrency for the daemon.
- `ported_fn_names_match_py.rs` — the drift gate (Appendix F.4), asserting every `/// Port of ...` doccomment names a real entry in `docs/powerline_py_functions.txt`.
- `contract_colorscheme_theme.rs`, `contract_dict_merge_gradient.rs`, `contract_round4_misc.rs` — contract tests pinning the colorscheme/theme resolution, the merge-and-gradient behavior, and assorted edges.
- `lib_edge_cases_boundary.rs`, `lib_edge_cases_round5.rs`, `lib_strwidth_fallback.rs` — the numeric and width edge cases (saturation, UTF-8 byte encoding, surrogate max, the `strwidth N/Na` fallback of Chapter 7).
- `bin_powerline_client.rs` — the client's wire marshalling and fallback.
- `vim_e2e.rs` — the vim renderer end to end.
- **The `*.sh` gate scripts** — repository-hygiene checks (final newlines, no trailing whitespace, man-page synopsis sections, README badges, docs structure) that keep the shipped artifacts consistent.

Reading the test map alongside the source is the fastest way to confirm any specific claim in this book: nearly every behavior described here has a test that pins it, and the two together — the prose that explains the design and the test that proves it holds — are what make the book's account of powerliners something you can check rather than something you have to take on faith.

Appendix T – The `segment_info` Reference

Every adapter reads the world through one map, `segment_info`, and this appendix documents what is in it, where each key comes from, and which segments read it — because “the daemon cannot see the calling shell” is only half the story; the other half is exactly what the client *does* send and where a segment finds it. It is grounded in the wire parse (`parse_args`, `powerline_daemon.rs:249`), the renderer’s environment assembly (`get_segment_info`, `renderer.rs:642`), and the per-target overrides in the tmux and vim renderers.

T.1 What the wire deposits

The client sends `argv + cwd + env` (Appendix E.1), and the daemon reconstructs three things into `segment_info`:

- **environ** — the full forwarded environment as a map. Segments that read it: `environment` (by its `variable` arg), `virtualenv` (`VIRTUAL_ENV/CONDA_DEFAULT_ENV`), `hostname`, `user`, `kubecontext` (`$KUBECONFIG`), `aws.context` / `gcp.context` (their `CLOUDSDK_*` / `AWS_*` overrides), and the cache/version segments (their `$*_HOME` / `$XDG_*` resolution chains). This is why a render reflects the *caller’s* environment, not the daemon’s stale one.
- **getcwd** — the forwarded working directory, aliased from `PWD` when the `cwd` field was empty. Segments that read it: `cwd`, `branch`, `stash`, `git_status`, `ci_status` — anything path-relative. `get_segment_info` (`renderer.rs:642`) sets `getcwd` from the request so a path-relative segment anchors on the shell’s directory rather than the daemon’s `chdir("/")`.
- **The parsed client arguments** — deposited under `args.*`: `args.jobnum` (`--jobnum`), `args.last_exit_code` (`--last-exit-code`), and `args.last_pipe_status` (`--last-pipe-status`). Segments that read them: `jobnum`, `last_status`, and `last_pipe_status` respectively. These are the shell-local facts the daemon has no other way to learn, which is the whole reason the wire protocol forwards `argv`.

T.2 The `mode` and per-target keys

`get_segment_info` also sets `mode` — the current editor/shell mode — which the colorscheme’s `mode_translations` key on, so vim’s insert mode or zsh’s vi-command mode can recolor the line. The per-target renderers patch in their own keys: the tmux renderer resolves `getcwd` from `pane_current_path` or a `TMUX_PWD_<pane_id>` environ var and sets `width_adjust` (`tmux.rs:217`); the vim renderer’s binding forwards `mode`, `bufnr`, `winnr`, and `buf_name` (Appendix I.4), which the vim segments and the tabline lists read; the pdb renderer patches `pdb`, `initial_stack_length`, and `curframe`. The general-purpose channel for anything else is `--renderer-arg KEY=VAL (-R)`, which lands in `segment_info` for a custom segment to read.

T.3 Why one map, not many arguments

Threading a single `segment_info` map through every adapter — rather than a fixed argument list — is what lets the segment signature stay `fn(args, info) -> Option<Value>` across 58 adapters with wildly

different needs: a `date` segment ignores `info` entirely, a `cwd` segment reads `getcwd`, a `last_pipe_status` segment reads `args.last_pipe_status`, and a custom `exec` segment can read whatever `--renderer-arg` deposited. The map is the extensibility seam: a new segment needs no new plumbing, only a new key to read, and the lister mechanism (Appendix K) works by *patching* this same map per item, which is how a per-workspace segment sees its workspace's data in exactly the place a normal segment would look.

Appendix U – Worked Segment Outputs

Tokens and format strings are abstract until you see the string a segment actually emits. This appendix collects concrete output examples for the segments whose formatting is most configurable, so a theme author can predict what a `format` change does. All are grounded in the adapters' `render_format` helpers.

U.1 kubecontext and hide_default

With `format "{k8s-glyph} {context}:{namespace}"` in context `prod-cluster`, namespace `kube-system`, and the default Nerd Font glyph, the segment emits `☼ prod-cluster:kube-system`. Now the subtle case: the same format in the `default` namespace with `hide_default` on (the default). `render_format_drop_ns (k8s.rs:128)` does not merely blank the `{namespace}` token — it also strips the adjacent separator, so `{context}:{namespace}` becomes `prod-cluster`, not `prod-cluster:`. This is the “stay quiet in the boring case, speak up when it could bite you” behavior: a forgotten `kube-system` shows nothing, but a switch to `production` shows `prod-cluster:production` in full. An unknown token is left intact (`render_format` only substitutes the tokens it knows), so `{context}/{namespace} v{version}` renders the literal `v{version}` verbatim — a deliberate choice so a typo degrades to visible text rather than a silent drop.

U.2 docker.containers

With `format "{docker-glyph} {running}/{total}"` and three running of eight total, the segment emits `🐳 3/8`. The available tokens are `{running}`, `{total}`, `{images}`, and `{stopped}` (the last derived as total minus running), so `"R{running} S{stopped} I{images}"` on the same state emits `R3 S5 I12`. With `show_when_zero` off (the default) and nothing running and no images, the segment returns `None` and vanishes rather than showing `0/0`.

U.3 process_count and the zombie warning

With `format "{proc-glyph} {total}"` and 214 processes, the segment emits `214`. The tokens `{total}` `{running}` `{sleeping}` `{zombie}` `{dwait}` `{stopped}` let a fuller readout — `"{total}p {running}R {zombie}Z"` — emit `214p 2R 0Z`. The interesting behavior is `warn_zombie` (default on): when a defunct process appears, the highlight chain gains `"process_count_zombie"` at the front, so a colorscheme can flip the whole segment to a warning color the moment a zombie shows up — a standing signal that something is not reaping its children.

U.4 The cache-size tokens

The cache segments distinguish *disk allocation* from *logical content size*, and the tokens make both available. For a `fusevm.jit_cache` with 40 entries totaling 1.2 MiB of content but 1.5 MiB of block allocation, `"{fusevm} {entries} {size}"` emits `☼ 40 1.5M` (the `{size}/{bytes}` tokens report `du -sh`-style disk allocation, the fix from `1a709a30dc`), while `"{logical_size}"` reports `1.2M` (the `stat`-style content sum). The distinction

matters for a cache you are watching grow: block allocation is what actually consumes the disk, which is why it is the default `{size}`.

Appendix V – The tmux Conf Files and Version Dispatch

The tmux binding is the most-used one, and its install path has a subtlety worth documenting: tmux changed its status-format syntax across versions, so a single conf cannot serve every tmux. This appendix documents how powerliners handles that, grounded in [src/ported/bindings/tmux/](#) and Chapter 8.

V.1 The eight embedded confs

The binary embeds eight tmux conf files via `include_str!`: `powerline-base.conf` (the version-independent wiring) plus seven version-gated variants — `powerline_tmux_1.7_plus.conf`, `powerline_tmux_1.8.conf`, `powerline_tmux_1.8_minus.conf`, `powerline_tmux_1.8_plus.conf`, `powerline_tmux_1.9_plus.conf`, `powerline_tmux_2.1_plus.conf`, and `powerline.conf`. `powerline-config tmux setup` extracts them to `$XDG_CACHE_HOME/powerliners/tmux/` on first call and sources the ones appropriate to the running tmux version, detected at setup time. The version split exists because tmux’s status-format capabilities changed — the `#{...}` format expansion, the per-window status format, and the `status-interval` behavior differ across the 1.7-through-2.1+ range — so the correct wiring depends on which tmux is running.

V.2 Why embedding beats install-time path baking

Upstream powerline baked config-file paths at install time, which broke when the tool was moved or installed a different way (a `pip` install, a distro package, a manual copy each put the confs somewhere different, and the baked path could be wrong). powerliners embeds the confs *in the binary* and extracts them to a cache dir keyed off `$XDG_CACHE_HOME`, so there is no compile-time path to get wrong: `powerline-config tmux setup` works identically whether the binary arrived via `brew install`, `cargo install`, or a manual `cp` into `$PATH`. This is the install-method-agnostic bundling of Chapter 8, and it is the same mechanism the vim plugin and the default config tree use (`bundled_config.rs`) — the artifacts that a user needs travel *inside* the one static binary, which is the deployment payoff of being native carried through to the last detail.

V.3 The setup handshake

`powerline-config tmux setup` is `setenv` then `source (src/ported/commands/config.rs)`: `setenv` exports the `_POWERLINE_*` environment variables tmux’s status formats reference, and `source` loads the version-appropriate conf that points `status-left/status-right/window-status` at the `powerline` client. From then on every `status-interval` tick and every window switch runs the client, which forwards to the warm daemon — the per-window multiplication (eight windows, eight renders per tick) that made the Python startup tax so painful and where the warm daemon pays off most.

Appendix W – Design-Rationale FAQ

Some questions recur, and answering them in one place captures the reasoning that is otherwise spread across the chapters. Each answer is the book's position stated compactly.

Why a daemon rather than just a faster binary? Because a fresh process cannot amortize its own startup: any model built on fresh-process-per-render pays startup every time by construction. A Rust binary that still forked per render would pay a (small) Rust startup every render — better than Python, but not sub-millisecond. The daemon is the load-bearing idea; the native language is what makes the daemon's client cheap and its cold fallback tolerable. The three wins are separate: the daemon deletes per-render startup, the native binary makes the client and cold path cheap, and the single static binary makes deployment trivial (Chapter 5).

Why keep the redundant `function_name_dup` field? Because the port is a port. Upstream's `gen_segment_getter` returns a five-tuple in which the function name appears twice, and preserving the shape — even where it is redundant — is what “faithful port” means when the goal is structure-and-byte fidelity rather than a cleaner API. Cleaning it up would be a behavioral change (Chapter 2).

Why not implement `SIGHUP` reload and the `truncate` callable? Because the port names its gaps rather than faking them. `SIGHUP` re-exec is not implemented; config changes are picked up by the mtime-staleness reload and `--replace` instead (Appendix E.8). The per-segment `truncate` callable cannot be invoked off a serde `Value`, so the port drops whole segments rather than shortening them (Appendix P). Both are documented divergences, tracked in the open, which is the same honesty the whole port is built on.

Why is `email_imap_alert` a placeholder? Because the honest state of the code is that its TLS IMAP probe is not wired in, so it emits a stub rather than a real unread count (Appendix A.9). It is present as an API-compatible stub so a theme that names it does not error, and the book says so rather than describing a capability the code does not yet have.

Is “1:1 port” marketing? No — it is a claim checked from four independent angles on every CI run: the `//py:NNN` citations (legibility and the density metric), the DONE tier (declared *and* demonstrated correspondence per function), the parity harness (behavior against the live interpreter, which found and fixed eleven real bugs), and the drift gate (declared correspondence against the canonical function set). No one of them alone suffices; together they make the phrase inspectable rather than aspirational (Chapter 10, Appendix F).

What did the project give up for “zero Python runtime”? Dynamic Python-module extensibility (replaced by the two subprocess `exec` hatches, Appendix J) and in-process REPL embedding (replaced by the external client hook, Appendix I.5). Both are niches whose result is reproduced by a different mechanism, and both are named plainly. Against them stands the elimination of the startup tax that motivated the whole project — a trade the design takes deliberately and describes honestly (Chapter 18).

Appendix X – The Divider Math, Worked

Chapter 7 states the divider rule; this appendix runs it across a full line, because the soft-versus-hard decision and the width accounting are where a “looks right” implementation and a byte-faithful one part ways. It is grounded in `_render_length` (`renderer.rs:1075`) and `_render_segments` (`renderer.rs:1237`), whose divider decision is `divider_type = "soft" if compare_bg == seg_bg else "hard"` (`renderer.rs:1186 / 1358`).

X.1 A four-segment left side

Take a left side of four segments with these resolved backgrounds (foregrounds omitted; they do not affect the divider decision):

#	segment	bg
1	mode	darkblue
2	branch	gray0
3	git_status	gray0
4	cwd	gray2

The renderer walks the boundaries left to right, comparing each segment’s background to the *next* non-literal segment’s background:

- 1→2 (darkblue vs gray0): different → **hard** divider. A solid chevron drawn `fg = darkblue` (segment 1’s bg) on `bg = gray0` (segment 2’s bg), producing the arrow that appears to point the blue block into the gray one.
- 2→3 (gray0 vs gray0): same → **soft** divider. A thin separator drawn *within* the gray block, in the muted `divider_highlight` color (the `divider_highlight` group if the segment set `divider_highlight_group`, else the segment’s own highlight). This is why two adjacent same-colored segments read as one block with a hairline between them, not two arrows.
- 3→4 (gray0 vs gray2): different → **hard** divider, `fg = gray0` on `bg = gray2`.

So the same four segments produce two hard chevrons and one soft separator, and the theme author placed *none* of them — they fell out of the coloring. Re-color `branch` to a different background and the 1→2 and 2→3 boundaries both change type automatically, which is the property that makes powerline themes composable: dividers re-flow when colors change.

X.2 The width accounting

`_render_length` must know every cell to make the truncation math (Appendix P) correct, so it sums, per segment: `outer_padding` on the first-left and last-right segments (`theme.outer_padding`), the `divider_spaces` around each divider (`theme.get_spaces()`), and the divider’s own display width from `compute_divider_widths` (which ran `strwidth` over each divider glyph once at the top of the render). A soft divider and a hard divider can have different widths — a multi-character hard chevron versus a single-character soft bar — so the accounting looks up the *specific* width for the boundary’s chosen type. The README lists `outer_padding`, `spaces`, and the multi-char/empty/single-char divider cases among the byte-identical behaviors precisely because this accounting has to match: a prompt that padded one cell differently would render off by

one and stop being a drop-in. The unglamorous sum of pads and divider widths is as much a fidelity surface as the colors are.

Appendix Y – Upstream Lineage and the Vendor Fixture

A faithful port is faithful to a *specific* upstream at a *specific* version, and this appendix pins that relationship so the parity claim has a definite referent. It is grounded in `src/ported/version.rs`, the `Cargo.toml` version, and the `vendor/` tree.

Y.1 The two version numbers

`powerliners` carries two distinct versions and it is worth not conflating them. The **product version** is the crate's own, `0.2.18` at the time of writing (`Cargo.toml`), bumped on each release of the port. The **upstream version** is the `powerline-status` version the port reproduces, `__version__ = "2.8.4"` (`src/ported/version.rs:17`), which `get_version` exposes exactly as upstream's `version.py` does — the port even reproduces the optional `b<count>` git-revision suffix and the fallback-with-`traceback` behavior when git is unavailable. So a `powerline-config` that prints a version prints upstream's `2.8.4`, because that is the version of the tool it is from a config's point of view, while the crate that shipped it is `0.2.18`.

Y.2 The vendored parity fixture

`vendor/powerline` is a checked-in copy of the upstream `powerline-status` source, and it exists for exactly one reason: the parity harness (Appendix F.1) runs the *live* Python interpreter on it. A golden-file snapshot could go stale — upstream changes, the snapshot does not, and the test now asserts old behavior — but running the real interpreter on the vendored source means the harness always compares against the actual upstream code the port claims to reproduce. This is more expensive (the tests need Python and the vendored tree, which is why `vendor/` is excluded from the *published* crate but kept in the repo, `Cargo.toml`'s `exclude` list) but it is the only way to make “byte-identical to upstream” a claim that stays true as the referent is pinned. The `vendor/powerline/client/powerline.c` in that tree is also the source the wire protocol is copied from, line-referenced in the daemon's comments (`sh:NN` citations), so the C client and the Rust client are provably the same protocol.

Y.3 What “tracking upstream” commits the port to

Pinning a version is also a maintenance contract. When upstream releases past `2.8.4`, the port's `// py:NNN` citations point maintainers at exactly which Rust lines correspond to the changed Python lines, so the port can be updated surgically rather than re-read wholesale; the drift gate (Appendix F.4) flags any ported function whose name no longer matches the canonical list; and the parity harness, run against the updated `vendor/` tree, immediately surfaces any behavior that moved. The lineage is not a one-time act of copying — it is a live correspondence, versioned on both sides, that the four verification mechanisms keep honest as the upstream moves.

Appendix Z — Man-Page Cross-Reference

This book is the continuous account; the man pages are the fast-fact reference, and the two are complementary. This appendix maps them so a reader knows which to reach for. `powerliners` ships five man pages, one per binary, and each corresponds to a region of this book.

- `powerline(1)` — the client. The extension/side arguments and `--socket`. Book: Chapter 4 (the client and its fallback), Appendix I (the bindings that invoke it), Appendix L.1.
- `powerline-daemon(1)` — the server. The five flags, the socket paths, the lifecycle. Book: Chapter 4, Appendix E (the full protocol and lifecycle), Appendix L.3.
- `powerline-render(1)` — the direct renderer. The full argparse surface (`ext`, `side`, `-r/-w/--last-exit-code/--last-pipe-status/--jobnum/-c/-t/-p/-R`). Book: Chapter 7 (the render loop), Appendix D (the encoders), Appendix L.2.
- `powerline-config(1)` — setup and dispatch. The `tmux/shell/vim` subcommands. Book: Chapter 8 (the bindings), Appendix I, Appendix L.4, Appendix V (the `tmux` confs).
- `powerline-lint(1)` — the validator. The check pipeline and its flags. Book: Chapter 9 (fidelity in failure), Appendix L.5.

For the config *format* rather than the CLI, the comprehensive man page (the `powerline(1)` family's configuration section) is the enumerated reference; this book's Chapter 6 and Appendices B and C are the explained version. The rule of thumb: reach for a man page when you need one flag, one path, or one key fast; reach for this book when you want to understand *why* the flag, the path, or the key is shaped the way it is. The man pages tell you what `powerliners` does; this book tells you what it is.

Appendix AA — Reading the Bundled Shell Theme

The best way to consolidate the whole model is to read a real config file with it, so this appendix walks the bundled `themes/shell/default.json` — the theme a fresh install renders — line by line. It is the exact file under `src/ported/config_files/themes/shell/`.

AA.1 The file

```
{ "segments": {
  "left": [
    { "function": "powerline.segments.shell.mode" },
    { "function": "powerline.segments.common.net.hostname", "priority": 10 },
    { "function": "powerline.segments.common.env.user", "priority": 30 },
    { "function": "powerline.segments.common.env.virtualenv", "priority": 50 },
    { "function": "powerline.segments.shell.cwd", "priority": 10 },
    { "function": "powerline.segments.shell.jobnum", "priority": 20 }
  ],
  "right": [
    { "function": "powerline.segments.shell.last_pipe_status", "priority": 10 },
    { "function": "powerline.segments.common.vcs.stash", "priority": 50 },
    { "function": "powerline.segments.common.vcs.branch", "priority": 40 }
  ]
}
```

AA.2 The left side, read

Six segments, each a `function` segment (the default `type`) named by its fully-qualified dotted path, which the three-tier `ADAPTERS` lookup (Chapter 3) resolves. `mode` carries no priority, so it is *never* dropped on a narrow prompt — the mode indicator is deemed essential. `hostname` (priority 10), `user` (30), `virtualenv` (50), `cwd` (10), and `jobnum` (20) all carry priorities, so on a narrow terminal they drop in descending-priority order: `virtualenv` first (50), then `user` (30), then `jobnum` (20), and the two priority-10 segments (`hostname`, `cwd`) last — a sensible sacrifice order that keeps the working directory and host visible longest while shedding the decorative context first. Each segment requests its own highlight group (`hostname` asks for `["hostname"]`, `user` for `["superuser", "user"]`, and so on), and several return `None` in the common case — `virtualenv` when no environment is active, `jobnum` when there are no background jobs — so a plain interactive shell collapses this six-segment left side to just the mode, host, and cwd.

AA.3 The right side, read

Three segments. `last_pipe_status` (priority 10) is the most important — it shows a failed pipeline stage even when the last stage succeeded — and it reads `args.last_pipe_status` off the `segment_info` the binding forwarded, rendering `None` (nothing) when the whole pipeline succeeded. `branch` (40) and `stash` (50) are the VCS pair: `branch` shows the git/hg/bzr branch and returns `None` outside a repo, `stash` shows the stash count and returns `None` at zero stashes. So in a non-repo directory after a successful command, the

entire right side is three segments all returning `None` — the right prompt is empty, which is exactly right, and cost almost nothing because three `None`-returning adapters is close to free (Chapter 3’s graceful-absence convention).

AA.4 What the theme does not say

Read what is *absent* as much as what is present. The theme names no colors — those come from the paired `colorschemes/shell/default.json`, which maps each segment’s highlight group to a `{fg, bg, attrs}` (or an alias), so swapping the colorscheme restyles this exact theme without touching it. It names no dividers here — the shell theme inherits divider glyphs from the cascade — and it places no dividers between segments, because the hard/soft decision falls out of the resolved colors at render time (Appendix X). It sets no `args` on most segments, taking each adapter’s defaults (Appendix A). This is the whole compatibility argument made concrete in one file: nine segment references and their priorities, and everything else — the colors, the encoding, the dividers, the layout — is supplied by the colorscheme, the renderer, and the adapters, each a faithful port. A user who wrote this file years ago for Python powerline gets the identical prompt from powerliners, in under a millisecond, having changed nothing. That is the book’s thesis, resting on one real theme: same prompt, no Python, sub-millisecond, proven.

Appendix AB — Common Pitfalls and Their Fixes

The troubleshooting table of Appendix I.6 covers the binding-level symptoms; this appendix covers the subtler ones — the behaviors that are *correct* per the port's design but surprise a user who does not know the design. Each is a place a chapter explains in full; here they are collected as a debugging checklist.

AB.1 A custom `exec` segment renders in the wrong color

The symptom: an `exec` segment shows its contents but ignores the `highlight_groups` you set in the theme. This is the documented `gen_segment_getter` divergence (Chapter 3, Appendix J.4): a function segment uses its resolved `function_name` as its *only* highlight group, ignoring the theme override. The fix is to name the colorscheme group after the function — "`exec`" for Pattern A, or the dotted path's trailing component for Pattern B — or to emit a JSON-array stdout that carries explicit `highlight_groups`. It is not a bug; it is a known port limitation with two clean workarounds.

AB.2 A wide-glyph theme is one cell off

The symptom: a statusline packed with CJK text or wide emoji truncates or aligns a cell short. This is the `strwidth` gap (Chapter 7, Appendix D.1): the port counts Unicode scalar values, not display cells, so a two-cell ideograph measures as one. The fix, until wide-character width lands, is to avoid relying on cell-exact alignment of wide glyphs in a width-constrained line; the overwhelming common case (ASCII, powerline glyphs, Latin text) is exact. It is a bounded, documented divergence, not a silent one.

AB.3 Everything after one segment is bold (or the wrong color)

The symptom, on tmux: a bold segment leaves every following segment bold. If you see this with the shipped renderer it is a colorscheme problem, not a renderer one — the renderer always emits the explicit `nobold`/`noitalics`/`nounder_score` off-forms (Appendix D.2) precisely to prevent it. Check that a hand-written custom `exec` segment emitting raw tmux markup is not itself setting `bold` without a matching `nobold`; the port's own segments cannot cause this, but a JSON-array `exec` script that hand-rolls markup can.

AB.4 A live-data segment shows a different number than Python did

The symptom: `cpu_load_percent` (or `network_load`, or `mem_usage`) reads a slightly different value than the Python powerline did. This is expected and is one of the three documented divergences (Chapter 9, Appendix Q.4): the port samples via OS probes on demand, Python samples via `psutil` on a different cadence, and two samplers at two instants read two numbers. The *markup* is byte-identical; only the sampled number

differs, and only by a sampling window. If the number matters exactly, it was never going to be reproducible — it is a live measurement.

AB.5 A network segment stalls the prompt

The symptom: the prompt occasionally hangs for a moment. Almost always this is an `external_ip`, `weather`, or `email_imap_alert` segment on the interactive path: the port samples on demand rather than in a background thread (Chapter 5), so a network round-trip lands on the render that triggers it. The fix is placement, not configuration — move the expensive samplers to the timer-refreshed statusline and keep the per-keystroke prompt to cheap segments (identity, time, git). Appendix A.14’s cost gradient is the guide.

AB.6 The daemon “isn’t being used” after a swap

The symptom: you swapped in the Rust daemon but the prompt feels the same speed as before. Confirm which daemon is actually serving: `ps -p $(cat /tmp/powerline-ipc-$(UID).pid)` on macOS/BSD, or `ss -x | grep powerline-ipc` on Linux (the abstract socket has no file to `ls`). If the client cannot reach the socket it silently `execvps powerline-render` (Appendix E.9) — a *working* but cold ~30 ms render — so a prompt that renders correctly but slowly is the tell that the daemon is down and the client is falling back. Restart it with `powerline-daemon --replace -q`.

Appendix AC – Quick-Reference Tables

The appendices above are the prose reference; this one is the lookup reference — three tables that collapse the whole book to something scannable. Everything here indexes material grounded earlier; this is the at-a-glance version.

AC.1 The segment quick-reference

Every built-in segment, its registry key (short form), its cost class, and where its full entry lives (Appendix A). Cost class: **cheap** (in-process / clock, safe on the keystroke path), **probe** (a subprocess or `/proc` read, sub-ms to a few ms), **network** (a network or authenticated call — statusline only).

Segment	key	cost	full entry
CPU load	<code>cpu_load_percent</code>	probe	A.1
System load	<code>system_load</code>	cheap	A.1
Uptime	<code>uptime</code>	cheap	A.1
Memory / swap	<code>mem_usage / mem_swap (+_percent)</code>	probe	A.2
Disk usage	<code>disk_usage (+_percent)</code>	probe	A.3
Disk I/O	<code>disk_io</code>	probe	A.3
GPU load / VRAM	<code>gpu_usage_percent / gpu_vram</code>	probe	A.3
Thermal	<code>thermal</code>	probe	A.4
Process count	<code>process_count</code>	probe	A.4
Git (rich)	<code>git_status</code>	probe	A.5
Branch / stash	<code>branch / stash</code>	probe	A.5
Job count	<code>jobnum</code>	cheap	A.6
Exit status	<code>last_status / last_pipe_status</code>	cheap	A.6
Cwd	<code>cwd</code>	cheap	A.7
User	<code>user</code>	cheap	A.7
Env var	<code>environment</code>	cheap	A.7
Virtualenv	<code>virtualenv</code>	cheap	A.7
Hostname	<code>hostname</code>	cheap	A.7
Internal IP	<code>internal_ip</code>	probe	A.7
External IP	<code>external_ip</code>	network	A.7
Network load	<code>network_load</code>	probe	A.7
Date / time	<code>date / fuzzy_time</code>	cheap	A.8
Battery	<code>battery</code>	probe	A.8
Players	<code>mpd/spotify/cmus/itunes/...</code>	probe	A.9
Mail	<code>email_imap_alert</code> (stub)	network	A.9
Weather	<code>weather</code>	network	A.10
Kubernetes	<code>kubecontext</code>	probe	A.11
Docker	<code>containers</code>	probe	A.11
GitHub CI	<code>ci_status</code>	network	A.11
AWS / GCP	<code>aws.context / gcp.context</code>	cheap	A.11
Cache stats	<code>fusevm.jit_cache / *.rkyv_cache</code>	cheap	A.12

Segment	key	cost	full entry
Tool versions	<code>zshrs/stryke/awkrs.version</code>	probe	A.12
Custom	<code>exec</code> (+ dotted-path dispatch)	probe	A.13

AC.2 The render-pipeline stages

The full path from wire request to bytes, with the function that owns each stage and the chapter that explains it. This is the spine the whole book hangs on.

Stage	Function / location	Chapter
Client marshals request	<code>powerline.rs:88-130</code>	4 / E
Daemon reads to <code>\0\0</code>	<code>do_read, powerline_daemon.rs:296</code>	4 / E
Parse wire request	<code>parse_args, :249</code>	4 / E
Select warm orchestrator	<code>PowerlineKey, :117</code>	5 / E
Locate config files	<code>_find_config_files, mod.rs:104</code>	6 / B
Load + merge config	<code>load_json_config + mergedicts</code>	6 / B
Build colorscheme	<code>Colorscheme::new, colorscheme.rs:109</code>	6 / C
Resolve segment getter	<code>gen_segment_getter, segment.rs:934</code>	2
Run adapter	<code>process_segment, segment.rs:692</code>	2 / A
Resolve colors	<code>get_highlighting, colorscheme.rs:292</code>	6 / C
Fan out lists	<code>process_segment_lister, segment.rs:578</code>	2 / K
Measure + truncate	<code>do_render / _render_length, renderer.rs:792/1075</code>	7 / D / P
Decide dividers	<code>bg-compare, renderer.rs:1186/1358</code>	7 / X
Encode to target	<code>per-renderer hlstyle/hl</code>	7 / D
Reply / fallback	<code>socket write / execvp powerline-render</code>	4 / E

AC.3 The source-file map

Where to look in the tree for each subsystem, so a claim in this book can be traced to code.

Subsystem	Path
Segment model	<code>src/ported/segment.rs</code>
Adapter registry	<code>src/bin/shared/render_runtime.rs</code>
Ported segments	<code>src/ported/segments/</code>
Extension segments	<code>src/extensions/</code>
Colorscheme	<code>src/ported/colorscheme.rs</code>
Config cascade	<code>src/ported/mod.rs,</code> <code>src/ported/lib/{dict,config}.rs</code>
Theme grammar	<code>src/ported/theme.rs</code>
Render core	<code>src/ported/renderer.rs</code>
Renderers	<code>src/ported/renderers/</code>
Daemon	<code>src/ported/scripts/powerline_daemon.rs</code>
Client	<code>src/bin/powerline.rs</code>
Command binaries	<code>src/ported/commands/</code> , <code>src/bin/*.rs</code>
Lint	<code>src/ported/lint/</code>
Listers	<code>src/ported/listers/</code>
Bundled config	<code>src/ported/config_files/</code> , <code>src/extensions/bundled_config.rs</code>
Parity fixture	<code>vendor/powerline/</code>
Tests	<code>tests/</code> (integration/parity), in-module <code>#[cfg(test)]</code> (unit)

Read this book front to back for the *why*; use these three tables and the man pages for the *what* and the *where*. Between them, every claim about powerliners is either explained in the prose, indexed in a table, or pinned by a test — which is the whole point of a book that set out to be grounded in the program rather than in a description of it.

Appendix AD – The `mergedicts` Deep-Merge, Worked

Chapter 6 named `mergedicts` as the function that collapses the config cascade into one effective tree, and Appendix N walked the cascade at the file level. This appendix works the merge itself at the value level, because the deep-merge semantics are subtle enough that a theme author who overrides “just one field” and gets a surprise almost always tripped over a rule stated here.

AD.1 The three merge flavors

The port carries three merge functions, matching upstream’s three (`src/ported/lib/dict.rs`), and choosing the wrong mental model for which one runs is the usual source of confusion. `mergedicts(d1, d2)` mutates `d1` in place, letting `d2` win on conflict — this is the cascade merge, where a later, higher-priority file overrides an earlier one. `mergedefaults(d1, d2)` is the inverse bias: it fills gaps in `d1` from `d2` without ever overwriting a value `d1` already holds, which is how the bundled defaults supply keys a user config omits without clobbering the keys it sets. `mergedicts_copy(d1, d2)` is the non-mutating variant, returning a fresh tree so the inputs survive for a later render. The cascade uses the first; default-filling uses the second; anywhere the port must merge without scribbling on a cached dict uses the third.

AD.2 The recursion rule, stated exactly

For each key `k` in the incoming dict `d2`, `mergedicts` applies exactly one of three actions, tested in this order (`dict.rs:117`): if `d1[k]` and `d2[k]` are *both* objects, recurse and merge them key-by-key; else if `d2[k]` carries the removal sentinel and removal is enabled, delete `k` from `d1`; else overwrite `d1[k]` with `d2[k]` wholesale. The load-bearing word is *both*: the recursion happens only when both sides are objects. If the base holds an object and the override holds a scalar — or an array — the override *replaces* the object entirely; there is no merge into a scalar and, critically, **no element-wise merge of arrays**. A theme’s `segments.left` is an array, so an override that supplies its own `segments.left` replaces the whole list, it does not splice into it. This is the single most common merge surprise, and it is not a bug: arrays are opaque values to the merge, and only objects recurse.

AD.3 The REMOVE sentinel

The overwrite rule can set or replace a key but cannot, by itself, *delete* an inherited one — and sometimes a user config needs to. Upstream solved this with a REMOVE marker; the port spells it `__powerliners_remove_this_key__` and honors it in the second branch of the recursion. When an override sets a key to that sentinel and removal is enabled, the merge deletes the key from the base rather than overwriting it, so a bundled default that a user does not want is not merely shadowed but excised. This matters because shadowing and deletion differ downstream: a shadowed key still exists (with the override’s value), while a deleted key is gone, and code that iterates the effective dict sees one or zero entries accordingly. The sentinel is the only way for a cascade — which is additive by nature — to be subtractive.

AD.4 A worked merge

Take a bundled `themes/tmux/default.json` fragment as the base `d1`:

```
{ "segments": {
  "left": [ { "function": "hostname" }, { "function": "cwd" } ] },
  "segment_data": {
    "cwd": { "args": { "max_depth": 3, "shorten_home": true } } }
```

and a user `~/.config/powerline/themes/tmux/default.json` fragment as the override `d2`:

```
{ "segment_data": {
  "cwd": { "args": { "max_depth": 5 } },
  "hostname": { "args": { "only_if_ssh": "__powerliners_remove_this_key__" } } }
```

`mergedicts(d1, d2)` walks `d2`'s keys. `segment_data` is an object on both sides, so it recurses. Inside, `cwd` is an object on both sides, so it recurses again: `args` is an object on both sides, recurse once more; `max_depth` exists on both as a scalar, so `5` overwrites `3`; `shorten_home` exists only in the base, so it survives untouched. The net `cwd.args` is `{ "max_depth": 5, "shorten_home": true }` — the override changed one field and left the sibling alone, which is exactly the promise of a deep merge. Under `hostname.args`, `only_if_ssh` carries the sentinel, so if it existed in the base it would be deleted; here it did not, so the branch is a no-op. Crucially, `d2` never mentioned `segments`, so the base's `segments.left` array is untouched and both segments render — but had `d2` supplied its own `segments.left`, that whole array would have replaced the base's two-element list, because arrays do not merge. One override, three rules, and an effective config that differs from the base in precisely the fields the user named.

Appendix AE – The Divider Glyph Cascade and the Fallback Chain

Appendix X worked the divider *math* — when a boundary is hard versus soft, and how the arrow fills. This appendix covers the divider *glyphs*: which characters the renderer actually emits, the four slots they occupy, and the fallback chain that keeps a line legible on a terminal with no powerline font installed.

AE.1 The four divider slots

A powerline line has not one divider glyph but four, because “hard versus soft” crosses with “left side versus right side.” The theme (or the cascade) supplies a `dividers` object with two keys, `left` and `right`, each holding a `hard` and a `soft` glyph — four glyphs in total. The left-side hard divider is the familiar right-pointing chevron that fills with the *previous* segment's background; the right-side hard divider is its mirror, pointing left and filling with the *next* segment's background, so a right-aligned statusline's arrows point back toward the center. The soft dividers are the thin separators drawn between two segments that share a background, one per side. The renderer picks the slot by side and by the hard/soft decision of Appendix X; the theme author supplies the glyphs and never picks the slot.

AE.2 Why the fill direction matters

The hard divider is not merely a glyph — it is a glyph painted in a specific two-color scheme that produces the arrow illusion, and the direction of the fill is what makes the chevron read as “segment A flowing into segment B” rather than a stray character. On the left side, the hard divider is drawn in the *previous* segment's background as its own foreground, over the *next* segment's background, so the solid part of the chevron matches the segment behind it and the arrow appears to extrude forward. On the right side the roles swap. Getting the fill direction wrong does not crash anything — it produces a chevron that points the right way but is colored backward, a subtle wrongness that the parity harness catches because the emitted bytes differ even when the glyph is identical. The port reproduces upstream's fill assignment exactly for this reason.

AE.3 The ASCII fallback

Not every terminal has a patched powerline font, and a line full of missing-glyph boxes is worse than no dividers at all, so powerline ships — and powerliners carries — an ASCII fallback theme in which the hard divider is a plain `>` (or `<` on the right) and the soft divider is a `|`. This is not automatic detection; it is a *configuration* choice, selected by pointing the cascade at the ASCII divider set (upstream's `powerline_config` distinguishes the unicode and ascii dividers, and the bundled config files carry both). The consequence for a migrating user is reassuring: a config that already selected ASCII dividers under Python selects them identically under powerliners, because the divider glyphs are ordinary config values resolved through the same cascade as everything else, not a hardcoded renderer constant.

AE.4 Spaces, padding, and the glyph's neighbors

A divider never touches the segment text directly; there is padding between them, and the padding is part of what a faithful port must reproduce. Each segment carries a **before** and **after** string and a left/right padding count, and the renderer lays out **before** + padding + contents + padding + **after** + divider for each segment in turn. The soft divider, in particular, is drawn *inside* a shared-background run, so it sits between two segments' padding rather than at a color boundary, which is why a soft divider is a thin muted separator and a hard divider is a full-height arrow: one lives inside a color region and one lives at its edge. The width these paddings and glyphs consume is exactly what the truncation pass (Appendix P) must account for when it decides whether a line fits, which is why the divider machinery and the width machinery are two halves of one problem.

Appendix AF – The Mode Machine, End to End

Chapter 6 walked how a colorscheme’s `mode_translations` recolors a line for a given mode, and Appendix AA noted the `mode` segment on the shell theme’s left. This appendix connects the two ends: where a mode *comes from*, how it rides the wire into the render, and how it fans out into both a segment and a colorscheme override.

AF.1 What a mode is

A “mode” in powerline is a short string naming the state of the program the prompt belongs to. In vim it is the editor mode — `n` for normal, `i` for insert, `v/V/^V` for the visual family, `R` for replace, `c` for command-line — read from vim’s own `mode()` at statusline-evaluation time. In a shell the concept is thinner: the shell reports a mode only where it has one to report (a vi-mode zsh line editor distinguishes insert from command), and otherwise the mode defaults to a single neutral value. tmux has no editor modes, so its statusline renders in the default mode throughout. The mode is therefore a vim-centric feature that the other extensions carry trivially, which is why the visible payoff — “the prompt turns green in insert mode” — is almost always a vim story.

AF.2 How the mode reaches the render

The mode is supplied by the binding as part of the render request, not discovered by the renderer. A vim statusline expression evaluates `mode()` and passes the result to the client; a vi-mode shell prompt passes its editor state; the tmux and default cases pass the neutral mode. On the daemon path the mode travels in the request alongside the argv and environment (Chapter 4), so the daemon renders against the caller’s *current* mode rather than any stale notion of its own — the same freshness argument that governs the cwd and environment applies to the mode. This is why a warm daemon can serve a vim that is flipping between normal and insert dozens of times a second: each render carries the mode live, and the daemon simply resolves against whatever it was handed.

AF.3 The two consumers: the segment and the colorscheme

Once inside the render, the mode fans out to two independent consumers, and keeping them distinct clarifies a lot of theme behavior. The first is the `mode segment` (`powerline.segments.shell.mode` and its vim analogue): a function segment whose contents are the mode string, formatted through a `mode_translations` display map that turns `n` into `NORMAL`, `i` into `INSERT`, and so on. This is the visible label. The second is the colorscheme’s `mode_translations` (Chapter 6, Appendix B.5), which recolors *any* group for the current mode — including groups that have nothing to do with the mode segment. These are orthogonal: you can have the recoloring without the label (delete the `mode` segment, keep the colorscheme translations) or the label without the recoloring (keep the segment, define no translations). A theme that turns green in insert mode but shows no `NORMAL/INSERT` text is doing the second without the first, and a theme that shows the text on an unchanging background is doing the first without the second.

AF.4 Why `mode` is never truncated

Appendix AA noted a detail worth its own line here: in the bundled shell theme the `mode` segment carries *no* priority, which means the truncation pass (Appendix P) never drops it, no matter how narrow the terminal. This is deliberate and it is a correctness choice, not an aesthetic one. The mode indicator tells you whether your next keystroke is a command or literal text; a prompt that silently dropped the mode label on a narrow window to save two cells would be actively dangerous in a vi-mode editor, because you would lose the one piece of state that tells you what typing will do. So the mode is the segment most protected from truncation — priority-less, and therefore undroppable — which is the truncation algorithm being used to encode a judgment about which information a user cannot afford to lose.

Appendix AG — Signal Handling and Async-Signal-Safety in the Daemon

Chapter 4 named the daemon's `SIGTERM` and `SIGINT` handling in passing. A long-lived server's signal discipline is subtle enough — and easy enough to get subtly wrong — that it earns a full accounting, because the difference between a correct and an almost-correct signal handler is the difference between a daemon that dies cleanly and one that corrupts its own shutdown.

AG.1 The two termination signals

The daemon must exit cleanly on two signals and it treats them differently in intent though similarly in effect. `SIGTERM` is the polite kill — what `powerline-daemon --replace` sends the old daemon, what a service manager sends on stop, what a `kill` with no signal sends. The port installs `sigterm_handler` (`powerline_daemon.rs:1069`), which unlinks the pidfile and calls `_exit` so the process leaves no stale lock and no half-flushed state. `SIGINT` is the terminal interrupt — Ctrl-C on a `--foreground` daemon — and the port maps it to a clean exit with status 0, mirroring upstream's `KeyboardInterrupt` → `SystemExit(0)`: an interactive interrupt of a foreground daemon is a normal, expected way to stop it, not an error, so it exits 0 rather than with a signal-death status.

AG.2 Why the handler must be async-signal-safe

A signal handler runs at an arbitrary point in the program — potentially in the middle of a `malloc`, a `printf`, or a `HashMap` insert — and the set of functions it may legally call is tiny: only the async-signal-safe ones, which excludes essentially all allocation, all buffered I/O, and most of the standard library. A handler that calls `println!`, formats a string, or frees memory can deadlock or corrupt the heap if the signal arrived while the main thread held an allocator lock. This is why `sigterm_handler` does the two narrowest possible things — an `unlink(2)` on a fixed path and an `_exit(2)` — and nothing else. `unlink` and `_exit` are both on the async-signal-safe list; anything richer would be a latent crash that fires only under the precise timing of a signal landing inside an unsafe critical section, which is exactly the kind of bug that passes every test and fails in production once a year.

AG.3 `_exit` versus `exit`

The choice of `_exit` over the ordinary `exit` is deliberate and is the subtle part. `exit(3)` runs `atexit` handlers, flushes and closes buffered streams, and runs C++/Rust destructors registered for process teardown — all of which allocate, take locks, and touch buffered state, and none of which is async-signal-safe. `_exit(2)` does none of that: it is the raw syscall that terminates the process immediately, letting the kernel reclaim memory and close descriptors without running any userspace teardown. In a signal handler that is precisely what you want — the process is ending anyway, and the only teardown that *must* happen (removing the pidfile) is done explicitly by hand before the `_exit`. Running the full `exit` path from a signal handler would reintroduce every hazard the handler is trying to avoid, so the port terminates the hard way, on purpose.

AG.4 The pidfile and the RAII guard, reconciled

There is an apparent tension worth resolving: Chapter 4 said the pidfile is released by a `PidLock` RAII guard on `Drop`, and this appendix says the signal handler unlinks it by hand. Both are true, and they cover different exit paths. On a *normal* exit — the EOF sentinel, a clean end of the accept loop — the `PidLock` value is dropped as its scope unwinds, and its `Drop` impl removes the pidfile; this is the ordinary Rust path and it runs safely because it is not in a signal context. On a *signal* exit the stack is not unwound — `_exit` skips destructors entirely — so `Drop` never runs, and the handler must remove the pidfile itself before exiting. The two mechanisms are complementary: RAII for the paths where destructors run, an explicit `unlink` for the path where they cannot. A daemon that relied on RAII alone would leak a stale pidfile on every `SIGTERM`; one that relied on the handler alone would leak on every clean exit. The port carries both, which is why a stale pidfile from a crashed daemon is the *only* case that leaves one behind — and Chapter 4's lock-the-pidfile mechanism is exactly what keeps that case from blocking a restart.

Appendix AH — The poll Loop and What Changed from select.select

Chapter 4 noted that the daemon’s accept loop uses `libc::poll` where upstream used `select.select`, and called the substitution “documented and behavior-neutral.” Behavior-neutral is a claim, and a claim about an event loop deserves to be shown rather than asserted, because `select` and `poll` differ in ways that a careless port would leak into observable behavior.

AH.1 What the loop must do

The accept loop (`main_loop`, `powerline_daemon.rs:854`) has one job: wait until *something* is ready — either the listening socket has a new connection to accept, or one of the live client connections has bytes to read (or has hung up) — then service exactly the ready descriptors and go back to waiting. It must never busy-spin, never block on one slow client while others wait, and never miss a hangup. `select` and `poll` are two kernel interfaces for the same “tell me which of these descriptors is ready” question, and either can implement this loop; the port chose `poll` and the question is whether the choice changed anything a client can see.

AH.2 The three differences that matter

`select` and `poll` differ in three ways, and the port neutralizes each. First, `select` uses a fixed-size bitset (`fd_set`) capped at `FD_SETMAX` (typically 1024), so a daemon with many connections could in principle exceed it; `poll` takes a vector of `pollfd` with no such cap, so the port is strictly *more* robust here, not less — a difference that can only remove a failure mode, not add one. Second, `select` reports readiness as three separate sets (read, write, error) while `poll` reports per-descriptor event bitmasks; the port asks for `POLLIN` and reads back `POLLIN | POLLHUP | POLLERR`, which brings us to the third and subtlest difference. `select` folds a peer hangup into the read set — a hung-up socket reads as “readable,” and the subsequent `read` returns 0 (EOF); `poll` reports the hangup separately as `POLLHUP`. To stay behavior-identical, the port explicitly folds `POLLHUP` back into “readable” (Chapter 4), so a peer that closes its end is serviced exactly as `select` would have serviced it: treated as readable, read, and found to be at EOF. Without that fold, a hung-up connection would sit ignored in the poll set forever, a busy-loop or a leak depending on the surrounding code — the port avoids both by making `poll` report the way `select` did.

AH.3 EINTR and the retry

A signal can interrupt a blocking `poll` or `accept`, causing it to return `EINTR` rather than a result. Naively this looks like an error and a naive loop would either die or drop the event; the correct behavior is to retry the interrupted call, because `EINTR` means “a signal arrived, nothing went wrong, ask again.” The port routes its blocking calls through `eintr_retry_call` (Chapter 4), which retries on `EINTR` and propagates any other error, so a `SIGCHLD` from a reaped subprocess or a `SIGWINCH` from a foreground terminal resize does not perturb the loop. This mirrors upstream’s implicit behavior — Python’s socket calls retry `EINTR` under the hood on modern versions — and making it explicit in the port is part of why the substitution is behavior-neutral rather than merely similar: every place `select` would have transparently resumed, `poll` explicitly resumes.

AH.4 The read timeout

One more loop detail governs liveness: `do_read` does not block forever waiting for a request to finish arriving. It loops reading until the buffer `ends_with(b"\0\0")` or a two-second timeout elapses (Chapter 4), so a client that connects, sends a partial request, and stalls cannot pin a connection open indefinitely. Two seconds is generous for a local UNIX-socket write of a few kilobytes — the real request completes in microseconds — so the timeout never fires on a healthy client and fires promptly on a broken one. This is the defensive counterpart to the accept loop's fairness: the loop will not block on one slow client because `poll` multiplexes, and a single request will not block its own connection forever because `do_read` has a deadline. Together they are what let one single-threaded daemon serve every shell, tmux, and vim on the machine without a stall in one starving the rest.

Appendix AI — The `powerline-lint` Check Catalog

`powerline-lint` earns mentions throughout the book — Chapter 4 lists it among the five binaries, Chapter 9 leans on it for fidelity, Appendix L details it as a binary — but the checks it actually runs are never enumerated in one place. This appendix is that enumeration, because a linter is only as useful as the specific mistakes it catches, and knowing the catalog is how a theme author reads a lint failure.

AI.1 What the linter is for

The linter exists because a powerline config is a graph of cross-references — themes name segments, segments name highlight groups, colorschemes name colors, formatting strings name fields — and a broken reference does not fail at load time in an obvious way; it fails at *render* time, on a live prompt, often as a missing segment or a loud color error in the middle of your statusline. The linter's whole purpose is to move those failures from render time to a deliberate check you run before deploying a config (Appendix M's migration runbook runs it first, on purpose). It parses the config with a location-tracking JSON loader — the `markedjson` loader (Appendix L) — so every complaint can point at the exact file, line, and column of the offending value, not just report that *something* is wrong.

AI.2 The reference checks

The core of the catalog is reference validation, and there are four kinds. **Unknown segment names:** a theme that names a `function` the `ADAPTERS` registry does not contain is flagged, because that segment would silently render nothing (or error) live. **Missing color references:** a colorscheme group that names a color absent from `colors.json` is flagged, because it would fail the `get_highlighting` lookup at render time (Chapter 6). **Missing highlight groups:** a theme whose segments request groups that no active colorscheme defines is flagged — the loud “Highlighting groups not found” error of Chapter 6, caught statically instead of live. **Dangling aliases:** a colorscheme group aliased to another group that does not exist is flagged, because the alias chase (Appendix AF's cousin in Chapter 6) would dead-end. Each of these is a cross-reference the render path assumes holds, lifted into a check that runs before the render path ever runs.

AI.3 The formatting checks

The second family validates the `before`, `after`, and `format` strings that segments carry, because a malformed format string is a class of bug that is invisible until the exact segment renders with the exact data that trips it. The linter checks that a segment's format placeholders name fields the segment actually emits — a `{branch}` in a segment that produces no `branch` key is flagged — and that the `before/after` decorations are well-formed strings rather than, say, an accidental object. These are the checks that catch a copied-and-edited theme where a format string outlived the segment it was written for.

AI.4 Reading a lint result

A lint run either prints nothing and exits 0 — the config is clean — or prints one diagnostic per problem, each anchored to a file, line, and column by the markedjson loader, and exits non-zero. The non-zero exit is what makes the linter scriptable: the migration runbook (Appendix M) runs `powerline-lint` as a gate before swapping daemons, so a config with a dangling reference is caught by the exit code before it can become a broken prompt. The discipline the linter enforces is the same one the whole port enforces on itself — make the failure loud, early, and located, rather than silent, late, and mysterious — applied to the user's config instead of to the port's own source.

Appendix AJ – The `segment_info` Environment Contract

Adapters read the world through a single map, `segment_info`, and Appendix A stated its three main key families in one paragraph. This appendix is the full contract: every key an adapter may read, who populates it, and why the indirection exists at all. An adapter that reads anything *not* in this map is reading the daemon's stale process state rather than the caller's, which is the bug the contract exists to prevent.

AJ.1 Why adapters do not read the process environment directly

The obvious way to write a `hostname` segment is to call `gethostname`, and the obvious way to write a `cwd` segment is to call `getcwd`. Both are wrong in a daemon. The daemon is a long-lived process with its own hostname (fine, it does not move) but its own working directory, its own environment, and its own notion of the controlling terminal — all of which are stale relative to the shell that requested the render. An adapter that read `getcwd()` directly would report the daemon's directory (`/`, after the `chdir` of daemonization) on every render, for every shell, regardless of where the user actually is. So adapters do *not* read the process; they read `segment_info`, which the render path populates from the *caller's* forwarded argv, cwd, and environment (Chapter 4). The contract is the mechanism that makes a shared daemon render faithfully for a caller it does not share state with.

AJ.2 The key families

`segment_info` carries a small, fixed set of keys, grouped by source. `environ` is the caller's full environment map, demarshalled from the wire request — every `KEY=VAL` the client sent — and it is what `environment`, `virtualenv`, `kubecontext` (via `$KUBECONFIG`), and every other environment-reading segment consults. `getcwd` is the caller's working directory, the `cwd` field of the wire request (with the `$PWD`-then-`/` fallback of Chapter 4), and it anchors `cwd`, `branch`, and every path-relative segment. `args` holds the parsed client arguments that segments need: `args.jobnum` (the background-job count the shell passed), `args.last_exit_code` (the exit status of the last command), and `args.last_pipe_status` (the per-stage exit codes of the last pipeline). `mode` is the current mode (Appendix AF). Each of these is data the *caller* had and the daemon did not, forwarded explicitly precisely because the daemon could not have known it.

AJ.3 The shell-status keys in detail

The three `args` status keys deserve their own note because they are the ones a shell binding must work to supply. A shell knows its own last exit code (`$?`), its own job count (`jobs` builtin), and, in a pipeline-aware shell, its own per-stage pipe status (`${pipestatus[@]}` in zsh, `${PIPESTATUS[@]}` in bash). The binding captures these at prompt time and passes them to the client as `--last-exit-code`, `--jobnum`, and `--last-pipe-status` arguments (Appendix I's cookbook shows the exact invocations), which the daemon parses into `args`. This is why the order of operations in a shell prompt hook matters: the exit code must be captured *first*, before any other command in the hook overwrites `$?`, or the segment reports the hook's exit code instead

of the user's command's. The contract's `args` keys are only as correct as the binding's capture discipline, which is why Appendix I is careful about hook ordering.

AJ.4 The patched environment of a lister

There is one place `segment_info` is not merely forwarded but *extended*, and it is the lister fan-out of Chapter 2. When `process_segment_lister` renders one subsegment per workspace or window, it folds that item's data into a *copy* of `segment_info` before processing the subsegment — so a subsegment can read the specific workspace's name or the specific window's index through the same `segment_info` interface it uses for everything else. The contract is uniform: whether a segment is a top-level prompt segment reading the caller's cwd or a lister subsegment reading a workspace's title, it reads through `segment_info`, and the render path is responsible for populating that map with the right data for the segment's context. The adapter never knows whether its `segment_info` came straight off the wire or was patched by a lister; it just reads its keys, which is exactly the decoupling that lets one adapter contract serve both cases.

Appendix AK — Building, Installing, and Cross-Compiling the Suite

Chapter 11 covered packaging from the user’s side — `cargo install`, the five binaries, the deployment story. This appendix is the builder’s side: how the suite compiles, what it links against, and what it takes to produce the static artifact that makes the deployment story true. The claims of Chapter 11 — “a single static binary, no runtime to install” — rest on build choices worth naming.

AK.1 One crate, five binaries

The suite is one Cargo crate with five `[[bin]]` targets (Chapter 4), which is why `cargo build` produces all five in one pass and they share one compiled copy of the library code — the segment model, the colorscheme resolver, the renderer, the config cascade — rather than five copies. The shared `render_runtime.rs` (Chapter 3) is compiled once and linked into both the daemon and the direct renderer, so the daemon and `powerline-render` cannot drift in how they resolve a segment: they are the same code. This is a build-level guarantee of the parity between the warm and cold paths that Chapter 5 asserts behaviorally — they render identically because, below the transport, they *are* identical.

AK.2 What it links against

The dependency surface is deliberately thin, because every dependency is something a deployment must carry or a static link must absorb. The port leans on `libc` for the syscalls a daemon needs — `fork`, `setsid`, `poll`, `unlink`, the socket calls — and on a small set of pure-Rust crates for JSON, serialization, and the like; it does *not* link a Python interpreter, a UI toolkit, or a heavy async runtime, because none of those is needed for a synchronous, single-threaded, UNIX-socket server. The absence is the point: the fewer shared libraries the binary needs at runtime, the closer it gets to the “copy the binary and run it” deployment of Chapter 11. On most platforms the only runtime dependency is `libc` itself, which every system already has.

AK.3 The static-linking path

The strongest form of the deployment promise is a *fully* static binary — one that needs not even a system `libc`, and runs on any kernel of the right architecture regardless of the distribution’s library versions. On Linux this is the `x86_64-unknown-linux-musl` (or `aarch64-...-musl`) target: `musl libc` links statically cleanly where `glibc` resists it, producing a binary with no dynamic dependencies at all. `cargo build --release --target x86_64-unknown-linux-musl` yields five artifacts that run on a bare container, an old distribution, or a rescue environment with equal ease. On macOS the platform `libc` is not statically linkable by policy, so the “static” claim is bounded to the system libraries every mac has — which is still a single self-contained binary with no `pip` tree, just not a zero-dynamic-dependency one. The port’s thin dependency surface is what makes the `musl` path straightforward: a crate that pulled in C libraries with their own link requirements would fight the static target, and this one does not.

AK.4 Cross-compilation and the release matrix

Because the suite is pure Rust over `libc`, cross-compiling it is the ordinary Rust cross-compile — add the target with `rustup target add`, build with `--target`, and for the musl and cross-arch cases use a cross linker (the `cross` tool or a configured `CC/linker` in `.cargo/config.toml`). There is no per-platform C build step to orchestrate, no interpreter to match to the target, no native extension to rebuild — the things that make cross-compiling a Python or a C-extension project painful are simply absent. A release matrix of `{linux-musl, linux-gnu, macos-arm, macos-x86}` is four `cargo build --target` invocations producing twenty artifacts (five binaries times four targets), each a self-contained native executable. This is the build-side reason the packaging story is short: compilation produces exactly what deployment needs and nothing that deployment must then assemble.

Appendix AL – The Cost of Absence: None-Returning Segments

The book repeatedly notes that a segment can return `None` and that a `None`-returning segment is “nearly free” (Chapter 3’s graceful-absence convention, Appendix AA.3’s empty right side). “Nearly free” is a performance claim and, like the latency budget of Chapter 5, it deserves to be shown rather than asserted, because a prompt full of conditionally-absent segments is the common case and its cost is what determines whether you can afford to *configure* segments you only sometimes see.

AL.1 Why segments return nothing

A large fraction of the segment library is conditional by design: `virtualenv` shows a Python environment only when one is active, `jobnum` shows a count only when there are background jobs, `stash` shows a stash count only when the stash is non-empty, `branch` shows a branch only inside a repository, `last_pipe_status` shows a failure only when the pipeline failed. In every one of these cases the common state is *absence* — most directories are not repos, most shells have no background jobs, most pipelines succeed — so the segment’s most frequent behavior is to render nothing. A prompt is configured with more segments than it usually shows, and the ones it does not show are the ones whose condition is currently false.

AL.2 What “nothing” costs

The cost of a `None`-returning segment is the cost of the check that decided it should return nothing, and the port keeps that check cheap for the common cases precisely because they are common. `jobnum` reads an integer the binding already forwarded in `args` and compares it to zero — no syscall, a branch. `last_pipe_status` reads the forwarded pipe-status array and checks whether any stage is non-zero — no syscall, a scan of a tiny array. `virtualenv` reads an environment variable from the forwarded `environ` — no syscall, a map lookup. These are the cheap-check absences, and a prompt full of them costs, in aggregate, a handful of comparisons: effectively free, which is why the shell theme can carry six left segments and three right segments (Appendix AA) and collapse to three visible ones with no measurable penalty for the six that vanished.

AL.3 The absences that are not free

Not every absence is cheap, and the distinction is exactly the segment-placement decision of Chapter 3. `branch` returning `None` outside a repo is *not* a pure environment check — deciding whether the current directory is inside a git working tree means walking up the directory tree looking for a `.git`, which is a sequence of `stat` calls. It is fast (a few syscalls) but it is not free, and it runs on *every* render whether or not you are in a repo, because the only way to know you are not in a repo is to look. `git_status` is heavier still. The lesson the book draws repeatedly is visible here in miniature: the cheap absences (integer and environment checks) can be configured freely, but the probe absences (filesystem walks, subprocesses) cost their probe every render regardless of outcome, so a prompt tuned for the keystroke path keeps the probe segments on the timer-refreshed statusline (Appendix AB.5) even though they usually render nothing — because “usually renders nothing” and “usually costs nothing” are not the same claim, and the difference is a `stat` loop.

AL.4 Why absence is rendered, not skipped in configuration

A reasonable question is why a user configures a segment that is absent most of the time rather than adding it only when needed — and the answer is that the prompt is static configuration evaluated against dynamic state, so the *configuration* names every segment that could ever appear and the *render* decides which ones do. You cannot “add `branch` only when in a repo” because the config is written once and the repo-ness changes per directory per render; the segment must be configured unconditionally and return `None` when its condition is false. This is the whole reason the `None`-returning convention exists: it is what lets a single static theme describe a prompt whose visible shape changes continuously as you move through directories, start and finish jobs, activate environments, and succeed or fail commands. The absence is not a degenerate case of the segment model; it is the segment model's answer to configuring a prompt for states that come and go.

Appendix AM – The Config Acceptance Envelope

Chapter 10 names four things the verification methodology cannot check: input coverage, the live-sampled boundary, the host's interpretation, and the extensions tree. Each is a limit on a mechanism that is otherwise working. This appendix describes a fifth limit that is different in kind, because it is not a gap in what the mechanisms check but a gap in what they can be *pointed at*.

Every mechanism in the methodology compares two renders of the same configuration. The parity harness feeds a config to the port and to the live interpreter and diffs the bytes. The citation rule pins a function to an upstream line. The tier classifier grades files. The drift gate matches a function inventory. All four presuppose that a configuration exists in both worlds — that the same JSON text produced a config tree on both sides, so that there is something to compare. **A configuration that one parser accepts and the other rejects produces no comparison at all.** There is no diff, because one side has no render. There is no assertion, because an assertion requires two values.

This is the acceptance envelope: the set of byte sequences a given implementation will turn into a working config tree. Two implementations are drop-in compatible on the intersection of their envelopes and are undefined outside it, and nothing in the methodology measures the envelopes' edges. That is worth documenting precisely, because the migration promise of Appendix M — install, verify, swap, and roll back if anything looks wrong — is a promise about renders, and a config outside the intersection fails before there is a render to inspect.

AM.1 Where the envelope has edges

A config file passes through three gates on its way to affecting output, and each has its own acceptance rule.

The first is **lexical**: is this byte sequence JSON at all? Chapter 6 notes that `load_json_config` has tests asserting both the happy parse and the error-on-bad-JSON path, so the port has a defined rejection behavior and exercises it. What the tests establish is that malformed input fails cleanly rather than corrupting state. What they do not establish is that the boundary between malformed and well-formed sits in the same place as the interpreter's, and that boundary is where an envelope divergence would live.

The second is **shape**: the parse succeeded and produced a tree, but is the tree the shape the consumer expects? A theme whose `segments` key holds a string instead of an array is valid JSON and invalid config. This gate is partly the linter's job and partly the consumers', and the two do not have identical coverage — the linter checks what it checks, and a consumer encountering an unexpected shape at render time has its own behavior.

The third is **reference**: the shape is right and the names in it must resolve. This is the gate `powerline-lint` is squarely aimed at, per Appendix A1: unknown segment names, missing color references, malformed formatting strings. It is the best-covered of the three, which is also why it is the least interesting here — a reference failure is a config error rather than an implementation divergence, and both implementations will have one.

The envelope question is almost entirely about the first gate, occasionally about the second, and never about the third.

AM.2 The cases where two JSON parsers can honestly disagree

JSON is a small format with a large number of under-specified corners, and a strict parser and a permissive one can both be defensible. The corners that matter for a config file, in rough order of how likely they are to appear in a real theme:

Duplicate keys in one object. The format does not define which wins, or whether it is an error at all. A theme that names the same segment key twice — easy to produce by copy-pasting a block and editing one copy — is a document whose meaning depends entirely on the parser's policy. This is the most likely corner to be hit accidentally and the one whose divergence would be most confusing, because a last-wins parser and a first-wins parser both render a plausible-looking prompt from the same file. Nothing looks broken; the wrong half of the file is simply in effect.

Non-finite numbers. The JSON grammar has no representation for infinity or not-a-number, and implementations differ on whether to emit and accept them as bare identifiers anyway. A config would only contain one by generation rather than by hand, but config files do get generated.

Number range and precision. JSON numbers are unbounded decimal literals; the values a program stores them in are not. A very large integer, or a decimal with more precision than a binary float can hold, is a document that two implementations may read as two different values. In a config that expresses widths, priorities, and gradient levels, the practical exposure is small — those are small numbers — but a `priority` given as a large integer is the kind of thing that would sit unnoticed until it decided a truncation order.

Escape sequences and lone surrogates. A `\uD800`-style escape with no partner is a sequence JSON permits writing and Unicode does not permit representing. Chapter 21 explains in detail why the two implementations' string models handle undecodable *bytes* differently; this is the same fault line reached through a text escape rather than through the filesystem, and it lands on a config file rather than on a path. It is the rarest corner listed here and the one most likely to produce an outright failure rather than a silent difference.

Leading byte-order marks, trailing bytes, and encoding. A file that begins with a BOM or ends with a stray newline plus junk is not strictly a JSON document, and parsers vary in tolerance. A BOM in particular is the classic way for a config file to acquire an invisible prefix from an editor that meant well.

Comments and trailing commas. Neither is JSON. Both are things people write anyway, because so many JSON-adjacent config formats accept them. The book's config schema is straight JSON, so a comment is a rejection in principle — but “in principle” is exactly the kind of statement an envelope makes concrete, and a reader with a comment in their theme would like to know before they migrate, not after.

AM.3 The two directions are not symmetric

This is the part that determines how much any of the above matters, and it is a genuinely asymmetric analysis.

If the port's parser is **more permissive** than the interpreter's, the failure mode is: a config works on powerliners and fails on Python. For the drop-in promise this is nearly harmless in the forward direction — the user is migrating *to* powerliners, and a config that works is a config that works. It becomes a problem only on rollback. Appendix M's whole value proposition is that you can go back, and going back with a config that the previous implementation cannot parse means going back to a broken prompt at exactly the moment you were trying to reach a known-good state. A permissive parser therefore silently degrades the rollback guarantee for any user who edits their config after migrating, which is every user who migrates and then keeps using the software.

If the port's parser is **stricter**, the failure mode is: a config that has worked for years stops parsing at install time. This is loud, immediate, and attributable — the user installs, the prompt breaks, the cause is one file — and it is a much better failure than the first one even though it looks worse, because it is discovered in the one moment the user is prepared to discover it. It is also the failure that the migration runbook's lint-first step is positioned to catch before the swap.

So the ranking is counter-intuitive and worth stating plainly: **being stricter than upstream is a bad first impression; being more permissive than upstream is a latent trap in the rollback path.** A port choosing between the two should prefer strictness, and a reader auditing their own config should care most about anything in it that is not uncontroversially valid JSON.

AM.4 How to test your own envelope

The methodology cannot check this for you, and the reason is structural rather than a matter of effort — but you can check it for your own tree in one operation, which is the practical resolution.

The migration runbook already has you render the same request both ways and diff the bytes before swapping anything. That procedure tests the intersection of the envelopes on the one config that matters to you: if both sides produce output, both parsed, and the diff answers the fidelity question. A parse divergence shows up as one side producing an error where the other produced markup, which is not a subtle diff. The step people skip is running it again *after* editing the config, and the rollback asymmetry above is the reason it is worth repeating — the config you verified at swap time is not the config you have three months later.

For the specific corners in AM.2, the check is a read rather than a run. Duplicate keys are findable by inspection of any block you copy-pasted. A BOM is findable by looking at the first bytes of every file the cascade resolves. Comments and trailing commas will announce themselves to any strict parser you have lying around. None of this is elaborate, and the value of the appendix is not the procedure but the awareness that the envelope exists as a compatibility surface distinct from every surface the four mechanisms cover.

AM.5 Why this belongs in the honesty ledger

Chapter 10 argues that enumerating blind spots is what makes a fidelity claim falsifiable, and the four it names each have a compensating mechanism. This fifth one has a weaker compensation — the linter covers the reference gate thoroughly and the lexical gate barely, and the migration diff covers only the configs you actually run it on — so the honest statement is narrower than the four in Chapter 10.

The claim powerliners can make about the envelope is: *for configs both implementations accept, the renders are byte-identical on the exercised inputs, and that is what the harness proves.* The claim it cannot make is that the sets of accepted configs are identical, because nothing in the methodology compares them and the comparison would require a fuzzing campaign against two parsers rather than a diff of two renders. Saying so costs nothing and prevents a reader from believing a stronger claim than the evidence supports, which is the same reason every other limit in this book is written down.

Appendix AN – Degenerate Inputs and Boundary Geometry

The worked examples in this book use ordinary inputs: a five-segment line, a thirty-six-cell pane, a theme with a left side and a right side. Ports break at the edges rather than in the middle, because the middle is where the fixtures are. This appendix walks the boundaries of the render pipeline — the empty line, the single segment, the width that cannot fit anything, the color that resolves to nothing — and asks what the documented algorithms do there.

Two of these have answers already stated elsewhere in the book, and they are included because they establish the pattern: a degenerate case with a *defined* answer is a fidelity asset. The falsy-hex fallback of Appendix C.1 is one — a color whose hex value is missing or zero falls back to the cterm index even on a truecolor terminal, which is a defined answer to a degenerate input and is listed among the byte-identical behaviors precisely because it is defined. The empty string at the end of the script-extension list, noted in Chapter 22, is another: it looks like a placeholder and is in fact the entry that lets a bare extensionless filename match. Degenerate inputs having deliberate answers is a sign of a port that read the original rather than reimplementing the happy path.

AN.1 The empty line

A theme can produce no segments at all, in three distinct ways, and they are not interchangeable.

The first is a theme whose segment array for a side is empty. Nothing is configured, nothing resolves, nothing renders. This is the trivial case and the only reason to name it is to contrast it with the second.

The second is a theme with segments, all of which returned nothing. Appendix AL is entirely about this case and establishes that it is the *designed* state rather than an error: a prompt is configured with more segments than it usually shows, and a line where every condition happens to be false is the extreme of an ordinary situation. Appendix AA.3 walks a real instance of it — in a non-repo directory after a successful command, all three of the bundled shell theme's right-side segments return nothing and the right prompt is empty — so this is the common case rather than a corner. The important structural point is that this and the first case must produce identical output, because the renderer's job is to lay out the segments that exist, and after resolution neither line has any.

The third is different in kind: a render that failed. If a colorscheme cannot resolve any of a segment's highlight groups, Appendix C.5 says the resolution errors with an explicit message naming the groups, and Chapter 4 explains that errors travel in band — the reply is the error text, on the same channel the markup would have used. So an empty line and a failed line are distinguishable at the byte level, loudly, and that is the design's intent: Chapter 4 argues that a prompt renderer whose errors are visible in the prompt is one whose errors get fixed. The degenerate case that renders nothing and the degenerate case that renders an error message are the two answers, and the design never produces a silent blank in the failure case.

AN.2 One segment, and the arithmetic of dividers

Dividers exist between things. A line with one segment has no interior boundary, so the whole hard/soft divider mechanism that Appendix X works through in detail has nothing to decide, and the only glyphs are whatever the outer edges call for.

This is worth stating because it makes the divider logic's shape visible. Appendix X's width accounting adds up contents, padding, and dividers; the divider count on a line of n segments is a function of n with an interior term that vanishes at n equals one. Any implementation that computed dividers as "one per segment" rather than "one per interior boundary" would be off by one on every line and would look correct on none of them, so this is not an edge case anybody ships wrong. The interesting version is the one Appendix P creates: truncation removes segments, and the dividers re-flow around the survivors because the drop happens before the boundary comparisons run. A line truncated down to one segment therefore has to arrive at the same divider arrangement as a line that was configured with one segment. That equivalence — truncation output being indistinguishable from equivalent configuration — is the property that makes the truncation loop trustworthy, and it holds precisely because dropping happens first and encoding happens after.

AN.3 A width nothing can fit

Here is the boundary case with the most interesting consequence, and it is derivable directly from the algorithm Appendix P states.

The truncation loop orders the droppable segments by descending priority value, drops them one at a time, and re-measures until the line fits. Appendix P states the crucial exclusion: segments with a null priority are never dropped. Appendix AA notes the same thing about the mode segment, and Appendix AF explains why — a mode indicator that could vanish under narrowness would be worse than useless, so it is left priority-less to pin it.

Combine those two facts. If a line's priority-less segments alone are wider than the available width, **the loop cannot make the line fit**, because it will exhaust the droppable set and still be over budget. Overflow is reachable by construction. This is not a bug and it is not even a design compromise; it is the correct behavior for a mechanism whose contract is "drop what the author marked as droppable." An author who marks nothing droppable has told the renderer that every segment is essential, and the renderer's only options are to obey and overflow or to disobey and drop something the author pinned. Obeying is right.

The practical consequence for a theme author is a rule Appendix P does not quite state: **leave at least one segment droppable if the line has any chance of being wider than its surface**. A prompt whose every segment is priority-less is a prompt that will wrap on a narrow pane, and the wrapping is the author's decision even if they did not know they were making it. The corollary is that the mode segment's priority-lessness is safe specifically because it is short — pinning a wide segment has a much larger overflow floor than pinning a narrow one, and the floor is what determines the narrowest surface the line can survive.

AN.4 No width at all

Distinct from a width too small is a width that was never supplied. Chapter 26 covers the plumbing; the boundary question here is what the loop does when it has no budget.

A truncation loop with no width has no comparison to make and therefore drops nothing. Every configured segment that resolved appears. This is the right default — a renderer that invented a width would be guessing about a surface it cannot see, and guessing wrong in the direction of dropping content is worse than emitting a line that the host wraps — but it means "nothing ever drops" has two possible causes that look identical: a theme whose priorities are all unset, and a binding that never passes a width. Distinguishing them takes one experiment, which is to render directly with an explicitly narrow width and see whether anything drops. If segments drop under an explicit width, the priorities are fine and the binding is the problem. If nothing drops even then, the priorities are the problem. That two-outcome test is the fastest available resolution and it belongs in the debugging repertoire of Chapter 17 alongside the config-versus-render bisection.

AN.5 Degenerate colors

Color has its own boundaries and they produce symptoms that do not look like color problems.

A highlight group whose foreground and background resolve to the same value renders text that is present, correctly sized, correctly divided, and invisible. Every stage of the pipeline is working. The width accounting is right. The dividers meet. The segment simply cannot be read. This is worth naming because the natural diagnosis is “the segment disappeared,” which sends the reader toward the segment adapter and the `None`-returning logic, and the actual cause is two entries in a colorscheme. The distinguishing observation is that an absent segment takes its padding and dividers with it and an invisible one does not, so the line’s *spacing* tells you which you have — an unexplained gap of the right width is an invisible segment, and no gap at all is an absent one.

A gradient with one entry is the other case. Appendix C.4 gives the index computation as the level scaled by one less than the list length, divided by a hundred, and rounded with Python’s banker’s rounding. A single-entry list makes the scaling factor zero, so every level maps to the only entry — a gradient that is a flat color, which is the sensible answer and which the formula produces without any special-casing. That the arithmetic degrades gracefully rather than requiring a guard is a small piece of evidence that the formula was ported rather than reconstructed.

Appendix C.4 also states, explicitly, that the level is *not* bounds-checked, and that this matches upstream. That sentence is doing more work than it appears to. It says the domain of the gradient function is levels from zero to a hundred, that the port declines to extend the domain, and that declining is a fidelity decision rather than an oversight. A segment emitting a level outside that range is outside the defined behavior of *both* implementations, which means it is not a place where the port owes an answer — it is a place where the segment owes an in-range value. That distinction matters for anyone writing a native segment under Chapter 16’s contract: emitting a gradient level is emitting a value into an unguarded computation, and clamping is the segment’s job.

AN.6 Why the boundaries are where ports break

The through-line is that every case above is a place where an implementation must make a decision that no ordinary input exercises, and where two independent implementations would plausibly decide differently. That is the definition of a fidelity risk, and it is why Chapter 10’s citation rule is a *stronger* guarantee than the parity harness for exactly this class of behavior. The harness proves the port matches on the inputs it runs; a fixture set built from realistic themes contains realistic themes, so the empty line, the one-segment line, the unfittable width, and the single-entry gradient are unlikely to be in it. The citation rule covers them anyway: a function whose branches are pinned line-by-line to upstream’s has its degenerate branches ported for the same reason as its ordinary ones, whether or not any fixture ever reaches them.

Chapter 10 makes this argument in general terms when it says coverage of behavior comes from the harness and coverage of code comes from the citations. The boundary cases in this appendix are the concrete population that argument is about — the branches that exist, that matter when they fire, and that no realistic fixture will ever reach.

Appendix A0 – Remote, Nested, and Containerized Sessions

Every deployment this book has described is one machine, one user, one daemon, and a set of hosts on that machine asking it for prompts. Real sessions are messier: shells reached over a network, editors inside multiplexers inside other multiplexers, containers with their own process trees and their own idea of what the machine is. This appendix works through what the architecture implies for those situations. Nothing here is a special mode or a supported feature — it is the consequence of properties established elsewhere, applied to topologies the earlier chapters did not name.

A0.1 The daemon is per-UID per-kernel, and a remote shell is on the far kernel

The socket is named for a user identity, and a UNIX socket is a kernel object. Both halves matter. The naming makes the daemon per-user; the kernel object makes it per-machine. There is no network transport in the design at any layer — the wire protocol is spoken over a local socket and nothing else — so a shell on another machine cannot reach your daemon and would not want to.

The consequence is the correct one and it is worth stating so nobody goes looking for configuration that does not exist. A remote shell renders its prompt using the remote machine's powerliners, against the remote machine's config cascade, talking to the remote machine's daemon, reporting the remote machine's CPU and disk and battery. That is what a prompt on that machine should say. A prompt is a description of the session you are in, and the session is over there.

Two operational facts follow. First, the performance argument applies independently on each machine: a remote host without a running daemon pays the cold path on every prompt, and the cold path over a link with latency is on top of whatever the link costs. A machine you shell into frequently deserves its own daemon in its own rc file for the same reason your workstation does. Second, upgrades are per-machine, which composes badly with Chapter 25's skew: a fleet upgraded at different times has daemons of different generations, each frozen at whenever that machine's session started, and the pidfile-versus-binary-timestamp check has to be run per host.

A0.2 A shared home directory means one config and many daemons

The config cascade is rooted in the user's configuration directory, and on a fleet with a network-mounted home directory that root is one tree shared by every machine. This is a common arrangement and it produces a specific pattern worth understanding.

The tree is shared, so an edit made anywhere is an edit everywhere. The daemons are not shared, so each machine's daemon independently notices. Chapter 13's invalidation is mtime-keyed and pull-based, which handles this without modification: each daemon compares the mtimes it recorded against what it now sees, and each rebuilds on the first render after the edit. There is no coordination and none is needed, because there is no shared state between the daemons — only shared inputs.

The one property to be aware of is that mtime comparison across a network filesystem inherits that filesystem's timestamp semantics, including whatever granularity and clock agreement it provides between the writing client and the reading one. Chapter 13 already argues that the check errs toward rebuilding, and that error direction is the right one here too: a timestamp that appears to move when nothing changed costs one setup, and the alternative error costs correctness. On a shared home directory the “unnecessary rebuild” case is simply more likely, and it remains the cheap error.

The other consequence is a nice one. Because the config is shared and the render is deterministic given the config, every machine in the fleet renders the same prompt shape from the same theme — and the segments that differ are exactly the ones describing the machine, which is the useful difference. The hostname segment earns its place in this topology in a way it does not on a single workstation.

AO.3 Containers, and the namespace the abstract socket lives in

Chapter 14 establishes that the Linux abstract socket's scope is the network namespace. That sentence is written there as a security observation. In a container context it is a connectivity fact, and it is the single most consequential thing in this appendix.

A container with its own network namespace cannot reach an abstract socket in the host's namespace, and vice versa. The socket is not hidden by a permission check, it is not unreachable due to a path — it is *not in that namespace at all*, which is a stronger form of separation than any access control. A shell inside such a container will therefore fail to connect, and Chapter 4's fallback will do exactly what it exists for: the client execs the direct renderer and the prompt renders correctly and more slowly. This is the failure mode working as designed, and the visible symptom is that prompts inside the container are slower than prompts outside it while looking identical.

The remedy, if the container is a long-lived development environment where prompt latency matters, is to run a daemon inside it. That is a per-namespace daemon serving the shells in that namespace, which is the same model as always — the container is simply a machine for this purpose, and the architecture treats it as one because the kernel does.

The filesystem-socket platforms behave differently and the difference is instructive. There the socket is a path, so reachability is determined by whether the path is visible and permitted, and a container that shares the relevant filesystem view can reach a daemon across the boundary where an abstract socket would have prevented it. The two namespaces of Chapter 4 therefore have genuinely different topological behavior, not merely different access-control stories, and a reader reasoning about container isolation should know which one their platform gives them.

One more consequence for anyone building images: the config cascade inside a container is the container's cascade. A container that does not carry the user's configuration directory resolves to whatever the bundled defaults provide, and a prompt that looks unexpectedly plain inside a container is almost always this rather than anything to do with the daemon.

AO.4 Nesting, and two hosts drawing on one surface

A multiplexer inside a multiplexer produces two statuslines, each rendered by its own invocation, each laid out against its own width, neither aware of the other. There is nothing special in the architecture about this because there is nothing shared between the two renders — they are separate requests carrying separate context, and the daemon has no notion that they came from related processes. Chapter 4's statement that there is no session and no per-client state is what makes nesting a non-event.

What nesting does affect is the geometry of Chapter 26. The inner multiplexer's surface is smaller than the outer one's by the outer one's chrome, and each host reports its own surface correctly, so both lines are laid

out correctly for the space they actually have. The inner line is narrower and will truncate earlier, which is right. A reader confused about why a statusline drops segments in a nested session and not in a plain one has found the answer: those are different widths, and priority truncation is doing precisely what it was asked to.

The related case is one multiplexer session attached from two terminals of different sizes. The session has one statusline configuration but the attached clients have different geometries, and how that is resolved is entirely the multiplexer's business — it is the party that owns the surface, and Chapter 26 puts geometry with the party that has the information. Whatever width it reports is the width the render uses. This is a place where the honest answer is that powerliners has no policy because it has no standing to have one, and any apparent oddity in the layout under multi-attach is a question for the host's documentation rather than the renderer's.

AO.5 What a session-aware prompt should actually show

The placement rule this appendix produces is a variant of the one Chapter 15 gives on latency grounds and Chapter 20 gives on sampling grounds, and its recurrence in a third analysis is the same evidence of structural rightness that Chapter 20 notes.

A prompt should say what distinguishes the session it is in from the sessions it is not in. On a single workstation that is almost nothing, which is why a hostname segment is usually noise there — Appendix A's hostname entry carries an argument to show the name only when the session is remote, which is exactly this rule encoded as a default. In a fleet, in containers, and in nested sessions the distinguishing facts multiply, and each one is worth a segment only to the degree that confusing two sessions would cost you something. The classic expensive confusion is running a destructive command on the wrong host, and the classic cheap one is not knowing which container you are in, which is why identity segments earn their space in proportion to how many identities are in play.

The cost side of that judgment is the same as everywhere else. An identity segment reading a forwarded environment variable is one of the cheap-check absences of Appendix AL — a map lookup, no syscall — so an identity segment that shows nothing on your workstation and something over a remote connection is very nearly free in the case where it shows nothing. That is the ideal shape for a segment in this class: silent and free at home, informative and still cheap away.

AO.6 The identifiers that are only locally meaningful

A closing detail that is easy to get wrong when reasoning about fleets. The socket name contains a numeric user identity, and numeric user identities are assigned per machine. Two users on two machines can share a number and two accounts for the same person can have different ones. None of this causes a collision, because the sockets are kernel objects that never leave their machine — the name only ever has to be unique within the kernel that holds it, and it is.

The pidfile is the piece that touches a filesystem, and on the filesystem platforms it lives in the temporary directory alongside the socket. It is per-machine for the same reason, and the only arrangement in which it could collide is one where two machines share that directory, which is not an arrangement anyone builds deliberately. The reason to write this down is that “a global identifier derived from a local number” is a pattern that usually deserves suspicion in distributed reasoning, and here it does not, because nothing is distributed. The daemon, its socket, its pidfile, and its clients are all on one kernel by construction, and the identifier only has to be unique in the scope it is used in. Naming that explicitly is what lets a reader stop worrying about it.

Appendix AP — The Reproducibility Ledger

The book is candid throughout about where powerliners' output is byte-identical to upstream and where it legitimately cannot be (Chapter 9, Appendix AB.4, Appendix Q.4). Those admissions are scattered across the chapters that raise them; this final appendix collects them into one ledger, because the precise boundary between “must match exactly” and “cannot match and should not claim to” is the book's central honesty and deserves a single place a reader can check it.

AM.1 The three tiers of reproducibility

Every byte powerliners emits falls into one of three tiers. **Structurally reproducible** output must be byte-identical to upstream and is proven so by the parity harness (Appendix F): the tmux/shell/vim markup, the #-doubling and escape encoding, the color resolution to (`cterm`, `hex`), the divider placement and fill, the truncation order, the divider math. Given the same config and the same inputs, these produce the same bytes, and a diff that finds a difference is a bug. **Input-dependent but deterministic** output depends on state the caller forwards — the cwd, the environment, the exit code, the mode — and is reproducible *given the same forwarded state*: two renders with the same `segment_info` produce the same bytes, and the parity harness controls the inputs precisely so it can assert this. **Live-sampled** output depends on a measurement taken at render time — CPU load, memory use, network counters, the current time, a weather reading — and is *not* reproducible even in principle, because two samplers reading a moving quantity at two instants read two values.

AM.2 The live-sampled set, named

Honesty requires naming the third tier exactly rather than gesturing at it, because a reader deserves to know which specific segments will differ. The live-sampled segments are the metrics and clocks: `cpu_load_percent`, `system_load`, `mem_usage/mem_swap`, `disk_usage/disk_io`, `gpu_usage_percent/gpu_vram`, `thermal`, `network_load`, `battery`, `uptime`, `date/fuzzy_time`, `external_ip` and `weather` (network reads), and the mail and player segments. For every one of these, the *markup* — the colors, the format, the gradient bucket the value lands in — is structurally reproducible, and only the *number inside the markup* is not. A parity assertion on `cpu_load_percent` therefore checks the encoding and the gradient selection against a fixed injected value, not against a live sample, because a live sample was never going to match across two processes sampling at two instants. This is not a gap in the port; it is a property of measuring a moving world, and the port marks it rather than pretending a clock reads the same twice.

AM.3 The documented divergences

Beyond the live-sampled tier there are exactly three *behavioral* divergences the book flags as known and bounded, and collecting them closes the ledger. First, the `strwidth` gap (Chapter 7, Appendix AB.2): the port measures Unicode scalar values, not display cells, so a line packed with wide CJK or emoji glyphs can be one cell off in truncation and alignment; the common case (ASCII, powerline glyphs, Latin text) is exact. Second, the `gen_segment_getter` highlight-group divergence (Chapter 3, Appendix AB.1): a `"type"`:

"function" segment uses its resolved `function_name` as its sole highlight group rather than honoring a theme's explicit `highlight_groups` override, with the two documented workarounds. Third, the daemon's config reload (Chapter 4): upstream's `SIGHUP`-driven re-exec is replaced by the per-`ext` mtime-staleness reload plus `--replace`, a lifecycle difference with no output consequence. These three are the whole of the behavioral divergence surface — everything else is either structurally identical or a live measurement — and the port names all three in the open rather than letting a user discover them as surprises.

AM.4 Why the ledger is the honesty

The reason this ledger is the right note to end the appendices on is that it *is* the book's thesis stated as a boundary. powerliners claims to be a faithful port, and a faithful-port claim is only meaningful if it is falsifiable — if there is a precise statement of what "faithful" covers and a mechanism that would catch a violation. The ledger is that statement: structurally-reproducible output is byte-checked against the live interpreter and any difference is a bug; input-dependent output is reproducible given the inputs and the harness controls them; live-sampled output is honestly excluded because it is not reproducible in physics, not merely in this implementation. A port that claimed byte-identical output including live CPU load would be lying, and a port that hid its three behavioral divergences would be hiding the places a careful user could catch it. powerliners does neither. It draws the line exactly, proves fidelity on the correct side of it, and names the exceptions on the other — which is, in the end, the difference between a program that says it is a faithful port and one that has made its faithfulness auditable. Same prompt, no Python, sub-millisecond, proven — and the word *proven* has a ledger behind it.

```
// END OF BOOK // PROMPT LOCKED, NATIVE //
```

Colophon

This book was written alongside the port it describes, and it is generated from one source. `docs/book.md` is the manuscript; `scripts/book_pdf.sh` renders it to PDF through pandoc and LuaLaTeX with the project's own theme, and `scripts/book_epub.sh` renders the same manuscript — plus this apparatus — to a reflowable EPUB 3. The cover is not hand-placed either: `scripts/book_cover.sh` draws it at 1600×2560 from the same palette the PDF theme uses, so the cover on a storefront and the chapter headings inside the book cannot drift apart.

The display faces are Orbitron and Share Tech Mono; body text is set in Arimo. All three are licensed under the SIL Open Font License 1.1.

There is no generated reference manual for this book. `powerliners` ships man pages — `powerline(1)`, `powerline-daemon(1)`, `powerline-config(1)`, `powerline-render(1)`, `powerline-lint(1)` — rather than a gen-docs crate, so this book is the long-form deliverable and its appendices carry the reference matter: the segment catalog, the config schema, the colour and highlight-group reference, the renderer reference, and the daemon protocol.

Every figure in these pages was taken from the source tree at this edition by running the command that produces it, and is cited to that command so a reader can check it rather than trust it. The book is deliberate about the difference between a count and a result: Chapter 34 publishes the whole test distribution rather than a headline, and Chapter 51 states, per figure, which numbers are stable and which will move. Two qualifications travel with the port-tier ratio everywhere it appears, because neither is visible in the number itself — the DONE tier grades citation and doccomment density rather than executed behaviour, and four of the graded rows are files outside the crate's module tree that never compile.

Reproducing the Book's Claims

The book is written to be checkable. With the powerliners repository cloned, these are the commands behind its central claims. Chapter 34 and Appendix G give the full method; these are the load-bearing ones.

The library test suite, which Chapter 34 reports as a distribution rather than a constant:

```
cargo test --lib
```

The parity harness — the same inputs through the real Python powerline, running on the vendored upstream source, and through powerliners, compared byte for byte:

```
cargo test --test parity_against_upstream
```

The drift gate, which fails the build if a ported function appears with no counterpart upstream:

```
cargo test --test ported_fn_names_match_py
```

The daemon's own suites — byte-for-byte scenarios against upstream, malformed and hostile wire input, socket lifecycle, and concurrency under backpressure:

```
cargo test --test daemon_parity
cargo test --test daemon_fuzz
cargo test --test daemon_e2e
cargo test --test daemon_stress
```

The performance claims are the one place the book asks for a method rather than a command, and Appendix G gives it: the same `powerline shell left` invocation, same config, timed three ways — Python cold, powerliners cold through `powerline-render`, and powerliners warm through the daemon — over enough repetitions to report a distribution. The three tiers have three different causes, and a benchmark that collapses them into one number cannot say which lever did what.

Also from MenkeTechnologies

powerliners is one program in a stack that is documented the same way — from source, with the numbers cited to the tools that produce them, and with the upstream named plainly wherever a program is a port. Companion volumes cover the shell, the terminal emulator, the terminal multiplexer, the bytecode virtual machine the language frontends share, and the wider command-line fleet.

Titles and availability change; the current catalog is published at github.com/MenkeTechnologies.